

台灣自編大學教科書

@prof

## 機器人學導論

機構、感知、控制與具身人工智慧

### 第三篇

#### 動力學、軌跡與控制

第 13 章 操作臂靜力學與拉格朗日動力學

第 14 章 Newton-Euler 動力學與模擬引擎

第 15 章 軌跡生成與運動曲線

第 16 章 線性回授與 PID 控制

第 17 章 操作臂非線性控制

第 18 章 最佳控制與 Riccati 方程

第 19 章 可控性與穩定性

第 20 章 力控制、阻抗控制與柔順操作

第 21 章 抓取與操作

編著：李世淵 ( 機器人叫獸 )

助教：李天宇、李宇晴

2026 年 9 月

## 第三篇 篇章導讀

第二篇教你機器人**能不能到那裡**，這一篇要教你**該用多大的力氣到那裡**。

第二篇的所有分析都是運動學的——只談位置、速度與幾何，隱含一個假設：關節可以瞬間達到任何我們指定的速度。這在低速時尚可接受，但真實的手臂有質量、有慣性、受重力作用。它在加速時會抵抗你、在轉彎時會產生你沒預料到的力、在停止時因為慣性而繼續往前衝。第三篇的九章，就是要處理這些「力」的問題。

**第 13、14 章（動力學）**建立「力與運動」的數學模型。前者用拉格朗日法從能量出發，讓我們**看懂**方程的結構——慣性、科氏力、離心力與重力四類項各自從何而來；後者改用 Newton-Euler 遞迴，把複雜度由  $O(n^3)$  降到  $O(n)$ ，讓我們**算得夠快**，能在毫秒級的控制迴路中即時執行。

**第 15 章（軌跡生成）**決定「該走哪條路」。同一條路徑，不同的時間安排會帶來截然不同的結果——慢慢加速反而更快，因為粗暴的運動會激發振動，等它平息的時間比省下來的還多。這一章也首次把動力學模型用於檢驗軌跡的可行性。

**第 16、17 章（控制）**讓機器人真的走上那條路。第 16 章從最基本的 PID 出發，建立回授的基礎與極限；第 17 章則把動力學模型放進控制器，用計算力矩控制突破 PID 的限制。核心哲學是一句話：**用模型處理已知的部分，用回授處理未知的部分**。

**第 18、19 章（最佳性與穩定性）**回答兩個更深的問題。增益該怎麼選才是最好的？——第 18 章用代價函數與 Riccati 方程給出可計算的答案。我們憑什麼相信控制器一定不會失控？——第 19 章用 Lyapunov 方法給出嚴格的證明。

**第 20、21 章（接觸與抓取）**讓機器人走出自由空間。一旦與環境接觸，前面的邏輯就完全反轉：高增益的精確位置控制反而成為最危險的設定。這兩章的核心洞見是——**與其消除不確定性，不如設計一個對不確定性不敏感的系統**。

### 這一篇要回答的七個問題

- 手臂有重量，加速時需要多大的力矩？關節之間又如何互相干擾？
- 六軸手臂的動力學方程展開有上萬項，控制器如何在一毫秒內算完？
- 同樣的路徑，為什麼「慢慢加速」反而比「全速衝過去」更快？

- 為什麼低速調校完美的 PID，一提速就失準？調高增益為何治標不治本？
- 控制器的增益該取多少才是「最好」的？這個問題有沒有標準答案？
- 我們憑什麼相信一個控制器一定不會失控？
- 機器人碰到東西時，為什麼精確的位置控制反而會壓碎雞蛋？

### 三條貫穿全篇的線索

**線索一：模型的價值不在完美，而在減負。** 第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍。追求完美的模型往往不划算，因為剩下的部分交給回授即可。

**線索二：理論的各部分互相支撐。** 第 13 章那個當時看似無用的「 $\dot{M} - 2C$  為斜對稱」性質，兩百頁後成為第 19 章穩定性證明的樞紐；第 11 章的雅可比對偶，則先後成為第 17 章操作空間控制的核心、第 20 章力映射的基礎、第 21 章抓取矩陣的原型。

**線索三：機構與控制必須協同設計。** 第 20 章的插銷入孔靠一個成本幾乎為零的倒角，就把容許誤差放寬 41 倍；第 21 章的欠驅動手用機構取代了大量的感測與計算。反覆出現的教訓是：在把問題丟給控制器之前，先問能不能用機構把問題變簡單。

### 讀完這一篇之後

你將能夠：建立機器人的完整動力學模型並在微秒級計算它、規劃兼顧速度與平順度且通過力矩檢驗的軌跡、設計並實作從 PID 到計算力矩控制的各級控制器、用最佳控制理論計算增益並用 Lyapunov 方法證明其穩定性、以及讓機器人安全地與環境接觸、溫柔地抓取物體。

但第三篇的所有技術，都建立在一個前提上：**機器人知道自己在哪裡、知道目標在哪裡**。固定基座的手臂靠編碼器就知道自己的姿態，工件的位置則由治具保證。一旦機器人要移動、要面對未知的環境，這個前提就不成立了。

第四篇「感知、定位與自主移動」將處理這些問題。屆時你會發現，第三篇的工具並未被拋棄——控制理論仍是無人機姿態穩定與四足步態的基礎。只是問題的重心，從「精確地執行」轉向了「正確地理解」。

### 本篇章節配置

| 章次     | 章名                    | 核心內容                                     | 叫獸開講                                  |
|--------|-----------------------|------------------------------------------|---------------------------------------|
| 第 13 章 | 操作臂靜力學與拉格朗日動力學        | 慣性張量、拉格朗日方程、標準形式、慣性矩陣、科氏力與重力項            | 「知道手臂要往哪動，為什麼還不知道馬達要出多少力？」——手臂是一桶會晃的水 |
| 第 14 章 | Newton-Euler 動力學與模擬引擎 | 前向與後向遞迴、 $O(n)$ 複雜度、正向動力學、數值積分、接觸模型      | 「每毫秒都要算一次力矩，怎麼算才夠快？」——一萬四千項的正確答案      |
| 第 15 章 | 軌跡生成與運動曲線             | 多項式、梯形與 S 形曲線、急跳度、混成、時間縮放、動力學檢驗          | 「到達同一位置，為什麼有些動作像機器？」——慢慢加速，反而更快       |
| 第 16 章 | 線性回授與 PID 控制          | 開閉迴路、自然頻率與阻尼比、積分飽和、數位實作、調參方法             | 「機器手臂到了目標位置，為什麼還在左右發抖？」——手上立竹竿的三個祕訣   |
| 第 17 章 | 操作臂非線性控制              | 計算力矩控制、回授線性化、重力補償、操作空間控制、滑模與自適應          | 「搬起重物後，原本調好的控制器為什麼失準？」——讓末端像一顆點質量     |
| 第 18 章 | 最佳控制與 Riccati 方程      | Bellman 原理、價值函數、HJB 方程、LQR、權重選擇、倒立擺      | 「倒立搖桿如何自己找出最省力的平衡之道？」——把「什麼叫好」寫成公式    |
| 第 19 章 | 可控性與穩定性               | 可控性矩陣、三種穩定性、Lyapunov 直接法、LaSalle 原理、伏筆回收 | 「控制器算出完美指令，系統卻仍翻車？」——彈珠一定會滾到碗底        |
| 第 20 章 | 力控制、阻抗控制與柔順操作         | 環境剛度、自然與人為約束、混合控制、阻抗與導納、接觸穩定性、RCC        | 「拿雞蛋不能只知道位置，還要知道用了多少力。」——控制兩者之間的關係    |
| 第 21 章 | 抓取與操作                 | 摩擦錐、抓取矩陣、力封閉、形封閉與關入籠、品質指標、欠驅動手           | 「戴著厚手套、閉著眼睛，為什麼還是抓得起來？」——柔順勝過精準       |

## 第三篇 動力學、軌跡與控制

# 第 13 章

## 操作臂靜力學與拉格朗日動力學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一位服務生端著滿滿一盤飲料快步轉彎的照片，杯中液面明顯傾斜，並以箭頭標示慣性、離心力與科氏力三個方向；右半為六軸手臂高速轉彎的多重曝光影像，末端軌跡以紅色虛線標出偏離理想路徑的部分。中央以四色方塊並列標註動力學方程的四項： $M(q)\ddot{q}$ （藍）、 $C(q,\dot{q})\dot{q}$ （橙）、 $G(q)$ （紅）、 $F(q)$ （灰）。標題文字：手臂不是沒有重量的幾何模型——它是一桶會晃的水。

## 叫獸開講 「知道手臂要往哪裡動，為什麼還不知道馬達要出多少力？」

實驗室的六軸手臂已經調校了兩週，末端的重複精度達到  $\pm 0.05$  公釐，在低速下畫圓、走直線都完美無缺。今天要驗收的是最後一項：把速度調到額定值的八成，執行同一段軌跡。

小玆按下啟動。手臂加速——然後在轉彎處明顯地「甩」了一下，末端偏離軌跡將近三公釐，接著在停止時抖了兩三下才穩住。示教器上跳出黃色警告：追隨誤差過大。

「可是低速時明明完美啊，」小玆翻著控制器參數，「是不是 PID 的增益要再調高一點？」

叫獸沒有回答，反而問了一個看起來無關的問題：「你有沒有提過裝滿水的水桶走路？」

小玆點頭。「走慢的時候，水面很平靜；一旦走快、或是突然轉彎，水就會晃出來——而且你必須用力抵抗那個晃動，否則會被水的慣性拉得踉蹌。」叫獸指著手臂，「你的機器手臂就是那桶水。前面十二章，我們把它當成一個沒有重量的幾何模型——關節說轉幾度就轉幾度，說停就停。但真實的手臂有質量、有慣性。它在加速時會抵抗你、在轉彎時會產生你沒預料到的力、在停止時因為慣性而繼續往前衝。」

他在白板上寫下三個詞：慣性、科氏力、重力。「你的 PID 控制器只知道『現在差了多少度』，它不知道『這個姿態下第二軸需要多大的力矩才能加速』，更不知道『第三軸轉動時會給第二軸一個額外的干擾力』。所以它只能事後補救——誤差出現了才修正。速度越快，補救得越不及時。」

小玆問：「那該怎麼辦？把增益調到很高？」

「那會讓它變得很硬，撞到東西時很危險，而且會激發機構的共振。」叫獸搖頭，「正確的作法是讓控制器事先知道：我要在這個姿態下、以這個加速度運動，需要多大的力矩？把這個力矩直接送給馬達，PID 就只需要修正那一點點模型誤差。這叫做前饋，而要算出那個力矩，就需要一個東西——動力學模型。」

他把筆放下：「第二篇我們問的是『手臂能不能到那裡』。從今天開始，第三篇要問的是：『該用多大的力氣到那裡』。」

## 本章為什麼重要

第二篇的運動學分析有一個隱含假設：關節可以瞬間達到任何指定的速度。這在低速時尚可

接受，但在高速運動、重負載或需要精確力控制時就會失效——因為機器人有質量、有慣性、受重力作用。

動力學回答的是「力與運動的關係」：要產生某個加速度，各關節需要輸出多少力矩？反之，給定力矩，機器人會怎麼運動？這個模型是三件事的基礎：**高精度高速控制**（第 17 章的計算力矩控制與前饋）、**力控制與碰撞偵測**（第 20 章，必須先扣除慣性與重力的貢獻才能辨識外力）、以及**模擬**（第 30 章的物理引擎、第 31 章的強化學習訓練環境）。

本章採用拉格朗日方法——它從能量出發，不需畫受力圖、不需處理內力，只要寫出動能與位能，微分幾次就得到運動方程。這個方法在推導與理解上極為優雅，適合建立物理直覺；第 14 章則會介紹計算效率更高的 Newton-Euler 遞迴法，兩者互補。

## 學習目標

- 能定義質心、轉動慣量與慣性張量，並說明其物理意義。
- 能運用平行軸定理計算物體對任意軸的轉動慣量。
- 能寫出機器人連桿的動能與位能表達式。
- 能敘述拉格朗日方程並說明其相對於牛頓法的優勢。
- 能推導單擺與平面 2R 手臂的完整動力學方程。
- 能辨識並解釋動力學方程中的慣性項、科氏力項、離心力項與重力項。
- 能說明慣性矩陣的物理意義及其正定、對稱、姿態相依三個性質。
- 能解釋耦合效應：為何一個關節的運動會影響其他關節。
- 能說明摩擦力模型及其對控制的影響。
- 能寫出機器人動力學的標準形式並說明各項的角色。
- 能以 Python 實作動力學方程並驗證能量守恆。

## 先備知識

第 9 章的正向運動學（用於計算各連桿質心的位置與速度）、第 11 章的雅可比矩陣（連桿質心的速度由雅可比計算）。物理方面需要高中的功、能量、動量與轉動慣量概念；數學方面需要偏導數與鏈鎖律。

本章的推導量是全書最大的一章之一。建議讀法：先確實掌握 13.5 節的單擺範例（每一步都親手算過），再進入 13.6 節的平面 2R 手臂。若中途卡住，多半是能量表達式沒寫對——回頭檢查速度的計算，而非懷疑微分。

## 核心名詞中英對照

表 13-1 本章核心名詞

| 中文     | 英文                           | 說明                          |
|--------|------------------------------|-----------------------------|
| 質心     | Center of Mass, CoM          | 物體質量分布的平均位置                 |
| 轉動慣量   | Moment of Inertia            | 物體抵抗角加速度的量度                 |
| 慣性張量   | Inertia Tensor               | 描述剛體三維轉動慣性的 $3 \times 3$ 矩陣 |
| 平行軸定理  | Parallel Axis Theorem        | 由質心軸的慣量求任意平行軸慣量的定理          |
| 廣義座標   | Generalized Coordinates      | 能完整描述系統位形的獨立變數              |
| 拉格朗日量  | Lagrangian                   | 動能減位能， $L = T - U$          |
| 拉格朗日方程 | Euler-Lagrange Equation      | 由能量推導運動方程的方程式               |
| 慣性矩陣   | Inertia Matrix / Mass Matrix | $M(q)$ ，連結關節加速度與力矩          |
| 科氏力    | Coriolis Force               | 兩軸同時轉動時產生的耦合力               |
| 離心力    | Centrifugal Force            | 單軸轉動時因圓周運動產生的力              |
| 重力向量   | Gravity Vector               | $G(q)$ ，平衡重力所需的關節力矩         |
| 耦合     | Coupling                     | 一個關節的運動影響其他關節的現象            |
| 黏滯摩擦   | Viscous Friction             | 與速度成正比的摩擦力                  |
| 庫倫摩擦   | Coulomb Friction             | 與速度方向相反、大小固定的摩擦力            |
| 逆動力學   | Inverse Dynamics             | 由運動求所需力矩                    |
| 正向動力學  | Forward Dynamics             | 由力矩求所產生的運動                  |

## 13.1 質量、質心與轉動慣量

### 13.1.1 三個基本的慣性參數

要描述一根連桿的動力學特性，需要三組參數。

**質量  $m$** ：一個純量，描述物體抵抗線加速度的能力。由牛頓第二定律  $F = ma$ ，質量越大，同樣的力產生的加速度越小。



**質心位置  $p_c$** ：一個三維向量，描述質量分布的平均位置。它的重要性在於：**重力可視為全部作用在質心上的單一力**，而剛體的平移運動也可以用質心的運動來描述。對均勻的細長連桿，質心在中點；對實際的機器人連桿（一端有馬達、另一端有減速機），質心往往明顯偏離幾何中心。

**慣性張量  $I$** ：一個  $3 \times 3$  的對稱矩陣，描述物體抵抗**角**加速度的能力。它是轉動版本的「質量」，但比質量複雜得多——因為繞不同軸旋轉的難易程度不同。

### 13.1.2 為什麼轉動慣量是矩陣

線運動的慣性只需一個數（質量），因為無論往哪個方向推，抵抗都相同。但轉動不然：一根長桿繞其長軸旋轉很容易，繞垂直於長軸的方向旋轉則困難得多。因此需要一個矩陣來描述各方向的差異：

$$I = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

慣性張量。對角元素稱為**轉動慣量**（繞各軸的慣性），非對角元素稱為**慣性積**（描述軸之間的耦合）。矩陣恆為對稱，故只有六個獨立元素。

角動量與角速度的關係為  $L = I \cdot \omega$ 。注意這裡有一個關鍵差異：**角動量的方向未必與角速度相同**。只有當旋轉恰好繞著慣性主軸時，兩者才平行。這解釋了一個常見現象——旋轉一個形狀不規則的物體時，它會自己「翻滾」，因為角動量守恆而角速度方向不斷改變。

實務上，若能選擇適當的座標軸（慣性主軸），慣性積會全部為零，矩陣化為對角形式。CAD 軟體通常能直接輸出連桿的質量、質心與慣性張量——這是建立動力學模型最實用的資料來源。

### 13.1.3 常見形狀的轉動慣量

實務估算中最常用的三個公式為：

$$\text{細長桿繞質心（垂直於桿）：} I = (1/12) m L^2$$

$$\text{細長桿繞一端（垂直於桿）：} I = (1/3) m L^2$$

$$\text{質點繞距離 } r \text{ 的軸：} I = m r^2$$

第 5.10.3 節在馬達選型時已用過前兩式，此處給出完整脈絡。

### 13.1.4 平行軸定理

已知物體繞其質心的轉動慣量  $I_c$ ，要求繞另一條平行軸（距離  $d$ ）的轉動慣量：

$$I = I_c + m d^2$$

平行軸定理。這是動力學計算中使用頻率最高的定理之一。

驗證一致性：細長桿繞質心為  $(1/12)mL^2$ ，質心到端點距離  $d = L/2$ ，故繞端點為  $(1/12)mL^2 + m(L/2)^2 = (1/12 + 1/4)mL^2 = (1/3)mL^2$  ✓ 與上述公式相符。

平行軸定理在機器人動力學中的角色極為關鍵：連桿的慣量參數通常以質心座標系給出（CAD 輸出），但動力學方程需要的是繞關節軸的慣量——兩者之間就靠這條定理連結。而  $m d^2$  這一項也解釋了第 5.10.2 節的設計原則：把重物往轉軸移近，慣量以距離的平方下降。

## 13.2 動能與位能

### 13.2.1 剛體的動能

一個運動中的剛體，其動能包含兩部分：質心的平移動能，加上繞質心的轉動動能。

$$T = \frac{1}{2} m \mathbf{v}_c^T \mathbf{v}_c + \frac{1}{2} \boldsymbol{\omega}^T I \boldsymbol{\omega}$$

剛體動能。第一項為平移（ $\mathbf{v}_c$  為質心速度）、第二項為轉動（ $\boldsymbol{\omega}$  為角速度、 $I$  為繞質心的慣性張量）。

這個「平移 + 轉動」的分解稱為 König 定理，它的價值在於把複雜的剛體運動拆成兩個獨立的部分——這正是我們能用有限個參數描述動力學的原因。

對整支機器手臂，總動能是各連桿動能的總和：

$$T = \sum_i \left[ \frac{1}{2} m_i \mathbf{v}_{c_i}^T \mathbf{v}_{c_i} + \frac{1}{2} \boldsymbol{\omega}_i^T I_i \boldsymbol{\omega}_i \right]$$

$n$  根連桿的動能總和。

### 13.2.2 用雅可比表達動能

上式中的  $\mathbf{v}_{c_i}$  與  $\boldsymbol{\omega}_i$  都是關節速度  $\dot{\mathbf{q}}$  的函數。運用第 11 章的雅可比概念，可以把每一根連桿質心的速度寫成：

$$\mathbf{v}_{c_i} = J_{v_i}(\mathbf{q}) \cdot \dot{\mathbf{q}} \quad \boldsymbol{\omega}_i = J_{\omega_i}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

$J_{v_i}$  與  $J_{\omega_i}$  是「以連桿  $i$  的質心為末端」的雅可比矩陣，可用第 11.4 節的幾何法直接構造。

代入動能表達式並整理：

$$T = \frac{1}{2} \mathbf{q}^T \left[ \sum_i (m_i \mathbf{J}_{-v_i}^T \mathbf{J}_{-v_i} + \mathbf{J}_{-\omega_i}^T \mathbf{I}_i \mathbf{J}_{-\omega_i}) \right] \mathbf{q}' = \frac{1}{2} \mathbf{q}^T \mathbf{M}(\mathbf{q}) \mathbf{q}'$$

定義中括號內的矩陣為慣性矩陣  $\mathbf{M}(\mathbf{q})$ 。

這是本章第一個重要成果。它顯示：機器人的動能永遠可以寫成  $\frac{1}{2} \mathbf{q}^T \mathbf{M}(\mathbf{q}) \mathbf{q}'$  的二次型，其中  $\mathbf{M}(\mathbf{q})$  是一個只與姿態有關（與速度無關）的  $n \times n$  矩陣。這個矩陣將是整個動力學方程的核心，13.7 節會詳細討論它的性質。

### 13.2.3 位能

機器人的位能主要來自重力（若有彈簧則另計）：

$$U = \sum_i m_i g h_{-c_i}(\mathbf{q}) \quad (h_{-c_i} \text{ 為第 } i \text{ 根連桿質心的高度})$$

位能只與姿態  $\mathbf{q}$  有關，與速度無關。這個「與速度無關」的性質，稍後會簡化拉格朗日方程的推導。

更一般的寫法是用向量形式： $U = -\sum_i m_i \mathbf{g}^T \mathbf{p}_{-c_i}(\mathbf{q})$ ，其中  $\mathbf{g}$  是重力加速度向量（通常為  $[0, 0, -9.8]^T$ ）。這個寫法不需假設重力沿特定方向，適用於傾斜安裝或移動平台上的機器人。

## 13.3 拉格朗日方程

### 13.3.1 方程式與意義

拉格朗日力學的核心是一個純量函數——拉格朗日量：

$$L(\mathbf{q}, \mathbf{q}') = T(\mathbf{q}, \mathbf{q}') - U(\mathbf{q})$$

動能減位能。注意是「減」而非「加」，這一點常令初學者困惑，但它是變分原理推導的必然結果。

系統的運動方程則由拉格朗日方程（又稱 Euler-Lagrange 方程）給出：

$$\frac{d}{dt} \left( \frac{\partial L}{\partial \mathbf{q}_i'} \right) - \frac{\partial L}{\partial \mathbf{q}_i} = \tau_i \quad (i = 1, 2, \dots, n)$$

對每一個廣義座標各寫一式，共  $n$  個方程。 $\tau_i$  為作用在第  $i$  個廣義座標上的廣義力（旋轉關節為力矩、移動關節為力）。

### 13.3.2 為什麼這個方法更好用

與牛頓法（逐一分析每個剛體的受力）相比，拉格朗日法有四個決定性的優勢。

**優勢一：不需畫受力圖。**牛頓法必須逐一找出每根連桿上的所有作用力，包括關節處的內力（連桿之間互相推擠的力）。對六軸手臂而言，這是數十個力向量的分析。

**優勢二：內力自動消去。**關節處的約束力（constraint force）不做功，因此完全不會出現

在能量表達式中。這是最大的簡化——牛頓法必須算出這些力再消去，拉格朗日法直接跳過。

**優勢三：座標系自由。**可以使用任何一組能描述系統位形的獨立變數（廣義座標），不必侷限於直角座標。對機器人而言，關節角度是最自然的廣義座標。

**優勢四：純量運算。**動能與位能都是純量，比向量運算不易出錯。

代價是：需要先寫出能量表達式（要算出每根連桿質心的速度，這需要雅可比），且微分過程對多自由度系統會相當繁瑣。這正是第 14 章引入 Newton-Euler 遞迴法的原因——它在計算效率上遠勝拉格朗日法。

#### ◎ 兩種方法的分工

拉格朗日法適合**推導與理解**：它能給出結構清晰的解析表達式，讓你看見慣性矩陣、科氏項與重力項各自的來源。Newton-Euler 法適合**計算與實作**：它以遞迴方式運算，複雜度為  $O(n)$ ，是控制器內部即時計算的標準作法。兩者得到的方程完全相同，只是路徑不同。本書第 13 章用前者建立理解，第 14 章用後者處理實作。

### 13.3.3 推導的四個步驟

運用拉格朗日法推導機器人動力學，可依下列步驟進行：

1. **選定廣義座標**：對串聯機器手臂，直接取關節變數  $q = [q_1, \dots, q_n]^T$ 。
2. **寫出動能 T**：計算每根連桿質心的速度（線速度與角速度），代入 13.2.1 節的公式並加總。
3. **寫出位能 U**：計算每根連桿質心的高度，乘上質量與重力加速度後加總。
4. **代入拉格朗日方程並微分**：對每個  $q_i$  計算  $\partial L / \partial q_i$ 、對時間微分、再減去  $\partial L / \partial q_i$ ，即得該關節的運動方程。

## 13.4 最簡單的例子：單擺

### 13.4.1 建立模型

在處理機器手臂之前，先用最簡單的例子熟悉整個流程。考慮一根長度  $L$ 、質量  $m$  的均勻細桿，一端以旋轉關節固定，可在垂直平面內自由擺動。關節處施加力矩  $\tau$ 。

**步驟一：廣義座標。**取  $\theta$  為桿與垂直向下方向的夾角，這是唯一的自由度。

步驟二：動能。桿繞固定端旋轉，其繞該端的轉動慣量為  $I = (1/3)mL^2$  ( 13.1.3 節 )：

$$T = \frac{1}{2} I \dot{\theta}^2 = \frac{1}{2} \cdot (1/3) m L^2 \cdot \dot{\theta}^2 = (1/6) m L^2 \dot{\theta}^2$$

純轉動的動能。此處直接用繞固定端的慣量，省去分解為平移加轉動的步驟。

步驟三：位能。質心在距關節  $L/2$  處，其高度 ( 以關節為基準 ) 為  $-(L/2)\cos \theta$ ：

$$U = -m g (L/2) \cos \theta$$

$\theta = 0$  ( 垂直向下 ) 時位能最低，符合物理直覺。

### 13.4.2 推導運動方程

步驟四：代入拉格朗日方程。先寫出拉格朗日量：

$$L = T - U = (1/6) m L^2 \dot{\theta}^2 + m g (L/2) \cos \theta$$

拉格朗日量。

計算對廣義速度的偏導數：

$$\partial L / \partial \dot{\theta} = (1/3) m L^2 \dot{\theta}$$

動能對速度的偏導數，物理上就是角動量。

對時間微分：

$$d/dt (\partial L / \partial \dot{\theta}) = (1/3) m L^2 \ddot{\theta}$$

角動量的變化率。

計算對廣義座標的偏導數：

$$\partial L / \partial \theta = -m g (L/2) \sin \theta$$

位能的負梯度，物理上就是重力產生的力矩。

代入拉格朗日方程  $d/dt(\partial L / \partial \dot{\theta}) - \partial L / \partial \theta = \tau$ ：

$$(1/3) m L^2 \ddot{\theta} + m g (L/2) \sin \theta = \tau$$

單擺的完整運動方程。

### 13.4.3 解讀結果

這條方程可以逐項解讀，而這三項恰好對應到機器人動力學的三大類項目：

**第一項  $(1/3)mL^2\ddot{\theta}$ ：**慣性項。它是「轉動慣量  $\times$  角加速度」，代表要產生角加速度所需的

力矩。注意係數  $(1/3)mL^2$  是常數——這是單擺的特殊性，一般機器手臂的慣性係數會隨姿態變化。

**第二項  $mg(L/2)\sin \theta$** ：重力項。它隨  $\theta$  變化：垂直向下 ( $\theta=0$ ) 時為零、水平 ( $\theta=90^\circ$ ) 時最大。這正是第 5.6.1 節「以水平姿態為最壞情況」計算馬達扭力的理論依據。

**右式  $\tau$** ：外加的關節力矩。

由此可以立即回答一個實務問題：**要讓單擺靜止在水平位置，需要多大的力矩？** 令  $\dot{\theta} = 0$ 、 $\theta = 90^\circ$ ，得  $\tau = mg(L/2)$ ——這就是重力補償（第 17 章）所需的力矩。

此外，若令  $\tau = 0$  且假設小角度 ( $\sin \theta \approx \theta$ )，方程化為  $\ddot{\theta} + (3g/2L)\theta = 0$ ，這是簡諧運動方程，其自然頻率為  $\omega_n = \sqrt{3g/2L}$ 。這個結果在第 16 章討論控制器頻寬時會再出現——**控制器的頻寬必須高於系統的自然頻率**，否則無法有效抑制振動。

## 13.5 平面 2R 手臂的完整推導

### 13.5.1 模型與符號

現在處理真正的機器手臂。考慮平面 2R 手臂：連桿長度  $L_1$ 、 $L_2$ ，質量  $m_1$ 、 $m_2$ ，質心分別位於距各自關節  $l_1$ 、 $l_2$  處（對均勻桿則  $l_i = L_i/2$ ），繞各自質心的轉動慣量為  $I_1$ 、 $I_2$ 。

這個例子的價值在於：它是最簡單但仍具備完整結構的機器人動力學——包含慣性、科氏力、離心力與重力四類項，且各項之間有耦合。理解它，就理解了六軸手臂動力學方程的結構。

### 13.5.2 第一步：各連桿質心的位置與速度

連桿 1 的質心位置與速度（由第 9 章的正向運動學）：

$$\begin{aligned} x_{c1} &= l_1 c_1, & y_{c1} &= l_1 s_1 \\ \dot{x}_{c1} &= -l_1 s_1 \dot{\theta}_1, & \dot{y}_{c1} &= l_1 c_1 \dot{\theta}_1 \end{aligned}$$

質心 1 的速度平方為  $v_{c1}^2 = l_1^2 \dot{\theta}_1^2$ （因  $s^2 + c^2 = 1$ ）。

連桿 2 的質心位置：

$$x_{c2} = L_1 c_1 + l_2 c_{12}, \quad y_{c2} = L_1 s_1 + l_2 s_{12}$$

其中  $c_{12} = \cos(\theta_1 + \theta_2)$ 。注意第一項用的是連桿 1 的全長  $L_1$ ，第二項用的是連桿 2 的質心距離  $l_2$ 。這是初學者最常出錯的地方。

對時間微分並計算速度平方（過程中利用  $\cos(A-B)$  的和差角公式化簡）：

$$v_c^2 = L_1^2 \dot{\theta}_1^2 + l_2^2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + 2L_1 l_2 \dot{\theta}_1 (\dot{\theta}_1 + \dot{\theta}_2) c_2$$

質心 2 的速度平方。最後一項含  $\cos \theta_2$ ，這正是耦合的來源——它使動能無法拆解成兩個獨立的部分。

### 13.5.3 第二步：動能與位能

兩根連桿的動能總和（各含平移與轉動兩部分）：

$$T = \frac{1}{2} m_1 l_1^2 \dot{\theta}_1^2 + \frac{1}{2} I_1 \dot{\theta}_1^2 + \frac{1}{2} m_2 [L_1^2 \dot{\theta}_1^2 + l_2^2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + 2L_1 l_2 \dot{\theta}_1 (\dot{\theta}_1 + \dot{\theta}_2) c_2] + \frac{1}{2} I_2 (\dot{\theta}_1 + \dot{\theta}_2)^2$$

總動能。注意連桿 2 的角速度是  $\theta_1 + \theta_2$ （絕對角速度），而非僅  $\theta_2$ 。

位能（設重力沿  $-Y$  方向）：

$$U = m_1 g l_1 s_1 + m_2 g (L_1 s_1 + l_2 s_{12})$$

總位能。

### 13.5.4 第三步：代入拉格朗日方程

推導過程冗長，此處直接給出整理後的結果（讀者可對照本章程式範例的符號運算驗證）。

為簡化書寫，定義三個常數組合：

$$\begin{aligned} \alpha &= m_1 l_1^2 + I_1 + m_2 (L_1^2 + l_2^2) + I_2 \\ \beta &= m_2 L_1 l_2 \\ \delta &= m_2 l_2^2 + I_2 \end{aligned}$$

這三個組合是機構的固有參數，與姿態無關。

第一關節的方程：

$$\tau_1 = (\alpha + 2\beta c_2) \dot{\theta}_1 + (\delta + \beta c_2) \dot{\theta}_2 - \beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \dot{\theta}_1 \dot{\theta}_2 + g_1(q)$$

含慣性、離心、科氏與重力四類項。

第二關節的方程：

$$\tau_2 = (\delta + \beta c_2) \dot{\theta}_1 + \delta \dot{\theta}_2 + \beta s_2 \dot{\theta}_1^2 + g_2(q)$$

第二關節的方程較簡單，但仍含來自第一關節的耦合項。

其中重力項為：

$$\begin{aligned} g_1(q) &= (m_1 l_1 + m_2 L_1) g c_1 + m_2 l_2 g c_{12} \\ g_2(q) &= m_2 l_2 g c_{12} \end{aligned}$$

重力產生的關節力矩，只與姿態有關、與速度無關。

### 13.5.5 整理成標準形式

把上述兩式寫成矩陣形式，就得到機器人動力學的標準形式：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

機器人動力學方程的標準形式。這是本章、乃至整個第三篇最重要的一條式子。

其中三個矩陣（向量）分別為：

$$M(q) = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix}$$

慣性矩陣，對稱且姿態相依。

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -\beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \dot{\theta}_1 \dot{\theta}_2 \\ \beta s_2 \dot{\theta}_1^2 \end{bmatrix}$$

科氏力與離心力項。含速度的平方或乘積。

$$G(q) = \begin{bmatrix} (m_1 l_1 + m_2 L_1)g c_1 + m_2 l_2 g c_{12} \\ m_2 l_2 g c_{12} \end{bmatrix}$$

重力項。

圖 13-1 動力學方程各項的物理意義

建議配圖：中央為平面 2R 手臂圖示，四周以四個色塊分別標註四類項目及其物理來源。 $M(q)\ddot{q}$ （藍）：加速時抵抗的慣性，圖示為手臂加速的箭頭。離心力（綠）：單軸旋轉時連桿被「甩」向外的傾向，圖示為旋轉中的手臂與向外的虛線箭頭。科氏力（橙）：兩軸同時轉動時產生的耦合力，圖示為兩個關節同時旋轉並在末端產生側向力。 $G(q)$ （紅）：重力使手臂下垂的力矩，圖示為向下的重力箭頭與其力臂。下方標註標準形式方程式。

## 13.6 慣性矩陣 $M(q)$

### 13.6.1 四個關鍵性質

慣性矩陣是動力學方程中最重要的物件，它有四個必須理解的性質。

**性質一：對稱。** $M = M^T$  恆成立。這來自動能是二次型的事實（ $\frac{1}{2}\dot{q}^T M \dot{q}$ ），任何非對稱的部分在二次型中都會自動抵消。對稱性使  $n \times n$  矩陣只有  $n(n+1)/2$  個獨立元素，並可用於驗算——若你推導出的  $M$  不對稱，一定是算錯了。

**性質二：正定。**對任意非零的  $\dot{q}$ ，恆有  $\dot{q}^T M \dot{q} > 0$ 。這在物理上是必然的——動能不可能為零或負值（除非完全靜止）。正定性帶來一個極重要的推論： **$M$  必定可逆**。這保證了正向動力學（由力矩求加速度）永遠有唯一解：

$$\ddot{q} = M^{-1}(\tau - C\dot{q} - G)$$

正向動力學。因  $M$  正定必可逆，此式永遠成立——這是物理模擬（第 30 章）能夠進行的數學保證。

**性質三：姿態相依。** $M$  是  $q$  的函數。以平面 2R 為例， $M_{11} = \alpha + 2\beta \cos \theta_2$  隨  $\theta_2$  變化：手



臂伸直 ( $\theta_2 = 0$ ) 時  $M_{11} = \alpha + 2\beta$  最大、手臂折疊 ( $\theta_2 = 180^\circ$ ) 時最小。這正是開頭故事中「同樣的關節速度，在不同姿態下需要不同力矩」的數學根源，也是為何固定增益的 PID 在高速時表現不佳。

**性質四：有界。**  $M$  的特徵值有上下界，且與位置無關（對只有旋轉關節的機器人）。這個性質在第 19 章的穩定性證明中扮演關鍵角色。

### 13.6.2 對角元素與非對角元素

慣性矩陣的元素有清楚的物理意義。

**對角元素  $M_{ii}$ ：**關節  $i$  單獨加速時所感受到的等效慣量。它包含了該關節下游所有連桿的貢獻——這解釋了為何靠近基座的關節（第一、二軸）需要遠大於手腕關節的馬達（第 5 章的選型考量）。

**非對角元素  $M_{ij}$  ( $i \neq j$ )：**耦合項。它代表「關節  $j$  的加速度，會對關節  $i$  產生多少力矩」。以平面 2R 為例， $M_{12} = \delta + \beta \cos \theta_2$  不為零，意味著：即使第一關節完全不動 ( $\dot{\theta}_1 = 0$ )，只要第二關節加速，第一關節仍需輸出力矩來維持不動。

這個耦合是機器人控制困難的核心來源。若把每個關節當成獨立系統各自用 PID 控制（第 16 章的作法），耦合項就成為每個關節眼中的「未知干擾」。速度越快、加速度越大，這個干擾就越顯著——這正是開頭故事中低速完美、高速失準的原因。

#### ◎ 耦合的直觀體驗

把手臂平伸，然後只彎曲手肘（前臂快速上抬）。你會感覺到**肩膀也需要用力**——即使肩關節完全沒有轉動。這就是非對角慣性項  $M_{12}$  的作用。再試一次，但這次把前臂盡量靠近身體（縮短力臂），你會發現肩膀需要的力氣小得多——因為  $\beta = m_2 L_1 l_2$  隨質心距離下降。這正是第 5 章「把重物往轉軸移近」設計原則的動力學解釋。

## 13.7 科氏力與離心力

### 13.7.1 兩種速度相關的力

$C(q, \dot{q})\dot{q}$  這一項包含兩種物理上不同的效應，可由它們對速度的依賴形式區分。

**離心力項：**係數含  $\dot{q}^2$ （單一速度的平方）。物理意義是：當連桿繞某軸旋轉時，它有「被甩出去」的傾向，這會在其他關節上產生力矩。以平面 2R 為例， $\tau_2$  中的  $\beta s_2 \dot{\theta}_1^2$  就是離心力項——

—第一關節旋轉時，連桿 2 被向外甩，在第二關節上產生力矩。

**科氏力項**：係數含  $\dot{q}_i \dot{q}_j$  (兩個不同速度的乘積)。它只在**兩個關節同時運動**時出現。以平面 2R 為例， $\tau_1$  中的  $-2\beta s_2 \dot{\theta}_1 \dot{\theta}_2$  就是科氏力項。

科氏力的物理直覺較難掌握，但可以用一個經典的例子理解：**在旋轉的旋轉木馬上往中心走**。你會感覺到一個側向的力把你往旁邊推——這就是科氏力。它源自「你的旋轉半徑正在改變，因此角動量守恆要求切線速度改變，而改變速度就需要力」。機器人的第二關節彎曲時，連桿 2 的質心到第一軸的距離正在改變，因此產生同樣的效應。

### 13.7.2 為何常被忽略、又何時不能忽略

這兩類力都與速度的平方成正比，因此：

**科氏力與離心力  $\propto \dot{q}^2$**

*速度加倍，這些力變成四倍。*

這解釋了實務上的經驗法則：**低速時可以忽略、高速時絕對不能忽略**。以工業機器為例，在每秒 10 度的示教速度下，這些力可能只佔總力矩的百分之幾；但在每秒 200 度的生產速度下，可能達到百分之三十以上。

這正是開頭故事的完整解釋：低速調校時，控制器面對的幾乎只有慣性項與重力項，PID 足以應付；一旦提高到八成速度，科氏力與離心力暴增為原來的數十倍，成為 PID 無法及時補償的大干擾，於是出現軌跡偏差與振動。

### 13.7.3 C 矩陣的不唯一性與斜對稱性

有一個技術細節值得注意：**C 矩陣的寫法不唯一**。因為  $C(q, \dot{q})\dot{q}$  是一個向量，把它「分解」成矩陣乘向量的方式有很多種。不同教科書可能給出不同的 C，但  $C \cdot \dot{q}$  的結果必定相同。

在眾多寫法中，有一種特別有價值——採用 Christoffel 符號定義的 C，會使下列性質成立：

**$\dot{M}(q) - 2C(q, \dot{q})$  是斜對稱矩陣 ( skew-symmetric )**

*即  $x^T(\dot{M} - 2C)x = 0$  對任意  $x$  成立。*

這個性質看似抽象，卻是第 19 章證明控制器穩定性的關鍵工具。它的物理意義是**能量守恆**：機器人內部的科氏力與離心力不會產生或消耗能量，只是在各關節之間重新分配。這與現實相符

——這些力源自慣性，而非外部能量來源。

## 13.8 重力項與摩擦

### 13.8.1 重力項 $G(q)$

重力項是四類項中最單純的：它只與姿態有關，與速度、加速度完全無關。

$$G(q) = \partial U(q) / \partial q$$

*重力項就是位能對關節角度的梯度。*

儘管單純，它在實務上卻極為重要，原因有三。

**原因一：它通常是最大的一項。**在中低速運動中，重力項往往佔總力矩的一半以上。一支水平伸展的手臂，光是「維持不動」就需要相當大的力矩。

**原因二：它是恆定的偏差。**與科氏力不同，重力項在靜止時仍然存在。若控制器不補償它，PID 的積分項必須持續累積來對抗重力，導致響應變慢且容易超越。這是**重力補償**（第 17 章）成為標準作法的原因。

**原因三：它是碰撞偵測的最大干擾源。**要由關節力矩推算外力（第 11.6.3 節），必須先精確扣除重力項。若重力模型有 5% 的誤差，在需要 50 N·m 重力補償的姿態下，就會產生 2.5 N·m 的殘差——這可能大於真實碰撞的訊號。

### 13.8.2 摩擦模型

摩擦力不出現在拉格朗日方程中（它是非保守力，不能由位能導出），必須另外加入。實務上常用的模型有三層：

**庫倫摩擦 (Coulomb)：**大小固定、方向與速度相反。

$$\tau_c = F_c \cdot \text{sign}(\dot{q})$$

*在速度過零時會有不連續跳變，這是控制上的難題（造成低速爬行現象，stick-slip）。*

**黏滯摩擦 (Viscous)：**與速度成正比。

$$\tau_v = F_v \cdot \dot{q}$$

*來自潤滑油的阻力，是最容易補償的一項（線性）。*

**靜摩擦 ( Stiction )**：靜止時需要克服的最大摩擦力，通常大於運動中的庫倫摩擦。它造成「明明給了力矩卻不動，稍微加大就突然衝出去」的現象。

加入摩擦後，完整的動力學方程為：

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + F(q) = \tau$$

含摩擦的完整形式。  $F(q)$  通常取  $F_v\dot{q} + F_c\text{sign}(\dot{q})$ 。

### 13.8.3 摩擦為何難以處理

摩擦是動力學模型中最不可靠的一環，原因有四：它隨溫度變化（機器暖機後摩擦下降）、隨潤滑狀態變化（保養前後不同）、隨磨損變化（使用數年後改變）、且在低速時高度非線性（stick-slip 現象）。

這對兩件事有直接影響。第一，**高減速比的關節摩擦特別大**——減速機內部有大量齒面接觸，其摩擦力矩可能達到額定輸出的百分之十以上。第二，**碰撞偵測的靈敏度受限於摩擦的不確定性**——這正是第 11.6.3 節知識框中「協作機器人減速比不能太高」的另一個理由。

實務上的處理方式有三：一是透過實驗辨識摩擦參數（讓關節以不同速度等速運動，記錄所需力矩，擬合出  $F_v$  與  $F_c$ ）；二是在低速時加入 **dither**（高頻微小振動）打破靜摩擦；三是採用低摩擦的機構設計（如直驅或低減速比方案）。

## 13.9 逆動力學與正向動力學

### 13.9.1 兩個方向的問題

有了標準形式  $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$ ，可以往兩個方向使用它。

**逆動力學 ( Inverse Dynamics )**：已知運動，求所需力矩。

$$\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q)$$

直接代入即可，不需解方程。計算量為  $O(n)$  或  $O(n^2)$ ，可即時執行。

這是控制上最常用的方向。給定規劃好的軌跡（ $q, \dot{q}, \ddot{q}$  都已知），逆動力學算出「理論上需要多大的力矩」，把它當作前饋送給馬達——這正是開頭故事中叫獸提到的解決方案，也是第 17 章計算力矩控制的核心。

**正向動力學 ( Forward Dynamics )**：已知力矩，求所產生的運動。

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{q}) [ \boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} - \mathbf{G}(\mathbf{q}) ]$$

需要求  $\mathbf{M}$  的反矩陣 ( 或解線性方程組 )，計算量較大。

這是**模擬**所需的方向。物理引擎在每個時間步驟中，由當前的力矩算出加速度，再積分得到速度與位置。第 30 章的 Gazebo、Isaac Sim 與 MuJoCo，內部做的就是這件事。

表 13-2 逆動力學與正向動力學的比較

| 面向    | 逆動力學                                                                                        | 正向動力學                                                                                              |
|-------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 問題    | 已知運動 → 求力矩                                                                                  | 已知力矩 → 求運動                                                                                         |
| 公式    | $\boldsymbol{\tau} = \mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G}$ | $\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C}\dot{\mathbf{q}} - \mathbf{G})$ |
| 是否需求逆 | 否 ( 直接代入 )                                                                                  | 是 ( $\mathbf{M}$ 正定故必可逆 )                                                                          |
| 計算複雜度 | $O(n)$ ( Newton-Euler 遞迴 )                                                                  | $O(n^3)$ 或 $O(n)$ ( 特殊演算法 )                                                                        |
| 主要用途  | 控制前饋、力矩預測                                                                                   | 物理模擬、預測行為                                                                                          |
| 本書章節  | 第 14、17 章                                                                                   | 第 14、30 章                                                                                          |

### 13.9.2 為何正向動力學一定有解

回到 13.6.1 節的性質二： $\mathbf{M}$  是正定矩陣，因此必定可逆。這保證了正向動力學永遠有唯一解——給定任何一組力矩，機器人的運動都是確定的。

這個看似理所當然的結論，其實有深刻的物理意義：它排除了「同樣的力產生不同運動」或「某些力矩組合無法產生任何運動」的可能。物理世界的因果關係，在數學上由慣性矩陣的正定性所保證。

## 13.10 參數辨識與模型誤差

### 13.10.1 參數從哪裡來

動力學模型需要每根連桿的三組參數：質量、質心位置、慣性張量。對  $n$  軸機器人，這是  $10n$  個參數 (  $1 + 3 + 6$  )。它們的來源有三：

**來源一：CAD 模型。**設計階段直接由 3D 模型計算。優點是免費且完整；缺點是與實體有落差——線材、螺絲、油脂、加工公差都未計入，誤差通常在 5 ~ 15%。

**來源二：實驗辨識。**讓機器人執行特定的激勵軌跡，記錄關節力矩與運動資料，再用最小平方方法反推參數。這是最準確的方法，但需要專門的實驗設計。

**來源三：廠商提供。**部分廠商會提供動力學參數（尤其是協作機器人，因為力控制需要）。這是最省事的方式。

### 13.10.2 線性參數化：辨識的關鍵

動力學方程有一個極為有用的性質：**它對慣性參數是線性的。**

$$\tau = Y(q, \dot{q}, \ddot{q}) \cdot \pi$$

$Y$  稱為迴歸矩陣 (regressor)，只與運動有關； $\pi$  是由慣性參數組成的向量。

這個性質使參數辨識變成一個**線性最小平方問題**——只要收集足夠多的 (運動, 力矩) 資料，就能用線性代數直接解出  $\pi$ ，不需非線性最佳化。這是動力學辨識能夠實用化的關鍵。

但有一個複雜之處：**並非所有參數都可辨識**。某些參數的組合永遠以固定的方式一起出現（如 13.5.4 節的  $\alpha$ 、 $\beta$ 、 $\delta$ ），無法個別分離。可辨識的組合稱為**基本參數** (base parameters)，其數目少於  $10n$ 。實務上只需辨識這些基本參數即可，因為動力學行為只取決於它們。

### 13.10.3 模型誤差的影響

動力學模型永遠不完美，主要誤差源有五：慣性參數誤差（5 ~ 15%）、摩擦的不確定性（最大的一項）、關節彈性與背隙（剛體假設失效）、連桿變形（高速或重載時）、以及未建模的元件（線材、氣管的拉扯力）。

這帶來一個重要的工程觀點：**動力學模型的價值不在於完美，而在於把大部分的力矩需求「預測」出來**。即使模型只有 80% 的準確度，前饋控制也能把 PID 需要處理的誤差降低五倍——這已足以帶來顯著的性能提升。第 17 章的計算力矩控制正是建立在這個思路上：**用模型處理已知的部分，用回授處理未知的部分**。

## 13.11 由能量邁向遞迴

### 13.11.1 本章的成果

本章建立了機器人動力學的完整框架。你現在能夠：計算連桿的慣性參數與能量、運用拉格朗日方程推導運動方程、辨識並解釋慣性項、科氏力項、離心力項與重力項、理解慣性矩陣的四個性質及其工程意義、並說明逆動力學與正向動力學的用途。

最重要的是那條標準形式的方程：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + F(q) = \tau$$

這條式子將在第三篇的每一章出現：第 15 章用它檢驗軌跡的可行性、第 16~17 章用它設計控制器、第 19 章用它證明穩定性、第 20 章用它分離外力。

### 13.11.2 接下來：從優雅到高效

拉格朗日法在理解上極為優雅，但它有一個實務上的致命問題：**計算效率**。要得到六軸手臂的解析方程，符號運算會產生數千項的表達式；即使得到了，每次求值也需要大量運算。控制器需要在每一毫秒完成一次逆動力學計算，這條路走不通。

**第 14 章 (Newton-Euler 遞迴)** 將提供解方。它放棄追求解析表達式，改用遞迴的數值計算：先由基座往末端傳遞速度與加速度（前向遞迴），再由末端往基座傳遞力與力矩（後向遞迴）。整個過程的複雜度只有  $O(n)$ ，六軸手臂的一次完整計算只需數微秒。兩種方法得到的答案完全相同，但一個適理解、一個適合實作。

**第 15 章 (軌跡生成)** 則會回頭使用本章的成果：規劃軌跡時不能只考慮幾何，還必須確認所需的力矩不超過馬達能力——而這正是逆動力學的工作。

換句話說：第 13 章讓你**看懂**動力學的結構，第 14 章讓你**算得夠快**，第 15 章之後則開始用**它來做事**。

## 完整計算例題

### 例題 13-1 單擺的動力學計算

題目：一根均勻細桿，長  $L = 0.6 \text{ m}$ 、質量  $m = 2.0 \text{ kg}$ ，一端以旋轉關節固定，可在垂直平面內擺動。(a) 求繞固定端的轉動慣量。(b) 當桿呈水平且靜止時，維持不動所需的力矩。(c) 當桿在水平位置且需產生  $5 \text{ rad/s}^2$  的角加速度時，所需的總力矩。(d) 求小角度擺動時的自然頻率。

解 (a)：均勻細桿繞一端的轉動慣量 (13.1.3 節)：

$$I = (1/3) m L^2 = (1/3)(2.0)(0.6^2) = (1/3)(2.0)(0.36) = 0.24 \text{ kg}\cdot\text{m}^2$$

繞固定端的轉動慣量。

**驗算 (平行軸定理)**：繞質心  $I_c = (1/12)(2.0)(0.36) = 0.06$ ；質心距端點  $d = 0.3 \text{ m}$ ，故  $I = 0.06 + (2.0)(0.09) = 0.06 + 0.18 = 0.24 \checkmark$  兩法一致。



解 (b)：水平時  $\theta = 90^\circ$ ，重力力矩最大。由 13.4.2 節的方程，令  $\ddot{\theta} = 0$ ：

$$\tau = m g (L/2) \sin 90^\circ = (2.0)(9.8)(0.3)(1.0) = 5.88 \text{ N}\cdot\text{m}$$

維持水平所需的力矩，即重力補償力矩。

解 (c)：總力矩為慣性項加重力項：

$$\tau = I \ddot{\theta} + m g (L/2) \sin \theta = (0.24)(5.0) + 5.88 = 1.20 + 5.88 = 7.08 \text{ N}\cdot\text{m}$$

總需求力矩。

**工程觀察：**注意慣性項 (1.20 N·m) 只佔總力矩的 17%，重力項 (5.88 N·m) 佔了 83%。這印證了 13.8.1 節的說法——在中低加速度下，重力往往是最大的一項。若加速度提高到  $50 \text{ rad/s}^2$ ，慣性項變為 12 N·m，兩者的比例就反轉了。

解 (d)：小角度時  $\sin \theta \approx \theta$ ，且此處  $\theta$  由垂直向下算起。方程化為：

$$I \ddot{\theta} + m g (L/2) \theta = 0 \Rightarrow \ddot{\theta} + [m g L / (2I)] \theta = 0$$

簡諧運動方程。

$$\omega_n = \sqrt{(m g L / (2I))} = \sqrt{(2.0)(9.8)(0.6) / (2 \times 0.24)} = \sqrt{(11.76/0.48)} = \sqrt{24.5} \approx 4.95 \text{ rad/s}$$

自然頻率約  $4.95 \text{ rad/s}$ ，即  $0.79 \text{ Hz}$  (週期約 1.27 秒)。

**控制上的意義：**若要有效控制這個系統，控制器的頻寬應至少為自然頻率的 5~10 倍 (約 4~8 Hz)，對應的取樣頻率則應再高 10 倍以上 (40~80 Hz 以上)。這是第 16 章選擇控制週期的依據之一。

### 例題 13-2 平面 2R 手臂的動力學計算

題目：平面 2R 手臂參數為： $L_1 = 0.4 \text{ m}$ 、 $L_2 = 0.3 \text{ m}$ 、 $m_1 = 3.0 \text{ kg}$ 、 $m_2 = 2.0 \text{ kg}$ ，兩連桿皆均勻 (故  $l_1 = 0.2$ 、 $l_2 = 0.15 \text{ m}$ )， $I_1 = (1/12)m_1 L_1^2 = 0.04$ 、 $I_2 = (1/12)m_2 L_2^2 = 0.015 \text{ kg}\cdot\text{m}^2$ 。目前姿態  $\theta_1 = 30^\circ$ 、 $\theta_2 = 60^\circ$ ，關節速度  $\dot{\theta} = (1.0, -0.5) \text{ rad/s}$ ，加速度  $\ddot{\theta} = (2.0, 1.0) \text{ rad/s}^2$ 。求各關節所需力矩。

解：先計算三個常數組合 (13.5.4 節)：

$$\begin{aligned} \alpha &= m_1 l_1^2 + I_1 + m_2 (L_1^2 + l_2^2) + I_2 \\ &= 3.0(0.04) + 0.04 + 2.0(0.16 + 0.0225) + 0.015 \\ &= 0.12 + 0.04 + 0.365 + 0.015 = 0.540 \text{ kg}\cdot\text{m}^2 \end{aligned}$$



常數  $\alpha$ 。

$$\beta = m_2 L_1 l_2 = 2.0 \times 0.4 \times 0.15 = 0.120 \text{ kg}\cdot\text{m}^2$$

耦合常數  $\beta$ 。

$$\delta = m_2 l_2^2 + I_2 = 2.0(0.0225) + 0.015 = 0.045 + 0.015 = 0.060 \text{ kg}\cdot\text{m}^2$$

常數  $\delta$ 。

代入  $\theta_2 = 60^\circ$  :  $c_2 = 0.5$ 、 $s_2 = 0.866$ 。慣性矩陣為：

$$M = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix} = \begin{bmatrix} 0.540 + 0.120 & 0.060 + 0.060 \\ 0.120 & 0.060 \end{bmatrix} = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix}$$

慣性矩陣。注意其對稱性 ✓

慣性項： $M \cdot \ddot{q} =$

$$\begin{bmatrix} 0.660(2.0) + 0.120(1.0) \\ 0.120(2.0) + 0.060(1.0) \end{bmatrix} = \begin{bmatrix} 1.440 \\ 0.300 \end{bmatrix} \text{ N}\cdot\text{m}$$

慣性項。注意第一關節的 1.44 中，有 0.12 來自第二關節的加速度——這就是耦合。

科氏與離心項： $\beta s_2 = 0.120 \times 0.866 = 0.1039$ 。

$$\begin{aligned} \tau_1 \text{ 項} &= -\beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \theta_1 \dot{\theta}_2 = -0.1039(0.25) - 2(0.1039)(1.0)(-0.5) \\ &= -0.026 + 0.1039 = 0.0779 \text{ N}\cdot\text{m} \end{aligned}$$

第一關節的速度相關項（離心 + 科氏）。

$$\tau_2 \text{ 項} = \beta s_2 \theta_1^2 = 0.1039(1.0)^2 = 0.1039 \text{ N}\cdot\text{m}$$

第二關節的離心力項。

重力項： $\theta_1 = 30^\circ$  ( $c_1 = 0.866$ )、 $\theta_1 + \theta_2 = 90^\circ$  ( $c_{12} = 0$ )。

$$\begin{aligned} G_1 &= (m_1 l_1 + m_2 L_1)g c_1 + m_2 l_2 g c_{12} \\ &= (3.0 \times 0.2 + 2.0 \times 0.4)(9.8)(0.866) + 2.0(0.15)(9.8)(0) \\ &= (0.6 + 0.8)(9.8)(0.866) = 11.88 \text{ N}\cdot\text{m} \end{aligned}$$

第一關節的重力項。

$$G_2 = m_2 l_2 g c_{12} = 2.0(0.15)(9.8)(0) = 0 \text{ N}\cdot\text{m}$$

第二關節的重力項。因連桿 2 恰好垂直，重力力臂為零。

總力矩：

$$\begin{aligned} \tau_1 &= 1.440 + 0.078 + 11.88 = 13.40 \text{ N}\cdot\text{m} \\ \tau_2 &= 0.300 + 0.104 + 0 = 0.404 \text{ N}\cdot\text{m} \end{aligned}$$

各關節的總需求力矩。

工程分析：第一關節的力矩中，重力佔 88.7%、慣性佔 10.7%、速度相關項僅佔 0.6%。這

是典型的中低速情形。若把速度提高五倍 ( $\dot{\theta} = (5, -2.5)$ )，速度相關項會變成 25 倍 (約 1.95 N·m)，佔比升至約 13%——這就是開頭故事中高速時失準的原因。

### 例題 13-3 耦合效應的量化

題目：承上例的機構參數。(a) 若第一關節完全靜止 ( $\theta_1 = \dot{\theta}_1 = 0$ )，而第二關節以  $\ddot{\theta}_2 = 10 \text{ rad/s}^2$  加速，求第一關節需輸出多少力矩才能維持不動 (忽略重力)。(b)  $\theta_2$  為何值時這個耦合力矩最大？最小？(c) 由此提出一個降低耦合的機構設計建議。

解 (a)：由標準形式， $\tau_1 = M_{11}\ddot{\theta}_1 + M_{12}\ddot{\theta}_2 + (\text{速度項}) + G_1$ 。代入  $\ddot{\theta}_1 = 0$ 、忽略速度項與重力：

$$\tau_1 = M_{12} \cdot \ddot{\theta}_2 = (\delta + \beta c_2) \times 10 = (0.060 + 0.120 \times 0.5) \times 10 = 0.120 \times 10 = 1.20 \text{ N}\cdot\text{m}$$

第一關節必須輸出 1.20 N·m 才能「維持不動」。

這個結果值得深思：第一關節明明沒有運動，卻必須出力。若控制器只用 PID (只看位置誤差)，它必須等到第一關節真的被推動、產生誤差之後才會反應——這就是耦合造成的追隨誤差。

解 (b)： $M_{12} = \delta + \beta \cos \theta_2$  隨  $\theta_2$  變化：

$$\theta_2 = 0^\circ : M_{12} = 0.060 + 0.120 = 0.180 \text{ (最大, 耦合力矩 } 1.80 \text{ N}\cdot\text{m)}$$

$$\theta_2 = 180^\circ : M_{12} = 0.060 - 0.120 = -0.060 \text{ (絕對值最小)}$$

$$\theta_2 = 120^\circ : M_{12} = 0.060 - 0.060 = 0 \text{ (耦合完全消失)}$$

耦合項隨姿態變化，甚至可能為零或變號。

有趣的觀察：在  $\theta_2 = 120^\circ$  時耦合為零——此時兩個關節在動力學上完全解耦。這種姿態稱為「動力學解耦位形」，某些高精度應用會刻意在此姿態附近作業。

解 (c)：耦合強度由  $\beta = m_2 L_1 l_2$  決定。要降低耦合，可從三方面著手：減輕連桿 2 的質量  $m_2$  (用輕量材料)、縮短連桿 1 的長度  $L_1$  (但會犧牲工作空間)、或把連桿 2 的質心往其關節移近 (減小  $l_2$ )——最後一項最實用，作法是把第二關節的馬達與減速機盡量往關節處配置，或採用第 5.9.2 節的皮帶傳動把馬達移到基座。這正是為何高性能機器手臂會不惜成本把驅動器往基座移。

## 程式範例：動力學方程的實作與驗證

以下 Python 程式實作平面 2R 手臂的完整動力學，包含逆動力學、正向動力學、能量計算與模擬，並用能量守恆驗證模型的正確性。

```
import numpy as np

class Planar2R:
    """平面 2R 手臂的動力學模型"""
    def __init__(self, L1=0.4, L2=0.3, m1=3.0, m2=2.0, g=9.8):
        self.L1, self.L2, self.m1, self.m2, self.g = L1, L2, m1, m2, g
        self.l1, self.l2 = L1/2, L2/2          # 均勻桿，質心在中點
        self.I1 = m1*L1**2/12                  # 繞質心的轉動慣量
        self.I2 = m2*L2**2/12
        # 三個常數組合 (13.5.4 節)
        self.alpha = m1*self.l1**2 + self.I1 + m2*(L1**2 + self.l2**2) + self.I2
        self.beta = m2*L1*self.l2
        self.delta = m2*self.l2**2 + self.I2

    def M(self, q):
        """慣性矩陣 (13.6 節)"""
        c2 = np.cos(q[1])
        return np.array([[self.alpha + 2*self.beta*c2, self.delta + self.beta*c2],
                        [self.delta + self.beta*c2, self.delta]])

    def C_qd(self, q, qd):
        """科氏力與離心力項 C(q,qd)·qd (13.7 節)"""
        s2 = np.sin(q[1])
        return np.array([-self.beta*s2*qd[1]**2 - 2*self.beta*s2*qd[0]*qd[1],
                        self.beta*s2*qd[0]**2])

    def G(self, q):
        """重力項 (13.8 節)"""
        c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
        return np.array([
            (self.m1*self.l1 + self.m2*self.L1)*self.g*c1 + self.m2*self.l2*self.g*c12,
            self.m2*self.l2*self.g*c12])

    def inverse_dynamics(self, q, qd, qdd):
        """逆動力學：已知運動求力矩 (13.9.1 節)"""
        return self.M(q) @ qdd + self.C_qd(q, qd) + self.G(q)

    def forward_dynamics(self, q, qd, tau):
        """正向動力學：已知力矩求加速度"""
        return np.linalg.solve(self.M(q), tau - self.C_qd(q, qd) - self.G(q))

    def energy(self, q, qd):
        """回傳 (動能, 位能)"""
        T = 0.5 * qd @ self.M(q) @ qd          # 動能為二次型
        h1 = self.l1*np.sin(q[0])
        h2 = self.L1*np.sin(q[0]) + self.l2*np.sin(q[0]+q[1])
        U = self.m1*self.g*h1 + self.m2*self.g*h2
```

```

    return T, U

d, r2d = np.deg2rad, np.rad2deg
arm = Planar2R()

# ----- 驗證一：例題 13-2 -----
print("=== 例題 13-2：逆動力學計算 ===")
q = np.array([d(30), d(60)])
qd = np.array([1.0, -0.5])
qdd = np.array([2.0, 1.0])
print(f"常數:  $\alpha$ ={arm.alpha:.4f}  $\beta$ ={arm.beta:.4f}  $\delta$ ={arm.delta:.4f} kg·m2")
print(f"慣性矩陣 M = \n{np.round(arm.M(q), 4)}")
print(f"對稱性驗證: {np.allclose(arm.M(q), arm.M(q).T)}")
print(f"正定性驗證: {np.all(np.linalg.eigvals(arm.M(q)) > 0)}")
print(f"慣性項 M· $\ddot{q}$  = {np.round(arm.M(q) @ qdd, 4)} N·m")
print(f"科氏離心 C· $\dot{q}$  = {np.round(arm.C_qd(q, qd), 4)} N·m")
print(f"重力項 G = {np.round(arm.G(q), 4)} N·m")
tau = arm.inverse_dynamics(q, qd, qdd)
print(f"總力矩  $\tau$  = {np.round(tau, 3)} N·m")

# ----- 驗證二：逆動力學與正向動力學互為反運算 -----
print("\n=== 驗證：逆 / 正向動力學的一致性 ===")
qdd_back = arm.forward_dynamics(q, qd, tau)
print(f"由  $\tau$  反算加速度: {np.round(qdd_back, 6)}")
print(f"與原始  $\ddot{q}$  一致: {np.allclose(qdd, qdd_back)}")

# ----- 驗證三：例題 13-3 耦合效應 -----
print("\n=== 例題 13-3：耦合效應掃描 ===")
print(f"{' $\theta_2$ ':>6} {'M12':>9} {'耦合力矩':>10} ( $\ddot{\theta}_2 = 10 \text{ rad/s}^2$ )")
for t2 in (0, 30, 60, 90, 120, 150, 180):
    M12 = arm.M([0, d(t2)])[0, 1]
    print(f"{'t2':>6d} {'M12':>9.4f} {'M12*10':>10.3f} N·m"
          + (" ← 動力學解耦" if abs(M12) < 0.01 else ""))

# ----- 驗證四：能量守恆（模型正確性的最強驗證） -----
print("\n=== 能量守恆驗證（自由擺動、無外力、無摩擦） ===")
q_s = np.array([d(60), d(30)])
qd_s = np.array([0.0, 0.0])
dt = 1e-5
T0, U0 = arm.energy(q_s, qd_s)
E0 = T0 + U0
print(f"起始: T={T0:.6f} U={U0:.6f} E={E0:.6f} J")
for step in range(200000):
    # 模擬 2 秒
    qdd_s = arm.forward_dynamics(q_s, qd_s, np.zeros(2))
    qd_s = qd_s + qdd_s*dt
    # 半隱式歐拉積分
    q_s = q_s + qd_s*dt
    if (step+1) % 50000 == 0:
        T, U = arm.energy(q_s, qd_s)

```

```

    print(f"t={({step+1})*dt:.1f}s: T={T:8.5f} U={U:9.5f} "
          f"E={T+U:.6f} J 誤差={abs(T+U-E0)/abs(E0)*100:.4f}%")
print("> 總能量幾乎守恆，證明動力學模型與積分器皆正確")

# ----- 應用：重力補償力矩地圖 -----
print("\n=== 重力補償力矩（各姿態下維持靜止所需）===")
print(f"{'θ1':>5}{'θ2':>6} {'τ1':>9} {'τ2':>8}")
for t1 in (0, 45, 90):
    for t2 in (0, 90):
        G = arm.G([d(t1), d(t2)])
        print(f"{t1:5d}{t2:6d} {G[0]:9.3f} {G[1]:8.3f} N·m")
print("> θ1=0°（水平伸展）時重力負擔最大，與第 5 章選型的最壞情況一致")

```

請試著修改：（一）在能量守恆測試中加入摩擦項，觀察能量如何單調遞減（這驗證了摩擦是耗散力）。（二）把積分步長  $dt$  加大到  $1e-3$ ，觀察能量誤差如何增加——這說明了物理模擬為何需要小步長。（三）把  $m_2$  減半，觀察耦合項  $\beta$  與所需力矩的變化，驗證例題 13-3(c) 的設計建議。

## 實作：動力學參數的實驗辨識

本章實作是為實體手臂辨識動力學參數，並驗證模型的預測能力。建議使用第 9～12 章已建立運動學模型的手臂，且需能量測關節電流（第 5、6 章）。

- 幾何與質量量測：**拆下各連桿（或查閱 CAD），量測其質量、長度，並用「平衡點法」找出質心位置（把連桿放在細桿上，找到能平衡的位置即為質心）。
- 慣量估算：**對形狀規則的連桿，用 13.1.3 節的公式計算；對不規則者，可用「三線擺法」實測（懸掛連桿使其扭轉振盪，由週期反推慣量），或直接採用 CAD 值。
- 重力項驗證：**讓手臂在多個靜止姿態下維持不動，記錄各關節的電流，換算為力矩。與模型計算的  $G(q)$  比較，繪出「理論 vs 實測」散布圖並計算相關係數。
- 摩擦參數辨識：**讓單一關節以多種等速轉動（其餘關節固定），記錄各速度下的穩態力矩。扣除重力項後，剩餘的力矩對速度作圖——截距為庫倫摩擦  $F_c$ 、斜率為黏滯摩擦  $F_v$ 。
- 慣性項驗證：**讓關節執行已知加速度的運動（如梯形速度曲線的加速段），記錄力矩。扣除重力與摩擦後，與  $M(q)\ddot{q}$  比較。
- 耦合效應觀察：**固定第一關節（位置控制），讓第二關節快速加速，觀察第一關節的力矩是否上升。這是例題 13-3 的實驗版本。

7. **模型準確度評估**：執行一段包含加減速的完整軌跡，全程記錄實測力矩與模型預測力矩，計算誤差百分比。分析誤差主要來自哪一項（通常是摩擦）。

8. **前饋效果測試（進階）**：若控制器允許加入前饋力矩，比較「有無前饋」時的追隨誤差，量化動力學模型帶來的改善。

表 13-3 動力學參數辨識記錄表（第一列為範例）

| 項目         | 測試條件                           | 理論值       | 實測值       | 誤差   | 說明         |
|------------|--------------------------------|-----------|-----------|------|------------|
| 重力項 $G_1$  | $\theta=(0^\circ, 0^\circ)$ 靜止 | 11.88 N·m | 12.94 N·m | 8.9% | 含摩擦與質心估計誤差 |
| （自行填寫其餘項目） |                                |           |           |      |            |

## 國中小也看得懂：端著水桶走路的道理

提過裝滿水的水桶走路嗎？慢慢走的時候水面很平靜；但只要你**突然加速或急轉彎**，水就會晃出來——而且你必須用力抓住桶子，不然會被水拉得踉蹌一下。機器手臂也是一樣的道理喔！

前面幾章我們都假裝機器手臂「沒有重量」——說轉多少度就轉多少度。但真實的手臂是有重量的，這會帶來三件麻煩事：

**第一件：要加速就要用力。** 越重的手臂，要讓它動起來就要越用力，要讓它停下來也要越用力。這就是為什麼機器人的手臂要盡量做輕。

**第二件：重力一直在拉它。** 把手臂平舉在空中不動，其實馬達一直在用力！不信你自己把手臂平舉三十秒試試看，肩膀會很酸——那個「酸」就是你的肌肉在對抗重力。機器人的馬達也一樣。

**第三件：關節會互相影響。** 這個最神奇。把手臂平舉，然後只彎手肘（前臂快速抬起來），你會發現肩膀也要用力！明明肩膀沒動啊，為什麼要用力？因為前臂動起來的時候，會「拉」肩膀一下。機器人工程師必須算出這個互相影響有多大，機器人動起來才會又快又準。

這一章在做的事情，就是用數學算出這三件事。算清楚了，機器人就可以動得又快又穩——就像練過的服務生，端著滿滿一盤飲料快步走也不會灑出來。

## 叫獸提醒與常見錯誤

- 誤區一：把連桿長度  $L$  與質心距離  $l$  搞混。計算連桿 2 的質心位置時，第一項用  $L_1$ （全

長)、第二項用  $I_2$  (質心距離)，寫錯會導致整個模型偏差。

- 誤區二：忘記連桿 2 的絕對角速度是  $\theta_1 + \theta_2$  而非  $\theta_2$ 。這是動能計算最常見的錯誤，會使耦合項完全消失。
- 誤區三：慣性張量的參考點弄錯。CAD 輸出的通常是繞質心的慣量，用於動力學方程時需以平行軸定理轉換。
- 誤區四：推導出的慣性矩陣不對稱。 $M$  必為對稱矩陣，若算出不對稱，一定是推導有誤，可據此檢查。
- 誤區五：低速調校完就直接高速運行。科氏力與離心力與速度平方成正比，速度加倍則這些力變成四倍。
- 誤區六：忽略摩擦。摩擦不出現在拉格朗日方程中（非保守力），必須另外加入，且它往往是模型誤差的最大來源。
- 誤區七：以為模型必須完美才有用。80% 準確的模型就能把 PID 需處理的誤差降低五倍，這已帶來顯著改善。
- 誤區八：把  $C$  矩陣的特定寫法當成唯一解。 $C$  的分解方式不唯一，但  $C \cdot q$  的結果必定相同。
- 誤區九：碰撞偵測時未扣除完整的動力學項。必須扣除  $M \ddot{q}$ 、 $C \dot{q}$ 、 $G$  與摩擦，剩餘的才是外力，否則加速時會誤判為碰撞。
- 誤區十：模擬時積分步長取得太大。動力學模擬需要小步長（通常 1 ms 以下）才能維持能量守恆，否則系統會「無中生有」地發散。

## 課堂討論

1. 開頭故事中，小好想把 PID 增益調高來解決高速失準。請說明這個作法為何治標不治本，並列出它可能帶來的三個副作用。
2. 拉格朗日法「內力自動消去」是最大的優勢。請以平面 2R 手臂為例，說明若改用牛頓法需要處理哪些內力，以及為何它們在能量法中不會出現。
3. 例題 13-3 發現在  $\theta_2 = 120^\circ$  時耦合完全消失。若你要設計一個高精度的裝配工作站，是否應該把作業點都安排在這個姿態附近？請討論其優缺點（提示：對照第 11、12 章的可操作度與奇異點）。
4. 動力學模型的誤差中，摩擦是最大也最難處理的一項。請討論三種降低摩擦影響的方法，並

比較它們在成本、性能與適用場合上的差異。

5. 第 30 章的物理模擬引擎內部執行的是正向動力學。請討論：為什麼模擬結果與真實機器人總會有落差 ( Sim-to-Real Gap , 第 2.7.3 節 ) ？本章的哪些內容能解釋這個落差？

## 章末摘要

- 動力學回答「力與運動的關係」，是高速控制、力控制與物理模擬的共同基礎。
- 剛體的慣性由三組參數描述：質量（抵抗線加速度）、質心位置（重力作用點）、慣性張量（抵抗角加速度的  $3 \times 3$  對稱矩陣）。
- 平行軸定理  $I = I_c + md^2$  連結質心慣量與任意平行軸慣量，是動力學計算中使用最頻繁的定理。
- 剛體動能可分解為平移與轉動兩部分（König 定理）；用雅可比表達後，機器人總動能恆為二次型  $T = \frac{1}{2} \dot{q}^T M(q) \dot{q}$ 。
- 拉格朗日量  $L = T - U$ ；拉格朗日方程  $d/dt(\partial L / \partial \dot{q}) - \partial L / \partial q = \tau$  由能量直接導出運動方程。
- 拉格朗日法的四大優勢：不需受力圖、內力自動消去、座標系自由、純量運算；代價是計算效率低於 Newton-Euler 法。
- 機器人動力學的標準形式： $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + F(q) = \tau$ ，四類項分別對應慣性、速度相關力、重力與摩擦。
- 慣性矩陣  $M(q)$  有四個性質：對稱、正定（故必可逆）、姿態相依、有界。正定性保證正向動力學永遠有唯一解。
- $M$  的對角元素是各關節的等效慣量；非對角元素是耦合項，代表一個關節的加速度會對其他關節產生力矩。
- 速度相關項含兩種力：離心力（係數含  $\dot{q}_i^2$ ）與科氏力（係數含  $\dot{q}_i \dot{q}_j$ ，兩軸同時運動時才出現）。
- 科氏力與離心力與速度平方成正比，低速可忽略、高速絕不可忽略——這是低速調校完美而高速失準的根本原因。
- 採 Christoffel 符號定義的  $C$  使  $\dot{M} - 2C$  為斜對稱，此性質是第 19 章穩定性證明的關鍵，物理意義為內部力不產生能量。
- 重力項  $G(q) = \partial U / \partial q$  只與姿態有關，在中低速時往往是最大的一項，也是碰撞偵測的最大



干擾源。

- 摩擦含庫倫、黏滯與靜摩擦三部分，是模型中最不可靠的一環，會隨溫度、潤滑與磨損變化。
- 逆動力學 ( $\tau = M\ddot{q} + C\dot{q} + G$ ) 用於控制前饋；正向動力學 ( $\ddot{q} = M^{-1}(\tau - C\dot{q} - G)$ ) 用於物理模擬。
- 動力學方程對慣性參數是線性的 ( $\tau = Y \cdot \pi$ )，使參數辨識成為線性最小平方問題；可辨識者稱為基本參數。

## 觀念題與計算題

### 觀念題

1. 說明質量、質心與慣性張量三者各描述什麼物理性質，以及為何轉動慣量必須用矩陣表示。
2. 敘述平行軸定理，並說明它在機器人動力學中連結了哪兩種資料。
3. 寫出剛體動能的兩部分分解，並說明為何機器人總動能恆為  $\frac{1}{2}\dot{q}^T M(q)\dot{q}$  的形式。
4. 寫出拉格朗日方程，並列舉它相對於牛頓法的四個優勢與一個代價。
5. 寫出機器人動力學的標準形式，並說明四類項各自的物理意義。
6. 列舉慣性矩陣的四個性質，並說明正定性帶來的關鍵推論。
7. 區分慣性矩陣的對角元素與非對角元素的物理意義，並說明後者為何造成控制困難。
8. 區分離心力與科氏力，並各舉一個生活中的類比。
9. 為何說「低速時可忽略科氏力與離心力、高速時絕不可忽略」？請以定量關係說明。
10. 說明  $\dot{M} - 2C$  為斜對稱的物理意義，以及它在何處會被用到。
11. 為何重力項在實務上特別重要？請列舉三個理由。
12. 列舉三種摩擦模型，並說明摩擦為何是動力學模型中最不可靠的一環。
13. 比較逆動力學與正向動力學的公式、計算成本與主要用途。
14. 說明「動力學方程對慣性參數是線性的」這個性質，以及它對參數辨識的意義。

### 計算題

1. 一根均勻細桿長 0.8 m、質量 3.0 kg，一端固定作單擺。(a) 求繞固定端的轉動慣量 (並用平行軸定理驗算)。(b) 求桿在水平位置維持靜止所需的力矩。(c) 若要在水平位置產

生  $8 \text{ rad/s}^2$  的角加速度，求總力矩。(d) 求小角度擺動的自然頻率與週期。

2. 平面 2R 手臂： $L_1 = 0.5 \text{ m}$ 、 $L_2 = 0.4 \text{ m}$ 、 $m_1 = 4.0 \text{ kg}$ 、 $m_2 = 2.5 \text{ kg}$  (皆均勻)， $I_1 = (1/12)m_1 L_1^2$ 、 $I_2 = (1/12)m_2 L_2^2$ 。(a) 求常數  $\alpha$ 、 $\beta$ 、 $\delta$ 。(b) 當  $\theta_2 = 45^\circ$  時求慣性矩陣並驗證其對稱與正定。(c) 當  $\theta = (0^\circ, 45^\circ)$  靜止時，求重力補償所需的力矩。
3. 承上題。姿態  $\theta = (20^\circ, 70^\circ)$ 、速度  $\dot{\theta} = (2.0, -1.5) \text{ rad/s}$ 、加速度  $\ddot{\theta} = (3.0, 2.0) \text{ rad/s}^2$ 。(a) 分別計算慣性項、科氏離心項與重力項。(b) 求總力矩。(c) 計算三類項各佔總力矩的百分比。(d) 若速度全部加倍而其餘不變，重新計算並比較速度相關項的變化倍數。
4. 承上題的機構參數。(a) 寫出耦合項  $M_{12}$  隨  $\theta_2$  變化的表達式。(b) 求  $M_{12}$  的最大值、最小值及其發生的  $\theta_2$ 。(c) 求使  $M_{12} = 0$  (動力學解耦) 的  $\theta_2$  值。(d) 若把  $m_2$  由  $2.5 \text{ kg}$  減為  $1.5 \text{ kg}$ ，重新計算最大耦合力矩 (設  $\dot{\theta}_2 = 15 \text{ rad/s}$ )，並計算改善的百分比。

## 動手做與專題延伸

### 本章實作：動力學模型模組

延續第 9~12 章的程式庫，加入動力學模組。這將是第三篇所有章节的共同基礎。

- **階段一：**實作平面 2R 手臂的完整動力學 (M、C、G、逆動力學、正向動力學)，並用能量守恆驗證。
- **階段二：**加入摩擦模型 (黏滯 + 庫倫)，驗證加入摩擦後能量單調遞減。
- **階段三：**實作三連桿或空間手臂的動力學。可用符號運算 (SymPy) 自動推導，並與數值微分的結果比對驗證。
- **階段四：**實作簡易的物理模擬器：給定初始狀態與力矩輸入，用正向動力學與數值積分模擬運動，並繪出動畫。這將是第 16 章測試控制器的環境。

### 專題延伸

- **能量守恆的積分器比較：**以自由擺動的平面 2R 手臂為測試案例，比較四種數值積分法 (顯式歐拉、半隱式歐拉、四階 Runge-Kutta、Verlet) 在不同步長下的能量漂移。繪出「步長 vs 能量誤差」的對數圖，並討論物理模擬器為何多採用半隱式或辛積分法。
- **力矩需求地圖：**對整個工作空間掃描，計算「以固定加速度執行運動」所需的關節力矩，繪成熱力圖。標出超過馬達額定扭力的區域——這張圖是第 15 章軌跡規劃的重要參考，也能回頭驗證第 5 章的馬達選型是否足夠。

- **進階：前饋控制的效益量化。**在你的模擬器中實作兩種控制器：（甲）純 PID；（乙）PID + 動力學前饋（計算力矩控制的簡化版）。讓兩者執行同一段高速軌跡，比較追隨誤差、最大力矩與穩定時間。接著故意讓前饋模型的參數有 20% 誤差，觀察性能下降多少——這個實驗會讓你直觀理解第 17 章「用模型處理已知、用回授處理未知」的設計哲學。

## 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson ( 第 6 章：操作臂動力學，本章主要參考 ) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley ( 第 7 章：動力學，拉格朗日法的完整處理 ) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer ( 第 7 章：動力學，含參數辨識與線性參數化 ) 。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press ( 第 8 章：動力學，含旋量觀點的表述 ) 。
- Featherstone, R., Rigid Body Dynamics Algorithms, Springer ( 動力學演算法的權威專著，含高效實作 ) 。
- Khalil, W. and Dombre, E., Modeling, Identification and Control of Robots, Butterworth-Heinemann ( 參數辨識的系統性方法 ) 。
- Goldstein, H., Poole, C. and Safko, J., Classical Mechanics, 3rd ed., Addison-Wesley ( 拉格朗日力學的經典教材 ) 。
- Swevers, J., Verdonck, W. and De Schutter, J., "Dynamic Model Identification for Industrial Robots," IEEE Control Systems Magazine, 2007 ( 工業機器人動力學辨識的實務回顧 ) 。
- Pinocchio 與 RBDL 函式庫技術文件：高效的剛體動力學計算實作，支援逆 / 正向動力學與參數辨識。
- MuJoCo 與 Bullet 物理引擎文件：正向動力學在模擬中的實作與數值積分策略。

## 第 14 章

# Newton-Euler 動力學與模擬引擎

編著：李世淵 ( 機器人叫獸 )

助教：李天宇、李宇晴

【章首情境圖建議】上半部為一列火車的側視示意圖，車頭往前加速，動力沿車廂一節節向後傳遞 ( 藍色箭頭 ) ；同時最後一節的重量透過連結器一節節往前累積 ( 紅色箭頭 ) ，越靠近車頭的連結器承受越大的力。下半部為一支六軸手臂，以完全相同的兩色箭頭標註前向遞迴 (  $\omega$ 、 $\dot{\omega}$ 、 $a$  ) 與後向遞迴 (  $f$ 、 $n$  ) ，每個關節旁標出取軸向投影得到的  $\tau_i$ 。右下角並列兩個對比框：「拉格朗日展開：14,000 項」與「Newton-Euler 遞迴：20 行程式」。

## 叫獸開講 「六軸機器人每毫秒都要算一次力矩，怎麼算才夠快？」

上完第 13 章之後，小翰花了整個週末，用 SymPy 推導六軸手臂的拉格朗日動力學方程。星期一早上他抱著筆電衝進研究室，臉上混合著興奮與絕望。

「老師，我算出來了.....但是。」他把螢幕轉過來。第一關節的力矩方程式佔滿了整個畫面，往下捲了將近兩分鐘還沒到底。「這一條式子有一萬四千多項。」

叫獸看了一眼，笑了：「恭喜你，你推導出了正確的答案——一個完全沒有人會使用的正確答案。」

「為什麼？它是對的啊。」

「因為控制器每一毫秒就要算一次。」叫獸在白板上寫下一個數字：**1000 Hz**。「這是伺服控制迴路的典型頻率。你的公式如果要花十毫秒才算得完，控制器就只能等——而在那十毫秒裡，手臂已經移動了好幾公釐。**再正確的模型，算不快就沒有價值。**」

小翰洩氣地問：「那第 13 章學的都白費了？」

「當然不是。」叫獸搖頭，「第 13 章讓你**看懂**動力學的結構——你現在知道什麼是慣性矩陣、什麼是科氏力、為什麼關節之間會耦合。這些理解是無法取代的。但如果要**計算**，我們需要另一條路。」

他在白板上畫了一支手臂，然後畫了兩個方向的箭頭：一個由基座指向末端，一個由末端指回基座。

「想像你在一列火車上。第一節車廂加速，第二節就會感受到；第二節再傳給第三節.....速度與加速度**由前往後傳遞**。」他指著向外的箭頭。「反過來，最後一節車廂的重量，會透過連結器一節一節往前壓；每一節車廂承受的力，等於它自己的重量加上後面所有車廂的重量。**力由後往前累積**。」他指著向內的箭頭。

「機器手臂完全一樣。先從基座往末端傳遞速度與加速度，算出每根連桿的運動狀態；再從末端往基座回推，算出每個關節需要承受多少力與力矩。整個過程只需要  **$O(n)$**  次運算——六軸手臂大約幾微秒就算完了。」

小翰盯著白板：「所以我週末推導的一萬四千項.....」

「用這個方法，二十行程式碼就能得到同樣的答案，而且快一萬倍。」叫獸放下筆，「這就是今天要講的 Newton-Euler 遞迴法。第 13 章教你看懂，今天教你算得夠快。」

## 本章為什麼重要

第 13 章的拉格朗日法在理解上優雅，但它有一個致命的實務問題：計算量隨自由度呈指數成長。六軸手臂的解析方程可達上萬項，無法在毫秒級的控制迴路中即時求值。

Newton-Euler 遞迴法解決了這個問題。它放棄追求解析表達式，改用兩趟遞迴的數值計算：**前向遞迴**由基座往末端傳遞速度與加速度、**後向遞迴**由末端往基座累積力與力矩。整個過程的複雜度只有  $O(n)$ ——這意味著即使是二十自由度的人形機器人，也能在微秒級完成一次完整的逆動力學計算。

這個效率上的突破，使三件事成為可能：**即時前饋控制**（第 17 章的計算力矩控制需要每毫秒算一次逆動力學）、**高速物理模擬**（第 30 章的模擬器與第 31 章的強化學習訓練，需要每秒模擬數千步）、以及**線上碰撞偵測**（第 20 章需即時扣除慣性與重力的貢獻）。可以說，沒有高效的動力學演算法，現代機器人的許多能力都無從談起。

## 學習目標

- 能寫出剛體的牛頓方程與尤拉方程，並說明兩者的分工。
- 能說明角速度與角加速度在連桿之間的傳遞關係。
- 能推導質心加速度的三個組成項（切向、離心、平移）。
- 能執行前向遞迴，逐一計算各連桿的速度與加速度。
- 能執行後向遞迴，逐一計算各關節的力與力矩。
- 能說明為何遞迴法的複雜度為  $O(n)$  而非  $O(n^3)$ 。
- 能運用遞迴法計算慣性矩陣、科氏項與重力項（複合剛體法）。
- 能比較拉格朗日法與 Newton-Euler 法的適用場合。
- 能說明正向動力學的三種演算法及其複雜度。
- 能說明物理模擬引擎的組成與數值積分的選擇。
- 能以 Python 實作 Newton-Euler 遞迴並與拉格朗日法交叉驗證。

## 先備知識

第 13 章的全部內容（尤其是慣性張量、標準形式方程與四類項的意義）、第 9 章的正向運動學與 DH 參數（遞迴需要各連桿之間的變換矩陣）、第 8 章的旋轉矩陣與座標系轉換、第 11 章的角速度概念與反對稱矩陣。

本章的推導以向量與矩陣運算為主，需要熟悉外積、座標系轉換與矩陣乘法。相較第 13 章，本章的數學較「機械」但概念較直觀——只要理解「傳遞」的物理圖像，公式就會自然成立。

## 核心名詞中英對照

表 14-1 本章核心名詞

| 中文     | 英文                                    | 說明                                                    |
|--------|---------------------------------------|-------------------------------------------------------|
| 牛頓方程   | Newton's Equation                     | 描述質心平移的方程 $F = ma$                                    |
| 尤拉方程   | Euler's Equation                      | 描述剛體轉動的方程 $N = I\dot{\omega} + \omega \times I\omega$ |
| 前向遞迴   | Forward Recursion / Outward Iteration | 由基座往末端傳遞運動量                                           |
| 後向遞迴   | Backward Recursion / Inward Iteration | 由末端往基座累積力與力矩                                          |
| 連桿間力   | Inter-link Force                      | 相鄰連桿之間互相作用的力                                          |
| 切向加速度  | Tangential Acceleration               | 角加速度造成的線加速度分量                                         |
| 向心加速度  | Centripetal Acceleration              | 角速度造成的指向轉軸的加速度                                        |
| 複合剛體法  | Composite Rigid Body Algorithm, CRBA  | 計算慣性矩陣的高效演算法                                          |
| 關節體演算法 | Articulated Body Algorithm, ABA       | $O(n)$ 複雜度的正向動力學演算法                                   |
| 重力技巧   | Gravity Trick                         | 以基座加速度模擬重力的技巧                                         |
| 數值積分   | Numerical Integration                 | 由加速度推算速度與位置的方法                                        |
| 辛積分法   | Symplectic Integrator                 | 能長期保持能量守恆的積分方法                                        |
| 剛度     | Stiffness                             | 微分方程中變化率差異極大的性質                                       |
| 接觸模型   | Contact Model                         | 描述物體碰撞與摩擦的數學模型                                        |
| 物理引擎   | Physics Engine                        | 執行剛體動力學模擬的軟體                                          |

## 14.1 牛頓方程與尤拉方程

### 14.1.1 剛體運動的兩條基本定律

Newton-Euler 法的名稱來自它所依據的兩條方程。任何剛體的運動，都可以完整地由這兩

條式子描述。

**牛頓方程**描述質心的平移運動：

$$\mathbf{F} = m \cdot \mathbf{a}_c$$

$F$  為作用在剛體上的合力、 $m$  為質量、 $a_c$  為質心的線加速度。這是大家熟悉的  $F = ma$ 。

**尤拉方程**描述繞質心的轉動運動：

$$\mathbf{N} = \mathbf{I} \cdot \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I} \cdot \boldsymbol{\omega})$$

$N$  為對質心的合力矩、 $I$  為繞質心的慣性張量、 $\boldsymbol{\omega}$  與  $\dot{\boldsymbol{\omega}}$  為角速度與角加速度。

### 14.1.2 尤拉方程的第二項從何而來

牛頓方程的形式很直觀，但尤拉方程多出了一項  $\boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$ ，這是初學者最困惑的地方。它的來源可以這樣理解：

角動量為  $\mathbf{L} = \mathbf{I} \cdot \boldsymbol{\omega}$ 。由角動量定理，力矩等於角動量的變化率： $\mathbf{N} = d\mathbf{L}/dt$ 。若在固定座標系中觀察，慣性張量  $\mathbf{I}$  會隨剛體轉動而改變（因為物體的方位在變），這使微分變得複雜。

因此我們改在**隨剛體轉動的座標系**中計算，此時  $\mathbf{I}$  是常數。但代價是：在旋轉座標系中對向量微分時，必須加上一個修正項：

$$d\mathbf{L}/dt|_{\text{固定}} = d\mathbf{L}/dt|_{\text{旋轉}} + \boldsymbol{\omega} \times \mathbf{L} = \mathbf{I} \cdot \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I} \cdot \boldsymbol{\omega})$$

第二項稱為**陀螺項 (gyroscopic term)**，是座標系旋轉所帶來的修正。

這一項的物理效應在生活中隨處可見：轉動中的陀螺不會倒下、騎自行車時車輪的角動量幫助維持平衡、以及旋轉一個形狀不對稱的物體時它會自己翻滾（Dzhanibekov 效應）。在機器人中，它是科氏力與離心力的來源之一。

**重要的簡化：**當剛體繞著其慣性主軸旋轉，且角速度沿該軸時， $\boldsymbol{\omega}$  與  $\mathbf{I}\boldsymbol{\omega}$  平行，外積為零，陀螺項消失。這也是為何平面問題（所有轉軸平行）的動力學比空間問題簡單得多。

### 14.1.3 與拉格朗日法的關係

值得強調的是：Newton-Euler 法與拉格朗日法得到的答案完全相同。它們只是推導路徑不同——前者從力的平衡出發（向量法），後者從能量出發（純量法）。

兩者的差異在於「過程中間產生了什麼」。拉格朗日法直接得到關節力矩，中間的連桿間內



力被自動消去；Newton-Euler 法則會顯式地算出每一根連桿之間的作用力。這看似多餘，實際上是一個重要的副產品——它讓我們能知道每個關節軸承受多大的力，這對機構設計與強度分析極為有用。

## 14.2 運動量的傳遞：前向遞迴

### 14.2.1 基本圖像

前向遞迴的物理圖像是開頭故事的火車：連桿  $i$  的運動狀態，等於連桿  $i-1$  的運動狀態，加上關節  $i$  自己的貢獻。因此只要從基座（運動狀態已知，通常為靜止）出發，就能一節一節往外推算。

需要傳遞的量有四個：角速度  $\omega$ 、角加速度  $\dot{\omega}$ 、連桿原點的線加速度  $a$ 、以及質心的線加速度  $a_c$ 。

### 14.2.2 角速度的傳遞

由第 11.3.3 節，角速度可以直接相加。因此連桿  $i$  的角速度等於連桿  $i-1$  的角速度（轉換到連桿  $i$  的座標系後），加上關節  $i$  自己造成的轉動：

$${}^i\omega_i = {}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1} + \dot{\theta}_i \cdot \hat{z}_i$$

旋轉關節的角速度傳遞。 ${}^iR_{i-1}$  是座標系轉換矩陣、 $\hat{z}_i$  是關節軸的單位向量（在連桿  $i$  座標系中通常就是  $[0,0,1]^T$ ）。

若關節  $i$  是移動關節，它不產生轉動，因此第二項消失：

$${}^i\omega_i = {}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1} \quad (\text{移動關節})$$

移動關節不改變角速度。

### 14.2.3 角加速度的傳遞

把角速度的式子對時間微分，會多出一個外積項：

$${}^i\dot{\omega}_i = {}^iR_{i-1} \cdot {}^{i-1}\dot{\omega}_{i-1} + ({}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1}) \times (\dot{\theta}_i \hat{z}_i) + \dot{\theta}_i \hat{z}_i$$

角加速度的傳遞。中間的外積項來自「在旋轉座標系中微分」，它是科氏效應的直接來源。

這個外積項的物理意義是：即使關節  $i$  以固定的角速度轉動（ $\dot{\theta}_i = 0$ ），只要連桿  $i-1$  本身也在轉動，連桿  $i$  仍會產生角加速度——因為它的轉軸方向正在改變。這正是陀螺效應在遞迴中

的體現。

#### 14.2.4 線加速度的傳遞

線加速度的傳遞來自剛體運動學的基本公式。連桿  $i$  的原點相對於連桿  $i-1$  的原點，有一個固定的位置向量  $P$ （在連桿  $i-1$  座標系中固定）。因此：

$${}^i\mathbf{a}_i = {}^i\mathbf{R}_{\{i-1\}} [ {}^{i-1}\boldsymbol{\omega}'_{\{i-1\}} \times {}^{i-1}\mathbf{P}_i + {}^{i-1}\boldsymbol{\omega}_{\{i-1\}} \times ({}^{i-1}\boldsymbol{\omega}_{\{i-1\}} \times {}^{i-1}\mathbf{P}_i) + {}^{i-1}\mathbf{a}_{\{i-1\}} ]$$

旋轉關節的線加速度傳遞，共三項。

這三項各有明確的物理意義，值得逐一辨識：

- $\boldsymbol{\omega}' \times \mathbf{P}$ （切向加速度）：角加速度造成的加速度，方向垂直於半徑，大小與角加速度成正比。這是「轉得越來越快」所產生的加速度。
- $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{P})$ （向心加速度）：即使角速度固定不變，做圓周運動的點仍有指向轉軸的加速度，大小為  $\omega^2 r$ 。這是離心力的來源。
- $\mathbf{a}_{\{i-1\}}$ （平移加速度）：上游連桿本身的加速度，直接傳遞下來。

質心的加速度用同樣的公式計算，只是把位置向量  $P$  換成「連桿原點到質心的向量」 $P_c$ ：

$${}^i\mathbf{a}_{ci} = {}^i\boldsymbol{\omega}'_i \times {}^i\mathbf{P}_{ci} + {}^i\boldsymbol{\omega}_i \times ({}^i\boldsymbol{\omega}_i \times {}^i\mathbf{P}_{ci}) + {}^i\mathbf{a}_i$$

質心加速度。注意此處全部在連桿  $i$  的座標系中計算，不需再轉換。

#### 14.2.5 重力的巧妙處理

重力需要特別處理，但有一個極為優雅的技巧：把基座的線加速度設為重力加速度的反方向。

$${}^0\mathbf{a}_0 = -\mathbf{g} = [0, 0, +9.8]^T \quad (\text{若重力沿 } -Z \text{ 方向})$$

重力技巧。設定基座「向上加速」，等效於整個系統受到向下的重力。

為什麼可行？這是愛因斯坦等效原理的一個工程應用：均勻重力場中的靜止觀察者，與無重力空間中等加速度運動的觀察者，物理上無法區分。因此讓基座「向上加速  $9.8 \text{ m/s}^2$ 」，所有連桿都會自動獲得正確的重力慣性力，不需在每一根連桿上另外加入重力項。

這個技巧的實用價值極高：它讓重力完全融入遞迴流程，不需任何特殊處理。程式中只需修改一個初始值，就能開關重力——這對模擬與測試極為方便。

##### ◎ 驗證重力技巧

把手臂靜止不動（所有  $\mathbf{q}' = \mathbf{q}'' = 0$ ），只設定基座加速度為  $[0, 0, 9.8]^T$ ，執行完整的遞迴。你得到的

關節力矩，恰好就是第 13 章的重力項  $G(q)$ 。這是驗證 Newton-Euler 程式是否正確的第一個測試——它不需要任何複雜的軌跡，卻能檢查出座標轉換與遞迴邏輯的大部分錯誤。

## 14.3 力的累積：後向遞迴

### 14.3.1 基本圖像

完成前向遞迴後，我們知道了每一根連桿的加速度。現在用牛頓與尤拉方程，算出每根連桿需要多少力與力矩才能產生那樣的運動：

$$\begin{aligned} {}^iF_i &= m_i \cdot {}^ia_{ci} \quad (\text{作用在連桿 } i \text{ 質心上的合力}) \\ {}^iN_i &= {}^iI_i \cdot {}^i\dot{\omega}_i + {}^i\omega_i \times ({}^iI_i \cdot {}^i\omega_i) \quad (\text{對質心的合力矩}) \end{aligned}$$

由牛頓與尤拉方程直接計算。這是每根連桿「所需」的淨力與淨力矩。

但這個「合力」是從哪裡來的？答案是：來自上游連桿的推力，減去傳給下游連桿的力。因此可以由末端往基座回推——這就是後向遞迴。

### 14.3.2 力與力矩的平衡

考慮連桿  $i$  的受力平衡。它受到兩個相鄰連桿的作用：連桿  $i-1$  透過關節  $i$  施加的力  $f_i$ ，以及連桿  $i+1$  透過關節  $i+1$  施加的反作用力  $-f_{i+1}$ 。兩者之和必須等於淨力：

$${}^if_i = {}^iR_{i+1} \cdot {}^{i+1}f_{i+1} + {}^iF_i$$

力的後向遞迴。連桿  $i$  所受的力，等於下游傳來的力，加上自己所需的淨力。

力矩的平衡較複雜，因為力作用在不同位置會產生額外的力矩：

$${}^in_i = {}^iN_i + {}^iR_{i+1} \cdot {}^{i+1}n_{i+1} + {}^iP_{ci} \times {}^iF_i + {}^iP_{i+1} \times ({}^iR_{i+1} \cdot {}^{i+1}f_{i+1})$$

力矩的後向遞迴，共四項：自身所需的淨力矩、下游傳來的力矩、自身慣性力產生的力矩、以及下游傳來的力所產生的力矩。

最後兩項是外積形式，代表「力乘上力臂」。這正是後向遞迴中最容易出錯的部分——必須確認每個力的作用點與力臂向量是否正確。

### 14.3.3 取出關節力矩

遞迴完成後，得到的是每個關節處的完整力與力矩向量（六維）。但關節只能在其轉軸方向出力，其餘方向由軸承承擔。因此關節的驅動力矩就是力矩向量在轉軸上的投影：

$$\tau_i = {}^in_i^T \cdot \hat{z}_i \quad (\text{旋轉關節})$$

$$\tau_i = {}^i\mathbf{f}_i^T \cdot \hat{\mathbf{z}}_i \quad (\text{移動關節})$$

取關節軸方向的分量。這一步是「六維力」轉為「一維關節力矩」的關鍵。

而其餘的分量（垂直於轉軸的力與力矩）雖然不是驅動力矩，卻是極有價值的資訊——它們正是軸承與結構必須承受的負荷。機構設計時，這些數值用於選定軸承規格與驗算連桿強度。這是 Newton-Euler 法相對於拉格朗日法的一個重要優勢（14.1.3 節）。

圖 14-1 Newton-Euler 兩趟遞迴的流程

建議配圖：一支三連桿手臂的示意圖，上方以藍色箭頭由基座往末端標示前向遞迴，每個關節旁標註傳遞的量（ $\omega$ 、 $\dot{\omega}$ 、 $a$ 、 $a_c$ ）；下方以紅色箭頭由末端往基座標示後向遞迴，每個關節旁標註累積的量（ $f$ 、 $n$ ），並在每個關節處以綠色標出取軸向投影得到的  $\tau_i$ 。左側標註「已知： $q, \dot{q}, \ddot{q}$ 」、右側標註「輸出： $\tau$ 」。基座處特別註明「重力技巧： $a_0 = -g$ 」。

## 14.4 完整演算法與複雜度分析

### 14.4.1 演算法總覽

把前後兩趟遞迴組合起來，就是完整的 Newton-Euler 逆動力學演算法：

1. 初始化：設  $\omega_0 = 0$ 、 $\dot{\omega}_0 = 0$ 、 $a_0 = -g$ （重力技巧）。
2. 前向遞迴（ $i = 1$  到  $n$ ）：依序計算  $\omega_i$ 、 $\dot{\omega}_i$ 、 $a_i$ 、 $a_{ci}$ ，再由牛頓尤拉方程得  $F_i$ 、 $N_i$ 。
3. 邊界條件：設末端所受的外力  $f_{n+1}$  與力矩  $n_{n+1}$ （無外力時為零；若末端接觸環境則填入實測或期望的力）。
4. 後向遞迴（ $i = n$  到  $1$ ）：依序計算  $f_i$ 、 $n_i$ 。
5. 取投影： $\tau_i = n_i^T \hat{\mathbf{z}}_i$ （旋轉關節），得到最終的關節力矩向量。

整個流程只需要基本的向量運算（加法、外積、矩陣乘向量），沒有任何矩陣求逆或符號運算。這正是它高效的原因。

### 14.4.2 為何是 $O(n)$

複雜度分析很直接：兩趟遞迴各執行  $n$  次，每次的運算量是固定的（幾個  $3 \times 3$  矩陣乘法與外積，與  $n$  無關）。因此總運算量與  $n$  成正比，即  $O(n)$ 。

這與拉格朗日法形成鮮明對比。若直接展開拉格朗日方程的解析式，慣性矩陣有  $n^2$  個元素、科氏項有  $n^3$  個 Christoffel 符號，複雜度為  $O(n^3)$  甚至更差（若考慮符號展開後的項數，會呈指數成長）。

表 14-2 兩種動力學方法的複雜度比較

| 方法              | 逆動力學複雜度      | 六軸手臂的典型運算時間       | 是否得到解析式      | 主要用途             |
|-----------------|--------------|-------------------|--------------|------------------|
| 拉格朗日 ( 符號展開 )   | $O(n^3)$ 或更差 | 數毫秒 ( 且需事先推導數小時 ) | 是 ( 但可達上萬項 ) | 理論分析、教學、小自由度系統   |
| Newton-Euler 遞迴 | $O(n)$       | 數微秒               | 否 ( 純數值 )    | 即時控制、物理模擬、大自由度系統 |

以實際數字說明這個差距的意義：一個 20 自由度的人形機器人，用  $O(n)$  演算法約需 20 個單位的運算；用  $O(n^3)$  則需要 8000 個單位——相差 400 倍。這正是為何所有實用的機器人軟體 ( Pinocchio、RBDL、KDL、MuJoCo ) 內部都採用遞迴演算法。

### 14.4.3 可以順便得到什麼

Newton-Euler 遞迴的一個重要副產品，是它能用同一套程式碼算出動力學方程的各個組成部分，只要適當設定輸入：

表 14-3 用 Newton-Euler 計算各項的技巧

| 要計算的量                        | 設定 $\ddot{q}$ | 設定 $\dot{q}$   | 設定重力 | 得到的結果                             |
|------------------------------|---------------|----------------|------|-----------------------------------|
| 完整力矩 $\tau$                  | 實際值           | 實際值            | 開啟   | $\tau = M\ddot{q} + C\dot{q} + G$ |
| 重力項 $G(q)$                   | 0             | 0              | 開啟   | $G(q)$                            |
| 科氏離心項 $C(q, \dot{q})\dot{q}$ | 實際值           | 0              | 關閉   | $C(q, \dot{q})\dot{q}$            |
| 慣性矩陣第 $i$ 行                  | 0             | $e_i$ ( 單位向量 ) | 關閉   | $M$ 的第 $i$ 行                      |

最後一列的技巧特別實用：把加速度設為第  $i$  個單位向量 ( 只有第  $i$  個元素為 1，其餘為 0 )，並關閉重力與速度，則得到的力矩就恰好是慣性矩陣的第  $i$  行。執行  $n$  次即可得到完整的  $M(q)$ ——這稱為**單位向量法**，複雜度為  $O(n^2)$ 。若追求更高效率，則使用專門的**複合剛體演算法** ( CRBA )，複雜度同樣是  $O(n^2)$  但常數更小。

## 14.5 正向動力學的三種演算法

### 14.5.1 問題的不對稱性

Newton-Euler 遞迴解決的是**逆動力學**：已知運動求力矩。但模擬需要的是相反方向——已

知力矩求加速度。這兩個問題的難度並不對稱。

逆動力學可以直接遞迴計算，因為運動狀態完全已知，力可以一節一節累積。正向動力學則不然：要算出連桿 1 的加速度，必須知道下游連桿施加的反作用力；但那個反作用力又取決於下游連桿的加速度，而後者又受連桿 1 影響——形成了循環依賴。這正是為何正向動力學需要更精巧的演算法。

### 14.5.2 方法一：組合法 ( $O(n^3)$ )

最直接的作法是利用第 13 章的標準形式。先用 Newton-Euler 分別算出各項，再解線性方程組：

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} [ \boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{G}(\mathbf{q}) ]$$

*組合法。需要三個步驟：計算偏差項、計算慣性矩陣、解線性方程組。*

具體流程為：一、用一次 Newton-Euler 遞迴（設  $\dot{\mathbf{q}} = 0$ 、開啟重力與實際速度）得到偏差項  $\mathbf{b} = \mathbf{C}\dot{\mathbf{q}} + \mathbf{G}$ ；二、用單位向量法或 CRBA 建構慣性矩陣  $\mathbf{M}$  (  $O(n^2)$  )；三、解  $\mathbf{M} \cdot \ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{b}$  (  $O(n^3)$ ，通常用 Cholesky 分解，因  $\mathbf{M}$  正定 )。

總複雜度由第三步主導，為  $O(n^3)$ 。對六軸手臂完全可接受 (  $n^3 = 216$  )，但對高自由度系統（如 30 自由度的人形機器人， $n^3 = 27000$ ）效率就不理想。

### 14.5.3 方法二：關節體演算法 ( $O(n)$ )

Featherstone 於 1983 年提出的關節體演算法 ( Articulated Body Algorithm, ABA ) 將複雜度降到  $O(n)$ 。它的核心概念稱為關節體慣量 ( articulated body inertia )：

把「連桿  $i$  以及它下游的所有連桿」視為一個整體，問一個問題：若對連桿  $i$  施加一個力，它會產生多大的加速度？由於下游連桿是透過關節連接（而非剛性固定），它們對連桿  $i$  呈現的「等效慣量」與剛性連接時不同——這個等效慣量就是關節體慣量。

ABA 用三趟遞迴完成計算：第一趟（前向）計算速度；第二趟（後向）計算關節體慣量與偏差力；第三趟（前向）由基座往末端計算加速度。每一趟都是  $O(n)$ ，因此總複雜度為  $O(n)$ 。

實務上，ABA 在自由度超過約 9 時開始優於組合法。現代的模擬器 ( MuJoCo、Bullet、Pinocchio ) 多半兩種都實作，依系統規模自動選擇。

### 14.5.4 方法三：最大座標法

前兩種方法都採用**最小座標**（以關節變數描述系統），約束自動滿足。另一條路線是**最大座標法**：把每根連桿都視為自由的六自由度剛體（共  $6n$  個變數），再用約束方程強制它們透過關節連接。

$$\mathbf{M} \ddot{\mathbf{q}} = \boldsymbol{\tau} + \mathbf{J}^c \boldsymbol{\lambda}, \quad \mathbf{J}^c \ddot{\mathbf{q}} = -\dot{\mathbf{J}}^c \dot{\mathbf{q}}$$

$\boldsymbol{\lambda}$  為約束力（拉格朗日乘子）、 $\mathbf{J}^c$  為約束雅可比。需同時解出加速度與約束力。

這個方法變數多、需要解更大的方程組，且約束誤差會隨時間累積（連桿可能「散開」，需額外的穩定化處理）。但它有一個重要優勢：**能自然處理閉環機構與接觸約束**。因此電玩物理引擎與需要大量接觸的模擬（如抓取、行走）常採此法，而機器手臂的控制器則多用最小座標法。

表 14-4 三種正向動力學方法的比較

| 方法                        | 複雜度                  | 座標 | 閉環支援 | 數值穩定性 | 典型應用         |
|---------------------------|----------------------|----|------|-------|--------------|
| 組合法 ( $\mathbf{M}^{-1}$ ) | $O(n^3)$             | 最小 | 困難   | 佳     | 低自由度手臂、控制器   |
| 關節體演算法 ABA                | $O(n)$               | 最小 | 困難   | 佳     | 人形、四足、高自由度模擬 |
| 最大座標法                     | $O(m^3)$ ( $m$ 為約束數) | 最大 | 自然支援 | 需穩定化  | 接觸豐富的模擬、電玩引擎 |

## 14.6 數值積分

### 14.6.1 由加速度到運動

正向動力學給出的是加速度。要得到實際的運動軌跡，必須對時間積分兩次：

$$\ddot{\mathbf{q}} \rightarrow \dot{\mathbf{q}} \rightarrow \mathbf{q}$$

積分兩次。實務上以離散的時間步驟  $\Delta t$  逐步推進，這稱為數值積分。

積分方法的選擇對模擬品質有決定性的影響——這也是第 13 章程式範例中「能量守恆」測試如此重要的原因。

### 14.6.2 四種常見的積分器

**顯式歐拉法 (Explicit Euler)**：最簡單，但最不穩定。



$$\dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t, \quad q_{k+1} = q_k + \dot{q}_k \Delta t$$

全部使用舊值更新。誤差為  $O(\Delta t)$ ，且能量會單調增加——模擬的鐘擺會越擺越高，最終發散。實務上幾乎不用。

**半隱式歐拉法 (Semi-implicit / Symplectic Euler)**：只改一個字，效果天差地遠。

$$\dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t, \quad q_{k+1} = q_k + \dot{q}_{k+1} \Delta t$$

位置更新時使用新的速度。這是一個辛積分器 (symplectic)，能長期保持能量在真值附近振盪而不發散。

這個看似微小的差異，是物理引擎的標準選擇。它的精度只有一階，但長期穩定性遠優於同階的顯式方法——對模擬而言，「不發散」比「每一步都很準」更重要。

**四階 Runge-Kutta (RK4)**：精度高（誤差  $O(\Delta t^4)$ ），每步需計算四次導數。適合需要高精度的短時間模擬（如驗證控制器），但不是辛積分器，長時間模擬仍會有能量漂移。

**Verlet 系列**：辛積分器，精度二階，在分子動力學與遊戲物理中廣泛使用。它的變體 (velocity Verlet) 能同時得到位置與速度，是精度與穩定性的良好折衷。

表 14-5 四種數值積分器的比較

| 方法              | 精度              | 每步計算量   | 能量行為     | 是否辛積分 | 適用場合         |
|-----------------|-----------------|---------|----------|-------|--------------|
| 顯式歐拉            | $O(\Delta t)$   | 1 次     | 單調增加（發散） | 否     | 不建議使用        |
| 半隱式歐拉           | $O(\Delta t)$   | 1 次     | 有界振盪     | 是     | 物理引擎標準選擇     |
| Velocity Verlet | $O(\Delta t^2)$ | 1 ~ 2 次 | 有界振盪     | 是     | 高精度模擬、分子動力學  |
| RK4             | $O(\Delta t^4)$ | 4 次     | 緩慢漂移     | 否     | 短時間高精度、控制器驗證 |

### 14.6.3 步長的選擇

步長  $\Delta t$  的選擇是精度與速度的權衡，但還受一個更嚴格的限制：**穩定性**。若系統中含有高頻動態（剛性接觸、高增益控制器、彈性關節），步長必須小於該動態的特徵時間，否則模擬會發散。

實務上的經驗值為：機器手臂的自由運動可用 1 ~ 5 ms；含接觸的模擬需要 0.1 ~ 1 ms；含高剛度彈簧或剛性碰撞則可能需要 0.01 ms 以下。這正是為何含接觸的模擬遠比自由運動昂貴——第 30 章討論模擬器選擇時會再回到這一點。

◎ 為何能量守恆是最好的驗證



動力學程式的錯誤（座標轉換寫反、外積順序顛倒、力臂用錯）往往不會造成明顯的異常，模擬看起來「還算合理」，卻是錯的。但能量守恆是一個極為嚴格的檢驗：只要有任何一處推導錯誤，能量幾乎必定會漂移或發散。因此標準的驗證流程是：關閉重力以外的所有外力與摩擦，讓系統自由運動數秒，檢查總能量是否維持在初值的 0.1% 以內。若通過此測試，模型與積分器幾乎可以確定是正確的。

## 14.7 接觸與碰撞

### 14.7.1 模擬中最困難的部分

到目前為止，我們處理的都是「自由運動」——機器人在空中揮舞，不接觸任何東西。但機器人的價值恰恰在於**接觸**：抓取物體、插入零件、行走時腳踩地面。而接觸正是物理模擬中最困難、最不可靠的部分。

困難的來源有三。**第一，接觸是不連續的**：物體要嘛接觸、要嘛不接觸，狀態瞬間切換，這對數值積分極不友善。**第二，接觸力極大且作用時間極短**：碰撞瞬間的力可達數千牛頓，持續時間卻只有毫秒，需要極小的步長才能捕捉。**第三，摩擦是非線性且不唯一的**：庫倫摩擦模型在某些配置下有多組解（甚至無解），這是著名的 Painlevé 悖論。

### 14.7.2 兩類接觸模型

**懲罰法（Penalty Method）**：允許物體輕微互相穿透，並依穿透深度產生一個彈簧—阻尼力把它們推開。

$$F_{\text{contact}} = k \cdot d + c \cdot \dot{d} \quad (d \text{ 為穿透深度})$$

懲罰法。實作簡單、力是連續的，適合梯度式的最佳化與學習。缺點是需要極大的剛度  $k$  才能避免明顯穿透，而大  $k$  導致方程剛性化，必須用極小的步長。

**約束法（Constraint-based）**：以不等式約束強制「不得穿透」，接觸力由求解一個線性互補問題（LCP）或凸最佳化問題得到。

優點是物理上更嚴謹、不會穿透、可用較大步長。缺點是求解成本高，且在多接觸點的情況下可能無唯一解。多數現代引擎（MuJoCo、Bullet 的部分模式）採用經過正則化的凸最佳化形式，在嚴謹性與可解性之間取得平衡。

### 14.7.3 為何模擬的抓取常常不準

這解釋了第 2.7.3 節提到的 Sim-to-Real Gap 中最頑固的一環：**在模擬中完美的抓取，到真機上常常滑落**。原因可歸納為四點：

- **摩擦係數不確定**：真實材料的摩擦係數隨表面狀態、濕度、壓力而變，模擬中通常設為一個固定值。
- **接觸面積被簡化**：真實的夾爪與物體之間是面接觸（且會變形），模擬中常簡化為數個點接觸。
- **柔軟性未建模**：真實的夾爪墊、物體表面都會變形，這種順應性對抓取穩定性極為關鍵，但難以精確模擬。
- **接觸模型本身的近似**：懲罰法的穿透、約束法的正則化，都與真實物理有落差。

因應之道是第 31 章會詳述的**領域隨機化**：在訓練時隨機改變摩擦係數、質量、接觸剛度等參數，強迫學到的策略對這些不確定性具備強健性。這是目前處理接觸不確定性最有效的實務方法。

## 14.8 物理模擬引擎

### 14.8.1 引擎的四大組成

一個完整的物理模擬引擎包含四個模組，動力學只是其中之一：

一、**碰撞偵測 ( Collision Detection )**：判斷哪些物體正在接觸，並計算接觸點、法向量與穿透深度。這通常是計算量最大的部分，分為粗略階段 ( broad phase，用包圍盒快速排除不可能碰撞的配對 ) 與精確階段 ( narrow phase，計算實際的接觸幾何 )。

二、**動力學求解 ( Dynamics Solver )**：即本章的正向動力學，計算在給定力與約束下的加速度。

三、**約束求解 ( Constraint Solver )**：處理關節約束、接觸約束與摩擦，通常以迭代方式求解（如投影高斯—賽德爾法）。

四、**數值積分 ( Integrator )**：14.6 節的內容，由加速度推進到下一個狀態。

### 14.8.2 常見引擎的比較

機器人領域常用的物理引擎各有取捨：

表 14-6 常見物理引擎的比較

| 引擎     | 特色                   | 接觸模型       | 速度      | 典型用途              |
|--------|----------------------|------------|---------|-------------------|
| MuJoCo | 為機器人與強化學習設計，接觸模型平滑可微 | 凸最佳化（軟接觸）  | 極快      | 強化學習訓練、控制研究       |
| Bullet | 開源、成熟、廣泛整合           | LCP / 順序脈衝 | 快       | 遊戲、通用機器人模擬        |
| ODE    | 歷史悠久、Gazebo 預設       | LCP        | 中       | ROS 生態系、教學        |
| PhysX  | NVIDIA 出品、GPU 加速     | 混合         | 極快（GPU） | Isaac Sim、大規模平行模擬 |
| DART   | 重視動力學精確性             | 約束法        | 中       | 學術研究、精確動力學        |

選擇的原則是：看重接觸的真實性選 DART 或 Bullet；看重速度與可微性（用於學習）選 MuJoCo；需要 GPU 大規模平行訓練選 Isaac Sim（PhysX）；需要與 ROS 2 整合則優先考慮 Gazebo 支援的引擎。第 30 章會從模擬應用的角度再次比較。

### 14.8.3 可微分模擬：新的方向

近年出現一個重要趨勢：可微分物理模擬。傳統模擬器只能回答「給定這些參數，會發生什麼」；可微分模擬器還能回答「若要得到期望的結果，參數該如何調整」——因為它能計算模擬結果對輸入參數的梯度。

這帶來三個應用：系統辨識（由觀測到的真實運動，用梯度下降反推物理參數，比第 13.10 節的傳統辨識更通用）、策略最佳化（直接對控制參數求梯度，比強化學習的隨機探索更有效率）、以及機構最佳化（把連桿尺寸也當作可微參數，自動設計出最佳機構）。

挑戰在於接觸的不連續性——碰撞的瞬間梯度不存在或無意義。因此可微分模擬器（如 MuJoCo 的軟接觸、Brax、Dojo）多半採用平滑化的接觸模型，以可微性換取部分物理真實性。這個取舍的合理性仍在研究中，是第 31、33 章會觸及的前沿主題。

## 14.9 兩種方法的分工與實務選擇

### 14.9.1 各自的舞台

走完第 13、14 兩章，可以清楚地劃分兩種方法的分工。

**拉格朗日法的價值在於「看見結構」**。它給出的解析表達式讓我們能辨識慣性矩陣、科氏項與重力項，理解耦合從何而來、哪些參數影響哪些效應。第 13 章例題 13-3 那個「 $\theta_2 = 120^\circ$  時耦合完全消失」的發現，只有透過解析式才看得出來。此外，控制理論的穩定性證明（第 19 章）也需要解析形式的性質（如  $\dot{M} - 2C$  的斜對稱性）。

**Newton-Euler 法的價值在於「算得夠快」**。所有需要即時計算的場合——控制器前饋、物理模擬、線上碰撞偵測——都採用它。它也是所有機器人動力學函式庫的內部實作。

因此實務上的標準流程是：**用拉格朗日法理解與分析，用 Newton-Euler 法實作與計算**。兩者互相驗證——若你的遞迴程式與解析式在同一組輸入下得到不同結果，必定有一方寫錯了。這正是本章程式範例的核心測試。

### 14.9.2 現代實務：直接用函式庫

必須誠實地說：在真實的工程專案中，很少有人從頭實作動力學演算法。成熟的開源函式庫已經處理了所有的細節與最佳化：

- **Pinocchio**：C++ 實作，效能極高，支援解析導數（可微分），是目前研究界的主流選擇。
- **RBDL**（Rigid Body Dynamics Library）：輕量、易於整合，廣泛用於人形機器人研究。
- **KDL**（Kinematics and Dynamics Library）：ROS 生態系的傳統選擇，功能完整但效能較普通。
- **Drake**：包含動力學、最佳化與控制的完整框架，由 MIT 與 TRI 開發。

那麼為何還要學這一章？三個理由。**理由一：除錯**。當函式庫的輸出不符預期時，你必須知道它在算什麼、可能哪裡出錯（最常見的是慣性參數的座標系定義不一致）。**理由二：非標準機構**。閉環、柔性關節、浮動基座等情形，往往需要自行擴充。**理由三：理解性能瓶頸**。知道哪些操作是  $O(n)$ 、哪些是  $O(n^3)$ ，才能在系統設計時做出正確的取捨。

## 14.10 由動力學邁向軌跡與控制

### 14.10.1 本章的成果

本章完成了動力學的計算工具。你現在能夠：運用牛頓與尤拉方程描述剛體運動、執行前向與後向遞迴計算逆動力學、理解  $O(n)$  複雜度的來源、選擇合適的正向動力學演算法與數值積分器、並理解物理模擬引擎的組成與接觸模型的限制。

結合第 13 章，你已經掌握了機器人動力學的完整圖像：知道方程長什麼樣、每一項代表什麼、以及如何在微秒內算出它。

### 14.10.2 接下來：把動力學用起來

第 13、14 章建立的是工具，接下來六章要把它用起來。

**第 15 章 (軌跡生成)**：規劃運動時不能只考慮幾何路徑，還必須確認所需的力矩不超過馬達能力、加速度不引起振動。這需要逆動力學來驗證軌跡的可行性。

**第 16 章 (線性回授與 PID)**：先從最簡單的控制器出發，理解回授的基本原理與極限——並看見為何純 PID 在高速時會失效 (正如第 13 章開頭的故事)。

**第 17 章 (非線性控制)**：把動力學模型加入控制器，用計算力矩控制與操作空間控制克服 PID 的限制。這是本章遞迴演算法最直接的應用場合。

**第 18、19 章 (最佳控制與穩定性)**：用更嚴謹的數學工具設計控制器並證明其穩定性。第 13 章提到的  $\dot{M} - 2C$  斜對稱性，將在此發揮關鍵作用。

**第 20 章 (力控制)**：當機器人接觸環境時，必須同時控制運動與力。動力學模型讓我們能區分「慣性造成的力」與「外界施加的力」。

換句話說：第 13 章讓你**看懂**、第 14 章讓你**算快**、第 15 章之後開始**用它做事**。

## 完整計算例題

### 例題 14-1 單連桿的 Newton-Euler 遞迴

題目：一根均勻連桿，長  $L = 0.5 \text{ m}$ 、質量  $m = 2.0 \text{ kg}$ ，繞固定端在垂直平面內轉動 (即第 13 章的單擺)。目前  $\theta = 60^\circ$  (由垂直向下量起)、 $\dot{\theta} = 2.0 \text{ rad/s}$ 、 $\ddot{\theta} = 3.0 \text{ rad/s}^2$ 。以 Newton-Euler 遞迴計算所需的關節力矩，並與第 13 章的拉格朗日結果比對。

解：設連桿座標系  $\{1\}$  的原點在關節處、X 軸沿連桿方向、Z 軸垂直於運動平面。質心在  $P_c = [L/2, 0, 0]^T = [0.25, 0, 0]^T$ 。繞質心的慣性張量中，只有 Z 方向的分量參與運動： $I_{zz} = (1/12)mL^2 = (1/12)(2.0)(0.25) = 0.04167 \text{ kg}\cdot\text{m}^2$ 。

**前向遞迴**。基座靜止， $\omega_0 = 0$ 、 $\dot{\omega}_0 = 0$ 。依重力技巧 (14.2.5 節)，設基座加速度為重力的反向。在連桿座標系中，需將重力向量轉換過來——當連桿與垂直方向夾  $60^\circ$  時：

$${}^1\mathbf{a}_1 = {}^1\mathbf{R}_0 \cdot (-\mathbf{g}^*) = [g \sin \theta, g \cos \theta, 0]^T = [9.8(0.866), 9.8(0.5), 0]^T \\ = [8.487, 4.900, 0]^T \text{ m/s}^2$$

關節處的線加速度 ( 僅含重力效應，因基座固定 ) 。

角速度與角加速度直接由關節給出：

$${}^1\boldsymbol{\omega}_1 = [0, 0, 2.0]^T \text{ rad/s}, \quad {}^1\boldsymbol{\omega}'_1 = [0, 0, 3.0]^T \text{ rad/s}^2$$

沿 Z 軸的轉動。

質心加速度 ( 14.2.4 節的三項 ) ：

$${}^1\mathbf{a}_{c1} = \boldsymbol{\omega}' \times \mathbf{P}_c + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{P}_c) + \mathbf{a}_1$$

三項相加。

$$\boldsymbol{\omega}' \times \mathbf{P}_c = [0, 0, 3.0] \times [0.25, 0, 0] = [0, 0.75, 0]^T \quad (\text{切向})$$

$$\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{P}_c) = -\omega^2 \mathbf{P}_c = [-(4.0)(0.25), 0, 0]^T = [-1.0, 0, 0]^T \quad (\text{向心})$$

前兩項。

$${}^1\mathbf{a}_{c1} = [-1.0 + 8.487, 0.75 + 4.900, 0]^T = [7.487, 5.650, 0]^T \text{ m/s}^2$$

質心總加速度。

牛頓與尤拉方程：

$${}^1\mathbf{F}_1 = m \cdot \mathbf{a}_{c1} = 2.0 \times [7.487, 5.650, 0]^T = [14.974, 11.300, 0]^T \text{ N}$$

淨力。

$${}^1\mathbf{N}_1 = I \boldsymbol{\omega}' + \boldsymbol{\omega} \times (I \boldsymbol{\omega}) = [0, 0, 0.04167 \times 3.0]^T + 0 = [0, 0, 0.125]^T \text{ N}\cdot\text{m}$$

淨力矩。因  $\boldsymbol{\omega}$  與  $I\boldsymbol{\omega}$  平行 ( 皆沿 Z )，陀螺項為零。

後向遞迴。末端無外力 (  $f_2 = 0$ 、 $n_2 = 0$  )，故：

$${}^1\mathbf{f}_1 = {}^1\mathbf{F}_1 = [14.974, 11.300, 0]^T \text{ N}$$

關節受力。

$${}^1\mathbf{n}_1 = {}^1\mathbf{N}_1 + \mathbf{P}_c \times {}^1\mathbf{F}_1 = [0, 0, 0.125]^T + [0.25, 0, 0] \times [14.974, 11.300, 0]^T$$

力矩累積。

$$\mathbf{P}_c \times \mathbf{F} = [0.25, 0, 0] \times [14.974, 11.300, 0] = [0, 0, 0.25 \times 11.300]^T = [0, 0, 2.825]^T$$

外積計算。

$${}^1\mathbf{n}_1 = [0, 0, 0.125 + 2.825]^T = [0, 0, 2.950]^T \text{ N}\cdot\text{m}$$

關節總力矩向量。

取投影：關節軸為 Z 軸，故  $\tau = n_1^T \hat{z} = 2.950 \text{ N}\cdot\text{m}$ 。

與拉格朗日法比對（第 13.4.2 節）：

$$\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta = (1/3)(2.0)(0.25)(3.0) + (2.0)(9.8)(0.25)(0.866) \\ = 0.500 + 2.450 = 2.950 \text{ N}\cdot\text{m} \checkmark$$

兩種方法完全一致，驗證成功。

觀察：Newton-Euler 法的步驟較多，但每一步都是簡單的向量運算；拉格朗日法一步到位，但需要事先推導出那條公式。對單連桿而言後者顯然較快，但當自由度增加到六時，情況會完全逆轉。

## 例題 14-2 複雜度的實際意義

題目：某控制器的迴路頻率為 1 kHz（每毫秒執行一次），處理器每秒可執行  $10^8$  次浮點運算。假設 Newton-Euler 遞迴每個自由度需 150 次浮點運算，而拉格朗日展開式的運算量約為  $30n^3$  次。（a）計算六軸手臂用兩種方法的運算時間。（b）計算 20 自由度人形機器人的情形。（c）判斷各情形是否能滿足即時要求。

解（a）：六軸手臂（ $n = 6$ ）。

$$\text{Newton-Euler : } 150 \times 6 = 900 \text{ 次運算} \rightarrow 900 / 10^8 = 9 \mu\text{s}$$

佔 1 ms 週期的 0.9%，遊刃有餘。

$$\text{拉格朗日 : } 30 \times 6^3 = 30 \times 216 = 6480 \text{ 次運算} \rightarrow 6480 / 10^8 = 64.8 \mu\text{s}$$

佔週期的 6.5%，仍可接受但已高出 7 倍。

解（b）：20 自由度人形機器人（ $n = 20$ ）。

$$\text{Newton-Euler : } 150 \times 20 = 3000 \text{ 次} \rightarrow 30 \mu\text{s}$$

佔週期的 3%，仍然輕鬆。

$$\text{拉格朗日 : } 30 \times 20^3 = 30 \times 8000 = 240000 \text{ 次} \rightarrow 2400 \mu\text{s} = 2.4 \text{ ms}$$

超過 1 ms 的控制週期，無法即時執行。

解（c）：六軸手臂兩種方法都能滿足即時要求；但 20 自由度系統只有 Newton-Euler 可行，拉格朗日法會使控制迴路超時。



**工程意義：**這個對比說明了為何複雜度分析不是學術遊戲。 $O(n)$  與  $O(n^3)$  的差別，在  $n = 6$  時只是「快 7 倍」，在  $n = 20$  時卻是「能做」與「不能做」的差別。而現代機器人（人形、四足、雙臂）的自由度普遍在 20 以上，且模擬時還需要以比實時快數十倍的速度執行——此時演算法效率就是決定性的。

### 例題 14-3 積分器的能量行為比較

題目：以單擺為例（ $L = 0.5 \text{ m}$ 、 $m = 2.0 \text{ kg}$ ，無摩擦、無驅動），由  $\theta = 90^\circ$ 、 $\dot{\theta} = 0$  靜止釋放。步長  $\Delta t = 0.01 \text{ s}$ 。（a）寫出顯式歐拉與半隱式歐拉的更新式。（b）計算第一步之後的狀態與總能量。（c）說明兩者長期行為的差異。

解（a）：單擺方程為  $\ddot{\theta} = -(mgL/2I)\sin\theta$ ，其中  $I = (1/3)mL^2 = 0.1667 \text{ kg}\cdot\text{m}^2$ 。故：

$$\ddot{\theta} = -[(2.0)(9.8)(0.25)/0.1667] \sin\theta = -29.4 \sin\theta$$

角加速度。

$$\text{顯式歐拉：}\theta_{k+1} = \theta_k + \dot{\theta}_k \Delta t, \quad \ddot{\theta}_{k+1} = \ddot{\theta}_k + \ddot{\theta}_k \Delta t$$

$$\text{半隱式歐拉：}\dot{\theta}_{k+1} = \dot{\theta}_k + \ddot{\theta}_k \Delta t, \quad \theta_{k+1} = \theta_k + \dot{\theta}_{k+1} \Delta t$$

唯一差別在位置更新使用舊速度或新速度。

解（b）：初始  $\theta_0 = \pi/2$ 、 $\dot{\theta}_0 = 0$ ，故  $\ddot{\theta}_0 = -29.4 \sin(90^\circ) = -29.4 \text{ rad/s}^2$ 。

初始能量： $T = 0$ 、 $U = -mg(L/2)\cos(90^\circ) = 0$ ，故  $E_0 = 0 \text{ J}$ （以水平位置為基準）。

顯式歐拉： $\dot{\theta}_1 = 0 + (-29.4)(0.01) = -0.294 \text{ rad/s}$ ； $\theta_1 = \pi/2 + (0)(0.01) = 1.5708 \text{ rad}$ （位置沒有改變，因為用的是舊速度 0）。

$$E_1 = \frac{1}{2}(0.1667)(0.294^2) + (-2.0 \times 9.8 \times 0.25 \times \cos 90^\circ) = 0.00721 + 0 = 0.00721 \text{ J}$$

能量憑空增加了！這正是顯式歐拉會發散的原因。

半隱式歐拉： $\dot{\theta}_1$  同樣為  $-0.294 \text{ rad/s}$ ；但  $\theta_1 = \pi/2 + (-0.294)(0.01) = 1.5679 \text{ rad}$ （位置有變化）。

$$E_1 = 0.00721 + (-4.9 \cos(1.5679)) = 0.00721 - 0.00436 = 0.00285 \text{ J}$$

誤差小得多，且不會單調累積。

解（c）：長期行為的差異是決定性的。顯式歐拉的能量每一步都增加，模擬中的單擺會越擺越高，數十秒後完全發散、失去物理意義。半隱式歐拉的能量在真值附近有界振盪，即使模擬



數小時也不會發散——擺動的幅度可能略有偏差，但物理行為始終合理。

這正是所有物理引擎採用辛積分器的原因：對長時間模擬而言，「不發散」比「每一步都精確」重要得多。

## 程式範例：Newton-Euler 遞迴的實作與交叉驗證

以下 Python 程式實作平面手臂的 Newton-Euler 遞迴逆動力學，並與第 13 章的拉格朗日解析式交叉驗證。同時示範用遞迴法萃取慣性矩陣、重力項，以及三種積分器的能量行為比較。

```
import numpy as np

class PlanarNE:
    """平面 n 連桿手臂的 Newton-Euler 遞迴動力學"""
    def __init__(self, L, m, g=9.8):
        self.L = np.array(L, float)      # 各連桿長度
        self.m = np.array(m, float)      # 各連桿質量
        self.n = len(L)
        self.lc = self.L / 2              # 均勻桿，質心在中點
        self.I = self.m * self.L**2 / 12  # 繞質心的轉動慣量 (Z 軸)
        self.g = g

    def inverse_dynamics(self, q, qd, qdd, gravity=True):
        """Newton-Euler 遞迴逆動力學 (14.4.1 節的五步驟)"""
        n = self.n
        # ----- 步驟 1：初始化 (重力技巧) -----
        w = 0.0                          # 角速度 (平面問題為純量，沿 Z)
        wd = 0.0                          # 角加速度
        # 基座線加速度：向上加速等效於重力 (14.2.5 節)
        a = np.array([0.0, self.g if gravity else 0.0])
        F, N, ac = [], [], []

        # ----- 步驟 2：前向遞迴 -----
        th = 0.0                          # 累積絕對角度
        for i in range(n):
            th_prev = th
            th += q[i]
            # 由前一連桿座標系轉到本連桿座標系 (旋轉 q[i])
            c, s = np.cos(q[i]), np.sin(q[i])
            R = np.array([[c, s], [-s, c]])  # ^i R_{i-1}
            a = R @ a                        # 先把上游加速度轉換過來
            w = w + qd[i]                    # 角速度累加 (14.2.2 節)
            wd = wd + qdd[i]                # 角加速度累加
            # 質心加速度：切向 + 向心 + 平移 (14.2.4 節)
            r = np.array([self.lc[i], 0.0])
            a_c = np.array([-w*w*r[0], wd*r[0]]) + a
```

```

ac.append(a_c)
F.append(self.m[i] * a_c)          # 牛頓方程
N.append(self.I[i] * wd)           # 尤拉方程 (平面・陀螺項為 0)
# 準備傳給下一連桿：連桿末端 (長度 L) 的加速度
rL = np.array([self.L[i], 0.0])
a = np.array([-w*w*rL[0], wd*rL[0]]) + a

# ----- 步驟 3-4：後向遞迴 -----
f = np.zeros(2)                    # 末端無外力
nz = 0.0
tau = np.zeros(n)
for i in range(n-1, -1, -1):
    if i < n-1:                    # 把下游的力轉回本座標系
        c, s = np.cos(q[i+1]), np.sin(q[i+1])
        R_next = np.array([[c, -s], [s, c]]) # ^i R_{i+1}
        f = R_next @ f
        r = np.array([self.lc[i], 0.0])
        rL = np.array([self.L[i], 0.0])
        # 力矩 = 自身淨力矩 + 下游力矩 + 兩個力臂項 (14.3.2 節)
        nz = N[i] + nz + np.cross(r, F[i]) + np.cross(rL, f)
        f = f + F[i]
        tau[i] = nz                # 步驟 5：取 Z 軸投影
return tau

# ----- 用遞迴法萃取各項 (14.4.3 節的技巧) -----
def gravity_vector(self, q):
    z = np.zeros(self.n)
    return self.inverse_dynamics(q, z, z, gravity=True)

def coriolis_vector(self, q, qd):
    return self.inverse_dynamics(q, qd, np.zeros(self.n), gravity=False)

def mass_matrix(self, q):
    """單位向量法：逐行萃取慣性矩陣"""
    M = np.zeros((self.n, self.n))
    z = np.zeros(self.n)
    for i in range(self.n):
        e = np.zeros(self.n); e[i] = 1.0
        M[:, i] = self.inverse_dynamics(q, z, e, gravity=False)
    return M

def forward_dynamics(self, q, qd, tau):
    b = self.inverse_dynamics(q, qd, np.zeros(self.n), gravity=True)
    return np.linalg.solve(self.mass_matrix(q), tau - b)

def energy(self, q, qd):
    T = 0.5 * qd @ self.mass_matrix(q) @ qd
    U, th, x, y = 0.0, 0.0, 0.0, 0.0
    for i in range(self.n):
        th += q[i]
        U += self.m[i]*self.g*(y + self.lc[i]*np.sin(th))
        x += self.L[i]*np.cos(th); y += self.L[i]*np.sin(th)
    return T, U

```

```

d = np.deg2rad

# ----- 驗證一：例題 14-1 單連桿・與拉格朗日比對 -----
print("=== 例題 14-1：單連桿交叉驗證 ===")
arm1 = PlanarNE([0.5], [2.0])
q, qd, qdd = np.array([d(60)]), np.array([2.0]), np.array([3.0])
tau_ne = arm1.inverse_dynamics(q, qd, qdd)
# 拉格朗日解析式 (第 13.4.2 節) :  $\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta$ 
m, L = 2.0, 0.5
tau_lag = (1/3)*m*L**2*qdd[0] + m*9.8*(L/2)*np.sin(q[0])
print(f"Newton-Euler : {tau_ne[0]:.4f} N·m")
print(f"拉格朗日解析 : {tau_lag:.4f} N·m")
print(f"兩者一致 : {np.isclose(tau_ne[0], tau_lag)}")

# ----- 驗證二：平面 2R・與第 13 章解析式比對 -----
print("\n=== 平面 2R：與第 13 章解析式交叉驗證 ===")
L1, L2, m1, m2 = 0.4, 0.3, 3.0, 2.0
arm2 = PlanarNE([L1, L2], [m1, m2])
l1, l2 = L1/2, L2/2
I1, I2 = m1*L1**2/12, m2*L2**2/12
alpha = m1*l1**2 + I1 + m2*(L1**2 + l2**2) + I2
beta = m2*L1*l2
delta = m2*l2**2 + I2

q = np.array([d(30), d(60)])
qd = np.array([1.0, -0.5])
qdd = np.array([2.0, 1.0])

# 解析式 (第 13.5.5 節)
c2, s2 = np.cos(q[1]), np.sin(q[1])
M_lag = np.array([[alpha + 2*beta*c2, delta + beta*c2],
                  [delta + beta*c2, delta]])
C_lag = np.array([-beta*s2*qd[1]**2 - 2*beta*s2*qd[0]*qd[1],
                  beta*s2*qd[0]**2])
c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
G_lag = np.array([(m1*l1 + m2*L1)*9.8*c1 + m2*l2*9.8*c12,
                  m2*l2*9.8*c12])
tau_lag2 = M_lag @ qdd + C_lag + G_lag

print(f"慣性矩陣 NE:\n{np.round(arm2.mass_matrix(q), 5)}")
print(f"慣性矩陣 拉格朗日:\n{np.round(M_lag, 5)}")
print(f"一致: {np.allclose(arm2.mass_matrix(q), M_lag)}")
print(f"重力項 NE: {np.round(arm2.gravity_vector(q), 5)}")
print(f"重力項 拉格朗日: {np.round(G_lag, 5)} 一致: "
      f"{np.allclose(arm2.gravity_vector(q), G_lag)}")
print(f"總力矩 NE: {np.round(arm2.inverse_dynamics(q, qd, qdd), 5)}")
print(f"總力矩 拉格朗日: {np.round(tau_lag2, 5)}")
print(f"一致: {np.allclose(arm2.inverse_dynamics(q, qd, qdd), tau_lag2)}")

```

```
# ----- 驗證三：例題 14-3 積分器能量行為 -----
print("\n=== 例題 14-3：三種積分器的能量行為 ===")
def simulate(method, steps=3000, dt=0.005):
    q, qd = np.array([d(90)]), np.array([0.0])
    T0, U0 = arm1.energy(q, qd); E0 = T0 + U0
    errs = []
    for k in range(steps):
        if method == "rk4":
            def deriv(s):
                qq, vv = s[:1], s[1:]
                return np.concatenate([vv, arm1.forward_dynamics(qq, vv, np.zeros(1))])
            s = np.concatenate([q, qd])
            k1 = deriv(s); k2 = deriv(s + dt/2*k1)
            k3 = deriv(s + dt/2*k2); k4 = deriv(s + dt*k3)
            s = s + dt/6*(k1 + 2*k2 + 2*k3 + k4)
            q, qd = s[:1], s[1:]
        else:
            qdd_ = arm1.forward_dynamics(q, qd, np.zeros(1))
            if method == "explicit":
                q_new = q + qd*dt # 用舊速度
                qd = qd + qdd_*dt
                q = q_new
            else: # semi-implicit
                qd = qd + qdd_*dt
                q = q + qd*dt # 用新速度
            if (k+1) % 600 == 0:
                T, U = arm1.energy(q, qd)
                errs.append(abs(T + U - E0))
    return errs

for m_ in ("explicit", "semi-implicit", "rk4"):
    e = simulate(m_)
    print(f" {m_:14s} 能量誤差演變: " +
          " ".join(f"{x:.2e}" for x in e))
print(" → 顯式歐拉單調發散；半隱式有界；RK4 最精確但非辛積分")
```

請試著修改：（一）把積分步長由 0.005 改為 0.02，觀察顯式歐拉多快發散、半隱式是否仍然穩定。（二）把 PlanarNE 擴充為三連桿，驗證遞迴程式是否仍與解析式一致（這是檢驗遞迴邏輯正確性的最佳測試）。（三）用 time 模組測量 mass\_matrix 與 inverse\_dynamics 的執行時間，驗證前者約為後者的  $n$  倍。

## 實作：動力學演算法的驗證與效能測試

本章實作以程式驗證為主，目標是建立一套可信賴的動力學計算工具，並量化其效能。

- 交叉驗證：**以平面 2R 手臂為測試案例，用 Newton-Euler 遞迴與第 13 章的拉格朗日解析式計算同一組輸入，比對  $M$ 、 $C \cdot q$ 、 $G$  與總力矩。在至少 100 組隨機的  $(q, \dot{q}, \ddot{q})$  下驗證兩者誤差小於  $1e-10$ 。

2. **擴充至三連桿**：把遞迴程式擴充到三連桿，並用 SymPy 自動推導對應的拉格朗日方程作為驗證基準。這一步能檢驗遞迴邏輯是否真的通用（很多錯誤只在三連桿以上才會顯現）。
3. **重力項單獨驗證**：設  $\dot{q} = \ddot{q} = 0$ ，執行遞迴，確認得到的力矩恰為重力補償力矩。用不同姿態測試，並與手算的  $\sum m_i g \cdot (\text{力臂})$  比對。
4. **能量守恆測試**：關閉重力以外的所有外力，自由模擬 10 秒，記錄總能量的漂移百分比。分別以三種積分器與三種步長（0.001、0.005、0.02 s）測試，製作 3×3 的比較表。
5. **效能量測**：用 time 模組量測逆動力學、慣性矩陣與正向動力學的執行時間。改變連桿數  $n = 2, 4, 6, 10, 20$ ，繪出「 $n$  vs 執行時間」的對數圖，驗證逆動力學為  $O(n)$ 、慣性矩陣為  $O(n^2)$ 。
6. **與函式庫比對（進階）**：安裝 Pinocchio 或 RBDL，用相同的機構參數建立模型，比對你的實作與函式庫的輸出。若有差異，最可能的原因是慣性張量的座標系定義不同。
7. **即時性評估**：計算你的實作在目標硬體（如 Raspberry Pi 或 STM32）上能支援的最高控制頻率，並判斷是否滿足 1 kHz 的需求。

表 14-7 動力學驗證記錄表（前兩列為範例）

| 測試項        | 測試條件                    | 預期結果         | 實測結果           | 判定 |
|------------|-------------------------|--------------|----------------|----|
| 交叉驗證       | 100 組隨機輸入，2R 手臂         | 誤差 $< 1e-10$ | 最大誤差 $3.2e-14$ | 通過 |
| 能量守恆       | 半隱式歐拉， $dt=0.005$ ，10 秒 | 漂移 $< 0.1\%$ | 0.03%          | 通過 |
| （自行填寫其餘測試） |                         |              |                |    |

## 國中小也看得懂：來回傳兩次的傳話遊戲

你有沒有玩過「傳話遊戲」？第一個人說一句話，一個一個傳下去，傳到最後一個人。機器手臂算力氣的方法，其實就像玩兩次傳話遊戲——一次往前傳，一次往後傳。

**第一次：從肩膀傳到手指（傳「動作」）。**想像你坐在火車上。第一節車廂動了，第二節就會跟著動；第二節再帶動第三節……機器手臂也一樣：肩膀轉動的時候，手肘會跟著被帶動，手腕又被手肘帶動。所以電腦先從最靠近身體的關節開始，一個一個算出「這一節現在動得多快」。

**第二次：從手指傳回肩膀（傳「力氣」）。**這次方向相反。想想看你手上拿著一本很重的

書：手指要出一點力氣抓住書、手腕要出更多力氣（因為要撐住書和手掌）、手肘要出更多、肩膀最辛苦（要撐住書和整條手臂）。越靠近身體的關節，要出的力氣越大——這就是為什麼機器手臂靠近底座的馬達總是最大顆的。

這個「來回傳兩次」的方法有多厲害呢？如果用另一種方法（直接寫出一條大公式），六軸機器人的公式會長到有一萬多項，印出來要好幾十頁！但用傳話的方法，只要二十行程式就寫完了，而且快一萬倍。這就是為什麼聰明的方法這麼重要——答案一樣，但一個做得到、一個做不到。

## 叫獸提醒與常見錯誤

- 誤區一：忘記尤拉方程的陀螺項  $\omega \times (I\omega)$ 。它在平面問題中為零，但在空間運動中不可忽略，遺漏會導致高速旋轉時的誤差。
- 誤區二：座標系轉換的方向弄反。前向遞迴用  ${}^iR_{i-1}$ （把上游量轉到本座標系），後向遞迴用  ${}^iR_{i+1}$ （把下游量轉回本座標系），兩者互為轉置。
- 誤區三：後向遞迴的力臂用錯。自身慣性力的力臂是質心向量  $P_c$ ，下游傳來的力的力臂是連桿全長向量  $P_{i+1}$ ，混用會造成明顯誤差。
- 誤區四：忘記重力技巧的符號。基座加速度應設為  $-g\vec{}$ （即向上為正），設反了會使重力方向顛倒。
- 誤區五：慣性張量的座標系不一致。慣性張量必須表示在該連桿自身的座標系中，若使用 CAD 輸出值，需確認其參考座標系與 DH 座標系是否一致。
- 誤區六：用顯式歐拉法做長時間模擬。它的能量單調增加，必定發散；應改用半隱式歐拉或其他辛積分器。
- 誤區七：模擬步長取得太大。含接觸或高剛度的系統需要極小步長（0.1 ms 以下），否則會穿透或發散。
- 誤區八：以為模擬中的抓取成功就代表實機可行。接觸模型的簡化（摩擦、變形、接觸面積）是 Sim-to-Real Gap 最頑固的來源。
- 誤區九：只驗證單連桿就相信程式正確。許多遞迴邏輯的錯誤只在三連桿以上才顯現，務必以多連桿與能量守恆雙重驗證。
- 誤區十：忽略複雜度分析。 $O(n)$  與  $O(n^3)$  在  $n=6$  時只差 7 倍，在  $n=20$  時卻是「能做」與

「不能做」的差別。

## 課堂討論

1. 開頭故事中小翰推導出一萬四千項的正確公式，叫獸卻說它「沒有價值」。請討論：在什麼情況下，解析式仍然是必要的？（提示：對照第 19 章的穩定性證明與第 13 章的耦合分析）
2. Newton-Euler 法會顯式算出連桿之間的內力，拉格朗日法則自動消去它們。請討論這個「多餘的資訊」在哪些工程場合反而是優點。
3. 重力技巧把重力轉化為基座的加速度，這用到了等效原理。請討論：這個技巧在什麼情況下會失效？（提示：考慮非均勻重力場，或裝在移動平台、太空站上的機器人）
4. 物理引擎採用辛積分器而非高階的 RK4，犧牲了單步精度換取長期穩定。請討論這個取舍在哪些應用中是正確的、哪些應用中反而應該選 RK4。
5. 可微分物理模擬能計算「結果對參數的梯度」，但必須把接觸平滑化。請討論這個近似可能帶來什麼問題，以及它對學習出來的策略在真機上的表現有何影響。

## 章末摘要

- Newton-Euler 法以牛頓方程 ( $F = ma_c$ ) 與尤拉方程 ( $N = I\dot{\omega} + \omega \times I\omega$ ) 為基礎，後者的陀螺項來自在旋轉座標系中微分角動量。
- 逆動力學以兩趟遞迴完成：前向遞迴由基座往末端傳遞  $\omega$ 、 $\dot{\omega}$ 、 $a$ 、 $a_c$ ；後向遞迴由末端往基座累積  $f$ 、 $n$ 。
- 質心加速度含三項：切向 ( $\omega \times P$ )、向心 ( $\omega \times (\omega \times P)$ ) 與平移（上游傳來的  $a$ ）。
- 重力技巧：把基座線加速度設為  $-g$ ，即可讓重力自動融入遞迴，不需在每根連桿另加重力項。
- 後向遞迴的力矩累積含四項：自身淨力矩、下游力矩、自身慣性力的力矩、下游力的力矩。
- 關節驅動力矩為力矩向量在轉軸上的投影；其餘分量是軸承與結構承受的負荷，是機構設計的重要資訊。
- 遞迴法的複雜度為  $O(n)$ ，遠優於拉格朗日展開的  $O(n^3)$ ； $n=6$  時差 7 倍， $n=20$  時則是能否即時的分界。
- 用同一套遞迴程式可萃取各項：設  $\ddot{q}=\dot{q}=0$  得  $G(q)$ ；設  $\ddot{q}=0$  且無重力得  $C\dot{q}$ ；設  $\ddot{q}=e_i$  得  $M$

的第  $i$  行 ( 單位向量法 ) 。

- 正向動力學有三法：組合法  $O(n^3)$ 、關節體演算法 ABA  $O(n)$ 、最大座標法 ( 自然支援閉環與接觸 ) 。
- 數值積分中，顯式歐拉能量單調增加必發散；半隱式歐拉是辛積分器，能量有界振盪，是物理引擎的標準選擇。
- 對長時間模擬而言，「不發散」比「單步精確」更重要——這是辛積分器優於 RK4 的理由。
- 接觸是模擬中最困難的部分：不連續、力大時間短、摩擦非線性且可能無唯一解。
- 接觸模型分懲罰法 ( 簡單、可微、需小步長 ) 與約束法 ( 嚴謹、不穿透、成本高 ) 兩類。
- 模擬抓取常與實機不符，源於摩擦不確定、接觸面積簡化、柔軟性未建模與接觸模型近似；因應之道是領域隨機化。
- 物理引擎含四模組：碰撞偵測、動力學求解、約束求解與數值積分；不同引擎在速度、精確性與可微性上各有取捨。
- 實務分工：用拉格朗日法理解與分析、用 Newton-Euler 法實作與計算，兩者互相驗證。

## 觀念題與計算題

### 觀念題

1. 寫出牛頓方程與尤拉方程，並說明尤拉方程中陀螺項  $\omega \times (I\omega)$  的來源與物理意義。
2. 說明前向遞迴傳遞哪四個量，以及角速度與角加速度的傳遞公式為何不同 ( 後者多出一項外積 ) 。
3. 列出質心加速度的三個組成項，並說明各自的物理意義。
4. 說明重力技巧的原理，以及它為何等效於在每根連桿加入重力。
5. 寫出後向遞迴中力矩累積的四項，並說明兩個力臂項的差別。
6. 為何關節驅動力矩只是力矩向量的一個分量？其餘分量有何工程價值？
7. 分析 Newton-Euler 遞迴的複雜度為何是  $O(n)$ ，並與拉格朗日展開比較。
8. 說明如何用同一套遞迴程式分別萃取  $M(q)$ 、 $C(q, \dot{q})\dot{q}$  與  $G(q)$ 。
9. 比較三種正向動力學演算法的複雜度、座標選擇與閉環支援能力。



10. 為何顯式歐拉法的能量會單調增加？半隱式歐拉只改一處為何就能解決？
11. 說明何謂辛積分器，以及為何物理引擎偏好它而非高階的 RK4。
12. 列舉接觸模擬困難的三個來源，並比較懲罰法與約束法的優缺點。
13. 說明模擬中的抓取為何常與實機不符，並提出因應之道。
14. 列舉物理引擎的四個組成模組，並說明哪一個通常是計算瓶頸。
15. 說明拉格朗日法與 Newton-Euler 法在實務上的分工，並各舉兩個適用場合。

## 計算題

1. 一根均勻連桿長 0.8 m、質量 3.0 kg，繞固定端在垂直平面轉動。目前  $\theta = 45^\circ$ （由垂直向下量起）、 $\dot{\theta} = 1.5 \text{ rad/s}$ 、 $\ddot{\theta} = 4.0 \text{ rad/s}^2$ 。（a）計算繞質心的轉動慣量。（b）以 Newton-Euler 遞迴計算質心加速度的三個分量。（c）求淨力與淨力矩。（d）求關節力矩，並與拉格朗日公式  $\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta$  比對。
2. 某控制器頻率為 2 kHz，處理器每秒可執行  $2 \times 10^8$  次浮點運算。設 Newton-Euler 每自由度需 180 次運算、拉格朗日展開需  $40n^3$  次。（a）計算 7 軸協作機器人兩種方法的運算時間與週期佔比。（b）計算 30 自由度人形機器人的情形。（c）若模擬需以實時的 50 倍速度執行，30 自由度系統用 Newton-Euler 是否可行？
3. 以單擺（ $L = 0.6 \text{ m}$ 、 $m = 2.5 \text{ kg}$ 、 $I = (1/3)mL^2$ ）由  $\theta = 60^\circ$ 、 $\dot{\theta} = 0$  靜止釋放，步長  $\Delta t = 0.02 \text{ s}$ 。（a）計算初始角加速度。（b）分別以顯式歐拉與半隱式歐拉計算第一步後的  $\theta$  與  $\dot{\theta}$ 。（c）計算兩者第一步後的總能量，並與初始能量比較。（d）預測長時間模擬後兩者的行為差異。
4. 某六軸手臂的動力學計算需求如下：逆動力學 900 次運算、慣性矩陣（單位向量法）為逆動力學的  $n$  倍、正向動力學需額外的 Cholesky 分解（約  $n^3/3$  次運算）。處理器每秒  $10^8$  次運算。（a）分別計算三者的運算時間。（b）若模擬迴路需同時執行正向動力學與碰撞偵測（假設後者需  $200 \mu\text{s}$ ），求可達的最高模擬頻率。（c）若要達到 1 kHz 的模擬頻率，碰撞偵測的時間預算最多為多少？

## 動手做與專題延伸

### 本章實作：動力學計算模組升級

延續第 13 章的動力學模組，加入 Newton-Euler 遞迴實作。這將是第 15 ~ 20 章所有控制

與規劃程式的計算核心。

- **階段一**：實作平面  $n$  連桿的 Newton-Euler 遞迴逆動力學，並與第 13 章的解析式交叉驗證（2R 與 3R 皆須通過）。
- **階段二**：實作 14.4.3 節的萃取技巧，用同一套遞迴取得  $M$ 、 $C \cdot \dot{q}$  與  $G$ ，並驗證  $\tau = M\ddot{q} + C\dot{q} + G$  恆成立。
- **階段三**：實作正向動力學與三種數值積分器，完成能量守恆的  $3 \times 3$  比較表（三種積分器  $\times$  三種步長）。
- **階段四**：擴充至空間手臂（含  $3 \times 3$  慣性張量與完整的座標轉換），並以第 9 章的 DH 參數表為輸入。

### 專題延伸

- **複雜度的實驗驗證**：為  $n = 2$  到 30 的平面手臂，實測逆動力學、慣性矩陣與正向動力學的執行時間，繪出對數—對數圖並用線性回歸求出斜率。驗證三者的斜率分別接近 1、2、3（對應  $O(n)$ 、 $O(n^2)$ 、 $O(n^3)$ ）。這個實驗能讓抽象的複雜度分析變成具體的曲線。
- **積分器的相圖比較**：以單擺為對象，用三種積分器各模擬 100 秒，在相平面（ $\theta$  對  $\dot{\theta}$ ）上繪出軌跡。理想的守恆系統應該是封閉曲線；你會看到顯式歐拉的軌跡向外螺旋發散、半隱式歐拉維持接近封閉的曲線、RK4 則緩慢向內收縮。這張圖是理解辛積分器價值的最佳視覺教材。
- **進階：接觸模擬的初探**。實作一個最簡單的接觸情境——單擺末端碰到地面。分別用懲罰法（彈簧阻尼）與速度反轉法（碰撞時速度乘上恢復係數）實作，比較兩者在不同步長下的行為。觀察懲罰法的剛度  $k$  增大時，穩定所需的步長如何急遽下降——這將讓你親身體會為何含接觸的模擬如此昂貴。

### 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 6 章：Newton-Euler 遞迴的完整推導，本章主要參考)。
- Featherstone, R., Rigid Body Dynamics Algorithms, Springer, 2008 (動力學演算法的權威專著，含 ABA 的完整推導)。
- Luh, J. Y. S., Walker, M. W. and Paul, R. P. C., "On-Line Computational Scheme for

Mechanical Manipulators," ASME J. Dynamic Systems, 1980 ( 遞迴 Newton-Euler 演算法的原始論文 ) 。

- Walker, M. W. and Orin, D. E., "Efficient Dynamic Computer Simulation of Robotic Mechanisms," ASME, 1982 ( 複合剛體演算法 CRBA ) 。
- Hairer, E., Lubich, C. and Wanner, G., Geometric Numerical Integration, Springer ( 辛積分器的數學理論 ) 。
- Todorov, E., Erez, T. and Tassa, Y., "MuJoCo: A Physics Engine for Model-Based Control," IROS 2012 ( MuJoCo 的設計理念與軟接觸模型 ) 。
- Stewart, D. E., "Rigid-Body Dynamics with Friction and Impact," SIAM Review, 2000 ( 接觸與摩擦的數學困難 · 含 Painlevé 悖論 ) 。
- Carpentier, J. et al., "The Pinocchio C++ Library," SII 2019 ( 高效動力學函式庫的設計與實作 ) 。
- Freeman, C. D. et al., "Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation," 2021 ( 可微分模擬的代表性工作 ) 。
- 各物理引擎官方文件：MuJoCo、Bullet、ODE、PhysX、DART 的技術手冊與接觸模型說明。

## 第 15 章

# 軌跡生成與運動曲線

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】上半部並列兩組四階曲線圖（位置／速度／加速度／急跳度），左組為梯形速度曲線（加速度呈階梯狀跳變，急跳度出現無限大的紅色脈衝），右組為 S 形七段曲線（加速度連續，急跳度為有限的階梯）。下半部為同一支手臂執行同一段路徑的兩次多重曝光：左側末端在終點處出現抖動殘影並標註「殘餘振動 0.65 秒」，右側平順停止並標註「立即穩定」。標題文字：同樣的路徑，不同的時間安排——慢慢加速，反而更快。

## 叫獸開講 「到達同一個位置，為什麼有些動作像機器，有些動作像人？」

叫獸在白板上畫了兩條一模一樣的路線：由 A 點到 B 點的一段直線。「這兩條路徑完全相同，」他說，「但我要讓機器人用兩種方式走過去。」

第一次執行：手臂突然啟動、以固定速度衝過去、到達 B 點瞬間停止。末端在停止時劇烈抖動，花了將近一秒才穩下來，示教器上跳出振動警告。

第二次執行：手臂緩緩加速、中段維持高速、接近終點時逐漸減速、平順地停在 B 點。全程沒有任何抖動，而且——總時間比第一次還短了 0.3 秒。

「同樣的路徑、同樣的起訖點，」叫獸說，「但一個又抖又慢，一個又穩又快。差別在哪裡？」

小好舉手：「在於**怎麼走**——什麼時候加速、什麼時候減速。」

「正確。這就是**路徑與軌跡**的差別。」叫獸在白板上寫下兩個詞。「路徑只說『經過哪些點』，是純幾何的問題，第二篇處理過了。軌跡則多了一個東西——**時間**。它說的是『在第幾秒鐘應該在哪裡』。」

他接著問了一個更尖銳的問題：「那為什麼第一次會抖？」

學生們沉默了一下。叫獸自己回答：「因為第一次的速度是瞬間跳到最大值的——這代表加速度是**無限大**。而力矩等於慣量乘上加速度，無限大的加速度需要無限大的力矩。馬達當然給不出來，於是它盡力而為，然後手臂就像被一鎚子敲了一下——**你激發了整個機構的振動**。」

他在白板上寫下四個詞，由上往下：**位置、速度、加速度、急跳度**。「第 13 章我們學了力矩正比於加速度。而加速度的變化率，叫做急跳度（jerk）。急跳度太大，就代表力矩瞬間改變——這會激發機構的彈性振動、傷害減速機、也讓乘客暈車。所以電梯、高鐵、機器手臂的運動曲線，都是為了控制急跳度而設計的。」

小翰問：「那為什麼第二次反而更快？」

「因為第一次的加速度超過馬達能力，控制器只好自動降速保護。」叫獸敲了敲白板，「這正是本章的核心：**軌跡規劃不只是幾何問題，它必須知道機器人的動力學極限**。你不能規劃一條馬達跑不動的軌跡。今天我們就來學，怎麼設計一條又快、又穩、又不超過極限的軌跡。」

## 本章為什麼重要

軌跡生成是連接「規劃」與「控制」的橋樑。第二篇告訴我們機器人能到哪裡、第 13 ~ 14 章告訴我們動起來需要多少力矩，而本章要決定的是：**在每一個時刻，機器人應該在哪裡、以多快的速度、多大的加速度運動。**

這個決定的品質，直接影響四件事。**節拍時間**：軌跡設計得好，同樣的路徑可以縮短數十個百分點的時間——這對每天執行數萬次的產線意義重大。**精度**：平滑的軌跡讓控制器容易追隨，粗糙的軌跡則會產生追隨誤差。**機構壽命**：頻繁的力矩突變會加速減速機與軸承的磨損。**振動**：如開頭故事所示，不當的軌跡會激發機構共振，使末端在停止後仍抖動不止。

本章也是第 13、14 章的第一個實際應用：規劃軌跡時，我們會用逆動力學檢驗「這條軌跡所需的力矩是否超過馬達能力」，並據此調整時間分配。這正是「模型」開始為「決策」服務的起點。

## 學習目標

- 能區分路徑與軌跡，並說明時間參數化的意義。
- 能區分關節空間與任務空間的軌跡規劃，並說明各自的優缺點。
- 能推導三次與五次多項式軌跡的係數，並說明邊界條件的作用。
- 能設計梯形速度曲線，並計算其時間分配與可行性條件。
- 能說明 S 形曲線（含急跳度限制）的優勢與代價。
- 能說明急跳度對振動、機構壽命與舒適度的影響。
- 能規劃通過多個路徑點的連續軌跡，並處理速度連續性。
- 能規劃笛卡兒空間的直線與圓弧軌跡。
- 能運用四元數 SLERP 規劃姿態軌跡。
- 能運用逆動力學檢驗軌跡的動力學可行性並進行時間縮放。
- 能以 Python 實作各類軌跡並比較其速度、加速度與急跳度曲線。

## 先備知識

第 9 ~ 10 章的正逆運動學（任務空間軌跡需要逐點求逆解）、第 11 章的雅可比（速度的轉

換與極限分析)、第 13~14 章的動力學(力矩可行性檢驗)、第 8 章的四元數(姿態內插)。

數學方面需要多項式、微分與積分的基本運算。本章的推導不難，但需要細心處理邊界條件——大部分的錯誤都來自於此。

## 核心名詞中英對照

表 15-1 本章核心名詞

| 中文      | 英文                           | 說明              |
|---------|------------------------------|-----------------|
| 路徑      | Path                         | 空間中的幾何曲線，不含時間資訊 |
| 軌跡      | Trajectory                   | 含時間參數化的路徑       |
| 時間參數化   | Time Parameterization        | 為路徑指定各點的通過時刻    |
| 路徑點     | Via Point / Waypoint         | 軌跡必須通過或接近的中間點   |
| 急跳度     | Jerk                         | 加速度的時間導數        |
| 梯形速度曲線  | Trapezoidal Velocity Profile | 等加速—等速—等減速的速度曲線 |
| S 形曲線   | S-curve                      | 加速度連續變化的平滑速度曲線  |
| 混成      | Blending                     | 在路徑點附近以曲線平順銜接   |
| 球面線性內插  | SLERP                        | 四元數之間的平滑姿態內插    |
| 時間最佳軌跡  | Time-optimal Trajectory      | 在限制下用時最短的軌跡     |
| 時間縮放    | Time Scaling                 | 調整整體執行時間以滿足限制   |
| 殘餘振動    | Residual Vibration           | 運動停止後仍持續的振動     |
| 輸入整形    | Input Shaping                | 以延遲疊加抵消振動的技術    |
| 路徑點混成半徑 | Blending Radius              | 在路徑點處偏離的容許距離    |
| 笛卡兒空間   | Cartesian Space              | 以末端位姿描述的空間      |

## 15.1 路徑與軌跡

### 15.1.1 多了一個維度：時間

如開頭故事所述，路徑與軌跡的差別在於時間。

路徑 (path) 是空間中的一條幾何曲線，通常以一個從 0 到 1 的參數  $s$  描述： $p(s)$ 。它只回答「經過哪些點、順序為何」，不涉及速度。第 26 章的路徑規劃演算法 ( $A^*$ 、RRT) 產生的就是路徑。

**軌跡 ( trajectory )** 則把時間加進來： $p(t)$ 。它回答「在第  $t$  秒鐘應該在哪裡」。有了時間，速度與加速度就自然定義了：

$$\text{速度 } v(t) = dp/dt, \quad \text{加速度 } a(t) = d^2p/dt^2, \quad \text{急跳度 } j(t) = d^3p/dt^3$$

*軌跡的各階導數。控制器需要位置與速度作為追隨目標，動力學則需要加速度來計算力矩。*

由路徑產生軌跡的過程稱為**時間參數化**：給定  $p(s)$ ，選擇一個函數  $s(t)$ ，使得  $p(t) = p(s(t))$ 。這個選擇有無限多種可能，而本章的核心正是「如何選得好」。

### 15.1.2 好軌跡的五個標準

什麼叫「好的軌跡」？工程上有五個標準，它們常常互相衝突。

- **平滑**：位置、速度、加速度應連續。加速度不連續代表力矩突變，會激發振動。
- **快速**：在滿足限制的前提下，總時間越短越好——這直接決定產線的節拍。
- **可行**：不超過關節的位置、速度、加速度與力矩極限。這是硬性條件。
- **精確**：末端應準確通過指定的路徑點，或在容許誤差內。
- **可預測**：軌跡應能事先完整計算，以便驗證與安全檢查。

「快速」與「平滑」的衝突最為明顯：要縮短時間就得加大加速度，而加大加速度就會增加力矩突變。整章的技術，本質上都是在這個張力之間尋找最佳點。

### 15.1.3 三個層次的規劃

完整的運動規劃通常分三層執行：

**第一層：路徑規劃**（第 26 章）。決定要經過哪些點，避開障礙物。輸出是一串路徑點或一條幾何曲線。

**第二層：軌跡生成**（本章）。為路徑加上時間，產生連續的位置、速度與加速度指令。

**第三層：軌跡追隨**（第 16~17 章）。控制器實際驅動馬達，使機器人盡可能貼合規劃的軌跡。

三層各司其職但互相影響：路徑若有尖角，軌跡就必須在該處減速；軌跡若超出動力學極限，控制器就無法追隨。本章的內容處於中間層，必須同時考慮上下兩層的需求。



## 15.2 關節空間與任務空間

### 15.2.1 兩種規劃方式

軌跡可以在兩個不同的空間中規劃，這是一個必須及早決定的關鍵選擇。

**關節空間規劃：**直接為每個關節角度設計時間函數  $q_i(t)$ 。作法是先用逆運動學求出起點與終點的關節角度，再在關節空間中內插。

**任務空間規劃：**先為末端位姿設計軌跡  $x(t)$ ，再逐點用逆運動學求出對應的關節角度。

### 15.2.2 各自的優缺點

**關節空間規劃的優點：**計算量小（不需逐點求逆解）、必定平滑（多項式在關節空間中直接定義）、不會遇到奇異點問題（因為完全不做逆運動學）、且不可能超出關節極限（可直接在關節空間設限）。

**關節空間規劃的缺點：**末端路徑不可預測。即使起訖點都正確，中間過程末端會走出一條奇怪的曲線——可能撞到障礙物、可能離開工作區域。這是它最大的問題。

**任務空間規劃的優點：**末端路徑完全可控（可以精確畫直線、圓弧），這對焊接、塗膠、切割等連續路徑作業是必要條件。

**任務空間規劃的缺點：**需逐點求逆運動學（計算量大）、可能在中途遇到奇異點（第 12 章）、可能因路徑超出工作空間而無解、且關節速度可能在某些姿態下暴增。

表 15-2 關節空間與任務空間規劃的比較

| 面向   | 關節空間             | 任務空間                   |
|------|------------------|------------------------|
| 規劃對象 | 各關節角度 $q(t)$     | 末端位姿 $x(t)$            |
| 計算量  | 小（直接內插）          | 大（逐點逆運動學）              |
| 末端路徑 | 不可預測（曲線）         | 完全可控（直線、圓弧）            |
| 奇異點  | 不受影響             | 可能遇到，需處理               |
| 關節極限 | 容易保證             | 需逐點檢查                  |
| 典型應用 | 點對點取放、避開障礙的大範圍移動 | 焊接、塗膠、切割、裝配            |
| 工業指令 | MOVJ（關節運動）       | MOVL / MOVC（直線 / 圓弧運動） |

### 15.2.3 實務上的選擇

工業機器人的程式中，兩種指令並存且各有明確用途：**MOVJ (關節運動)** 用於「從 A 大範圍移動到 B，中間怎麼走無所謂」的場合，因為它最快也最安全；**MOVL (直線運動)** 用於「必須沿直線」的場合，如焊縫、膠道。

一個典型的取放循環會混用兩者：由待機點 **MOVJ** 到取料點上方（快速移動）、**MOVL** 垂直下降取料（精確直線）、**MOVJ** 移動到放料點上方（快速移動）、**MOVL** 垂直下降放料。這樣既保證了必要的精度，又不浪費時間在不必要的直線運動上。

#### ◎ 關節空間規劃的意外路徑

一個常見的教學現場事故：學生用關節空間指令讓手臂從桌面左側移動到右側，起訖點都在桌面上方安全高度。執行時手臂卻**穿過桌面**——因為中間過程末端沿著一條下凹的曲線移動。這說明了關節空間規劃的風險：起訖點安全不代表過程安全。因應之道是加入中間路徑點，或改用任務空間規劃並做碰撞檢查。

## 15.3 多項式軌跡

### 15.3.1 三次多項式

最基本的軌跡形式是多項式。考慮單一關節由  $q_0$  移動到  $q_f$ ，總時間為  $T$ 。若要求起訖點的位置與速度都指定，共有四個邊界條件，因此需要四個係數——即三次多項式：

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

三次多項式軌跡。

邊界條件為： $q(0) = q_0$ 、 $\dot{q}(0) = 0$ 、 $q(T) = q_f$ 、 $\dot{q}(T) = 0$ （假設起訖皆靜止）。代入求解可得：

$$a_0 = q_0, \quad a_1 = 0, \quad a_2 = 3(q_f - q_0)/T^2, \quad a_3 = -2(q_f - q_0)/T^3$$

三次多項式的係數。

其速度與加速度為：

$$\dot{q}(t) = 6(q_f - q_0)(t/T^2)(1 - t/T) \quad \ddot{q}(t) = 6(q_f - q_0)(1 - 2t/T)/T^2$$

速度呈拋物線（中點最大）、加速度呈直線（由正變負）。

**關鍵觀察：**最大速度發生在中點，其值為：

$$v_{\max} = 1.5 \times (q_f - q_0)/T$$

最大速度是平均速度的 1.5 倍。這個係數在檢驗速度極限時很有用。

最大加速度發生在起點與終點，其值為  $6(q_f - q_0)/T^2$ 。注意加速度在  $t=0$  時由零瞬間跳到最大值——這代表急跳度無限大，會激發振動。這是三次多項式的主要缺點。

### 15.3.2 五次多項式

要解決加速度不連續的問題，可以再增加兩個邊界條件： $\ddot{q}(0) = 0$  與  $\ddot{q}(T) = 0$ 。共六個條件，需要六個係數，即五次多項式：

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

五次多項式軌跡。

在起訖皆靜止且加速度為零的條件下，係數為：

$$a_0 = q_0, a_1 = a_2 = 0, a_3 = 10\Delta q/T^3, a_4 = -15\Delta q/T^4, a_5 = 6\Delta q/T^5 \quad (\Delta q = q_f - q_0)$$

五次多項式的係數。

其最大速度與最大加速度為：

$$v_{\max} = 1.875 \Delta q/T \quad (\text{在中點}) \quad a_{\max} = 5.774 \Delta q/T^2$$

五次多項式的極值。相較三次多項式，速度峰值高了 25%、加速度峰值也較高。

**取捨：**五次多項式的加速度連續（急跳度有限），因此運動更平順、振動更小；代價是同樣的時間內需要更高的峰值速度與加速度。若馬達能力足夠，五次多項式是較好的選擇；若已接近極限，則需延長時間或改用其他曲線。

圖 15-1 三次與五次多項式軌跡的比較

建議配圖：四列子圖並排比較兩種多項式。第一列為位置（兩者皆為 S 形，五次略陡）；第二列為速度（三次為拋物線、五次為鐘形，峰值較高）；第三列為加速度（三次為直線且在  $t=0$  與  $t=T$  處有跳變，五次為連續的正弦狀曲線，兩端為零）；第四列為急跳度（三次為常數但在端點有無限大脈衝，五次為連續曲線）。在三次多項式的加速度跳變處以紅色圈標註「振動來源」。

## 15.4 梯形速度曲線

### 15.4.1 為何工業界偏好梯形

多項式軌跡在整段時間中速度都在變化，只有一瞬間達到最大速度。這在時間效率上並不理想——馬達大部分時間都沒有全力運轉。

**梯形速度曲線**則不然：它以固定加速度加速到最大速度、維持一段等速、再以固定減速度停止。速度曲線呈梯形，因而得名。它的優勢是在滿足相同的速度與加速度極限下，**用時最短**——這是時間最佳控制的直接結果（第 18 章會說明其理論根據）。

#### 15.4.2 時間分配計算

設總位移為  $\Delta q$ 、最大速度為  $v_{\max}$ 、最大加速度為  $a_{\max}$ 。先計算加速段所需的時間與位移：

$$t_a = v_{\max} / a_{\max}, \quad \Delta q_a = \frac{1}{2} a_{\max} t_a^2 = v_{\max}^2 / (2 a_{\max})$$

*加速段的時間與位移。減速段相同。*

若總位移足夠長 ( $\Delta q \geq 2\Delta q_a$ )，則存在等速段：

$$t_c = (\Delta q - 2\Delta q_a) / v_{\max} \quad \text{總時間 } T = 2t_a + t_c = \Delta q / v_{\max} + v_{\max} / a_{\max}$$

*有等速段的情形。總時間的公式很優雅：位移除以最大速度，再加上一個固定的「加減速代價」。*

#### 15.4.3 三角形退化情形

若位移太短 ( $\Delta q < 2\Delta q_a$ )，機器人還沒加速到  $v_{\max}$  就必須開始減速。此時等速段消失，速度曲線退化為三角形：

$$v_{\text{peak}} = \sqrt{(a_{\max} \cdot \Delta q)} \quad T = 2\sqrt{(\Delta q / a_{\max})}$$

*三角形情形。實際達到的峰值速度低於  $v_{\max}$ 。*

判斷條件為：

$$\text{若 } \Delta q \geq v_{\max}^2 / a_{\max} \rightarrow \text{梯形} ; \text{否則} \rightarrow \text{三角形}$$

*判斷準則。程式中必須先做此判斷，否則會算出負的等速時間。*

**實務意義：**短距離移動時，機器人根本達不到額定的最高速度。這解釋了一個常見現象——把機器人的速度參數從 80% 調到 100%，短距離的取放動作卻幾乎沒有變快。因為瓶頸在加速度而非速度，此時該調的是加速度參數。

#### 15.4.4 梯形曲線的缺點

梯形曲線的加速度是階梯狀的：在  $t=0$  瞬間由 0 跳到  $a_{\max}$ 、在  $t_a$  瞬間由  $a_{\max}$  跳到 0、在減速起點再跳到  $-a_{\max}$ 。每一次跳變都是無限大的急跳度。

這正是開頭故事中第一次執行會抖動的原因（該案例更極端，連加速段都沒有）。因此高性能的控制器多半採用下一節的 S 形曲線，在梯形的基礎上把每個轉角「磨圓」。

## 15.5 S 形曲線與急跳度限制

### 15.5.1 把梯形的角磨圓

S 形曲線（S-curve）的作法是在梯形速度曲線的每個轉折處，加入一段加速度線性變化的過渡。這使加速度成為連續函數，急跳度成為有限的階梯狀函數。

完整的 S 形曲線有七個階段：加加速（jerk 為正）、等加速、減加速（jerk 為負）、等速、加減速、等減速、減減速。因此又稱為**七段式速度曲線**。

急跳度： $j(t) \in \{+j_{\max}, 0, -j_{\max}\}$

加速度： $a(t)$  為連續的梯形

速度： $v(t)$  為連續且平滑的 S 形

三階曲線的層級關係。每往下積分一次，曲線就平滑一階。

### 15.5.2 時間分配

S 形曲線的時間計算比梯形複雜，因為要處理更多的退化情形（可能沒有等速段、可能沒有等加速段、甚至兩者都沒有）。基本的三個時間參數為：

$t_j = a_{\max} / j_{\max}$  （急跳段時間）

$t_a = v_{\max} / a_{\max} + t_j$  （完整加速段時間，含兩段急跳）

$T = \Delta q / v_{\max} + v_{\max} / a_{\max} + a_{\max} / j_{\max}$  （總時間，完整七段時）

S 形曲線的時間分配。注意總時間比梯形多出  $a_{\max}/j_{\max}$  這一項——這就是平滑的代價。

代價的量化：若  $a_{\max} = 5 \text{ m/s}^2$ 、 $j_{\max} = 50 \text{ m/s}^3$ ，則多出的時間為 0.1 秒。對一個原本 1 秒的動作而言，這是 10% 的節拍損失。但換來的是振動大幅降低——實務上往往因為不必等待振動衰減，**總體反而更快**（如開頭故事所示）。

### 15.5.3 急跳度為何重要

急跳度雖然是三階導數，聽起來很抽象，但它有四個非常具體的工程意義。

**意義一：力矩的變化率。**由  $\tau = M \ddot{q}$ ，力矩正比於加速度；因此急跳度正比於**力矩的變化率**。力矩瞬間改變會對減速機的齒面、軸承與聯軸器產生衝擊負荷，加速磨損。

**意義二：激發結構振動。**任何機構都有其自然頻率（第 13 章例題 13-1 算過單擺的自然頻率）。加速度的階梯狀跳變在頻域上含有寬廣的頻率成分，必定包含機構的共振頻率，因而激發振動。這正是開頭故事中「像被鎚子敲了一下」的物理解釋。

**意義三：乘坐舒適度。**人體對加速度的變化特別敏感。電梯與高鐵的加減速曲線都經過精心設計以限制急跳度——國際標準建議電梯的急跳度不超過約  $1.5 \text{ m/s}^3$ 。

**意義四：追隨精度。**控制器需要時間反應。急跳度大代表指令變化快，控制器來不及跟上，追隨誤差因而增加。

### 15.5.4 殘餘振動與輸入整形

即使限制了急跳度，機構的振動仍無法完全消除。有一個巧妙的技術可以進一步抑制：**輸入整形**（input shaping）。

它的原理是：既然一個脈衝會激發振動，那麼再送一個**時間錯開半個振動週期**的脈衝，兩者激發的振動恰好反相而互相抵消。具體作法是把原始的軌跡指令與一組脈衝序列做卷積：

**最簡單的 ZV 整形器：在  $t=0$  送出  $A_1$ 、在  $t = T_d/2$  送出  $A_2$  （ $T_d$  為阻尼振動週期）**

*ZV (Zero Vibration) 整形器。代價是總時間增加半個振動週期。*

輸入整形在半導體設備、晶圓搬運與大型龍門機構中應用廣泛，因為這些系統的結構柔性大、振動問題嚴重。它的限制是需要準確知道系統的自然頻率與阻尼比——頻率估計誤差超過約 10% 時效果就會明顯下降。

**表 15-3 四種軌跡形式的比較**

| 形式      | 加速度     | 急跳度    | 時間效率  | 振動 | 適用場合         |
|---------|---------|--------|-------|----|--------------|
| 三次多項式   | 線性、端點跳變 | 端點無限大  | 中     | 明顯 | 教學、低速動作      |
| 五次多項式   | 連續、端點為零 | 有限但不受控 | 較差    | 小  | 需平滑起停的動作     |
| 梯形速度    | 階梯狀跳變   | 跳變處無限大 | 最佳    | 明顯 | 一般工業運動、對時間敏感 |
| S 形（七段） | 連續梯形    | 有限且可設限 | 略差於梯形 | 最小 | 高速高精度、柔性機構   |

## 15.6 多路徑點與混成

### 15.6.1 通過多個點的問題

實際的作業往往需要通過一連串路徑點。最單純的作法是把每一段都當作獨立的點對點運動，在每個路徑點停止再啟動。這樣做正確但極沒效率——每次停止再加速都浪費時間，而且會產生大量的加減速衝擊。

因此需要**連續通過**：機器人在路徑點處不停止，而是以一定速度直接通過。這帶來一個新問題：在路徑點處的速度該設多少？

### 15.6.2 路徑點速度的決定

三種常見的策略：

**策略一：使用者指定**。由程式明確指定每個路徑點的速度。最靈活但最麻煩。

**策略二：啟發式規則**。若路徑點前後的運動方向相同，取兩段平均速度；若方向相反（轉折），則設為零。這是工業控制器常見的預設行為。

若  $\text{sign}(\Delta q_{\text{前}}) = \text{sign}(\Delta q_{\text{後}})$  :  $v_k = (v_{\text{前}} + v_{\text{後}})/2$  ; 否則  $v_k = 0$

*常見的啟發式規則。避免在轉折處產生不合理的速度。*

**策略三：整體最佳化**。把所有路徑點的速度視為未知數，以「總時間最短」或「加速度平方積分最小」為目標求解。這能得到最佳結果，但計算量大，通常用於離線編程。

### 15.6.3 混成：不精確通過反而更好

有一個重要的實務技巧：**允許機器人不精確通過路徑點**。

在轉折處，若要求精確通過，機器人必須減速到零再轉向（因為速度方向要瞬間改變，需要無限大的加速度）。但若允許它在路徑點附近「切彎」——沿一段圓弧或多項式曲線平順轉向——就能維持較高的速度通過。

這個容許的偏離距離稱為**混成半徑**（blending radius），在工業機器人程式中通常以參數指定（如 ABB 的 zone data、FANUC 的 CNT 值）。

混成半徑越大 → 速度越快、路徑偏離越多

混成半徑為零 → 精確通過但必須停止

*混成半徑是速度與精度的直接取舍參數。*

**實務原則**：在自由空間的移動段（避開障礙即可），使用大的混成半徑以提高速度；在作業段（如焊縫、裝配路徑），使用零混成以保證精度。一個典型的取放程式，可能在移動段用 100



mm 的混成、在取放點用精確定位。

圖 15-2 路徑點混成的效果

建議配圖：俯視圖，三個路徑點  $P1$ 、 $P2$ 、 $P3$  呈直角轉折。左圖為零混成：路徑精確通過  $P2$ ，但速度曲線顯示在  $P2$  處降為零再重新加速，總時間較長。右圖為有混成半徑  $r$ ：路徑在  $P2$  附近以圓弧切過（以虛線圖標示混成區域），末端與  $P2$  的最大偏離為  $r$ ，但速度曲線維持在較高水準，總時間顯著縮短。下方以數字標註兩者的時間差異。

## 15.7 笛卡兒空間軌跡

### 15.7.1 直線運動

任務空間中最基本的軌跡是直線。給定起訖位置  $p_0$  與  $p_f$ ，路徑為：

$$p(s) = p_0 + s(p_f - p_0), \quad s \in [0, 1]$$

直線路徑的參數化。

接著用前面任一種時間曲線為  $s$  賦予時間： $s(t)$  可以是三次多項式、五次多項式、梯形或 S 形。這種「幾何路徑 + 時間曲線」的分離設計極為實用——換一種時間曲線不需重新設計路徑。

直線運動的執行需要逐點逆運動學。實務上通常以固定的插補週期（如每  $1 \sim 4$  ms）計算一個點，求逆解後送給關節控制器。這意味著一秒鐘要執行數百次逆運動學——這正是第 10 章強調封閉解重要性的原因。

### 15.7.2 圓弧運動

圓弧運動需要三個點來定義：起點、中間點與終點（三點決定一個圓）。控制器由這三點計算出圓心與半徑，再沿圓弧內插。

圓弧運動的一個重要特性是向心加速度：

$$a_c = v^2 / R$$

沿圓弧運動時，即使速度大小不變，仍有指向圓心的加速度。半徑越小、速度越快，向心加速度越大。

這帶來一個實務限制：小半徑的圓弧必須減速。若不減速，向心加速度會超過機構或馬達的能力。多數控制器會自動依此公式限制圓弧段的速度——這也解釋了為何機器人在走小圓弧時明顯變慢。

### 15.7.3 姿態軌跡與 SLERP



末端不只有位置，還有姿態。姿態的內插不能用線性方式——如第 8 章所述，對旋轉矩陣的元素做線性內插，結果不再是旋轉矩陣；對歐拉角做線性內插，路徑會扭曲且可能經過萬向節鎖。

正確的作法是四元數球面線性內插 (SLERP)：

$$\mathbf{q}(s) = [\sin((1-s)\Omega) \cdot \mathbf{q}_0 + \sin(s\Omega) \cdot \mathbf{q}_f] / \sin \Omega, \quad \text{其中 } \cos \Omega = \mathbf{q}_0 \cdot \mathbf{q}_f$$

SLERP 公式。它產生的是繞固定軸的等速旋轉——兩個姿態之間最自然的路徑。

實作上有兩個要點。**要點一：處理雙重覆蓋**。由於  $\mathbf{q}$  與  $-\mathbf{q}$  代表同一姿態（第 8.5.1 節），若  $\mathbf{q}_0 \cdot \mathbf{q}_f < 0$ ，應先將  $\mathbf{q}_f$  取負號，否則會走「遠路」（繞超過 180 度）。**要點二：小角度時改用線性內插**。當  $\Omega$  很小時  $\sin \Omega \rightarrow 0$ ，公式會數值不穩定，此時用線性內插再正規化即可。

#### 15.7.4 位置與姿態的時間協調

一個容易被忽略的細節：位置與姿態應使用**同一個時間參數**  $s(t)$ ，才能保證兩者同步完成。若各自使用不同的時間曲線，可能出現「位置已到達但姿態還在轉」的情形，導致末端在終點附近做出奇怪的動作。

但有時這是刻意的設計。例如在噴塗作業中，可能希望姿態提前調整到位、位置再緩慢通過——此時就需要為兩者設計不同的時間曲線。這種情形應在程式中明確標註，避免誤解。

## 15.8 動力學可行性與時間縮放

### 15.8.1 軌跡必須通過動力學檢驗

到目前為止，我們規劃軌跡時只考慮了運動學極限（速度、加速度）。但真正的限制是**力矩**——而力矩與加速度的關係並非簡單的比例，它取決於姿態（第 13 章的  $M(\mathbf{q})$  隨姿態變化）與速度（科氏力與速度平方成正比）。

因此完整的軌跡規劃必須包含一個檢驗步驟：對規劃好的軌跡，逐點用**逆動力學**計算所需力矩，確認不超過馬達能力。

$$\text{對每個時刻 } t: \quad \boldsymbol{\tau}(t) = \mathbf{M}(\mathbf{q}(t))\ddot{\mathbf{q}}(t) + \mathbf{C}(\mathbf{q}(t), \dot{\mathbf{q}}(t))\dot{\mathbf{q}}(t) + \mathbf{G}(\mathbf{q}(t))$$

$$\text{檢驗: } |\tau_i(t)| \leq \tau_{\max,i} \quad \text{對所有 } i \text{ 與 } t \text{ 成立}$$

動力學可行性檢驗。這正是第 14 章的  $O(n)$  遞迴演算法派上用場的地方。

### 15.8.2 時間縮放：優雅的修正方法

若檢驗失敗（某處力矩超限），最直接的修正方法是**時間縮放**：把整條軌跡的執行時間拉長

k 倍。

時間縮放有一個優美的數學性質。設原軌跡為  $q(t)$ ，縮放後為  $\tilde{q}(t) = q(t/k)$ 。則：

$$\dot{\tilde{q}} = \dot{q}/k, \quad \ddot{\tilde{q}} = \ddot{q}/k^2$$

速度按  $1/k$  縮放、加速度按  $1/k^2$  縮放。

因此動力學方程中各項的縮放行為為：慣性項  $M \ddot{q}$  按  $1/k^2$  縮放、科氏項  $C \dot{q}$  也按  $1/k^2$  (因為含速度平方)、但重力項  $G$  完全不變 (它與速度無關)。

$$\tilde{\tau} = (1/k^2)(M \ddot{\tilde{q}} + C \dot{\tilde{q}}) + G$$

時間縮放後的力矩。這是本節最重要的關係式。

這個結果有兩個重要的推論。**推論一**：把時間拉長一倍 ( $k = 2$ )，動態部分的力矩降為四分之一——時間縮放對降低力矩極為有效。**推論二**：無論怎麼放慢，**重力項都無法降低**。若重力所需的力矩就已超過馬達能力，任何時間縮放都救不了，只能減輕負載或更換馬達。

### 15.8.3 時間最佳軌跡

既然時間縮放能保證可行，一個自然的問題是：在滿足所有限制的前提下，最短的時間是多少？

這稱為**時間最佳路徑參數化** (Time-Optimal Path Parameterization, TOPP)。它的核心想法是：沿著給定的路徑，在每一點計算「在此處最大可容許的速度與加速度」(受力矩極限、向心加速度、關節速度等限制)，再用類似「相平面分析」的方法找出最快的通過方式。

時間最佳解有一個典型特徵：在每一個時刻，至少有一個關節處於其極限 (力矩飽和或速度飽和)。這稱為 bang-bang 特性，與第 18 章的最佳控制理論一致。若你的軌跡在整段時間中都沒有任何一個關節達到極限，那它一定不是時間最佳的——還有加速的空間。

#### ◎ 為何實務上很少用時間最佳軌跡

時間最佳軌跡在理論上很吸引人，但實務應用有限，原因有三。一、**不夠平滑**：bang-bang 的力矩切換會產生大的急跳度，激發振動。二、**沒有餘裕**：所有關節都在極限運轉，任何模型誤差或外部干擾都會導致追隨失敗。三、**模型依賴**：需要精確的動力學參數，而現實中它們有 10~20% 的不確定性。因此實務上多採「次佳但穩健」的作法：用 80~90% 的極限值規劃 S 形曲線，保留餘裕給控制器與不確定性。

## 15.9 多軸協調與同步

### 15.9.1 最慢的關節決定一切

多軸運動時有一個基本問題：各關節的移動量不同、能力也不同，若各自以最快速度執行，它們會在不同時刻到達終點——這會使末端走出不可預測的路徑。

正確的作法是**同步**：讓所有關節同時開始、同時結束。作法是先計算每個關節「單獨執行時所需的最短時間」，取其中最大者作為共同的執行時間，再讓其餘關節放慢配合。

$$T = \max_i (T_i), \text{ 其中 } T_i \text{ 為關節 } i \text{ 的最短可行時間}$$

*同步時間。決定總時間的那個關節稱為主導軸 ( dominant axis )。*

**實務意義**：這解釋了為何「調快某一軸的速度參數」往往沒有效果——若它不是主導軸，調快也沒用。要縮短節拍，必須先找出主導軸，再針對它最佳化（減少該軸的移動量、或提高該軸的加減速能力）。

### 15.9.2 三種同步等級

工業控制器通常提供三種同步等級，精細程度不同：

**無同步**：各軸獨立執行，最快但路徑不可預測。已很少使用。

**終點同步**：所有軸同時結束，但中間過程的速度比例不固定。實作簡單，是關節運動 ( MOVJ ) 的標準行為。

**全程同步**：所有軸的位置在整個過程中維持固定比例（即所有軸的正規化時間曲線完全相同）。這使關節空間中的路徑成為一條直線，末端路徑雖仍是曲線但完全可預測。多數現代控制器採此作法。

### 15.9.3 外部軸與多機協同

當系統包含外部軸（如第 4 章提到的第七軸行走軸、旋轉工作台）或多台機器人時，同步問題更為複雜。此時需要一個共同的時間基準，並確保所有設備的插補週期對齊。

這正是第 3 章提到的 EtherCAT 等現場匯流排的價值所在——它們提供微秒級的**分散式時鐘同步**，使多個驅動器能在同一時刻執行指令。若同步誤差達到毫秒級，高速協同作業（如兩台機器人共同搬運一個大工件）就會產生內應力甚至損壞工件。

## 15.10 實務上的軌跡規劃流程

### 15.10.1 完整的七個步驟

整合本章內容，一個完整的軌跡規劃流程如下：

1. **取得路徑**：由使用者示教、離線編程或路徑規劃演算法（第 26 章）取得一串路徑點。
2. **選擇空間**：依任務需求決定各段使用關節空間或任務空間（15.2.3 節）。
3. **設定混成**：為每個路徑點指定混成半徑——作業段設零、移動段設大值（15.6.3 節）。
4. **時間參數化**：選擇時間曲線（梯形或 S 形），依速度與加速度極限計算各段時間。
5. **多軸同步**：計算各軸所需時間，取最大值統一，找出主導軸（15.9.1 節）。
6. **動力學檢驗**：逐點計算逆動力學，確認力矩不超限；若超限則時間縮放並回到步驟 5（15.8 節）。
7. **輸出指令**：以固定的插補週期（1~4 ms）產生位置、速度、加速度序列，送給控制器。

步驟 6 與 5 之間的迭代是關鍵。實務上通常從一個保守的時間估計開始，逐步縮短直到某處觸及極限，這稱為**時間最佳化迴圈**。

### 15.10.2 線上與離線規劃

**離線規劃**：在執行前完整計算整條軌跡。優點是可以做完整的檢驗（碰撞、力矩、奇異點）、可以做全域最佳化。缺點是無法應對環境變化。適用於固定的產線作業。

**線上規劃**：邊執行邊計算，通常只規劃未來數十毫秒。優點是能即時反應（如視覺伺服追蹤移動的物件、或人突然接近時減速）。缺點是無法做全域最佳化，且必須保證計算能在控制週期內完成。

現代的協作機器人多採**混合式**：離線規劃出名目軌跡，執行時再依感測資訊做線上調整（如偵測到人接近就即時做時間縮放降速）。這種「離線骨架 + 線上微調」的架構，兼顧了最佳性與反應性。

## 15.11 由軌跡邁向控制

### 15.11.1 本章的成果

本章完成了「規劃」到「控制」之間的橋樑。你現在能夠：區分路徑與軌跡、在關節空間與

任務空間之間做出正確選擇、設計多項式與梯形 / S 形速度曲線、規劃通過多個路徑點的連續軌跡與笛卡兒直線圓弧、用 SLERP 內插姿態、以逆動力學檢驗可行性並用時間縮放修正、以及處理多軸同步。

更重要的是，你理解了幾個貫穿全章的取舍：**快速與平滑的張力**（急跳度限制的代價）、**精度與速度的取舍**（混成半徑）、以及**最佳與穩健的權衡**（為何實務不用時間最佳軌跡）。這些取舍的判斷力，是工程師與程式產生器的差別所在。

### 15.11.2 接下來：讓機器人真的走上這條軌跡

但軌跡只是**指令**。規劃出完美的軌跡，不代表機器人就會照著走——馬達有摩擦、機構有彈性、負載會變化、外界有干擾。把指令變成實際運動，是**控制**的工作。

**第 16 章（線性回授與 PID）**：從最基本的控制器出發。我們將分析一維點質量的 PD 與 PID 控制，理解比例、微分、積分三項各自的作用，以及為何增益不能無限調高。這一章會解釋第 13 章開頭故事中「調高增益」為何治標不治本。

**第 17 章（非線性控制）**：把第 13、14 章的動力學模型加入控制器。計算力矩控制（computed torque control）用逆動力學做前饋，把非線性的機器人「線性化」，使 PID 得以有效運作。這正是第 13 章叫獸提到的解方。

**第 18~19 章**：用最佳控制與 Lyapunov 理論，設計更好的控制器並嚴格證明其穩定性。

換句話說：第 15 章決定**該走哪條路**，第 16 章之後決定**如何真的走上去**。

## 完整計算例題

### 例題 15-1 多項式軌跡的設計與檢驗

題目：某關節需由  $0^\circ$  移動到  $90^\circ$ ，時間  $T = 2.0$  秒，起訖皆靜止。關節的速度極限為  $60^\circ/\text{s}$ 、加速度極限為  $120^\circ/\text{s}^2$ 。（a）設計三次多項式軌跡並求其最大速度與加速度。（b）改用五次多項式，重新計算。（c）判斷兩者是否滿足極限。（d）若五次多項式不可行，求所需的最短時間。

解（a）： $\Delta q = 90^\circ$ ， $T = 2.0 \text{ s}$ 。由 15.3.1 節：

$$v_{\max} = 1.5 \Delta q / T = 1.5 \times 90 / 2.0 = 67.5^\circ/\text{s}$$

三次多項式的最大速度。

$$a_{\max} = 6 \Delta q/T^2 = 6 \times 90/4.0 = 135 \text{ }^\circ/\text{s}^2$$

三次多項式的最大加速度 (發生在起訖點)。

解 (b) : 由 15.3.2 節 :

$$v_{\max} = 1.875 \Delta q/T = 1.875 \times 90/2.0 = 84.4 \text{ }^\circ/\text{s}$$

五次多項式的最大速度。

$$a_{\max} = 5.774 \Delta q/T^2 = 5.774 \times 90/4.0 = 129.9 \text{ }^\circ/\text{s}^2$$

五次多項式的最大加速度 (發生在約  $t = 0.21T$  與  $0.79T$  處)。

解 (c) : 三次多項式 : 速度  $67.5 > 60$  (超限 12.5%)、加速度  $135 > 120$  (超限 12.5%)。

不可行。

五次多項式 : 速度  $84.4 > 60$  (超限 40.7%)、加速度  $129.9 > 120$  (超限 8.3%)。更不可行。

這個結果值得注意 : 五次多項式雖然更平滑 , 但峰值速度高出 25% , 反而更容易超出速度極限。平滑不是免費的。

解 (d) : 由時間縮放的關係 , 速度按  $1/k$ 、加速度按  $1/k^2$  縮放。分別求兩個限制所需的縮放倍數 :

$$\text{速度限制 : } k_v = 84.4/60 = 1.407$$

$$\text{加速度限制 : } k_a = \sqrt{(129.9/120)} = \sqrt{1.0825} = 1.040$$

兩個限制各自要求的縮放倍數。

取較大者  $k = 1.407$  (速度為主導限制) :

$$T_{\min} = 1.407 \times 2.0 = 2.81 \text{ 秒}$$

五次多項式的最短可行時間。此時速度恰好達到  $60 \text{ }^\circ/\text{s}$  , 加速度為  $129.9/1.407^2 = 65.6 \text{ }^\circ/\text{s}^2$  (remaining 餘裕充足)。

**工程觀察 :** 此案例中速度是瓶頸 , 加速度餘裕很大——這代表若改用梯形速度曲線 (能在較短時間內達到並維持最高速度) , 可以更快完成。下一個例題將驗證這一點。

## 例題 15-2 梯形速度曲線的設計

題目 : 承上題的關節 ( $\Delta q = 90^\circ$ 、 $v_{\max} = 60 \text{ }^\circ/\text{s}$ 、 $a_{\max} = 120 \text{ }^\circ/\text{s}^2$ )。(a) 設計梯形速

度曲線並求總時間。(b) 與例題 15-1 的五次多項式比較。(c) 若移動量改為  $20^\circ$ ，重新計算並說明差異。(d) 改用 S 形曲線 ( $j_{\max} = 600^\circ/\text{s}^3$ )，求總時間。

解 (a)：先檢查是否有等速段 (15.4.3 節)：

$$v_{\max}^2/a_{\max} = 60^2/120 = 3600/120 = 30^\circ < \Delta q = 90^\circ \rightarrow \text{有等速段 (梯形)}$$

判斷條件成立。

$$t_a = v_{\max}/a_{\max} = 60/120 = 0.5 \text{ s} \quad (\text{加速段時間})$$

$$T = \Delta q/v_{\max} + v_{\max}/a_{\max} = 90/60 + 60/120 = 1.5 + 0.5 = 2.0 \text{ 秒}$$

梯形曲線的總時間。

解 (b)：梯形曲線只需 2.0 秒，而五次多項式需 2.81 秒——梯形快了 29%。原因是梯形曲線讓馬達在大部分時間中都以最高速度運轉，而多項式只在瞬間達到峰值速度。

這印證了 15.4.1 節的說法：在相同的速度與加速度限制下，梯形速度曲線是時間最佳的。

解 (c)： $\Delta q = 20^\circ$ ，重新檢查：

$$v_{\max}^2/a_{\max} = 30^\circ > \Delta q = 20^\circ \rightarrow \text{無等速段 (三角形)}$$

退化為三角形曲線。

$$v_{\text{peak}} = \sqrt{a_{\max} \cdot \Delta q} = \sqrt{120 \times 20} = \sqrt{2400} = 49.0^\circ/\text{s}$$

$$T = 2\sqrt{\Delta q/a_{\max}} = 2\sqrt{20/120} = 2 \times 0.408 = 0.816 \text{ 秒}$$

三角形情形的峰值速度與總時間。

**關鍵觀察：**實際達到的峰值速度只有  $49^\circ/\text{s}$ ，遠低於額定的  $60^\circ/\text{s}$ 。因此提高速度參數對這個動作完全沒有效果——瓶頸在加速度。若要縮短時間，必須提高  $a_{\max}$ 。這正是 15.4.3 節提到的實務現象。

解 (d)：S 形曲線的總時間 (15.5.2 節，以  $\Delta q = 90^\circ$  計算)：

$$T = \Delta q/v_{\max} + v_{\max}/a_{\max} + a_{\max}/j_{\max} = 1.5 + 0.5 + 120/600 = 1.5 + 0.5 + 0.2 = 2.2 \text{ 秒}$$

S 形曲線的總時間。

比梯形多了 0.2 秒 (10%)，這就是平滑的代價。但換來的是加速度連續、急跳度有限、振動大幅降低。若原本因振動需要在終點等待 0.3 秒才能進行下一動作，改用 S 形反而總體更快——這正是開頭故事的情境。



**例題 15-3 動力學可行性與時間縮放**

題目：某平面 2R 手臂 ( 參數同第 13 章例題 13-2 :  $L_1=0.4$ 、 $L_2=0.3$  m ,  $m_1=3.0$ 、 $m_2=2.0$  kg ) 執行一段軌跡。在某個時刻，姿態  $\theta = (30^\circ, 60^\circ)$ 、速度  $\dot{\theta} = (2.0, -1.0)$  rad/s、加速度  $\ddot{\theta} = (4.0, 2.0)$  rad/s<sup>2</sup>。已知兩關節的力矩極限分別為  $\tau_{\max} = (15, 3)$  N·m。( a ) 計算此時刻所需的力矩。( b ) 判斷是否可行。( c ) 若不可行，求所需的時間縮放倍數。( d ) 驗證縮放後的力矩。

解 ( a )：沿用第 13 章的常數： $\alpha = 0.540$ 、 $\beta = 0.120$ 、 $\delta = 0.060$  kg·m<sup>2</sup>。 $\theta_2 = 60^\circ$  時  $c_2 = 0.5$ 、 $s_2 = 0.866$ 。

$$M = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix} \quad M \cdot \ddot{q} = \begin{bmatrix} 0.660(4.0) + 0.120(2.0) \\ 0.120(4.0) + 0.060(2.0) \end{bmatrix} = \begin{bmatrix} 2.880 \\ 0.600 \end{bmatrix}$$

慣性項。

$$\begin{aligned} \beta s_2 &= 0.1039 \\ C \text{ 項}_1 &= -0.1039(1.0)^2 - 2(0.1039)(2.0)(-1.0) = -0.104 + 0.416 = 0.312 \\ C \text{ 項}_2 &= 0.1039(2.0)^2 = 0.416 \end{aligned}$$

科氏與離心項。

重力項 ( 由第 13 章例題 13-2，同樣的姿態 )： $G = (11.88, 0)^T$  N·m。

$$\begin{aligned} \tau_1 &= 2.880 + 0.312 + 11.88 = 15.07 \text{ N}\cdot\text{m} \\ \tau_2 &= 0.600 + 0.416 + 0 = 1.02 \text{ N}\cdot\text{m} \end{aligned}$$

所需力矩。

解 ( b )： $\tau_1 = 15.07 > 15$  ( 超限 0.5% )、 $\tau_2 = 1.02 < 3$  ( 可行 )。因此第一關節超限，軌跡不可行。

解 ( c )：由 15.8.2 節，時間縮放後： $\tilde{\tau} = (1/k^2)(M\ddot{q} + Cq) + G$ 。對第一關節：

$$(1/k^2)(2.880 + 0.312) + 11.88 \leq 15 \Rightarrow 3.192/k^2 \leq 3.12 \Rightarrow k^2 \geq 1.023 \Rightarrow k \geq 1.011$$

所需的時間縮放倍數。

取  $k = 1.05$  ( 留一些餘裕 )。整條軌跡的時間拉長 5%。

解 ( d )：驗證縮放後的力矩：

$$\tilde{\tau}_1 = 3.192/1.05^2 + 11.88 = 3.192/1.1025 + 11.88 = 2.895 + 11.88 = 14.78 \text{ N}\cdot\text{m} \checkmark$$

低於 15 N·m 的極限，可行。



**重要觀察：**注意重力項  $11.88 \text{ N}\cdot\text{m}$  佔了極限  $15 \text{ N}\cdot\text{m}$  的 79%，動態部分只剩  $3.12 \text{ N}\cdot\text{m}$  的空間。這說明了兩件事：一、此姿態下馬達的餘裕極小，是整條軌跡的瓶頸；二、**若重力項本身就超過  $15 \text{ N}\cdot\text{m}$ ，任何時間縮放都無濟於事——這正是 15.8.2 節推論二的實例。**此時只能減輕負載、改變姿態，或更換更大的馬達。

## ■ 程式範例：軌跡生成器與可行性檢驗

以下 Python 程式實作四種軌跡形式（三次、五次、梯形、S 形），比較其速度、加速度與急跳度特性，並示範動力學可行性檢驗與時間縮放。

```
import numpy as np

# ===== 各類軌跡生成器 =====
def cubic(q0, qf, T, t):
    """三次多項式 ( 15.3.1 節 ) : 起訖靜止"""
    dq = qf - q0
    s = t / T
    q = q0 + dq*(3*s**2 - 2*s**3)
    qd = dq*(6*s - 6*s**2)/T
    qdd = dq*(6 - 12*s)/T**2
    return q, qd, qdd

def quintic(q0, qf, T, t):
    """五次多項式 ( 15.3.2 節 ) : 起訖靜止且加速度為零"""
    dq = qf - q0
    s = t / T
    q = q0 + dq*(10*s**3 - 15*s**4 + 6*s**5)
    qd = dq*(30*s**2 - 60*s**3 + 30*s**4)/T
    qdd = dq*(60*s - 180*s**2 + 120*s**3)/T**2
    return q, qd, qdd

def trapezoidal(q0, qf, vmax, amax, t):
    """梯形速度曲線 ( 15.4 節 )，自動處理三角形退化"""
    dq = qf - q0
    sgn = np.sign(dq) if dq != 0 else 1.0
    D = abs(dq)
    if D >= vmax**2/amax:
        # 有等速段
        ta = vmax/amax
        T = D/vmax + vmax/amax
        vp = vmax
    else:
        # 三角形退化
        vp = np.sqrt(amax*D)
        ta = vp/amax
        T = 2*ta
    if t <= 0:
        q, qd, qdd = 0.0, 0.0, 0.0
    elif t < ta:
        q, qd, qdd = 0.5*amax*t**2, amax*t, amax
    elif t < T-ta:
        q, qd, qdd = 0.5*vp*ta + vp*(t-ta), vp, 0.0
    elif t <= T:
        td = T - t
        q, qd, qdd = D - 0.5*amax*td**2, amax*td, -amax
```

```

else:
    q, qd, qdd = D, 0.0, 0.0
    return q0 + sgn*q, sgn*qd, sgn*qdd, T

def scurve_time(D, vmax, amax, jmax):
    """S 形曲線的總時間 (15.5.2 節·完整七段情形)"""
    return D/vmax + vmax/amax + amax/jmax

# ===== 驗證：例題 15-1、15-2 =====
print("=== 例題 15-1：多項式軌跡 ===")
dq, T = 90.0, 2.0
print(f"三次多項式: v_max={1.5*dq/T:6.2f} °/s   a_max={6*dq/T**2:7.2f} °/s²")
print(f"五次多項式: v_max={1.875*dq/T:6.2f} °/s   a_max={5.774*dq/T**2:7.2f} °/s²")
print(f"極限:      v_max= 60.00 °/s   a_max= 120.00 °/s²")
kv, ka = 1.875*dq/T/60, np.sqrt(5.774*dq/T**2/120)
print(f"五次多項式所需縮放: k_v={kv:.3f}   k_a={ka:.3f} → 取 k={max(kv,ka):.3f}")
print(f"最短可行時間: {max(kv,ka)*T:.2f} 秒")

print("\n=== 例題 15-2：梯形與 S 形曲線 ===")
for D in (90.0, 20.0):
    _, _, _, Ttrap = trapezoidal(0, D, 60.0, 120.0, 0)
    kind = "梯形" if D >= 60**2/120 else "三角形"
    vp = 60.0 if D >= 60**2/120 else np.sqrt(120*D)
    print(f"位移 {D:5.1f}°: {kind:4s}   峰值速度={vp:5.1f} °/s   總時間={Ttrap:.3f} s")
Ts = scurve_time(90.0, 60.0, 120.0, 600.0)
print(f"S 形曲線 (90°, j_max=600): 總時間={Ts:.3f} s   "
      f" ( 比梯形多 {(Ts-2.0)/2.0*100:.0f}%) ")

# ===== 四種軌跡的特性比較 =====
print("\n=== 四種軌跡的峰值比較 (Δq=90°, T=2.0s) ===")
ts = np.linspace(0, T, 2001)
for name, fn in (("三次多項式", cubic), ("五次多項式", quintic)):
    q, qd, qdd = np.array([fn(0, dq, T, t) for t in ts]).T
    jerk = np.gradient(qdd, ts)
    print(f"{name}: |v|max={np.abs(qd).max():6.2f}   "
          f"|a|max={np.abs(qdd).max():7.2f}   "
          f"|j|max={np.abs(jerk).max():8.1f} °/s³")

# ===== 動力學可行性檢驗 (例題 15-3) =====
print("\n=== 例題 15-3：動力學可行性與時間縮放 ===")
class Arm2R:
    def __init__(s, L1=0.4, L2=0.3, m1=3.0, m2=2.0, g=9.8):
        s.L1, s.L2, s.m1, s.m2, s.g = L1, L2, m1, m2, g
        s.l1, s.l2 = L1/2, L2/2
        s.I1, s.I2 = m1*L1**2/12, m2*L2**2/12
        s.a = m1*s.l1**2 + s.I1 + m2*(L1**2 + s.l2**2) + s.I2
        s.b = m2*L1*s.l2
        s.d = m2*s.l2**2 + s.I2
    def tau(s, q, qd, qdd):
        c2, s2 = np.cos(q[1]), np.sin(q[1])
        M = np.array([[s.a + 2*s.b*c2, s.d + s.b*c2],
                      [s.d + s.b*c2, s.d]])

```

```

C = np.array([-s.b*s2*qd[1]**2 - 2*s.b*s2*qd[0]*qd[1],
              s.b*s2*qd[0]**2])
c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
G = np.array([(s.m1*s.l1 + s.m2*s.l1)*s.g*c1 + s.m2*s.l2*s.g*c12,
              s.m2*s.l2*s.g*c12])
return M @ qdd + C + G, M @ qdd + C, G

arm = Arm2R()
q = np.deg2rad([30, 60])
qd = np.array([2.0, -1.0])
qdd = np.array([4.0, 2.0])
tau_max = np.array([15.0, 3.0])

tau, dyn, grav = arm.tau(q, qd, qdd)
print(f"動態部分 (Mq̈+Cq̇): {np.round(dyn, 3)} N·m")
print(f"重力部分 G: {np.round(grav, 3)} N·m")
print(f"總力矩 τ: {np.round(tau, 3)} N·m")
print(f"力矩極限: {tau_max} N·m")
print(f"是否可行: {np.all(np.abs(tau) <= tau_max)}")

# 時間縮放 (15.8.2 節：動態項按 1/k² 縮放，重力不變)
k_need = 1.0
for i in range(2):
    margin = tau_max[i] - abs(grav[i])
    if margin <= 0:
        print(f"關節{i+1}: 重力已超限，時間縮放無效！")
    elif abs(dyn[i]) > margin:
        k_need = max(k_need, np.sqrt(abs(dyn[i])/margin))
print(f"所需時間縮放倍數: k = {k_need:.4f}")

k = 1.05
tau_scaled = dyn/k**2 + grav
print(f"以 k={k} 縮放後的力矩: {np.round(tau_scaled, 3)} N·m")
print(f"縮放後是否可行: {np.all(np.abs(tau_scaled) <= tau_max)}")
print(f"重力佔極限比例: ")
    f"{abs(grav[0])/tau_max[0]*100:.1f}% ← 動態餘裕僅剩 "
    f"{(1-abs(grav[0])/tau_max[0])*100:.1f}%"

```

請試著修改：（一）把 S 形曲線的  $j_{\max}$  由 600 降到 200，觀察總時間如何增加。（二）在動力學檢驗中把姿態改為  $\theta = (0^\circ, 90^\circ)$ （手臂水平伸展），觀察重力項如何暴增、動態餘裕如何被壓縮。（三）繪出四種軌跡的位置、速度、加速度與急跳度四階曲線，直觀比較它們的平滑程度。

## 實作：軌跡品質的實驗評估

本章實作是在實體手臂上比較不同軌跡形式的實際表現，量化「平滑」的價值。

1. **基準設定**：選定一段點對點運動（建議移動量較大，如某關節轉  $90^\circ$ ），並確認手臂在該路徑上無碰撞風險。
2. **四種軌跡實作**：分別以三次多項式、五次多項式、梯形速度、S 形曲線規劃同一段運動，時間參數設為相同（或各自的最短可行時間）。
3. **振動量測**：在末端加裝加速度計（可用手機的感測器 App，第 6 章的實作已用過），記錄每種軌跡執行時的末端加速度。特別記錄**運動停止後**的殘餘振動——這是最能區分軌跡品質的指標。
4. **振動量化**：計算殘餘振動的峰值振幅與衰減至 5% 所需的時間（穩定時間）。製作四種軌跡的比較表。
5. **總時間比較**：記錄「開始運動」到「振動衰減至可接受範圍」的總時間。你可能會發現：S 形曲線雖然運動時間較長，但總時間反而最短——這是本章最重要的實驗結論。
6. **追隨誤差量測**：若控制器能輸出追隨誤差，記錄每種軌跡的最大誤差。平滑的軌跡應有較小的誤差。
7. **極限探索**：逐步縮短執行時間，找出每種軌跡開始出現異常（超限警告、明顯振動、追隨誤差暴增）的臨界時間。
8. **混成效果測試**：規劃一段含三個路徑點的直角轉折路徑，分別以零混成與不同混成半徑執行，記錄總時間與實際路徑偏離量，繪出「混成半徑 vs 時間」與「混成半徑 vs 偏離量」兩條曲線。

表 15-4 軌跡品質比較表（第一列為範例）

| 軌跡形式       | 運動時間   | 殘餘振動峰值 | 穩定時間   | 總時間    | 最大追隨誤差 |
|------------|--------|--------|--------|--------|--------|
| 梯形速度       | 2.00 s | 0.42 g | 0.65 s | 2.65 s | 1.8 mm |
| （自行填寫其餘三種） |        |        |        |        |        |

## 國中小也看得懂：為什麼捷運不會突然衝出去

坐過捷運嗎？有沒有注意過，車子**啟動的時候不會突然衝出去**，而是慢慢加速；快到站的時候也是慢慢減速，最後才停下來。為什麼不乾脆一下子加速到最快呢？因為那樣的話，站著的乘客會全部跌倒！

機器手臂也一樣。如果讓它「說走就走、說停就停」，會發生兩件壞事：第一，馬達根本做

不到（就像你沒辦法瞬間從站著變成全速跑步）；第二，整支手臂會**晃來晃去**，像用力甩一根竹竿之後，竿子還會抖好久。

所以工程師要幫機器人設計一條「**什麼時候該多快**」的計畫表。這張表叫做「軌跡」。好的軌跡長得像一座梯形的小山：慢慢加速上去、中間維持最快速度、再慢慢減速下來。

這裡有一件很有趣的事：**慢慢加速反而更快！**聽起來很奇怪對不對？但想想看——如果你猛力甩手臂，它會抖個不停，你必須等它不抖了才能做下一件事。如果你溫和地移動，它一停就穩住了，馬上可以做下一件事。加起來，反而是溫和的比較快。

下次你玩附錄的機器手臂時，試試看兩種方式：一次讓它「咻」地衝過去，一次讓它慢慢加速慢慢停。用眼睛看看哪一種抖得比較厲害，再用手機的碼表量量看，哪一種真正「穩下來」的時間比較短。你會親眼看到這一章的道理喔！

## 叫獸提醒與常見錯誤

- 誤區一：以為關節空間規劃的起訖點安全就代表全程安全。中間過程末端會走出曲線，可能穿過桌面或撞到障礙物。
- 誤區二：在需要直線的作業（焊接、塗膠）使用關節運動指令。末端路徑會偏離直線，造成焊縫或膠道歪斜。
- 誤區三：以為五次多項式一定比三次好。它雖然加速度連續，但峰值速度高出 25%，更容易超出速度極限。
- 誤區四：短距離移動時調高速度參數。此時瓶頸在加速度，速度根本達不到額定值，應調整的是加速度參數。
- 誤區五：忽略急跳度。加速度的跳變會激發機構共振、加速減速機磨損，且使追隨誤差增加。
- 誤區六：在轉折處要求精確通過。這會迫使機器人減速至零，大幅增加節拍時間；應在非作業段使用混成半徑。
- 誤區七：對姿態用歐拉角做線性內插。路徑會扭曲且可能經過萬向節鎖；應使用四元數 SLERP。
- 誤區八：SLERP 未處理雙重覆蓋。若  $q_0 \cdot q_f < 0$  而未把  $q_f$  取負，機器人會繞遠路（超過

180 度)。

- 誤區九：只做運動學檢驗而不檢驗力矩。速度與加速度在限制內，不代表力矩也在限制內——後者還取決於姿態與重力。
- 誤區十：以為時間縮放萬能。重力項不隨時間縮放而降低；若重力本身就超限，只能減輕負載或換馬達。
- 誤區十一：追求時間最佳軌跡。它沒有餘裕、對模型誤差敏感、且急跳度大；實務應保留 10 ~ 20% 的餘裕。
- 誤區十二：忽略多軸同步。各軸獨立執行會使末端路徑不可預測；且調快非主導軸的速度對節拍毫無幫助。

## 課堂討論

1. 開頭故事中，平滑的軌跡「反而更快」。請從三個角度解釋這個現象：控制器的降速保護、殘餘振動的等待時間、以及追隨誤差。
2. 混成半徑是速度與精度的直接取舍。請為以下三個情境各建議一個混成策略：(a) 點焊（只需在特定点停留）；(b) 連續弧焊（整條路徑都要精確）；(c) 從料架取料再放到輸送帶（自由空間移動）。
3. 時間縮放無法降低重力項。請討論這對「機器人在什麼姿態下最容易超出力矩極限」有何啟示，並提出三種設計層面的因應方式。
4. 實務上很少使用時間最佳軌跡。請討論：在什麼情況下追求時間最佳是值得的？需要哪些配套條件（提示：模型精度、控制器性能、機構剛性）？
5. 輸入整形以「延遲半個振動週期再送第二個脈衝」來抵消振動。請討論這個方法的三個前提條件，以及當這些條件不滿足時會發生什麼。

## 章末摘要

- 路徑是幾何曲線（不含時間），軌跡是時間參數化的路徑；由路徑產生軌跡的過程稱為時間參數化。
- 好軌跡的五個標準：平滑、快速、可行、精確、可預測；其中「快速」與「平滑」存在根本的張力。

- 關節空間規劃計算快、無奇異點問題，但末端路徑不可預測；任務空間規劃路徑可控，但需逐點逆解且可能遇奇異點。
- 三次多項式的  $v_{\max} = 1.5\Delta q/T$ 、 $a_{\max} = 6\Delta q/T^2$ ，加速度在端點跳變（急跳度無限大）。
- 五次多項式加速度連續，但  $v_{\max} = 1.875\Delta q/T$  高出 25%，更容易觸及速度極限——平滑不是免費的。
- 梯形速度曲線在相同的速度與加速度限制下用時最短： $T = \Delta q/v_{\max} + v_{\max}/a_{\max}$ 。
- 當  $\Delta q < v_{\max}^2/a_{\max}$  時退化為三角形，實際峰值速度低於額定值——此時提高速度參數無效，瓶頸在加速度。
- S 形曲線以七段結構限制急跳度，總時間多出  $a_{\max}/j_{\max}$ ，換取加速度連續與振動降低。
- 急跳度的四個意義：力矩變化率、激發結構振動、乘坐舒適度、追隨精度。
- 多路徑點的速度可由使用者指定、啟發式規則或整體最佳化決定；混成半徑是速度與精度的直接取舍參數。
- 笛卡兒直線採「幾何路徑 + 時間曲線」分離設計；圓弧須依  $a_c = v^2/R$  限制速度；姿態須用四元數 SLERP 並處理雙重覆蓋。
- 軌跡必須通過動力學檢驗：逐點計算  $\tau = M\ddot{q} + C\dot{q} + G$  並確認不超限。
- 時間縮放  $k$  倍後，速度按  $1/k$ 、加速度按  $1/k^2$ 、動態力矩按  $1/k^2$  縮放，但重力項完全不變。
- 時間最佳軌跡具 bang-bang 特性（恆有關節處於極限），但因不夠平滑、無餘裕、依賴模型，實務少用。
- 多軸同步取各軸所需時間的最大值，決定總時間的是主導軸；調快非主導軸對節拍無幫助。
- 實務流程七步驟：取得路徑、選擇空間、設定混成、時間參數化、多軸同步、動力學檢驗、輸出指令。

## 觀念題與計算題

### 觀念題

1. 區分路徑與軌跡，並說明時間參數化的意義。
2. 列舉好軌跡的五個標準，並說明哪兩者之間存在根本的張力。



3. 比較關節空間與任務空間規劃的優缺點，並各舉兩個適用場合。
4. 為何關節空間規劃「起訖點安全不代表全程安全」？請提出兩種因應方式。
5. 推導三次多項式的係數，並說明為何其加速度在端點會跳變。
6. 說明五次多項式相對於三次多項式的優勢與代價。
7. 寫出梯形速度曲線的總時間公式，並說明三角形退化的判斷條件與實務意義。
8. 說明 S 形曲線的七個階段，以及它相對於梯形曲線多出的時間代價。
9. 列舉急跳度的四個工程意義，並說明它與機構振動的關係。
10. 說明混成半徑的作用，並解釋為何「允許不精確通過」反而是好的設計。
11. 為何姿態內插必須使用 SLERP 而非歐拉角線性內插？實作時需注意哪兩個要點？
12. 說明圓弧運動為何必須依半徑限制速度，並寫出相關公式。
13. 寫出時間縮放後各項力矩的變化關係，並說明為何重力項不隨縮放降低。
14. 說明時間最佳軌跡的 bang-bang 特性，以及實務上少用它的三個原因。
15. 說明多軸同步的三種等級，以及「主導軸」的概念對節拍最佳化的意義。

### 計算題

1. 某關節需由  $-30^\circ$  移動到  $120^\circ$ ，速度極限  $90^\circ/\text{s}$ 、加速度極限  $200^\circ/\text{s}^2$ 。(a) 以梯形速度曲線設計，判斷是否有等速段並求總時間。(b) 求加速段的時間與位移。(c) 若改用五次多項式且時間相同，求其峰值速度與加速度，並判斷是否可行。(d) 若五次多項式不可行，求最短可行時間。
2. 承上題的關節。(a) 若移動量改為  $30^\circ$ ，重新判斷曲線形狀並求總時間與峰值速度。(b) 計算「把速度極限提高到  $120^\circ/\text{s}$ 」對此動作總時間的影響。(c) 計算「把加速度極限提高到  $300^\circ/\text{s}^2$ 」的影響。(d) 由此說明應優先調整哪個參數。
3. 某 S 形曲線的參數為  $v_{\max} = 100 \text{ mm/s}$ 、 $a_{\max} = 500 \text{ mm/s}^2$ 、 $j_{\max} = 5000 \text{ mm/s}^3$ ，位移  $300 \text{ mm}$ 。(a) 求急跳段時間  $t_j$  與完整加速段時間  $t_a$ 。(b) 求總時間。(c) 與梯形曲線比較，計算多出的時間百分比。(d) 若  $j_{\max}$  降為  $1000 \text{ mm/s}^3$ ，重新計算總時間。
4. 某平面 2R 手臂在某時刻的動態力矩為  $(M\ddot{q} + C\dot{q}) = (4.5, 1.2)^T \text{ N}\cdot\text{m}$ 、重力項  $G = (10.5, 0.8)^T \text{ N}\cdot\text{m}$ ，力矩極限為  $(14, 2.5)^T \text{ N}\cdot\text{m}$ 。(a) 計算總力矩並判斷可行性。(b) 計算各關節



的動態餘裕。(c) 求所需的時間縮放倍數。(d) 若把負載加倍使重力項變為  $(18, 1.4)^T$ ，重新判斷可行性並說明結論。

## 動手做與專題延伸

### 本章實作：軌跡生成模組

延續前面章節的程式庫，加入軌跡生成模組。這將是第 16 ~ 17 章控制器測試的訊號來源。

- **階段一**：實作四種基本軌跡（三次、五次、梯形、S 形），統一介面為  $\text{traj}(t) \rightarrow (q, \dot{q}, \ddot{q})$ ，並繪出四階曲線比較。
- **階段二**：實作多路徑點軌跡，支援使用者指定或啟發式的路徑點速度，並加入混成功能。
- **階段三**：實作笛卡兒空間的直線與圓弧軌跡，含 SLERP 姿態內插與逐點逆運動學。
- **階段四**：整合第 14 章的逆動力學，實作可行性檢驗與自動時間縮放的迴圈。
- **階段五**：實作多軸同步，找出主導軸並統一時間。

### 專題延伸

- **四階曲線視覺化**：為四種軌跡形式繪製位置、速度、加速度、急跳度的四階疊圖。特別標註三次多項式在端點的急跳度脈衝與梯形曲線在轉折處的跳變。這張圖是理解「平滑」概念最直觀的教材。
- **節拍最佳化實驗**：設計一個包含五個路徑點的取放循環，系統性地變化三個參數（速度比例、加速度比例、混成半徑），記錄總節拍時間與路徑偏離量。繪出三維的參數—性能圖，找出最佳的參數組合。這模擬了產線調機的真實工作。
- **進階：時間最佳路徑參數化 (TOPP) 的簡化實作**。對一條給定的路徑，在每個路徑點計算「受力矩極限所允許的最大加速度」，再用前向—後向掃描法（先由起點正向加速掃描、再由終點反向減速掃描、取兩者的較小值）求出時間最佳的速度剖面。與固定的梯形曲線比較節拍時間，並觀察最佳解的 bang-bang 特性。

## 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 7 章：軌跡生成，本章主要參考)。
- Biagiotti, L. and Melchiorri, C., Trajectory Planning for Automatic Machines and

Robots, Springer ( 軌跡規劃的完整專著，含各類運動曲線的詳細分析 ) 。

- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer ( 第 4 章：軌跡規劃 ) 。
- Bobrow, J. E., Dubowsky, S. and Gibson, J. S., "Time-Optimal Control of Robotic Manipulators Along Specified Paths," IJRR, 1985 ( 時間最佳路徑參數化的奠基論文 ) 。
- Pham, Q.-C., "A General, Fast, and Robust Implementation of the Time-Optimal Path Parameterization Algorithm," IEEE Trans. Robotics, 2014 ( 現代 TOPP 演算法的高效實作 ) 。
- Singer, N. C. and Seering, W. P., "Preshaping Command Inputs to Reduce System Vibration," ASME J. Dynamic Systems, 1990 ( 輸入整形技術的原始論文 ) 。
- Shoemake, K., "Animating Rotation with Quaternion Curves," SIGGRAPH 1985 ( SLERP 姿態內插 ) 。
- Kunz, T. and Stilman, M., "Time-Optimal Trajectory Generation for Path Following with Bounded Acceleration and Velocity," RSS 2012 ( 含加速度限制的軌跡生成 ) 。
- Ruckig 開源函式庫：即時的 S 形軌跡生成器，支援線上重新規劃，廣泛用於現代機器人系統。
- MoveIt 2 與 Ruckig、TOPP-RA 的整合文件：ROS 2 生態系中軌跡生成與時間參數化的實務作法。

## 第 16 章

# 線性回授與 PID 控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一位學生在手掌上平衡竹竿的三格連續照片：第一格蒙眼（竿子傾倒，標註「開迴路」）、第二格張眼但動作誇張（竿子劇烈擺盪，標註「只有  $P$ 」）、第三格動作細微而穩定（標註「 $PID$ 」）。右半為對應的階躍響應曲線圖，三條曲線分別為等幅振盪（ $\zeta=0$ ）、有超越的衰減（ $\zeta=0.3$ ）與平順到位（ $\zeta=0.707$ ），並以虛線標出  $\pm 2\%$  容許帶。中央以三個大字母  $P$ 、 $I$ 、 $D$  串連兩側，下方註記： $P$  看現在、 $I$  看過去、 $D$  看未來。

## 叫獸開講 「機器手臂到了目標位置，為什麼還在左右發抖？」

叫獸請小翰上台，蒙上眼睛，然後在他手上放了一根竹竿，竿頂立著一顆保麗龍球。「請你把球維持在竿子頂端不要掉下來。」

小翰試了三秒，球就掉了。「我看不到啊！」

「對，這就是**開迴路**。」叫獸解開眼罩，「你只能憑記憶動作，不知道現在的狀況。現在張開眼睛再試一次。」這次小翰撐了將近半分鐘——他能看到球往哪邊倒，就往哪邊移動竿子。

「這叫**閉迴路**：你量測誤差、據以修正。這就是所有控制的核心。」叫獸在白板上畫了一個圓圈：目標 → 比較 → 控制 → 系統 → 量測 → 回到比較。

接著他問了一個更深的問題：「小翰，你剛才是**怎麼**修正的？球往右倒，你就把竿子往右移——移多少？」

「憑感覺……倒得越多，我移得越多。」

「這就是**比例控制**。」叫獸寫下 P。「但如果只有這樣，會發生什麼事？」他請小翰再試一次，但這次要求「只能看球目前偏多少，不准預測」。小翰的動作變得非常誇張——球在左右之間劇烈擺盪，撐不到五秒。

「因為你反應太慢了。等球已經倒了才修正，它已經有速度了，你把它扳回來時它又衝過頭。」叫獸寫下第二個字母 D。「真正厲害的人會看球**倒的速度**，提前反應——球還沒倒多少，但倒得很快，就要趕快修。這叫**微分控制**，它的作用像剎車。」

最後他在竿子上綁了一小塊配重，讓竿子微微向左傾斜。小翰重新開始，這次球雖然沒掉，但總是停在偏左一點的位置，怎麼都回不到正中央。

「因為你的比例控制需要**誤差存在**才會出力。誤差變小，你出的力就變小，小到剛好抵消配重時就平衡了——於是留下一個永遠消不掉的偏差。」叫獸寫下第三個字母 I。「解法是：把過去累積的誤差加起來，只要偏差持續存在，修正量就持續增加，直到把它推回中央為止。」

白板上留下三個字母：P、I、D。「這三個字母，」叫獸說，「統治了全世界九成以上的工業控制迴路——從你家的冷氣、汽車的定速巡航，到六軸機器手臂的每一個關節。今天，我們就把它們徹底搞懂。」

## 本章為什麼重要

第 15 章規劃出了完美的軌跡，但那只是**指令**。真實的機器人有摩擦、有彈性、負載會變、外界有干擾——把指令變成實際運動，是控制器的工作。

PID 是控制領域最重要的實用工具。據統計，工業界九成以上的控制迴路採用 PID 或其變體，而所有的機器人關節伺服控制，內層都是 PID。理解它不只是為了會調參數，更是為了理解**回授的本質與極限**：為什麼增益不能無限調高、為什麼快速與穩定互相衝突、以及為什麼在機器人上單靠 PID 不夠（第 13 章開頭的故事）。

本章的分析對象刻意選擇最簡單的系統——一維點質量。這不是簡化的教學玩具，而是有深刻理由的：第 17 章的計算力矩控制，正是要把複雜的多軸機器人「線性化」成  $n$  個獨立的一維點質量。因此把一維系統徹底搞懂，就掌握了現代機器人控制的核心。

## 學習目標

- 能區分開迴路與閉迴路控制，並說明回授的三個價值。
- 能繪製控制系統的方塊圖並辨識各元件的角色。
- 能推導一維點質量在 P 控制下的閉環方程並說明其振盪特性。
- 能推導 PD 控制的閉環二階方程，並由係數求出自然頻率與阻尼比。
- 能說明過阻尼、臨界阻尼與欠阻尼三種響應的特徵與適用場合。
- 能說明積分項如何消除穩態誤差，以及它帶來的代價。
- 能解釋積分飽和現象並提出至少兩種防止方法。
- 能運用時域規格（上升時間、超越量、整定時間）進行增益設計。
- 能說明數位實作的三個議題：取樣週期、微分雜訊、離散化。
- 能說明前饋控制的原理及其與回授的分工。
- 能以 Python 實作 PID 控制器並在動力學模擬中測試。

## 先備知識

第 13 章的動力學（尤其是一維點質量與二階系統的概念）、第 15 章的軌跡生成（提供控制器的追隨目標）、第 7 章的嵌入式實作（PWM、中斷、控制週期）。

數學方面需要二階常微分方程的解法（特徵方程與根的性質）。若對此不熟悉，可先閱讀附錄 D 的微分方程速成——本章會以物理直覺重新說明每一個數學結果，不會讓推導成為理解的障礙。

## 核心名詞中英對照

表 16-1 本章核心名詞

| 中文   | 英文                            | 說明               |
|------|-------------------------------|------------------|
| 開迴路  | Open-loop                     | 不使用量測回授的控制方式     |
| 閉迴路  | Closed-loop                   | 以量測誤差修正輸出的控制方式   |
| 回授   | Feedback                      | 將輸出量測值送回輸入端比較    |
| 前饋   | Feedforward                   | 依模型預先計算所需輸入      |
| 誤差   | Error                         | 目標值與實際值之差        |
| 增益   | Gain                          | 控制器對誤差的放大倍數      |
| 比例控制 | Proportional Control          | 輸出與當前誤差成正比       |
| 微分控制 | Derivative Control            | 輸出與誤差變化率成正比      |
| 積分控制 | Integral Control              | 輸出與誤差累積量成正比      |
| 自然頻率 | Natural Frequency, $\omega_n$ | 系統無阻尼時的振盪頻率      |
| 阻尼比  | Damping Ratio, $\zeta$        | 描述振盪衰減快慢的無量綱參數   |
| 穩態誤差 | Steady-state Error            | 時間趨於無窮時殘留的誤差     |
| 超越量  | Overshoot                     | 響應超過目標值的最大百分比    |
| 整定時間 | Settling Time                 | 響應進入並維持在容許帶內所需時間 |
| 積分飽和 | Integral Windup               | 積分項過度累積導致的響應惡化   |
| 取樣週期 | Sampling Period               | 數位控制器執行一次的時間間隔   |

## 16.1 開迴路與閉迴路

### 16.1.1 兩種控制哲學

開迴路控制依據事先的計算或經驗直接下達指令，不檢查結果。例如：「馬達通電 3 秒，應該會轉 90 度。」它的優點是簡單、便宜、不會不穩定（因為沒有回授就沒有振盪的可能）。

缺點是任何未預期的因素都會造成誤差且無法修正：電池電壓下降、摩擦力變化、負載改變、

機構磨損——全部都會累積成無法察覺的偏差。第 5 章的步進馬達失步（5.4.2 節）正是開迴路的典型風險。

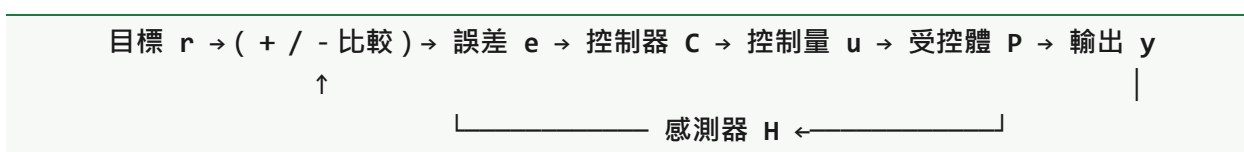
閉迴路控制則持續量測實際輸出，計算與目標的誤差，據以調整控制量。這帶來三個決定性的價值：

- **抗干擾**：外界的擾動造成誤差後會被偵測並修正。
- **降低對模型的依賴**：即使系統參數估計不準（如負載不明），回授仍能把誤差壓下來。
- **改變系統動態**：可以讓一個原本反應緩慢的系統變快、讓一個容易振盪的系統變穩。

代價則是：需要感測器（成本）、可能不穩定（增益過高會發散）、且引入雜訊（感測器的雜訊會被放大送進控制量）。

## 16.1.2 控制系統的方塊圖

標準的閉迴路控制系統包含五個元件：



閉迴路方塊圖。誤差  $e = r - y$ （若感測器增益為 1）。

對機器人的單一關節而言： $r$  是第 15 章規劃的目標角度、 $C$  是 PID 控制器、 $u$  是送給驅動器的力矩指令、 $P$  是關節的動力學（第 13 章）、 $y$  是實際角度、 $H$  是編碼器（第 6 章）。可以看到，本書前面各章的內容在此匯集成一個完整的迴路。

圖 16-1 機器人關節的控制迴路

建議配圖：完整的方塊圖。左側為軌跡生成器（第 15 章）輸出目標位置、速度與加速度；中央為 PID 控制器（三個並聯的方塊  $K_p$ 、 $K_i$ 、 $K_d$ ）；控制量經驅動器（第 5、7 章）送至馬達；受控體標註為關節動力學  $M(q)\ddot{q} + C\dot{q} + G$ （第 13 章）；輸出經編碼器（第 6 章）回授。圖中並以虛線標出第 17 章將加入的前饋路徑（由軌跡直接經逆動力學至控制量）。各方塊下方標註對應的章節編號。

## 16.2 一維點質量：所有控制的起點

### 16.2.1 為何從最簡單的系統開始

考慮一個質量為  $m$  的物體，在無摩擦的水平面上，受控制力  $u$  作用。其運動方程為：

$$m \ddot{x} = u$$

一維點質量。這是最簡單的動態系統，卻包含了控制問題的所有本質。

為何不直接研究六軸手臂？三個理由。**理由一：本質相同**。多軸系統的每個關節，在忽略耦合後就是一個一維系統。**理由二：可解析分析**。一維系統的閉環方程有封閉解，可以精確算出增益與性能的關係；多軸系統只能數值模擬。**理由三：第 17 章的目標**。計算力矩控制的核心思想，正是用動力學模型把多軸系統「化為」 $n$  個獨立的一維點質量——因此一維分析不是簡化，而是最終要達成的理想狀態。

### 16.2.2 控制目標

控制的目標是讓  $x(t)$  追隨目標  $x_d(t)$ 。定義誤差：

$$e = x_d - x$$

誤差。控制的任務就是讓  $e \rightarrow 0$ ，且過程要快、要穩、不要振盪。

最基本的情形是**定點調節** (regulation)： $x_d$  為固定值，把物體從某處移到目標點並停住。進階的情形是**軌跡追隨** (tracking)： $x_d(t)$  隨時間變化，物體必須持續跟上——這正是第 15 章規劃的軌跡所需要的。本章先分析定點調節，16.9 節再處理軌跡追隨。

## 16.3 比例控制與振盪

### 16.3.1 最直覺的控制律

最自然的想法是：誤差越大，出力越大。

$$u = K_p \cdot e = K_p (x_d - x)$$

比例控制 (P control)。  $K_p$  稱為比例增益。

代入運動方程並整理 (設  $x_d$  為常數，故  $\ddot{e} = -\ddot{x}$ )：

$$m \ddot{x} = K_p (x_d - x) \Rightarrow m \ddot{e} + K_p e = 0 \Rightarrow \ddot{e} + (K_p/m) e = 0$$

閉環方程。這是一個標準的簡諧振盪方程。

### 16.3.2 結果：永不停止的振盪

上式的解為：

$$e(t) = e_0 \cos(\omega t), \quad \text{其中 } \omega = \sqrt{K_p/m}$$

純比例控制的響應。誤差以固定振幅永遠振盪，不會衰減。



這個結果與物理直覺完全吻合：**比例控制器等效於一根彈簧**。誤差就是彈簧的伸長量， $K_p$  就是彈簧常數。一個質量掛在無阻尼的彈簧上，會永遠上下振盪——這正是我們得到的結果。

#### ◎ 比例控制 = 彈簧

這個類比極為有用，值得牢記。 $K_p$  越大，等效彈簧越硬，振盪頻率越高 ( $\omega = \sqrt{K_p/m}$ )，系統反應越快——但也越容易被激發振動。 $K_p$  越小，系統越「軟」，反應慢但也溫和。第 20 章的阻抗控制，正是刻意利用這個類比，把  $K_p$  設計成期望的機構剛度。

### 16.3.3 為何純 P 控制不可用

純比例控制有兩個致命問題。**問題一：永遠振盪**。理想情況下振幅不衰減；實際系統因有摩擦會緩慢衰減，但過程漫長且難以預測。這正是開頭故事中小翰「只看偏多少、不准預測」時球劇烈擺盪的原因。

**問題二：穩態誤差**。若系統受到持續的干擾力  $d$  (如重力、摩擦)，平衡時必須有  $K_p \cdot e = d$ ，因此：

$$e_{ss} = d / K_p$$

穩態誤差與干擾成正比、與增益成反比。要消除它只能把  $K_p$  調到無限大——但那會使系統極度振盪且放大雜訊。

這正是開頭故事中「綁上配重後球總是偏左」的現象。解決這兩個問題，分別需要  $D$  項與  $I$  項。

## 16.4 微分控制與阻尼

### 16.4.1 加入「剎車」

要抑制振盪，需要一個與**速度**反向的力——就像空氣阻力或油壓阻尼器。這就是微分控制：

$$u = K_p e + K_d \dot{e}$$

*PD 控制。 $K_d$  稱為微分增益。 $\dot{e}$  是誤差的變化率。*

注意：在定點調節中  $x_d$  為常數，故  $\dot{e} = -\dot{x}$ 。因此微分項實際上就是  $-K_d \dot{x}$ ——一個與運動速度反向的力，正是阻尼的定義。

### 16.4.2 閉環方程與二階系統

代入運動方程：

$$m \ddot{e} + K_d \dot{e} + K_p e = 0$$

PD 控制的閉環方程。這是機械工程中最著名的方程之一——質量—阻尼器—彈簧系統。

這個結果揭示了一個優美的對應關係：

$$m \leftrightarrow \text{質量} \quad K_d \leftrightarrow \text{阻尼係數} \quad K_p \leftrightarrow \text{彈簧常數}$$

PD 控制器把控制問題轉化為一個虛擬的機械系統。控制器設計，就是在「設計」這個虛擬系統的彈簧與阻尼器。

將方程改寫為標準二階形式：

$$\ddot{e} + 2\zeta\omega_n \dot{e} + \omega_n^2 e = 0$$

二階系統的標準形式。 $\omega_n$  為自然頻率、 $\zeta$  為阻尼比。

比較係數可得兩個關鍵公式：

$$\omega_n = \sqrt{K_p/m} \quad \zeta = K_d / (2\sqrt{K_p m})$$

由增益求系統參數。這兩式是 PD 控制器設計的核心。

16.4.3 三種響應

二階系統的行為完全由阻尼比  $\zeta$  決定，可分為三種情形：

欠阻尼 ( $\zeta < 1$ )：系統會超越目標再回擺，逐漸衰減。振盪頻率為  $\omega_d = \omega_n\sqrt{1-\zeta^2}$ 。反應快但有超越量。

臨界阻尼 ( $\zeta = 1$ )：不振盪的最快響應。這是理論上的最佳點——在所有不超越的情形中，它的整定最快。

過阻尼 ( $\zeta > 1$ )：完全不振盪，但反應遲緩。適用於絕對不容許超越的場合。

表 16-2 三種阻尼情形的比較

| 情形   | 阻尼比 $\zeta$     | 超越量               | 特徵       | 適用場合           |
|------|-----------------|-------------------|----------|----------------|
| 欠阻尼  | $0 < \zeta < 1$ | 有 ( $\zeta$ 越小越大) | 反應最快、有振盪 | 允許小幅超越、追求速度    |
| 臨界阻尼 | $\zeta = 1$     | 無                 | 不振盪的最快響應 | 理論最佳、常作為設計目標   |
| 過阻尼  | $\zeta > 1$     | 無                 | 反應遲緩     | 絕對禁止超越 (如插入作業) |
| 無阻尼  | $\zeta = 0$     | 100%              | 永不衰減的振盪  | 純 P 控制，不可用     |

圖 16-2 不同阻尼比的階躍響應

建議配圖：單一座標圖，橫軸為正規化時間  $\omega_n t$ 、縱軸為輸出（目標值為 1）。繪出五條曲線並標註阻尼比： $\zeta=0$ （等幅振盪）、 $\zeta=0.3$ （明顯超越與振盪）、 $\zeta=0.707$ （小幅超越約 4.3%，常用設計點）、 $\zeta=1$ （臨界阻尼，無超越且最快）、 $\zeta=2$ （過阻尼，緩慢）。以虛線標出  $\pm 2\%$  的容許帶，並標註  $\zeta=0.707$  與  $\zeta=1$  的整定時間比較。

#### 16.4.4 實務上的常用設計點

雖然  $\zeta = 1$  是「不超越的最快」，實務上最常用的設計點卻是  $\zeta = 0.707$ （即  $1/\sqrt{2}$ ）。原因是它允許約 4.3% 的小幅超越，換來的是明顯更短的上升時間與整定時間——在多數應用中這個交換是划算的。此外， $\zeta = 0.707$  在頻域上對應到最平坦的頻率響應（Butterworth 特性），對雜訊的處理也較為理想。

在機器人上則需依任務調整：取放動作可用  $\zeta \approx 0.7$  追求速度；精密插入應用  $\zeta \geq 1$  避免超越造成撞擊；力控制（第 20 章）則往往需要更高的阻尼以確保接觸穩定。

### 16.5 積分控制與穩態誤差

#### 16.5.1 消除頑固的偏差

16.3.3 節指出：在持續干擾下，比例控制必然留下穩態誤差  $e_{ss} = d/K_p$ 。要消除它，需要一個「只要誤差存在就持續增強」的機制——這就是積分項：

$$u = K_p e + K_i \int e \, dt + K_d \dot{e}$$

完整的 PID 控制律。 $K_i$  為積分增益。

積分項的關鍵性質是：只要誤差不為零，積分值就持續累積，控制量就持續增加。因此系統唯一可能的平衡狀態，就是誤差恰好為零——此時積分停止增長，而累積下來的值恰好提供了抵消干擾所需的力。

$$\text{平衡時：} K_i \int e \, dt = d, \quad \text{而 } e = 0$$

積分項「記住」了需要多大的力來抵消干擾，使誤差得以歸零。

這正是開頭故事中「把過去累積的誤差加起來」的作法。它的物理意義是：控制器學會了這個干擾的大小，並持續提供補償。

#### 16.5.2 積分的三個代價

積分項雖能消除穩態誤差，卻帶來三個明顯的代價。

**代價一：降低穩定性。**加入積分後，閉環系統由二階變為三階。三階系統若增益配置不當會不穩定——這是 PID 調參中最容易出問題的地方。粗略的規則是： $K_i$  越大越容易振盪。

**代價二：增加超越量。**積分項在誤差消除後仍保有累積值，這使系統傾向於「衝過頭」。因此加入積分幾乎必然使超越量增加。

**代價三：反應遲滯。**積分是對過去的累積，本質上是「向後看」的，因此反應較慢。這與微分項（向前預測）恰成對比。

三項作用可以這樣總結：**P 看現在、I 看過去、D 看未來**。三者互補，但也互相牽制。

### 16.5.3 積分飽和與防護

積分項有一個惡名昭彰的實務問題：**積分飽和**（integral windup）。

問題發生在控制量達到物理極限時（馬達力矩飽和、PWM 達到 100%）。此時即使控制器要求更大的輸出，實際系統也無法提供，誤差持續存在，積分項卻仍在不斷累積——可能累積到極大的值。

當系統終於接近目標時，這個巨大的積分值仍在推動系統，造成嚴重的超越；而要「消掉」這個積分值，必須讓誤差反向累積同樣長的時間。結果是巨大的超越加上漫長的振盪。

三種標準的防護方法：

- **條件積分（Conditional Integration）**：偵測到控制量飽和時，暫停積分累積。最簡單有效，是嵌入式系統的首選。
- **積分限幅（Clamping）**：直接限制積分項的最大值，通常設為控制量上限的某個比例。實作簡單但需調整限幅值。
- **反飽和回授（Back-calculation）**：把「要求輸出」與「實際輸出」的差值回饋去減少積分項，飽和越嚴重、積分被削減越多。效果最平順，是工業控制器的標準作法。

#### ◎ 一個常見的現場故障

某輸送設備在正常運轉時控制良好，但每次「開機時工件卡住」之後就會失控暴衝。原因就是積分飽和：卡住期間馬達力矩飽和、誤差持續、積分累積到極大值；工件一鬆脫，那個巨大的積分值立刻推動馬達全速運轉。這類故障的特徵是「平常都好，異常之後更糟」——只要看到這個模式，第一個該檢查的就是積分飽和防護。

## 16.6 增益設計與時域規格

### 16.6.1 由規格反推增益

實務上的設計流程不是「隨便試增益」，而是**先訂規格、再算增益**。常用的三個時域規格為：

**上升時間  $t_r$** ：輸出由 10% 上升到 90% 所需的時間。它衡量反應速度。

**超越量  $M_p$** ：最大超過目標值的百分比。它由阻尼比唯一決定：

$$M_p = \exp(-\pi\zeta / \sqrt{1-\zeta^2}) \times 100\%$$

超越量與阻尼比的關係。 $\zeta=0.707$  對應 4.3%、 $\zeta=0.5$  對應 16.3%、 $\zeta=0.3$  對應 37.2%。

**整定時間  $t_s$** ：輸出進入並維持在目標值  $\pm 2\%$  帶內所需的時間。近似公式為：

$$t_s \approx 4 / (\zeta \omega_n)$$

整定時間近似式（2% 準則）。注意它只與  $\zeta \omega_n$  的乘積有關——這個乘積就是指數衰減的速率。

### 16.6.2 設計流程

標準的三步驟設計流程：

1. 由超越量規格決定  $\zeta$ 。若要求超越量不超過 5%，則  $\zeta \geq 0.69$ ，取  $\zeta = 0.707$ 。
2. 由整定時間規格決定  $\omega_n$ 。由  $t_s \approx 4/(\zeta \omega_n)$  反推： $\omega_n \approx 4/(\zeta t_s)$ 。
3. 由  $\zeta$  與  $\omega_n$  計算增益： $K_p = m \omega_n^2$ 、 $K_d = 2 \zeta \omega_n m$ 。

這個流程把「調參數」從試誤變成了設計。例題 16-2 會完整走一遍。

### 16.6.3 增益的上限：三個限制

由  $K_p = m \omega_n^2$  可知，要反應更快（大  $\omega_n$ ）就需要更大的增益。那麼能否無限提高？答案是不能，有三個限制。

**限制一：致動器飽和**。大增益會要求大力矩，但馬達有極限（第 5 章）。一旦飽和，實際的閉環行為就與設計不符，且可能觸發積分飽和。

**限制二：雜訊放大**。微分項會放大感測器雜訊——若編碼器讀值有微小抖動，微分後會變成劇烈的雜訊，再乘上  $K_d$  送給馬達，造成馬達嗡嗡作響甚至發熱。這是  $K_d$  的主要限制。

**限制三：未建模動態**。最根本也最危險的限制。真實的機構並非剛體——連桿有彈性、減速機有扭轉柔度、皮帶會伸長。這些「未建模」的柔性有其自身的共振頻率。若控制器的頻寬

$\omega_n$  接近或超過這個共振頻率，就會激發共振，系統立刻失控。

**經驗法則：控制器頻寬  $\omega_n < (1/3 \sim 1/5) \times$  最低結構共振頻率**

*這是機電系統設計的黃金法則。想要更高的頻寬，必須先提高機構的剛性。*

這正是第 13 章開頭故事「調高增益治標不治本」的完整答案：增益調高會依序遇到雜訊放大（馬達嗡嗡叫）、致動器飽和（力矩不足）、最後是激發共振（劇烈振動甚至損壞）。真正的解方不是提高增益，而是加入前饋（16.9 節與第 17 章）。

## 16.7 數位實作

### 16.7.1 從連續到離散

前面的分析都在連續時間中進行，但實際的控制器是數位的——每隔一個固定的取樣週期  $T$  執行一次。這帶來三個必須處理的議題。

**議題一：微分的近似。**導數必須用差分近似：

$$\dot{e}_k \approx (e_k - e_{k-1}) / T$$

*後向差分。最簡單但對雜訊極為敏感——雜訊被除以一個小數  $T$ ，等於被放大了  $1/T$  倍。*

**議題二：積分的近似。**積分用累加近似：

$$\int e \, dt \approx \sum e_k \cdot T$$

*矩形積分。也可用梯形法提高精度，但實務上矩形法通常已足夠。*

**議題三：取樣延遲。**從量測到輸出至少延遲一個週期，這相當於在迴路中加入了純延遲，會降低穩定裕度。

### 16.7.2 取樣週期的選擇

取樣週期太長會導致三個問題：延遲增大（穩定性下降）、離散化誤差、以及可能的混疊（第 6.9 節）。太短則增加運算負擔。

經驗法則是：

$$\text{取樣頻率 } f_s \geq (10 \sim 20) \times \text{閉環頻寬 } f_{bw} \quad (f_{bw} = \omega_n / 2\pi)$$

*取樣頻率的選擇準則。若閉環頻寬為 10 Hz，取樣頻率應至少 100 ~ 200 Hz。*

這解釋了第 3 章的時間尺度金字塔：機器人的關節位置迴路通常在 1 kHz、速度迴路數 kHz、電流迴路可達數十 kHz——每一層的取樣頻率都遠高於其閉環頻寬。

### 16.7.3 微分項的雜訊處理

微分放大雜訊是 PID 數位實作最大的實務難題。三種標準對策：

**對策一：微分項濾波。**在微分後加一個一階低通濾波器（第 6.10.2 節）。這是最常見的作法，稱為「不完全微分」。

$$D_k = \alpha D_{k-1} + (1-\alpha) K_d (e_k - e_{k-1})/T$$

帶濾波的微分項。 $\alpha$  越大濾波越強，但延遲也越大。

**對策二：用速度量測取代微分。**若系統有速度感測器（或編碼器能可靠地計算速度），直接使用速度值，避免對位置微分。這是伺服驅動器的標準作法。

**對策三：對量測值微分而非誤差。**在追隨變化的目標時，目標值的突變會使誤差微分產生巨大脈衝（稱為 derivative kick）。改為只對輸出  $y$  微分（並取負號），可避免這個問題：

$$u = K_p e + K_i \int e dt - K_d \dot{y}$$

微分作用於輸出而非誤差。這是工業 PID 控制器的標準實作。

## 16.8 PID 調參方法

### 16.8.1 手動調參的標準流程

即使有 16.6 節的設計公式，實務上仍需微調——因為真實系統有摩擦、彈性與參數不確定性。標準的手動調參流程為：

1. **全部歸零：** $K_p = K_i = K_d = 0$ 。
2. **先調  $K_p$ ：**逐步增大直到系統出現持續的等幅振盪，記下此時的增益（稱為臨界增益  $K_u$ ）與振盪週期  $T_u$ ，然後把  $K_p$  降到約  $K_u$  的一半。
3. **再調  $K_d$ ：**逐步增大以抑制振盪，直到超越量符合規格。若馬達開始發出嗡嗡聲或高頻抖動，代表  $K_d$  過大（雜訊放大）。
4. **最後調  $K_i$ ：**只在確實存在穩態誤差時才加入，由小值開始逐步增大，直到誤差在可接受時間內消除。若出現緩慢的振盪或超越量明顯增加，代表  $K_i$  過大。



5. 加入防飽和：確認積分飽和防護已啟用（16.5.3 節）。
6. 全工作範圍驗證：在不同姿態、不同負載下測試——機器人的等效慣量隨姿態變化（第 13 章的  $M(q)$ ），單一組增益未必在所有姿態下都適用。

## 16.8.2 Ziegler-Nichols 法

最著名的經驗調參法是 Ziegler-Nichols 法。以臨界增益法為例：先求出臨界增益  $K_u$  與振盪週期  $T_u$ （步驟同上），再依下表設定：

表 16-3 Ziegler-Nichols 臨界增益法的參數建議

| 控制器類型 | $K_p$      | $K_i$           | $K_d$         |
|-------|------------|-----------------|---------------|
| P     | $0.5 K_u$  | —               | —             |
| PI    | $0.45 K_u$ | $1.2 K_p / T_u$ | —             |
| PID   | $0.6 K_u$  | $2 K_p / T_u$   | $K_p T_u / 8$ |

**必須注意的限制：**Ziegler-Nichols 法的設計目標是「對干擾的快速抑制」，其結果通常有較大的超越量（約 25%），對追求精度的機器人應用往往過於激進。它適合作為**起點**，之後再依 16.8.1 節的流程微調。此外，求臨界增益的過程需要讓系統實際振盪，這在實體機器人上有風險，應在低速、無負載、有防護的情況下進行。

## 16.8.3 機器人的特殊考量

將 PID 應用於機器人關節時，有三個特殊考量。

**考量一：增益隨姿態調整。**等效慣量  $M(q)$  隨姿態變化可達數倍（第 13.6.1 節）。若用固定增益，在低慣量姿態下會過於激進、在高慣量姿態下又太遲鈍。解法是**增益排程**（gain scheduling）：依姿態查表或計算調整增益，使  $\omega_n$  與  $\zeta$  維持恆定。

**考量二：重力補償優先。**重力是持續存在的干擾（第 13.8.1 節），若讓積分項去對抗它，會導致響應遲緩與超越。正確作法是用**前饋直接補償重力**，讓積分項只需處理摩擦等較小的殘餘干擾。這是第 17 章的第一個主題。

**考量三：關節間的耦合。**獨立的關節 PID 把耦合項（第 13.6.2 節的  $M_{12}$ ）視為未知干擾。低速時影響小，高速時則成為主要誤差來源——這正是第 13 章開頭故事的核心，也是第 17 章計算力矩控制要解決的問題。



## 16.9 軌跡追隨與前饋

### 16.9.1 追隨誤差：回授的先天限制

前面的分析都針對定點調節。但機器人的實際任務是追隨第 15 章規劃的軌跡——目標值持續變化。此時會出現一個純回授控制無法避免的問題：**追隨誤差**。

原因很根本：**回授控制必須先有誤差才會出力**。當目標以固定速度移動時，控制器需要持續輸出力量來維持運動，而這個力量來自  $K_p \cdot e$ ，因此誤差  $e$  不可能為零。速度越快、所需的力越大、誤差也越大。

$$\text{等速追隨時的穩態誤差：} e_{ss} \approx (m \ddot{x}_d + \text{摩擦}) / K_p$$

*追隨誤差與所需的力成正比、與增益成反比。這是純回授的先天限制。*

這解釋了一個常見現象：機器人在定點時精度很高（ $\pm 0.02 \text{ mm}$ ），但在運動中的動態精度差得多（ $\pm 1 \text{ mm}$  以上）。工業機器人型錄上標示的「重複精度」是靜態的，動態的「路徑精度」通常要差一到兩個數量級。

### 16.9.2 前饋：不等誤差發生就先出力

解決之道是**前饋**（feedforward）：既然我們**事先知道**目標軌跡（第 15 章已完整規劃），就能預先計算出所需的力，直接送給馬達，不必等誤差出現。

$$u = u_{ff} + u_{fb} = m \ddot{x}_d + [K_p e + K_d \dot{e}]$$

*前饋加回授的控制律。第一項由模型預先計算、第二項處理模型誤差與干擾。*

這個架構的思想極為重要，可以總結為一句話：**用模型處理已知的部分，用回授處理未知的部分**。

效果的量化：若模型完全正確，前饋項會提供全部所需的力，誤差理論上為零，回授項不需出力。若模型有 20% 的誤差，回授項只需處理那 20%——相較於純回授要處理 100%，誤差降為五分之一。

### 16.9.3 三個層次的前饋

前饋可以由粗到細分為三個層次，複雜度與效果遞增：

**表 16-4 三個層次的前饋**

| 層次      | 前饋項                               | 所需資訊       | 誤差改善             | 本書章節   |
|---------|-----------------------------------|------------|------------------|--------|
| 速度前饋    | $K_v \cdot \dot{x}_d$             | 目標速度       | 小 ( 僅補償黏滯摩擦 )    | 本章     |
| 加速度前饋   | $m \cdot \ddot{x}_d$              | 目標加速度、質量估計 | 中 ( 補償慣性 )       | 本章     |
| 完整動力學前饋 | $M(q)\ddot{q}_d + C\dot{q}_d + G$ | 完整動力學模型    | 大 ( 補償慣性、耦合、重力 ) | 第 17 章 |

第三層次正是第 17 章的**計算力矩控制**。它把第 13、14 章建立的動力學模型直接用於前饋，是本書「模型服務於控制」思想的完整實現。

### ◎ 前饋與回授的分工

一個生活化的比喻：開車轉彎時，有經驗的駕駛在**進入彎道前**就開始轉方向盤（前饋——依據對彎道的預知）；同時也持續看著車道線做微調（回授——處理路面、風、車況的不確定性）。新手駕駛只會做後者，於是總是壓線後才修正，開得又慢又晃。機器人控制完全相同：純 PID 是新手駕駛、加入前饋才是老手。

## 16.10 PID 的極限

### 16.10.1 PID 做得好的事

公允地說，PID 在多數場合表現極佳，這也是它統治工業界的原因。它的優勢是：**不需要模型**（只需調三個參數）、**實作簡單**（幾行程式碼）、**魯棒**（對參數變化有一定容忍度）、**直覺**（三項的作用清楚可解釋）。

對於變化緩慢、耦合弱、干擾小的系統，PID 幾乎總是足夠。這涵蓋了工業界九成以上的控制迴路。

### 16.10.2 PID 做不好的事

但在機器人上，PID 會遇到四個結構性的限制：

**限制一：非線性**。機器人的動力學是非線性的（第 13 章），而 PID 是線性控制器。在某個工作點調好的增益，換到另一個姿態就可能不適用。

**限制二：耦合**。獨立關節 PID 把耦合項視為干擾。低速時可忽略，高速時（科氏力與速度平方成正比）成為主要誤差源。

**限制三：追隨誤差。**如 16.9.1 節所述，純回授必然有追隨誤差，且與速度成正比。

**限制四：增益上限。**如 16.6.3 節所述，增益受雜訊、飽和與結構共振三重限制，無法靠調高增益解決上述問題。

這四點合起來，正是第 13 章開頭故事的完整診斷：**低速時 PID 完美（非線性與耦合效應小），高速時失準（四個限制同時發作），而調高增益碰到天花板。**真正的解方是引入模型——這就是第 17 章的主題。

## 16.11 由線性邁向非線性

### 16.11.1 本章的成果

本章建立了控制的基礎。你現在能夠：區分開迴路與閉迴路、理解 P、I、D 三項各自的物理意義（彈簧、記憶、阻尼）、由時域規格反推增益、辨識並防止積分飽和、處理數位實作的取樣與雜訊問題、以及理解前饋與回授的分工。

最重要的是三個核心洞見：**PD 控制等效於一個虛擬的彈簧—阻尼系統**（16.4.2 節）——這個觀點將在第 20 章的阻抗控制中發揮關鍵作用；**增益有三重上限**（16.6.3 節），因此「調高增益」不是萬能解；以及**用模型處理已知、用回授處理未知**（16.9.2 節）——這是現代控制的核心哲學。

### 16.11.2 接下來：把模型放進控制器

**第 17 章（非線性控制）**將直接處理本章留下的四個限制。核心方法是**計算力矩控制**：用第 13、14 章的動力學模型做前饋，把非線性的機器人「線性化」成  $n$  個獨立的一維點質量——而那正是本章分析得最透徹的系統。屆時你會發現，本章的所有結論（ $\zeta$ 、 $\omega_n$ 、增益設計）都能直接套用，只是作用在「已被線性化」的系統上。

同時第 17 章也會介紹**重力補償**（把  $G(q)$  直接前饋，這是最簡單也最有效的一步）與**操作空間控制**（直接在笛卡兒空間設計控制律）。

**第 18~19 章**則提供更嚴謹的工具：最佳控制（如何系統性地選擇「最好」的增益）與 Lyapunov 穩定性（如何**證明**控制器不會失控，而非只是實驗上看起來穩定）。

換句話說：第 16 章教你**不用模型也能控制**，第 17 章教你**用模型控制得更好**，第 18~19 章

教你證明它真的可靠。

## 完整計算例題

### 例題 16-1 PD 控制的參數分析

題目：某機器人關節的等效慣量  $m = 0.5 \text{ kg}\cdot\text{m}^2$  (此處以轉動慣量表示，控制量為力矩)。  
 設定  $K_p = 200 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 20 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。(a) 求自然頻率與阻尼比。(b) 判斷響應類型並估算超越量。(c) 求整定時間。(d) 若要達到臨界阻尼， $K_d$  應設為多少？

解 (a)：由 16.4.2 節的公式：

$$\omega_n = \sqrt{(K_p/m)} = \sqrt{(200/0.5)} = \sqrt{400} = 20 \text{ rad/s}$$

自然頻率，約 3.18 Hz。

$$\zeta = K_d / (2\sqrt{(K_p m)}) = 20 / (2\sqrt{(200 \times 0.5)}) = 20 / (2 \times 10) = 1.0$$

阻尼比恰為 1。

解 (b)： $\zeta = 1.0$  恰為臨界阻尼。這是不振盪的最快響應，超越量為 0%。

解 (c)：由 16.6.1 節的近似式：

$$t_s \approx 4/(\zeta \omega_n) = 4/(1.0 \times 20) = 0.2 \text{ 秒}$$

整定時間約 0.2 秒。

解 (d)：題目已是臨界阻尼，故  $K_d = 20$  即為答案。若要驗證：臨界阻尼要求  $\zeta = 1$ ，即  $K_d = 2\sqrt{(K_p m)} = 2\sqrt{100} = 20 \checkmark$

**延伸觀察：**若把  $K_p$  加倍到 400，則  $\omega_n = \sqrt{800} \approx 28.3 \text{ rad/s}$  (反應更快)，但若  $K_d$  維持 20， $\zeta$  會降為  $20/(2\sqrt{200}) = 0.707$ ——變成欠阻尼，出現約 4.3% 的超越。這說明了一個重要的實務要點：**調整  $K_p$  時必須同步調整  $K_d$ ，才能維持期望的阻尼比。**具體關係為  $K_d \propto \sqrt{K_p}$ 。

### 例題 16-2 由規格設計增益

題目：某關節等效慣量  $m = 0.8 \text{ kg}\cdot\text{m}^2$ 。設計規格要求：超越量不超過 5%、整定時間不超過 0.3 秒。(a) 由規格決定  $\zeta$  與  $\omega_n$ 。(b) 計算所需的  $K_p$  與  $K_d$ 。(c) 驗證設計。(d) 若該關節的最低結構共振頻率為 25 Hz，判斷此設計是否安全。

解 (a)：由超越量決定  $\zeta$ 。由 16.6.1 節的公式，要求  $M_p \leq 5\%$ ：

$$\exp(-\pi\zeta/\sqrt{1-\zeta^2}) \leq 0.05 \Rightarrow \pi\zeta/\sqrt{1-\zeta^2} \geq 3.0 \Rightarrow \zeta \geq 0.690$$

取常用值  $\zeta = 0.707$  (對應超越量 4.3% · 滿足規格)。

由整定時間決定  $\omega_n$ ：

$$t_s \approx 4/(\zeta\omega_n) \leq 0.3 \Rightarrow \omega_n \geq 4/(0.707 \times 0.3) = 18.86 \text{ rad/s}$$

取  $\omega_n = 20 \text{ rad/s}$  (留一些餘裕)。

解 (b)：由 16.6.2 節的公式：

$$K_p = m \omega_n^2 = 0.8 \times 400 = 320 \text{ N}\cdot\text{m/rad}$$

比例增益。

$$K_d = 2 \zeta \omega_n m = 2 \times 0.707 \times 20 \times 0.8 = 22.6 \text{ N}\cdot\text{m}\cdot\text{s/rad}$$

微分增益。

解 (c)：驗證  $\omega_n = \sqrt{(320/0.8)} = \sqrt{400} = 20 \checkmark$ 、 $\zeta = 22.6/(2\sqrt{(320 \times 0.8)}) = 22.6/(2 \times 16) = 0.707 \checkmark$ 。預期超越量  $4.3\% < 5\% \checkmark$ 、整定時間  $4/(0.707 \times 20) = 0.283 \text{ s} < 0.3 \text{ s} \checkmark$ 。設計符合所有規格。

解 (d)：控制器頻寬  $\omega_n = 20 \text{ rad/s} = 3.18 \text{ Hz}$ 。結構共振頻率為  $25 \text{ Hz}$ 。比值為  $25/3.18 \approx 7.9$  倍。

依 16.6.3 節的黃金法則 (頻寬應小於共振頻率的  $1/3$  到  $1/5$ ，即比值至少  $3 \sim 5$ )，本設計的比值  $7.9$  相當安全，還有提高頻寬的空間。若要進一步縮短整定時間，可把  $\omega_n$  提高到約  $30 \text{ rad/s}$  (比值仍有  $5.2$ )，此時  $t_s \approx 0.19$  秒。

**工程判斷：**這個餘裕分析是控制器設計中最容易被忽略卻最重要的一步。許多「調不好」的系統，實際上是因為要求的頻寬已經逼近結構共振——此時無論怎麼調參數都會振盪，唯一的解方是提高機構剛性或降低性能要求。

### 例題 16-3 追隨誤差與前饋的效益

題目：承例題 16-2 的關節 ( $m = 0.8$ 、 $K_p = 320$ 、 $K_d = 22.6$ )。現要追隨一段等加速度軌跡，加速度為  $\ddot{x}_d = 5 \text{ rad/s}^2$ 。(a) 求純 PD 控制下的追隨誤差。(b) 若加入加速度前饋且質量估計準確，求誤差。(c) 若質量估計有 20% 誤差 (估為  $0.64$ )，求誤差。(d) 比較三者並說明結論。

解 (a)：純 PD 控制時，穩態下系統以固定加速度運動，所需的力矩為  $m \cdot \ddot{x}_d$ ，這個力矩必須由  $K_p \cdot e$  提供（穩態時  $\dot{e} \approx 0$ ）：

$$K_p \cdot e_{ss} = m \ddot{x}_d \Rightarrow e_{ss} = m \ddot{x}_d / K_p = (0.8 \times 5) / 320 = 4.0 / 320 = 0.0125 \text{ rad}$$

追隨誤差約  $0.0125 \text{ rad} \approx 0.716^\circ$ 。

若關節後接一支 0.5 m 的連桿，末端誤差約為  $0.0125 \times 0.5 = 6.25 \text{ mm}$ ——這對精密作業是不可接受的。

解 (b)：加入前饋  $u_{ff} = m \ddot{x}_d = 0.8 \times 5 = 4.0 \text{ N}\cdot\text{m}$ 。此力矩恰好提供全部所需，回授項不需出力：

$$e_{ss} = 0 \text{ rad}$$

誤差理論上為零。這是完美模型下的理想結果。

解 (c)：質量估計為  $\hat{m} = 0.64$ （低估 20%），前饋提供  $0.64 \times 5 = 3.2 \text{ N}\cdot\text{m}$ ，尚缺  $4.0 - 3.2 = 0.8 \text{ N}\cdot\text{m}$ ，由回授補足：

$$e_{ss} = 0.8 / 320 = 0.0025 \text{ rad} \approx 0.143^\circ$$

誤差降為純回授的  $1/5$ ，末端誤差約  $1.25 \text{ mm}$ 。

解 (d)：三者比較如下表：

表 16-5 三種控制方式的追隨誤差比較

| 控制方式          | 前饋力矩    | 回授需補足   | 追隨誤差       | 末端誤差 (0.5 m 臂) |
|---------------|---------|---------|------------|----------------|
| 純 PD          | 0 N·m   | 4.0 N·m | 0.0125 rad | 6.25 mm        |
| 前饋 (模型準確)     | 4.0 N·m | 0 N·m   | 0 rad      | 0 mm           |
| 前饋 (模型誤差 20%) | 3.2 N·m | 0.8 N·m | 0.0025 rad | 1.25 mm        |

結論：這個例題量化了 16.9.2 節的核心論點——誤差的改善倍數，恰等於模型的準確度倍數。模型準確度 80%（誤差 20%），誤差就降為  $1/5$ 。這也說明了為何「不完美的模型仍然極有價值」：即使模型只有 80% 準確，效果也遠勝於把增益調高五倍（後者會遭遇 16.6.3 節的三重限制）。這正是第 17 章的核心思想。

## 程式範例：PID 控制器與階躍響應分析

以下 Python 程式實作完整的 PID 控制器（含防飽和與微分濾波），在一維點質量與平面 2R 手臂上測試，並比較不同增益、有無前饋的效果。

```
import numpy as np

# ===== PID 控制器 =====
class PID:
    """含防積分飽和與微分濾波的 PID 控制器"""
    def __init__(self, kp, ki=0.0, kd=0.0, dt=0.001,
                 u_max=np.inf, d_filter=0.0, derivative_on_output=True):
        self.kp, self.ki, self.kd, self.dt = kp, ki, kd, dt
        self.u_max = u_max
        self.alpha = d_filter          # 微分濾波係數 (0=不濾波)
        self.d_on_out = derivative_on_output  # 對輸出微分，避免 derivative kick
        self.integral = 0.0
        self.prev_e = None
        self.prev_y = None
        self.d_term = 0.0

    def reset(self):
        self.integral = 0.0
        self.prev_e = self.prev_y = None
        self.d_term = 0.0

    def update(self, r, y, ydot=None, u_ff=0.0):
        e = r - y
        # ----- 比例項 -----
        p_term = self.kp * e
        # ----- 微分項 -----
        if ydot is not None:          # 有速度量測則直接使用 (最佳)
            raw_d = -self.kd * ydot if self.d_on_out else 0.0
        elif self.d_on_out and self.prev_y is not None:
            raw_d = -self.kd * (y - self.prev_y) / self.dt
        elif self.prev_e is not None:
            raw_d = self.kd * (e - self.prev_e) / self.dt
        else:
            raw_d = 0.0
        self.d_term = self.alpha*self.d_term + (1-self.alpha)*raw_d
        # ----- 積分項 (含條件積分防飽和) -----
        i_term = self.ki * self.integral
        u_unsat = u_ff + p_term + i_term + self.d_term
        u = np.clip(u_unsat, -self.u_max, self.u_max)
        if abs(u_unsat - u) < 1e-9:    # 未飽和才累積積分
            self.integral += e * self.dt
        self.prev_e, self.prev_y = e, y
        return u

# ===== 一維點質量模擬 =====
def simulate_1d(kp, kd, ki=0.0, m=0.8, T=1.0, dt=0.001,
               target=1.0, disturbance=0.0, u_max=np.inf):
```

```

pid = PID(kp, ki, kd, dt, u_max=u_max)
x, v = 0.0, 0.0
ts, xs, us = [], [], []
for k in range(int(T/dt)):
    u = pid.update(target, x, ydot=v)
    a = (u + disturbance) / m          # 含外部干擾
    v += a*dt; x += v*dt              # 半隱式歐拉 (第 14 章)
    ts.append(k*dt); xs.append(x); us.append(u)
return np.array(ts), np.array(xs), np.array(us)

def analyze(ts, xs, target=1.0):
    """計算時域規格：超越量、整定時間、穩態誤差"""
    Mp = (xs.max() - target)/target*100 if xs.max() > target else 0.0
    band = 0.02*target
    idx = np.where(np.abs(xs - target) > band)[0]
    ts_settle = ts[idx[-1]] if len(idx) else 0.0
    return Mp, ts_settle, target - xs[-1]

# ----- 驗證：例題 16-1 與 16-2 -----
print("=== 例題 16-1：不同阻尼比的響應 ===")
m = 0.5
print(f"{'K_p':>6}{ 'K_d':>7}{ 'wn':>7}{ 'ζ':>7}{ '超越量':>9}{ '整定時間':>10}")
for kp, kd in ((200, 20), (200, 10), (200, 30), (400, 20)):
    wn = np.sqrt(kp/m); zeta = kd/(2*np.sqrt(kp*m))
    ts, xs, _ = simulate_1d(kp, kd, m=m, T=1.0)
    Mp, tset, _ = analyze(ts, xs)
    print(f"{kp:6.0f}{kd:7.1f}{wn:7.1f}{zeta:7.3f}{Mp:8.1f}%{tset:9.3f}s")

print("\n=== 例題 16-2：由規格設計增益 ===")
m2, zeta_d, wn_d = 0.8, 0.707, 20.0
kp = m2*wn_d**2; kd = 2*zeta_d*wn_d*m2
print(f"設計：ζ={zeta_d}, wn={wn_d} → K_p={kp:.1f}, K_d={kd:.2f}")
ts, xs, _ = simulate_1d(kp, kd, m=m2, T=0.8)
Mp, tset, _ = analyze(ts, xs)
print(f"實測：超越量={Mp:.2f}% (規格 <5%) 整定時間={tset:.3f}s (規格 <0.3s)")
print(f"結構共振 25 Hz vs 控制頻寬 {wn_d/(2*np.pi):.2f} Hz "
      f"→ 比值 {25/(wn_d/(2*np.pi)):.1f} 倍 (安全) ")

# ----- 穩態誤差與積分項 -----
print("\n=== 積分項消除穩態誤差 (持續干擾 -3 N) ===")
for ki, tag in ((0.0, "PD "), (300.0, "PID ")):
    ts, xs, _ = simulate_1d(kp, kd, ki, m=m2, T=2.0, disturbance=-3.0)
    _, _, ess = analyze(ts, xs)
    print(f" {tag}: 穩態誤差 = {ess:8.5f} "
          f" (理論 PD 值 = {3.0/kp:.5f}) ")

# ----- 積分飽和示範 -----
print("\n=== 積分飽和防護的效果 (力矩上限 5 N·m) ===")
for prot, tag in ((True, "有防護"), (False, "無防護")):
    pid = PID(kp, 300.0, kd, 0.001, u_max=5.0)
    x, v, xs2 = 0.0, 0.0, []

```



```

for k in range(3000):
    e = 1.0 - x
    p_t, d_t = pid.kp*e, -pid.kd*v
    i_t = pid.ki*pid.integral
    u_un = p_t + i_t + d_t
    u = np.clip(u_un, -5.0, 5.0)
    if prot and abs(u_un - u) > 1e-9:
        pass # 飽和時暫停積分
    else:
        pid.integral += e*0.001
        a = u/m2; v += a*0.001; x += v*0.001
        xs2.append(x)
xs2 = np.array(xs2)
print(f" {tag}: 最大超越 = {(xs2.max()-1.0)*100:6.2f}% "
      f"最終值 = {xs2[-1]:.4f}")

# ----- 例題 16-3：前饋的效益 -----
print("\n=== 例題 16-3：前饋控制的追隨誤差 ===")
acc = 5.0
for m_hat, tag in ((0.0, "純 PD"),
                  (0.8, "前饋(模型準確)"),
                  (0.64, "前饋(誤差 20%)")):
    pid = PID(kp, 0.0, kd, 0.001)
    x, v = 0.0, 0.0
    for k in range(1500):
        t = k*0.001
        xd, vd = 0.5*acc*t**2, acc*t # 等加速度目標
        u = pid.update(xd, x, ydot=v, u_ff=m_hat*acc)
        a = u/m2; v += a*0.001; x += v*0.001
        err = 0.5*acc*1.5**2 - x
    print(f" {tag}: 追隨誤差 = {err:9.6f} rad "
          f"= {np.rad2deg(err):7.4f}° 末端(0.5m) = {err*500:7.3f} mm")

```

請試著修改：（一）把  $K_d$  設為 0 觀察純 P 控制的等幅振盪。（二）在微分項加入模擬的感測雜訊（在  $y$  上加高斯雜訊），比較有無微分濾波的控制量抖動程度。（三）把質量  $m$  改為隨時間變化（模擬機器人姿態改變導致的慣量變化），觀察固定增益的性能如何劣化——這將說明第 17 章增益排程的必要性。

## 實作：實體關節的 PID 調參

本章實作是在實體平台上完成一次完整的 PID 調參，並量化各項的作用。建議使用附錄 F 的循跡車馬達、或任何具備編碼器回授的關節。

- 系統辨識**：先量測系統的等效慣量。作法是施加已知的階躍力矩，記錄角加速度，由  $\tau = J\alpha$  反推  $J$ 。重複三次取平均。

2. **理論設計**：依 16.6.2 節的流程，設定規格（建議：超越量  $< 10\%$ 、整定時間  $< 0.5\text{ s}$ ），計算理論增益  $K_p$  與  $K_d$ 。
3. **純 P 測試**：只設  $K_p$ （用理論值），觀察階躍響應。記錄振盪頻率並與理論值  $\omega_n = \sqrt{(K_p/J)}$  比較。這一步驗證你的慣量估計是否正確。
4. **加入  $K_d$** ：由 0 逐步增加到理論值，記錄每一步的超越量與整定時間。找出實際的臨界阻尼點，並與理論值比較（差異來自摩擦提供的額外阻尼）。
5. **雜訊上限探索**：繼續增大  $K_d$  直到馬達出現可聽見的嗡嗡聲或高頻抖動。記錄此臨界值——這就是雜訊放大的實際上限（16.6.3 節限制二）。
6. **穩態誤差量測**：在系統上施加一個固定的外力（如掛砝碼或用手輕壓），量測穩態誤差，驗證  $e_{ss} = d/K_p$  的關係。
7. **加入  $K_i$** ：由小值逐步增加直到穩態誤差在 1 秒內消除。觀察超越量的增加幅度。
8. **積分飽和實驗**：在系統啟動前先用手擋住不讓它動 3 秒，再放開。分別測試有無防飽和保護的行為差異——你會親眼看到 16.5.3 節描述的暴衝現象。
9. **前饋效益驗證**：規劃一段等加速度軌跡（第 15 章），分別以純 PID 與加入加速度前饋執行，比較追隨誤差。驗證例題 16-3 的結論。
10. **共振頻率探索**（選做，須謹慎）：在低增益下用手快速撥動連桿末端，用加速度計或觀察振動記錄其自然振動頻率。這就是結構共振頻率，據以檢驗你的控制頻寬是否安全。

表 16-6 PID 調參記錄表（前兩列為範例）

| 步驟             | $K_p$ | $K_d$ | $K_i$ | 超越量  | 整定時間   | 穩態誤差      | 觀察                     |
|----------------|-------|-------|-------|------|--------|-----------|------------------------|
| 純 P（理論值）       | 320   | 0     | 0     | 持續振盪 | —      | 0.012 rad | 頻率 3.2 Hz，接近理論 3.18 Hz |
| 加入 $K_d$ （理論值） | 320   | 22.6  | 0     | 5.8% | 0.31 s | 0.012 rad | 略高於理論，摩擦影響             |
| （自行填寫其餘步驟）     |       |       |       |      |        |           |                        |

## 國中小也看得懂：在手上立竹竿的三個祕訣

玩過在手掌上立一根竹竿的遊戲嗎？如果閉上眼睛，竿子一秒就倒了；張開眼睛，你就能一

直讓它立著。為什麼？因為你**看得到它往哪邊倒，就往哪邊移動手**。這個「看→修正」的循環，就是機器人控制最重要的道理。

但光是「往倒的方向移動」還不夠。有三件事要一起做，工程師叫它們 **P、I、D**：

**P 是「現在差多少」**。竿子歪得越多，你就要移動得越多。這很直覺對不對？可是只做這件事的話，竿子會左右晃個不停——因為你總是等它歪了才反應，扳回來的時候又衝過頭了。

**D 是「正在多快地變糟」**。高手會看竿子倒的速度：雖然現在只歪一點點，但倒得很快，就要趕快修！這就像騎腳踏車時的煞車——不是等快撞到才煞，而是看到前面有狀況就先減速。有了 D，晃動就會停下來。

**I 是「已經歪了多久」**。如果竿子上黏了一塊黏土讓它老是往左倒，你會發現不管怎樣它總是偏左一點點。這時候要記住：「我已經偏左很久了，要再多用一點力！」把過去累積的偏差加起來一起修正，就能真正回到正中央。

記住這句話就好：**P 看現在、D 看未來、I 看過去**。你家的冷氣、電梯、汽車的定速巡航，還有工廠裡所有的機器手臂，通通都在用這三個字母控制。你在附錄 F 做的循跡車，如果只用 P 會蛇行，加上 D 就會變得又直又穩——不信你試試看！

## 叫獸提醒與常見錯誤

- 誤區一：以為調高  $K_p$  就能提高精度。增益受雜訊放大、致動器飽和與結構共振三重限制，超過就會振盪或失控。
- 誤區二：調  $K_p$  卻不同步調  $K_d$ 。阻尼比  $\zeta = K_d / (2\sqrt{K_p m})$ ， $K_p$  加倍時  $K_d$  需乘以  $\sqrt{2}$  才能維持相同阻尼。
- 誤區三：忘記積分飽和防護。這是「平常都好、卡住之後暴衝」這類故障的第一嫌疑。
- 誤區四：對誤差微分而非對輸出微分。目標值突變時會產生 derivative kick，使控制量出現巨大脈衝。
- 誤區五：微分項未濾波。感測器雜訊被除以取樣週期（放大  $1/T$  倍）再乘上  $K_d$ ，會使馬達發熱與異音。
- 誤區六：取樣頻率不足。應至少為閉環頻寬的 10~20 倍，否則延遲會嚴重侵蝕穩定裕度。
- 誤區七：用同一組增益走遍所有姿態。機器人的等效慣量隨姿態變化可達數倍，應考慮增益

排程。

- 誤區八：用積分項去對抗重力。重力是可預測的，應以前饋直接補償；讓積分處理它會導致響應遲緩與超越。
- 誤區九：以為定點精度就是動態精度。純回授必然有追隨誤差，且與速度成正比，兩者可差一到兩個數量級。
- 誤區十：忽略結構共振。控制頻寬應低於最低共振頻率的  $1/3 \sim 1/5$ ；若已逼近，任何調參都無法解決，只能提高機構剛性。
- 誤區十一：直接套用 Ziegler-Nichols 的結果。它針對干擾抑制設計，超越量可達 25%，對機器人往往過於激進，應作為起點而非終點。
- 誤區十二：在實體機器人上求臨界增益。讓系統實際振盪有損壞風險，應在低速、無負載、有防護的條件下進行，或改用模型計算。

## 課堂討論

1. 開頭故事中，小翰蒙眼、只看位置、加上配重三種情境，分別對應 PID 的哪個面向？請為每種情境設計一個機器人上的對應實例。
2. PD 控制等效於一個虛擬的彈簧—阻尼系統。請討論這個觀點如何幫助我們設計「與人接觸時安全」的機器人（提示：對照第 20 章的阻抗控制與第 5 章的反向驅動性）。
3. 第 13 章的故事中，小好想調高增益解決高速失準。請用本章 16.6.3 節與 16.10.2 節的內容，完整說明為何這個方向行不通，並指出正確的解方。
4. 例題 16-3 顯示「模型準確度 80% 就能使誤差降為  $1/5$ 」。請討論：若模型準確度只有 50%，效益如何？在什麼情況下建立模型的成本會超過其效益？
5. 增益排程依姿態調整增益，使  $\omega_n$  與  $\zeta$  維持恆定。請討論這個作法與第 17 章「用動力學模型線性化」的關係——兩者是互補還是替代？

## 章末摘要

- 開迴路不用回授，簡單但無法修正誤差；閉迴路以量測誤差修正，帶來抗干擾、降低模型依賴與改變系統動態三個價值。
- 比例控制等效於彈簧： $u = K_p e$  使閉環成為簡諧振盪，永不衰減，且在持續干擾下留下穩

態誤差  $e_{ss} = d/K_p$ 。

- 微分控制等效於阻尼器；PD 控制的閉環方程  $m\ddot{e} + K_d \dot{e} + K_p e = 0$  是標準的質量—阻尼—彈簧系統。
- 由增益求系統參數： $\omega_n = \sqrt{(K_p/m)}$ 、 $\zeta = K_d/(2\sqrt{(K_p m)})$ ；調  $K_p$  時須同步調  $K_d$  以維持阻尼比。
- 阻尼比決定響應類型： $\zeta < 1$  欠阻尼（有超越）、 $\zeta = 1$  臨界阻尼（不振盪的最快響應）、 $\zeta > 1$  過阻尼；實務常用  $\zeta = 0.707$ （超越 4.3%）。
- 積分項消除穩態誤差，因其「記住」了抵消干擾所需的力；代價是降低穩定性、增加超越量、反應遲滯。
- P 看現在、I 看過去、D 看未來——三者互補但互相牽制。
- 積分飽和發生在控制量飽和時，防護方法有條件積分、限幅與反飽和回授；症狀是「異常之後更糟」。
- 設計流程：由超越量規格定  $\zeta$ 、由整定時間規格定  $\omega_n$ 、再計算  $K_p = m\omega_n^2$ 、 $K_d = 2\zeta\omega_n m$ 。
- 增益有三重上限：雜訊放大（限制  $K_d$ ）、致動器飽和、以及最根本的結構共振（頻寬應低於共振頻率的  $1/3 \sim 1/5$ ）。
- 數位實作三議題：微分放大雜訊（需濾波或用速度量測）、積分近似、取樣延遲；取樣頻率應為閉環頻寬的  $10 \sim 20$  倍。
- 微分應作用於輸出而非誤差，以避免目標突變時的 derivative kick。
- 純回授在軌跡追隨時必然有誤差  $e_{ss} \approx m \ddot{x}_d / K_p$ ，因為回授必須先有誤差才會出力。
- 前饋依模型預先計算所需的力： $u = u_{ff} + u_{fb}$ ，核心思想是「用模型處理已知的部分、用回授處理未知的部分」。
- 誤差的改善倍數約等於模型的準確度倍數——模型 80% 準確，誤差降為  $1/5$ 。
- PID 在機器人上的四個結構性限制：非線性、耦合、追隨誤差、增益上限；解方是引入動力學模型（第 17 章）。

## 觀念題與計算題

### 觀念題

1. 比較開迴路與閉迴路控制，並列舉回授的三個價值與三個代價。
2. 說明比例控制為何等效於彈簧，以及為何純 P 控制會產生等幅振盪。
3. 推導純 P 控制在持續干擾下的穩態誤差公式，並說明為何不能靠調高  $K_p$  消除。
4. 寫出 PD 控制的閉環方程，並說明它與機械系統的對應關係。
5. 寫出由增益求  $\omega_n$  與  $\zeta$  的公式，並說明調整  $K_p$  時  $K_d$  應如何配合。
6. 比較欠阻尼、臨界阻尼與過阻尼三種響應，並說明為何實務常用  $\zeta = 0.707$  而非  $\zeta = 1$ 。
7. 說明積分項消除穩態誤差的機制，以及它帶來的三個代價。
8. 解釋積分飽和的成因與典型症狀，並列舉三種防護方法。
9. 說明由時域規格設計增益的三個步驟。
10. 列舉限制增益的三個因素，並說明何者最為根本。
11. 說明數位實作的三個議題，以及微分項雜訊的三種對策。
12. 為何微分應作用於輸出而非誤差？這避免了什麼問題？
13. 說明為何純回授在軌跡追隨時必然有誤差，並寫出其近似公式。
14. 說明前饋的原理與「用模型處理已知、用回授處理未知」的思想。
15. 列舉 PID 在機器人應用上的四個結構性限制。

## 計算題

1. 某關節等效慣量  $m = 1.2 \text{ kg}\cdot\text{m}^2$ ，設定  $K_p = 300 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 36 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。(a) 求  $\omega_n$  與  $\zeta$ 。(b) 判斷響應類型並估算超越量。(c) 求整定時間。(d) 若要達到臨界阻尼， $K_d$  應改為多少？(e) 若改為  $\zeta = 0.707$ ， $K_d$  應為多少？
2. 某關節  $m = 0.6 \text{ kg}\cdot\text{m}^2$ 。規格要求超越量不超過 10%、整定時間不超過 0.25 秒。(a) 由超越量決定  $\zeta$  (提示： $M_p = 10\%$  對應  $\zeta \approx 0.591$ )。(b) 由整定時間決定  $\omega_n$ 。(c) 計算  $K_p$  與  $K_d$ 。(d) 若該關節的結構共振頻率為 18 Hz，判斷此設計是否安全。(e) 若不安全，應如何修改規格？
3. 承上題設計的控制器。系統受到  $2.5 \text{ N}\cdot\text{m}$  的持續干擾 (如重力)。(a) 求純 PD 的穩態誤差。(b) 若連桿長 0.4 m，求末端誤差。(c) 若加入積分項且要求誤差在 0.5 秒內消除，估算所需的  $K_i$  (提示：可由  $K_i \approx \omega_n^2 m / T_i$  估算， $T_i$  取 0.5 秒)。(d) 改用重力前饋補償，說明其相對於積分項的三個優點。

4. 某關節  $m = 0.9 \text{ kg}\cdot\text{m}^2$ 、 $K_p = 400$ 、 $K_d = 30$ ，追隨一段軌跡，其最大加速度為  $8 \text{ rad/s}^2$ 、最大速度為  $3 \text{ rad/s}$ 。(a) 求純 PD 在最大加速度處的追隨誤差。(b) 若質量估計為  $0.75$  (誤差  $16.7\%$ )，加入加速度前饋後的誤差為何？(c) 計算誤差改善的倍數。(d) 若連桿長  $0.6 \text{ m}$ ，比較兩者的末端誤差，並判斷是否滿足  $\pm 1 \text{ mm}$  的精度要求。

## 動手做與專題延伸

### 本章實作：PID 控制器模組

延續前面章節的程式庫，加入控制模組。這將與第 13~15 章的動力學與軌跡模組整合成完整的模擬平台。

- **階段一：**實作完整的 PID 類別，含條件積分防飽和、微分濾波、對輸出微分三項防護。
- **階段二：**實作階躍響應分析函式，自動計算超越量、上升時間、整定時間與穩態誤差。
- **階段三：**整合第 13 章的動力學模擬，在平面 2R 手臂上測試獨立關節 PID，觀察耦合造成的誤差。
- **階段四：**加入第 15 章的軌跡生成，測試軌跡追隨，並比較有無加速度前饋的效果。
- **階段五：**實作增益排程——依姿態計算等效慣量  $M(q)$  的對角元素，動態調整增益使  $\omega_n$  與  $\zeta$  維持恆定。比較固定增益與排程增益的性能差異。

### 專題延伸

- **參數空間地圖：**對一維點質量系統，在  $(K_p, K_d)$  平面上掃描數百組增益組合，計算每組的超越量與整定時間，繪成兩張熱力圖。在圖上標出  $\zeta = 0.707$  與  $\zeta = 1$  的等高線，以及「超越量  $< 5\%$  且整定時間  $< 0.3 \text{ s}$ 」的可行區域。這張圖能讓抽象的設計公式變成直觀的地圖。
- **雜訊與增益的取舍實驗：**在模擬的感測器讀值上加入不同強度的高斯雜訊，對每種雜訊強度找出「控制量抖動不超過額定  $10\%$ 」所允許的最大  $K_d$ 。繪出「雜訊強度 vs 最大可用  $K_d$ 」曲線，量化 16.6.3 節的限制二。
- **進階：耦合效應的量化研究。**在平面 2R 手臂上，讓第一關節維持定點 (PID 控制)，第二關節執行不同速度的正弦運動。記錄第一關節的位置誤差隨第二關節速度的變化，繪出曲線。你應該會看到誤差隨速度平方成長——這正是第 13 章科氏力的直接證據，也是第 17 章計算力矩控制必要性的實驗依據。



## 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson ( 第 9 章：操作臂的線性控制，本章主要參考 ) 。
- Åström, K. J. and Murray, R. M., Feedback Systems: An Introduction for Scientists and Engineers, 2nd ed., Princeton University Press ( 回授控制的現代入門經典，線上免費 ) 。
- Åström, K. J. and Hägglund, T., PID Controllers: Theory, Design, and Tuning, 2nd ed., ISA ( PID 的權威專著，含各類調參方法與防飽和技術 ) 。
- Ogata, K., Modern Control Engineering, 5th ed., Pearson ( 控制理論的標準教科書，時域規格與二階系統分析 ) 。
- Ziegler, J. G. and Nichols, N. B., "Optimum Settings for Automatic Controllers," ASME Transactions, 1942 ( 經驗調參法的原始論文 ) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley ( 第 6 章：獨立關節控制 ) 。
- Franklin, G. F., Powell, J. D. and Emami-Naeini, A., Feedback Control of Dynamic Systems, 8th ed., Pearson ( 含數位控制與離散化的完整處理 ) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer ( 第 8 章：運動控制，含前饋與增益排程 ) 。
- 各伺服驅動器廠商技術手冊：台達、三菱、安川等廠商的伺服調機指南，含實務的調參流程與共振抑制濾波器設定。
- MATLAB Control System Toolbox 與 Python 的 python-control 套件文件：控制系統設計與分析的實作工具。



# 第 17 章

## 操作臂非線性控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩張並列的追隨誤差曲線圖：上圖標註「固定增益  $PID$ 」，誤差隨姿態改變而劇烈起伏，加上負載後整體抬升；下圖標註「計算力矩控制」，誤差維持在一條平坦的低線，換姿態與加負載後幾乎不變。右半為一位工程師用單手輕推六軸手臂末端、手臂順著推力平順移動的照片，旁註「零力控制： $\tau = G(q)$ 」。中央以方塊圖串起兩層架構——內層「 $M(q)u + Cq + G$ 」抵消非線性、外層「 $\ddot{q}_d + K_d\dot{e} + K_p e$ 」做線性控制。標題文字：讓末端表現得像一顆一維點質量。

## 叫獸開講 「機器手臂搬起重物後，原本調好的控制器為什麼失準？」

小好把第 13 章那台手臂又推了出來。她花了三天，用 Ziegler-Nichols 方法把六個關節的 PID 都調到最佳——在額定速度的三成下，追隨誤差只有 0.08 公釐，漂亮極了。

「現在，」叫獸從架上拿下一個五公斤的配重塊，裝到末端法蘭上，「重跑同一段軌跡。」

手臂啟動，第一個彎道就明顯下墜，示教器上的追隨誤差跳到 2.4 公釐。小好皺眉：「那我重調一次增益。」

「可以。」叫獸點頭，「但我還有第二個問題。」他把配重塊拆掉，改把手臂的作業區從機器人的正前方移到側前方四十五度。同樣的軌跡、同樣的增益、沒有負載——誤差還是變成了 1.6 公釐。

小好愣住了：「連姿態換了都不行？」

「因為你調的那組增益，只對『某一個特定姿態、某一種特定負載』是最佳的。」叫獸在白板上寫下第 13 章的方程式： $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$ 。「你看這個  $M$ ——它是  $q$  的函數。手臂伸直時和折疊時，同一個關節的等效慣量可能差三倍。你的 PID 增益是固定的，但系統本身在變。」

「這就像，」他想了想，「你在學開車，教練幫你調好了方向盤的靈敏度。但這台車有時候是空車、有時候載滿貨、有時候在柏油路、有時候在冰上——同一組設定不可能都好用。」

小好問：「那怎麼辦？每個姿態各調一組增益？」

「那叫增益排程，確實有人這樣做，但六個關節乘上無限多種姿態，你調不完。」叫獸搖頭，「更好的辦法是——既然我們已經有動力學模型了，為什麼不直接用它？」

他在白板上畫了一個方塊圖：「讓控制器先用逆動力學算出『這個姿態、這個加速度，理論上需要多少力矩』，把它直接送出去。這叫前饋。剩下的模型誤差，才交給 PID 修正。」

「這樣做有一個很美的結果。」叫獸在方塊圖旁邊寫下一行字：讓末端表現得像一顆一維點質量。「第 16 章我們分析的那個最簡單的模型  $m\ddot{x} = f$ ，其實可以被『製造』出來——只要控制器把非線性的部分全部抵消掉，剩下的系統在數學上就真的變成了一顆點質量。而點質量的控制，我們上一章已經完全解決了。」

## 本章為什麼重要

第 16 章的 PID 建立在一個假設上：系統是線性、非時變的。但第 13 章告訴我們，機器人完全不是這樣——慣性矩陣隨姿態變化、科氏力與速度平方成正比、重力隨姿態改變、關節之間互相耦合。PID 之所以還能用，是因為它把這些通通當成「干擾」來被動修正；而干擾越大（高速、重載），修正就越跟不上。

本章的核心思想是**主動出擊**：既然我們已經在第 13、14 章建立了動力學模型，就應該用它預先算出所需的力矩，而不是等誤差出現才補救。這個轉變帶來三個層次的成果：**計算力矩控制**把非線性系統精確線性化、**操作空間控制**讓我們直接在任務空間設計行為、而**重力補償**則是其中最簡單卻最實用的一項——許多協作機器人的「手動導引」功能就只靠它。

本章也是全書「模型」與「控制」交會的地方。第 9~12 章的運動學、第 13~14 章的動力學，都將在這裡被真正地使用起來。

## 學習目標

- 能說明機器人系統的四個非線性來源，並解釋固定增益 PID 為何不足。
- 能推導計算力矩控制律，並說明其如何達成回授線性化。
- 能說明計算力矩控制的兩個層次：內層線性化與外層線性控制。
- 能實作重力補償，並說明它在手動導引與零力示教中的角色。
- 能說明逆動力學控制與計算力矩控制的關係。
- 能說明模型誤差對回授線性化的影響，並估算殘餘誤差。
- 能比較關節空間控制與操作空間控制的適用場合。
- 能推導操作空間的等效慣量  $\Lambda$  並說明其物理意義。
- 能說明滑模控制、自適應控制與魯棒控制的基本思想與適用時機。
- 能說明模型預測控制的基本架構與其相對於 PID 的優勢。
- 能以 Python 實作計算力矩控制並與 PID 比較追隨性能。

## 先備知識

第 13 章的動力學標準形式與各項的物理意義、第 14 章的 Newton-Euler 遞迴（實作前饋需

要它的計算效率)、第 16 章的 PD / PID 控制與二階系統分析、第 11 章的雅可比矩陣(操作空間控制的核心工具)、第 15 章的軌跡生成(提供  $\mathbf{q}_d$ 、 $\dot{\mathbf{q}}_d$ 、 $\ddot{\mathbf{q}}_d$ )。

本章是全書控制部分的關鍵轉折。建議讀法：務必先徹底理解 17.2 節的計算力矩控制——它是後續所有進階控制的基礎。若 17.7 節的操作空間控制推導感到吃力，可先跳過細節、掌握結論，之後回頭補。

## 核心名詞中英對照

表 17-1 本章核心名詞

| 中文     | 英文                                   | 說明               |
|--------|--------------------------------------|------------------|
| 計算力矩控制 | Computed Torque Control, CTC         | 以逆動力學抵消非線性的控制法   |
| 回授線性化  | Feedback Linearization               | 以非線性回授使閉環系統成為線性  |
| 逆動力學控制 | Inverse Dynamics Control             | 計算力矩控制的別稱        |
| 前饋     | Feedforward                          | 依模型預先計算的控制輸入     |
| 重力補償   | Gravity Compensation                 | 抵消重力所需的關節力矩      |
| 零力控制   | Zero-force Control                   | 使機器人可被輕易手動推動的模式  |
| 手動導引   | Hand Guiding / Lead-through          | 以手直接帶動機器人示教的方式   |
| 操作空間   | Operational Space                    | 以末端位姿描述的控制空間     |
| 操作空間慣量 | Operational Space Inertia, $\Lambda$ | 末端在任務空間中感受到的等效慣量 |
| 動態解耦   | Dynamic Decoupling                   | 使各方向的運動互不影響      |
| 滑模控制   | Sliding Mode Control                 | 以切換律強制系統沿滑動面運動   |
| 顫振     | Chattering                           | 滑模控制中高頻切換造成的振動   |
| 自適應控制  | Adaptive Control                     | 線上估測未知參數的控制法     |
| 魯棒控制   | Robust Control                       | 對模型不確定性維持性能的控制法  |
| 模型預測控制 | Model Predictive Control, MPC        | 以滾動時域最佳化求解控制輸入   |
| 滾動時域   | Receding Horizon                     | 每步重新求解未來一段時間的最佳解 |

## 17.1 非線性從何而來

### 17.1.1 四個非線性來源

回顧第 13 章的動力學方程，可以清楚辨識出四個非線性來源：

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + F(q) = \tau$$

機器人動力學標準形式。四項各自貢獻不同性質的非線性。

**來源一：慣性矩陣隨姿態變化。** $M(q)$  是  $q$  的函數。以第 13 章的平面 2R 為例， $M_{11} = \alpha + 2\beta\cos\theta_2$ ——當  $\theta_2$  由  $0^\circ$  變到  $180^\circ$ ， $M_{11}$  由 0.78 變到 0.30，相差 2.6 倍。這代表同樣的關節加速度，在不同姿態下需要差 2.6 倍的力矩。

**來源二：關節之間的耦合。** $M$  的非對角元素不為零，一個關節的加速度會對其他關節產生力矩（第 13 章例題 13-3 量化過：第一關節明明沒動卻要出 1.2 N·m）。對固定增益的獨立關節 PID 而言，這是無法預測的干擾。

**來源三：速度相關項與速度平方成正比。** $C(q,\dot{q})\dot{q}$  中的科氏力與離心力隨速度平方增長。速度加倍，這些力變成四倍——這正是低速調校完美、高速失準的根源。

**來源四：重力隨姿態變化。** $G(q)$  在水平伸展時最大、垂直時為零。PID 必須靠積分項慢慢累積來對抗它，導致響應遲鈍。

### 17.1.2 為何固定增益 PID 不足

第 16 章的分析建立在  $m\ddot{x} = f$  這個線性模型上，由此推出的自然頻率  $\omega_n = \sqrt{(K_p/m)}$  與阻尼比  $\zeta = K_d/(2\sqrt{(K_p \cdot m)})$ ，都假設  $m$  是常數。

但在機器人上，「 $m$ 」實際上是  $M(q)$ ——它會變。這意味著：

$$\omega_n(q) = \sqrt{(K_p/M(q))}, \quad \zeta(q) = K_d/(2\sqrt{(K_p \cdot M(q))})$$

有效的自然頻率與阻尼比隨姿態變化。同一組增益，在不同姿態下產生完全不同的動態行為。

具體而言：在**慣量小的姿態**（手臂折疊），有效  $\omega_n$  大、 $\zeta$  小——系統反應快但容易振盪；在**慣量大的姿態**（手臂伸展），有效  $\omega_n$  小、 $\zeta$  大——系統反應遲鈍。

這正是開頭故事的完整解釋。小好正在正前方調好的增益，換到側前方四十五度時， $M(q)$  改變了，於是阻尼比也變了——原本的臨界阻尼可能變成欠阻尼（振盪）或過阻尼（遲鈍）。加上五公斤負載後， $M$  更是整體放大，情況更嚴重。

#### ◎ 增益排程：一個治標的解法

面對  $M(q)$  的變化，一個直觀的作法是**增益排程**（gain scheduling）：預先為不同的姿態區域各調一組增益，執行時依當前姿態切換。這在單一自由度或姿態變化範圍小的系統上確實可行（如汽車

的變增益轉向助力)。但對六軸手臂而言，姿態空間是六維的，要涵蓋所有組合需要天文數字的參數，且切換時可能產生不連續。因此工業機器人幾乎都不採此路，而是直接使用本章的模型式方法。

## 17.2 計算力矩控制

### 17.2.1 核心理念：先抵消，再控制

計算力矩控制 (Computed Torque Control, CTC) 的想法極為優雅：既然我們知道非線性項是什麼，就用控制輸入把它們精確抵消掉。

具體作法是把控制力矩設計成兩部分：

$$\tau = M(q) \cdot u + C(q, \dot{q}) \dot{q} + G(q)$$

計算力矩控制律。後兩項抵消非線性， $u$  是待設計的「虛擬控制輸入」。

把它代入系統方程  $M \ddot{q} + C \dot{q} + G = \tau$ ：

$$M \ddot{q} + C \dot{q} + G = M u + C \dot{q} + G \quad \Rightarrow \quad M \ddot{q} = M u \quad \Rightarrow \quad \ddot{q} = u$$

驚人的結果：閉環系統變成了  $\ddot{q} = u$ 。

這個結果的意義極為重大。原本複雜的非線性耦合系統，經過這個控制律之後，變成了  $n$  個獨立的雙積分器——每個關節都是一個「加速度就是輸入」的簡單系統，彼此完全不耦合，且沒有任何非線性。

$$\ddot{q}_i = u_i \quad (i = 1, 2, \dots, n)$$

$n$  個獨立的雙積分器。這正是第 16 章的一維點質量模型 ( $m = 1$ )！

### 17.2.2 外層：線性控制器

既然系統已經變成雙積分器，第 16 章的所有分析就完全適用。設計  $u$  為 PD 形式：

$$u = \ddot{q}_d + K_d(\dot{q}_d - \dot{q}) + K_p(q_d - q)$$

外層控制律。第一項是前饋加速度，後兩項是 PD 回授。

代入  $\ddot{q} = u$  並整理，令誤差  $e = q_d - q$ ：

$$\ddot{e} + K_d \dot{e} + K_p e = 0$$

誤差動態方程。這是一個線性、非時變、且各關節完全解耦的二階系統。

這條式子的美妙之處在於：它不含  $M$ 、 $C$ 、 $G$  任何一項。無論機器人處於什麼姿態、以多快的速度運動、帶多重的負載，誤差的收斂行為都完全相同。這徹底解決了開頭故事的問題。

增益的選擇直接套用第 16 章的公式：

$$K_p = \omega_n^2, \quad K_d = 2\zeta\omega_n$$

由期望的自然頻率與阻尼比反推增益。通常取  $\zeta = 1$  (臨界阻尼) 以避免超越。

圖 17-1 計算力矩控制的兩層架構

建議配圖：方塊圖。左側輸入為軌跡規劃器輸出的  $q_d$ 、 $\dot{q}_d$ 、 $\ddot{q}_d$ 。外層 (藍色區塊) 為線性控制器： $\ddot{q}_d$  直接前饋，加上  $K_p$ 、 $K_d$  對誤差的回授，輸出虛擬控制  $u$ 。內層 (綠色區塊) 為線性化補償：以  $u$  乘上  $M(q)$ ，再加上  $C(q, \dot{q})\dot{q}$  與  $G(q)$ ，輸出實際力矩  $\tau$  送給機器人。機器人的  $q$ 、 $\dot{q}$  回授至兩層。圖下方標註：內層把非線性抵消成  $\ddot{q} = u$ ，外層再用第 16 章的 PD 設計。

### 17.2.3 為何叫「計算力矩」

這個名稱來自它的實作方式：控制器在每一個週期都要計算出「維持這個運動所需的力矩」——也就是第 14 章的逆動力學。這正是為何第 14 章的  $O(n)$  遞迴演算法如此重要：CTC 需要在每一毫秒完成一次完整的逆動力學計算。

值得注意的是，實作上通常不會分別計算  $M$ 、 $C$ 、 $G$  再組合，而是直接呼叫一次 Newton-Euler 遞迴，傳入  $(q, \dot{q}, u)$  作為輸入：

$$\tau = \text{InverseDynamics}(q, \dot{q}, u) \quad \text{其中 } u = \ddot{q}_d + K_d \dot{e} + K_p e$$

實作技巧。把  $u$  當作「期望加速度」傳入逆動力學，一次遞迴即可得到完整的  $\tau$ ，複雜度  $O(n)$ 。

## 17.3 回授線性化的觀點

### 17.3.1 更一般的框架

計算力矩控制是一個更廣泛的概念——回授線性化 (feedback linearization) ——在機器人上的特例。

回授線性化的一般想法是：對於一個非線性系統，若能找到一個非線性的回授控制律與座標變換，使閉環系統在新座標下成為線性系統，就能套用所有線性控制理論的工具。

機器人動力學特別適合這個方法，原因有二：一、 $M(q)$  恆為正定因此必可逆 (第 13.6.1 節的性質二)，這保證了  $\tau = M \cdot u + \dots$  這個變換永遠可行；二、控制輸入的維度等於自由度 (每個關節都有一個馬達)，這使系統是「完全驅動」的 (fully actuated)。

### 17.3.2 完全驅動與驅動不足

這裡有一個重要的分野。**完全驅動系統**（如固定基座的機械手臂）可以精確回授線性化；**驅動不足系統**（underactuated，如第 5.5 節提到的四足機器人騰空階段、兩輪自平衡車、四旋翼無人機）則不行——因為某些自由度沒有直接的控制輸入。

這解釋了為何本章的方法主要用於機械手臂，而人形機器人與無人機需要更複雜的控制策略（第 27、28 章）。對它們而言，「浮動基座」的六個自由度沒有馬達直接驅動，只能透過與地面或空氣的接觸間接控制。

### 17.3.3 與線性化的差異

必須釐清一個容易混淆的概念：回授線性化**不是**在某個工作點附近做泰勒展開的「近似線性化」。

**近似線性化**（第 19 章會用到）是在平衡點附近取一階近似，只在該點附近有效，離開就失準。**回授線性化**則是精確的：只要模型正確，在**整個狀態空間**中閉環系統都嚴格等於  $\ddot{q} = u$ ，沒有任何近似。

代價是：回授線性化需要**完整且準確的模型**。近似線性化只需要在一個點上的資訊。這個取捨，正是下一節要討論的。

## 17.4 模型誤差的影響

### 17.4.1 現實中的不完美

計算力矩控制的推導假設模型完全準確。但第 13.10.3 節已指出，實際的動力學模型有 10~20% 的誤差，主要來自慣性參數不準、摩擦難以建模、關節彈性與未建模的線材拉力。

設控制器使用的是估計模型  $\hat{M}$ 、 $\hat{C}$ 、 $\hat{G}$ ，實際系統為  $M$ 、 $C$ 、 $G$ 。控制律為：

$$\tau = \hat{M}u + \hat{C}\dot{q} + \hat{G}$$

使用估計模型的控制律。

代入真實系統  $M\ddot{q} + C\dot{q} + G = \tau$  並整理：

$$\ddot{q} = M^{-1}\hat{M}u + M^{-1}[(\hat{C} - C)\dot{q} + (\hat{G} - G)]$$

閉環方程。若模型完美（ $\hat{M} = M$  等），退化為  $\ddot{q} = u$ ；否則會有兩個偏差。



**偏差一：** $M^{-1}M \neq I$ 。這使有效增益偏離設計值。若  $M$  高估了 20%，有效的  $K_p$  與  $K_d$  都會被放大 1.2 倍——阻尼比不變（因為兩者同比例變化），但自然頻率上升 10%。這通常影響不大。

**偏差二：**括號中的殘差項。這是一個「等效干擾」，會造成穩態誤差與追隨誤差。它的大小正比於模型誤差與運動的劇烈程度。

### 17.4.2 為何仍然有效

關鍵的觀察是：即使模型有 20% 的誤差，CTC 仍然消除了 80% 的非線性。留給 PD 回授處理的殘差，只有原本的五分之一。

這帶來兩個好處。第一，**追隨誤差大幅降低**——干擾小了五倍，同樣的增益下誤差也小約五倍。第二，**增益可以調得更低**——因為不需要靠高增益去對抗大干擾，系統可以更「軟」，這對安全與抗雜訊都有利。

#### ◎ 模型式控制的正確心態

初學者常有一個誤解：認為模型式控制需要「精確」的模型才能用，因此在參數不確定時乾脆放棄。這是錯的。正確的心態是：**模型處理已知的部分，回授處理未知的部分**。一個 80% 準確的模型，就能讓回授的負擔降低五倍——這已經是巨大的改善。追求 99% 的模型精度往往不划算，因為剩下的 1% 由回授處理即可。第 13 章的結論在此再次適用：模型的價值不在完美，而在把大部分的需求「預測」出來。

### 17.4.3 實務上的折衷版本

完整的 CTC 需要即時計算  $M$ 、 $C$ 、 $G$  三項。在計算資源有限或模型不完整時，有兩個常用的簡化版本。

**版本一：只做重力補償**（17.5 節）。只計算  $G(q)$ ，忽略  $M$  與  $C$ 。這適用於中低速運動，因為此時重力往往佔總力矩的 70~90%（第 13 章例題 13-2 中佔 88.7%）。計算量極小，效果顯著。

**版本二：固定慣量近似**。用一個常數矩陣  $M$ （例如取工作區域中央姿態的  $M$  值）取代  $M(q)$ ，並忽略  $C$ 。這保留了「解耦」的好處，但省去了每週期重算  $M$  的成本。

這兩個版本在實務上極為常見——許多工業機器人控制器的預設模式，其實就是重力補償加上獨立關節 PID。完整的 CTC 通常只在高速或高精度需求時才啟用。

## 17.5 重力補償

### 17.5.1 最簡單也最實用的模型式控制

重力補償是計算力矩控制的極簡版本：只補償  $G(q)$ ，其餘交給 PD 回授。

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + G(q)$$

*PD 加重力補償。這是機器人控制中投資報酬率最高的一項改良。*

它的價值在於三件事。**第一，消除穩態誤差**。純 PD 控制在有重力時必定有穩態誤差——因為必須維持一個非零的誤差才能產生足以平衡重力的力矩：

$$\text{純 PD 的穩態誤差：} e_{ss} = G(q)/K_p$$

*誤差與重力成正比、與增益成反比。補償重力後， $e_{ss} = 0$ 。*

**第二，避免依賴積分項**。第 16 章提到積分項雖能消除穩態誤差，但會降低相位裕度、拖慢響應、並帶來積分飽和的風險。重力補償直接從源頭消除誤差，不需積分項。

**第三，計算量極小**。 $G(q)$  只需一次正向運動學加上簡單的加總（第 14.4.3 節的技巧：設  $\ddot{q} = 0$  執行一次遞迴即可），遠比完整的 CTC 便宜。

### 17.5.2 理論保證

第 16 章的知識框提過 Takegaki-Arimoto 定理，現在可以完整陳述：

#### ◎ PD 加重力補償的穩定性定理

對於任何機械手臂，控制律  $\tau = K_p(q_d - q) - K_d \dot{q} + G(q)$ （其中  $K_p$ 、 $K_d$  為任意正定矩陣）能保證定點調節的**全域漸近穩定**——無論初始位置在哪裡，機器人必定收斂到目標且誤差趨於零。這個結論不需要知道  $M(q)$  與  $C(q, \dot{q})$  的任何資訊，只需要  $G(q)$ 。這正是重力補償如此重要的理論依據，第 19 章會用 Lyapunov 方法給出完整證明。

必須注意這個定理的適用範圍：它保證的是**定點調節**（ $q_d$  為常數），不保證時變軌跡的追隨精度。要追隨快速的軌跡，仍然需要完整的 CTC 或至少加上加速度前饋。

### 17.5.3 零力控制與手動導引

重力補償有一個極為重要的應用：**零力控制**（zero-force control）。若把控制律中的 PD 項全部去掉，只保留重力補償：

$$\tau = G(q)$$

純重力補償。機器人恰好抵消自身重量，但不主動維持位置。

此時機器人會**浮在空中**——它不會因重力下墜，但也不抗拒外力。用手輕輕一推，它就順著推動的方向移動；放手後停在原處。這正是協作機器人「**手動導引示教**」( hand guiding / lead-through teaching ) 的原理：操作員直接抓著手臂把它帶到想要的位置，按下記錄鍵存下該點。

這個功能徹底改變了機器人的示教方式。傳統的示教器需要操作員理解座標系與運動方向，用按鈕一軸一軸地移動；手動導引則任何人都能上手——這是協作機器人易用性的關鍵，也是它們能進入中小企業的重要原因。

實作上有三個要點。**要點一：重力模型必須準確**。若補償不足，手臂會緩慢下墜；若過度補償，手臂會自己往上飄。兩者都會讓操作員感到不安。**要點二：必須補償摩擦**。減速機的摩擦會讓手臂「推不動」或「推了就停不下來」，因此高階系統會額外加入摩擦補償。**要點三：安全限制**。零力模式下機器人幾乎不受控，必須限制速度並設定虛擬牆（軟體邊界）避免碰撞。

#### 17.5.4 重力補償的機構替代方案

值得一提的是，重力也可以用**機構**而非控制來補償。第 5 章的專題延伸提過這個作法：在關節上加裝彈簧或配重，使其產生的力矩恰好抵消重力力矩。

這種被動重力補償的優點是：不消耗能量、斷電時手臂也不會掉落、且能大幅降低馬達的持續扭力需求（第 5 章例題 5-2 的改善方案二）。缺點是增加機構重量與慣量、只對特定的負載範圍有效、且設計複雜。

實務上兩者常並用：大型手臂用彈簧平衡器承擔大部分的重量，控制器再用軟體補償剩餘的部分。這是「機構與控制協同設計」的典型範例。

## 17.6 關節空間的軌跡追隨

### 17.6.1 完整的控制架構

把第 15 章的軌跡生成與本章的 CTC 結合，就得到完整的關節空間軌跡追隨架構：

1. **軌跡生成器**（第 15 章）：依任務需求產生  $q_d(t)$ 、 $\dot{q}_d(t)$ 、 $\ddot{q}_d(t)$  三者。注意必須提供加

速度——這是 CTC 前饋所必需的。

2. 誤差計算： $e = q_d - q$ ， $\dot{e} = \dot{q}_d - \dot{q}$ 。
3. 外層控制律： $u = \ddot{q}_d + K_d \dot{e} + K_p e$ 。
4. 內層線性化： $\tau = \text{InverseDynamics}(q, \dot{q}, u)$ ，一次 Newton-Euler 遞迴即可（第 14 章）。
5. 力矩限制：檢查並限制  $\tau$  不超過馬達能力，若觸及飽和則須有對應處理（見 17.6.3 節）。
6. 送出指令：把  $\tau$  轉換為電流指令送給驅動器（第 5 章的  $\tau = K_t I$ ）。

整個迴路通常以 1 kHz 執行。以第 14 章例題 14-2 的估算，六軸手臂的一次逆動力學約需 9  $\mu\text{s}$ ，佔週期的不到 1%——完全可行。

## 17.6.2 三種控制律的比較

為了看清楚各種方法的差異，把三種控制律並列：

表 17-2 三種關節空間控制律的比較

| 控制律       | 公式                                        | 所需模型      | 計算量             | 追隨性能        | 適用場合           |
|-----------|-------------------------------------------|-----------|-----------------|-------------|----------------|
| 獨立關節 PID  | $\tau = K_p e + K_d \dot{e} + K_i \int e$ | 無         | 極低              | 低速尚可、高速差    | 教學、低速、簡單系統     |
| PD + 重力補償 | $\tau = K_p e + K_d \dot{e} + G(q)$       | 只需 $G(q)$ | 低               | 定點調節優異、追隨中等 | 定位、手動導引、多數工業應用 |
| 計算力矩控制    | $\tau = M(q)u + C\dot{q} + G$             | 完整動力學     | 中<br>( $O(n)$ ) | 優異          | 高速、高精度、重載變化    |

實務上的選擇原則：若運動緩慢（額定速度 30% 以下）且負載固定，PD 加重力補償通常就足夠；若需要高速、或負載會大幅變化、或要求高追隨精度，則採用完整 CTC。

## 17.6.3 力矩飽和的處理

一個實務上必須處理的問題是**力矩飽和**：控制器算出的  $\tau$  可能超過馬達能力。此時若直接截斷，會產生兩個問題。

**問題一：線性化被破壞。**CTC 的推導假設  $\tau$  能被精確執行；一旦飽和， $\ddot{q} \neq u$ ，閉環不再是雙積分器，性能保證失效。

**問題二：積分飽和（若有積分項）。**誤差持續存在但控制器已無力修正，積分項會不斷累積（第 16 章的 windup 問題）。

因應之道有三：一、**事前預防**——在第 15 章的軌跡規劃階段就做動力學可行性檢驗（15.8 節），確保規劃的軌跡不會導致飽和。這是最好的方法。二、**動態降速**——偵測到接近飽和時，即時放慢軌跡的執行速度（時間縮放）。三、**抗飽和機制**——採用第 16 章的反算法或條件積分，避免積分項失控。

## 17.7 操作空間控制

### 17.7.1 為何要在任務空間控制

到目前為止，控制都在關節空間進行：規劃  $q_d(t)$ 、控制關節誤差。但許多任務的需求其實是在**任務空間**中描述的：「末端沿這條直線移動」、「末端對工件施加 10 N 的力」、「末端保持在這個位置不動，但可以被推」。

關節空間控制要達成這些需求，必須先把任務轉成關節軌跡（逐點逆運動學），再控制關節。這有三個缺點：一、**逆運動學的計算負擔**；二、**關節誤差與末端誤差的關係隨姿態變化**（同樣的關節誤差，在不同姿態下造成的末端偏差不同）；三、**難以直接指定末端的動態行為**（如剛度、阻尼）。

**操作空間控制**（operational space control，由 Khatib 於 1987 年提出）直接在任務空間中設計控制律，繞過這些問題。

### 17.7.2 推導：把動力學投影到任務空間

推導的起點是第 11 章的雅可比關係。末端速度與加速度為：

$$\dot{x} = J \dot{q} \quad \ddot{x} = J \ddot{q} + \dot{J} \dot{q}$$

任務空間與關節空間的速度、加速度關係。第二項來自雅可比隨姿態變化（第 11.9.3 節）。

我們希望設計控制律，使末端的行為像一顆在任務空間中的點質量：

$$\ddot{x} = u_x \quad (u_x \text{ 為任務空間的虛擬控制輸入})$$

目標：讓末端表現得像一顆一維點質量——正是開頭故事中叫獸說的那句話。

由  $\ddot{x} = J \ddot{q} + \dot{J} \dot{q} = u_x$  反解關節加速度：

$$\ddot{q} = J^{-1}(u_x - \dot{J} \dot{q}) \quad (\text{或用偽逆 } J^+, \text{ 若冗餘})$$

所需的關節加速度。

代入 CTC 的控制律  $\tau = M \ddot{q} + C \dot{q} + G$  :

$$\tau = M J^{-1}(\ddot{u}_x - \dot{J} \dot{q}) + C \dot{q} + G$$

操作空間控制律的初步形式。

Khatib 進一步證明，這可以改寫成一個更優雅、且直接以「末端力」為輸入的形式：

$$\tau = J^T [ \Lambda(q) \ddot{u}_x + \mu(q, \dot{q}) + p(q) ]$$

操作空間控制律。 $J^T$  把任務空間的力映射為關節力矩（第 11.6 節的對偶性）。

其中三項的意義分別為：

$$\Lambda(q) = (J M^{-1} J^T)^{-1} \quad (\text{操作空間慣量矩陣})$$

$$\mu(q, \dot{q}) = \Lambda J M^{-1} C \dot{q} - \Lambda \dot{J} \dot{q} \quad (\text{任務空間的科氏離心項})$$

$$p(q) = \Lambda J M^{-1} G \quad (\text{任務空間的重力項})$$

三個投影後的量。它們是關節空間的  $M$ 、 $C$ 、 $G$  在任務空間中的等效物。

### 17.7.3 操作空間慣量 $\Lambda$ 的物理意義

$\Lambda = (J M^{-1} J^T)^{-1}$  這個矩陣值得特別說明，它有一個非常直觀的物理意義：**末端在各方向上感受到的等效慣量**。

想像你用手推機器人的末端。往某些方向推很輕鬆（等效慣量小），往另一些方向推很沉重（等效慣量大）。 $\Lambda$  就精確描述了這個「各方向推起來多重」的分布——它是第 11.8 節速度橢球與力橢球概念在動力學層面的延伸。

$\Lambda$  的三個重要性質：一、**對稱正定**（繼承自  $M$  的正定性）；二、**隨姿態變化**；三、**在奇異點處退化**——當  $J$  失去滿秩時， $J M^{-1} J^T$  變成奇異， $\Lambda$  的某些元素趨於無窮大。這在物理上完全合理：在奇異點的失去方向上，末端「推不動」，等效慣量無限大。

### 17.7.4 動態解耦與冗餘

操作空間控制的一個關鍵成果是**動態解耦**：經過上述控制律，末端在任務空間的六個自由度變成互相獨立的雙積分器。這意味著可以為每個方向獨立設計控制器——例如在  $Z$  方向用高剛度（精確定位）、在  $X$ - $Y$  方向用低剛度（順應性），這正是第 20 章力控制與阻抗控制的基礎。

對冗餘機器人，操作空間控制還能自然地結合第 12 章的零空間投影：

$$\tau = J^T [\Lambda \ddot{u}_x + \mu + p] + (I - J^T J) \tau_{\text{null}}$$

第二項是動態一致的零空間投影，其中  $J = M^{-1}J^*$  為動態一致的廣義逆。 $\tau_{null}$  可用於次要目標（避關節極限、維持姿勢等）。

注意這裡用的是「動態一致」的零空間投影，而非第 12 章的運動學版本。兩者的差別在於：運動學版本保證零空間運動不影響末端位置，動態版本則保證不影響末端受力——後者在力控制中更為重要。

### 圖 17-2 關節空間控制與操作空間控制的架構對比

建議配圖：上下兩列方塊圖。上列（關節空間控制）：任務目標  $x_d \rightarrow$  逆運動學  $\rightarrow q_d \rightarrow$  關節 PD/CTC  $\rightarrow \tau \rightarrow$  機器人；標註「需逐點逆解、誤差關係隨姿態變化」。下列（操作空間控制）：任務目標  $x_d \rightarrow$  直接在任務空間計算誤差與  $u_x \rightarrow$  乘上  $\Lambda$  得任務空間力  $\rightarrow$  經  $J^T$  轉為關節力矩  $\tau \rightarrow$  機器人；末端位姿  $x$  由正向運動學回授。標註「不需逆運動學、末端動態解耦、可直接指定剛度」。右側另附冗餘機器人的零空間分支。

## 17.8 進階非線性控制概觀

### 17.8.1 滑模控制

計算力矩控制依賴準確的模型。當模型誤差較大時，一個強健的替代方案是滑模控制（sliding mode control）。

它的核心想法是：先定義一個「滑動面」——一個誤差空間中的超平面，只要系統狀態落在這個面上，誤差就會自動收斂：

$$s = \dot{e} + \lambda e = 0 \quad (\text{滑動面})$$

當  $s = 0$  時， $\dot{e} = -\lambda e$ ，誤差以指數衰減至零。 $\lambda$  決定收斂速度。

控制律的設計則是：無論系統在滑動面的哪一側，都用足夠強的力把它推回面上：

$$\tau = \tau_{eq} + K \cdot \text{sign}(s)$$

$\tau_{eq}$  為等效控制（依名目模型計算）， $K \cdot \text{sign}(s)$  為切換項。只要  $K$  大於模型誤差的上界，就能保證收斂。

**優點：**對模型不確定性與外部干擾極為強健，且不需要精確的參數，只需知道誤差的上界。

**缺點：** $\text{sign}$  函數造成控制輸入高頻切換，產生顫振（chattering）——這會激發機構振動、加速磨損、並產生噪音。

實務上的緩解方法是用飽和函數（saturation）取代  $\text{sign}$ ，在滑動面附近設一個「邊界層」使控制連續。代價是犧牲了理論上的有限時間收斂，改為收斂到一個小鄰域內。

### 17.8.2 自適應控制



自適應控制 ( adaptive control ) 採取另一條路線：既然參數不準，就線上估測它們。

它利用第 13.10.2 節的關鍵性質——動力學方程對慣性參數是線性的：

$$\tau = Y(q, \dot{q}, \ddot{q}) \cdot \pi$$

$Y$  為迴歸矩陣 ( 只與運動有關 )、 $\pi$  為慣性參數向量。這個線性結構是自適應控制可行的數學基礎。

控制器同時做兩件事：用當前的參數估計值  $\hat{\pi}$  計算控制力矩、並依追隨誤差調整  $\hat{\pi}$ 。經典的 Slotine-Li 自適應控制器能證明：追隨誤差保證收斂到零，即使初始的參數估計完全錯誤。

**優點：**能自動適應未知或變化的負載——例如機器人抓起一個未知重量的物件後，控制器會自動學會補償它。**缺點：**參數收斂需要「持續激勵」( 軌跡必須夠豐富才能辨識出所有參數 )；且收斂過程中性能較差；實作也較複雜。

### 17.8.3 魯棒控制

魯棒控制 ( robust control ) 的哲學介於兩者之間：不追求估測真實參數，也不用高頻切換，而是設計一個對「所有可能的參數組合」都能維持性能的控制器。

典型作法是在 CTC 的基礎上加入一個補償項，其大小依模型誤差的上界設計。它比滑模控制平滑 ( 無顫振 )、比自適應控制簡單 ( 無需線上估測 )，但性能通常較保守——因為必須為最壞情況做準備。

表 17-3 四種非線性控制方法的比較

| 方法     | 核心思想       | 模型需求            | 對不確定性     | 主要缺點      | 適用場合       |
|--------|------------|-----------------|-----------|-----------|------------|
| 計算力矩控制 | 用模型精確抵消非線性 | 完整且準確           | 敏感        | 依賴模型精度    | 模型可靠、高速高精度 |
| 滑模控制   | 強力推回滑動面    | 只需誤差上界          | 極強健       | 顫振        | 模型很不確定、干擾大 |
| 自適應控制  | 線上估測未知參數   | 需知結構 ( $Y$ 矩陣 ) | 強 ( 會收斂 ) | 需持續激勵、收斂慢 | 負載未知或會變化   |
| 魯棒控制   | 為最壞情況設計    | 需知誤差上界          | 強健        | 性能保守      | 不確定性有界且已知  |

### 17.8.4 模型預測控制

模型預測控制 ( Model Predictive Control, MPC ) 代表另一個層次的思路。前述方法都是



「當下反應」——依當前誤差計算控制輸入。MPC 則是「向前看」：

1. 在每個控制週期，用模型預測未來一段時間（預測時域）內系統的行為。
2. 求解一個最佳化問題：找出未來的控制序列，使某個代價函數最小，同時滿足所有約束（力矩極限、關節極限、避障）。
3. 只執行這個序列的**第一步**。
4. 下一個週期，以新的量測值重新求解——這稱為滾動時域（receding horizon）。

**MPC 最大的優勢是能明確處理約束。**前述所有方法都是「先算出控制輸入，再截斷到極限」；MPC 則在求解時就把極限納入考慮，因此不會發生飽和造成的性能崩壞。此外它天然支援多目標權衡與未來資訊（如已知的軌跡）。

**代價是計算量。**每個週期都要解一個最佳化問題，這在過去只能用於慢速系統（化工製程的控制週期以分鐘計）。但隨著求解器與硬體的進步，近年 MPC 已能在毫秒級執行——四足機器人的即時步態控制（第 28 章）就大量使用 MPC，這是近十年腿式機器人能力躍進的關鍵技術之一。

MPC 與第 18 章的最佳控制關係密切：它本質上是「有限時域的最佳控制問題，反覆求解」。第 18 章將建立最佳控制的理論基礎，屆時可以更完整地理解 MPC 的數學結構。

## 17.9 多任務的動態版本

### 17.9.1 由運動學到動力學的優先權

第 12.8 節介紹了分層逆運動學：以嚴格的優先權處理多個任務，低優先權的任務在高優先權任務的零空間中執行。那是**運動學**層次的處理——只考慮位置與速度。

操作空間控制讓我們能把同樣的想法推廣到**動力學**層次。差別在於：運動學版本保證「低優先權任務不影響高優先權任務的運動」；動力學版本則保證「不影響高優先權任務所受的力」。

以兩層任務為例（主任務雅可比  $J_1$ 、次任務  $J_2$ ）：

$$\tau = J_1^T F_1 + N_1^T J_2^T F_2, \quad \text{其中 } N_1 = I - J_1^T J_1^T$$

動態一致的分層控制。 $N_1$  是動態一致的零空間投影算子， $J_1 = M^{-1}J_1^T \Lambda_1$  為動態一致廣義逆。

這個架構是人形機器人**全身控制**（whole-body control）的核心。一台人形機器人可能同

時需要：維持平衡（最高優先權）、右手跟隨軌跡、左手保持不動、注視目標、以及所有關節遠離極限——分層架構讓這些任務能有序地共存。第 28 章會回到這個主題。

### 17.9.2 為何動態一致性重要

使用運動學版本的零空間投影，在動力學層次上會產生一個微妙的問題：次任務的力矩雖然不會改變主任務的**加速度**，卻可能改變主任務所需的**力**。

在純運動控制中這無妨；但在**力控制**（第 20 章）中就是嚴重問題——例如機器人一手抵著工件維持 10 N 的接觸力，另一手在做次要動作，若次要動作干擾了接觸力，工件可能被推移或損壞。動態一致的投影正是為了避免這種干擾。

## 17.10 實務上的控制架構

### 17.10.1 工業機器人的典型架構

真實的工業機器人控制器並非只用單一種方法，而是一個多層次的複合架構：

1. **軌跡插補層**（10~100 Hz）：依第 15 章的方法產生  $q_d$ 、 $\dot{q}_d$ 、 $\ddot{q}_d$ ，並做動力學可行性檢查。
2. **動力學前饋層**（1 kHz）：以逆動力學計算名目力矩  $\tau_{ff} = M \ddot{q}_d + C \dot{q}_d + G$ ，這是「理論上需要的力矩」。
3. **位置 / 速度回授層**（1~4 kHz）：PID 修正實際的追隨誤差，輸出  $\tau_{fb}$ 。
4. **電流迴路層**（10~20 kHz）：驅動器內部的 PI 控制，確保實際電流跟上力矩指令（第 5 章）。
5. **安全監控層**：檢查力矩、速度、位置是否超限，異常時觸發保護。

總力矩為  $\tau = \tau_{ff} + \tau_{fb}$ 。這個架構的精神就是本章反覆強調的：**前饋處理可預測的部分**，**回授處理不可預測的部分**。

### 17.10.2 三個實務要點

**要點一：前饋用期望值，回授用實際值。** $\tau_{ff}$  應以  $q_d$ 、 $\dot{q}_d$  計算（而非實際的  $q$ 、 $\dot{q}$ ）。理由是：若用實際值，感測雜訊會直接進入前饋通道被放大；用期望值則前饋是純粹的「開迴路預測」，平滑無雜訊。

**要點二：微分項用速度回授而非誤差微分。**回顧第 16 章，若對誤差微分，當  $\dot{q}_d$  快速變化時會產生大的尖峰。實務上多採  $\tau_{fb} = K_p \cdot e - K_d \cdot \dot{q}$ （直接對實際速度做阻尼），這稱為「速度回授」形式。

**要點三：模型與實機的參數必須同步。**若更換了夾具或負載，控制器的動力學參數必須同步更新，否則前饋會系統性偏差。多數協作機器人提供「負載辨識」功能，能自動估測末端負載的質量與質心。

#### ◎ 為何協作機器人特別需要模型式控制

協作機器人有兩個需求，使它們比傳統工業機器人更依賴動力學模型。**需求一：碰撞偵測**（第 11.6.3 節）。要由關節力矩推算外力，必須先精確扣除  $M \ddot{q}$ 、 $C \dot{q}$ 、 $G$  三項——模型誤差直接轉化為誤判或漏判。**需求二：手動導引**（17.5.3 節）。零力控制完全建立在重力與摩擦的精確補償上。因此協作機器人廠商通常會在出廠前做完整的動力學參數辨識，並提供線上的負載自動辨識功能——這在傳統工業機器人上並非標準配備。

## 17.11 由控制律邁向最佳性與穩定性

### 17.11.1 本章的成果

本章完成了模型式控制的核心。你現在能夠：辨識機器人的四個非線性來源、推導計算力矩控制並理解其回授線性化的本質、實作重力補償與零力控制、分析模型誤差的影響、在關節空間與操作空間之間選擇合適的控制架構、並理解滑模、自適應、魯棒與 MPC 四種進階方法的思想。

最重要的是，你理解了那個貫穿全章的設計哲學：**用模型處理已知的部分，用回授處理未知的部分**。這不只是控制技術，更是工程思維——面對複雜系統時，先用理解把問題縮小，剩下的才交給反饋修正。

### 17.11.2 兩個尚未回答的問題

但本章留下了兩個重要的問題。

**問題一：增益該怎麼選才是「最好」的？**第 16 章告訴我們  $K_p = \omega_n^2$ 、 $K_d = 2\zeta\omega_n$ ，但  $\omega_n$  該取多少？調高則反應快但控制量大、調低則省力但遲鈍。這中間有沒有一個「最佳」的權衡點？這需要先定義什麼叫「好」——**第 18 章（最佳控制與 Riccati 方程）**將以代價函數的形式回答這個問題，並導出 LQR 這個能自動算出最佳增益的方法。

**問題二：我們怎麼知道控制器一定不會失控？** 本章的推導都建立在「模型正確」的假設上，並以直覺論證了模型誤差的影響。但這不是嚴格的證明。**第 19 章（可控性與穩定性）**將引入 Lyapunov 方法，嚴格證明本章各種控制律的穩定性——包括 17.5.2 節那個「PD 加重力補償保證全域漸近穩定」的定理。

**第 20 章（力控制）**則會把本章的操作空間控制推進一步：不只控制末端的運動，還要控制末端與環境之間的力。屆時  $\Lambda$  矩陣與動態解耦的價值將完全展現。

換句話說：第 17 章告訴你**控制器該長什麼樣**，第 18 章告訴你**參數該怎麼選最好**，第 19 章告訴你**為什麼它一定有效**。

## 完整計算例題

### 例題 17-1 計算力矩控制的完整計算

題目：平面 2R 手臂（參數同第 13 章例題 13-2： $L_1=0.4$ 、 $L_2=0.3$  m， $m_1=3.0$ 、 $m_2=2.0$  kg， $\alpha=0.540$ 、 $\beta=0.120$ 、 $\delta=0.060$  kg·m<sup>2</sup>）。目前狀態  $q = (30^\circ, 60^\circ)$ 、 $\dot{q} = (1.0, -0.5)$  rad/s。軌跡要求  $q_d = (32^\circ, 58^\circ)$ 、 $\dot{q}_d = (1.2, -0.4)$  rad/s、 $\ddot{q}_d = (2.0, 1.0)$  rad/s<sup>2</sup>。控制增益取  $\omega_n = 20$  rad/s、 $\zeta = 1$ （臨界阻尼）。（a）求  $K_p$ 、 $K_d$ 。（b）計算虛擬控制  $u$ 。（c）計算實際力矩  $\tau$ 。（d）驗證閉環誤差動態。

解（a）：由第 16 章的公式，臨界阻尼時：

$$K_p = \omega_n^2 = 20^2 = 400 \quad (1/s^2) \quad K_d = 2\zeta\omega_n = 2(1)(20) = 40 \quad (1/s)$$

增益。注意 CTC 的增益是無量綱化的（作用在加速度層級），與直接 PID 的增益單位不同。

解（b）：先計算誤差（換算為弧度）：

$$\begin{aligned} e &= q_d - q = (32-30, 58-60)^\circ = (2, -2)^\circ = (0.0349, -0.0349) \text{ rad} \\ \dot{e} &= \dot{q}_d - \dot{q} = (1.2-1.0, -0.4-(-0.5)) = (0.2, 0.1) \text{ rad/s} \end{aligned}$$

位置與速度誤差。

$$\begin{aligned} u &= \ddot{q}_d + K_d \cdot \dot{e} + K_p \cdot e \\ &= (2.0, 1.0) + 40(0.2, 0.1) + 400(0.0349, -0.0349) \\ &= (2.0, 1.0) + (8.0, 4.0) + (13.96, -13.96) \\ &= (23.96, -8.96) \text{ rad/s}^2 \end{aligned}$$

虛擬控制輸入，即「期望的關節加速度」。

解（c）：把  $u$  當作期望加速度代入逆動力學。先計算各項（ $\theta_2=60^\circ$ ， $c_2=0.5$ 、 $s_2=0.866$ ）：

$$M = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix} = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix}$$

慣性矩陣。

$$M \cdot u = \begin{bmatrix} 0.660(23.96) + 0.120(-8.96) \\ 0.120(23.96) + 0.060(-8.96) \end{bmatrix} = \begin{bmatrix} 15.813 - 1.075 \\ 2.875 - 0.538 \end{bmatrix} = \begin{bmatrix} 14.738 \\ 2.337 \end{bmatrix}$$

慣性項。注意這裡用的是實際的  $\dot{q}$  計算  $C$  項，但用  $u$  取代  $\ddot{q}$ 。

科氏離心項（用實際速度  $\dot{q} = (1.0, -0.5)$ ， $\beta_{s_2} = 0.1039$ ）：

$$C \dot{q} = \begin{bmatrix} -0.1039(0.25) - 2(0.1039)(1.0)(-0.5) \\ 0.1039(1.0)^2 \end{bmatrix} = \begin{bmatrix} -0.026 + 0.104 \\ 0.104 \end{bmatrix} = \begin{bmatrix} 0.078 \\ 0.104 \end{bmatrix}$$

科氏與離心項。

重力項（由第 13 章例題 13-2，同姿態）： $G = (11.88, 0)^T \text{ N}\cdot\text{m}$ 。

$$\tau = M u + C \dot{q} + G = \begin{bmatrix} 14.738 + 0.078 + 11.88 \\ 2.337 + 0.104 + 0 \end{bmatrix} = \begin{bmatrix} 26.70 \\ 2.44 \end{bmatrix} \text{ N}\cdot\text{m}$$

最終送出的力矩。

解（d）：若模型完全準確，代入系統方程可得  $\ddot{q} = u$ 。因此誤差動態為：

$$\ddot{e} + 40\dot{e} + 400e = 0$$

閉環誤差方程。這是臨界阻尼的二階系統，其解為  $e(t) = (A + Bt)e^{-(20t)}$ 。

誤差的包絡線時間常數為  $1/20 = 0.05$  秒，因此約 4 個時間常數（0.2 秒）後誤差衰減至 2% 以下。重要的是：這個收斂行為與姿態、速度、負載完全無關——這正是 CTC 的價值。

## 例題 17-2 重力補償與穩態誤差

題目：承上手臂。（a）若採用純 PD 控制（無重力補償）， $K_p = 320 \text{ N}\cdot\text{m}/\text{rad}$ ，求在  $q = (0^\circ, 90^\circ)$  姿態下維持靜止的穩態誤差。（b）若加入重力補償，穩態誤差為何？（c）若要用純 PD 把誤差壓到  $0.5^\circ$  以下，需要多大的  $K_p$ ？（d）討論此增益的可行性。

解（a）：先計算該姿態的重力項。 $q = (0^\circ, 90^\circ)$  時， $c_1 = 1$ 、 $c_{12} = \cos(90^\circ) = 0$ 。

$$G_1 = (m_1 l_1 + m_2 l_1)g \cdot c_1 + m_2 l_2 g \cdot c_{12} = (3.0 \times 0.2 + 2.0 \times 0.4)(9.8)(1) + 0 = (0.6 + 0.8)(9.8) = 13.72 \text{ N}\cdot\text{m}$$

第一關節的重力力矩（此姿態為水平伸展，重力最大）。

純 PD 在穩態時（ $\dot{q} = 0$ 、 $\ddot{q} = 0$ ）： $K_p \cdot e = G$ ，故：

$$e_{ss} = G_1/K_p = 13.72/320 = 0.0429 \text{ rad} = 2.46^\circ$$

穩態誤差。手臂會比目標位置低  $2.46$  度。

解 (b)：加入重力補償後，控制律為  $\tau = K_p \cdot e + K_d \cdot \dot{e} + G(q)$ 。穩態時  $K_p \cdot e + G = G$ ，故  $e_{ss} = 0$ 。誤差完全消除。

解 (c)：要求  $e_{ss} \leq 0.5^\circ = 0.00873 \text{ rad}$ ：

$$K_p \geq G_1/e_{ss} = 13.72/0.00873 = 1572 \text{ N}\cdot\text{m/rad}$$

所需的比例增益，是原本的 4.9 倍。

解 (d)：這個增益不可行，理由有三。

**理由一：激發共振。**由  $\omega_n = \sqrt{(K_p/M)}$ ，取  $M_{11} \approx 0.66$  (該姿態下)： $\omega_n = \sqrt{(1572/0.66)} = 48.8 \text{ rad/s} \approx 7.8 \text{ Hz}$ 。若機構的第一共振頻率在 10~20 Hz (大型手臂常見)，控制器頻寬已太接近，會激發振動。

**理由二：放大雜訊。**編碼器的量化雜訊若為 0.001 rad，經  $K_p = 1572$  放大後產生 1.57 N·m 的力矩抖動——這會造成明顯的機械噪音與磨損。

**理由三：安全風險。**高增益使機器人變得極「硬」，碰撞時的衝擊力大幅增加，這對協作應用是不可接受的。

**結論：**這正是重力補償的價值——用一行模型計算，達成了增益調高五倍也達不到的效果，而且沒有任何副作用。這是本章「用模型處理已知部分」哲學最直接的例證。

### 例題 17-3 模型誤差的影響評估

題目：承例題 17-1 的情境。假設控制器使用的模型有誤差：末端多了一個未建模的 1.0 kg 負載 (安裝在連桿 2 末端，距第二關節 0.3 m)。(a) 計算真實的動力學參數變化。(b) 估算重力項的誤差。(c) 估算此誤差造成的穩態位置偏差 (設  $K_p = 400$ 、外層為 CTC)。(d) 提出兩種因應方式。

解 (a)：未建模負載  $m_L = 1.0 \text{ kg}$  位於  $r = 0.3 \text{ m}$  處。它對各項的影響為：

$$\begin{aligned}\Delta\delta &= m_L r^2 = 1.0(0.09) = 0.090 \text{ kg}\cdot\text{m}^2 \\ \Delta\beta &= m_L L_1 r = 1.0(0.4)(0.3) = 0.120 \text{ kg}\cdot\text{m}^2 \\ \Delta\alpha &= m_L(L_1^2 + r^2) = 1.0(0.16 + 0.09) = 0.250 \text{ kg}\cdot\text{m}^2\end{aligned}$$

慣性參數的增量。注意  $\beta$  增加了 100% (由 0.120 變為 0.240) ——耦合大幅增強。

解 (b)：重力項的增量 (在  $q = (30^\circ, 60^\circ)$ ， $c_{12} = \cos(90^\circ) = 0$ )：

$$\Delta G_1 = m_L(L_1 c_1 + r \cdot c_{12})g = 1.0(0.4 \times 0.866 + 0.3 \times 0)(9.8) = 1.0(0.3464)(9.8) =$$

$$\Delta G_2 = m_L \cdot r \cdot c_{12} \cdot g = 0 \text{ N}\cdot\text{m}$$

重力項的誤差。第一關節少補償了  $3.4 \text{ N}\cdot\text{m}$ 。

解 (c)：這個未補償的力矩，會被外層 PD 視為干擾。在 CTC 架構下，等效的干擾加速度為：

$$\Delta \ddot{q} = M^{-1} \cdot \Delta G \quad M^{-1} = (1/\det) \begin{bmatrix} 0.060 & -0.120 \\ -0.120 & 0.660 \end{bmatrix}, \det = 0.660(0.060) - 0.120^2 = 0.0252$$

慣性矩陣的反矩陣。

$$\Delta \ddot{q}_1 = (0.060 \times 3.395)/0.0252 = 0.2037/0.0252 = 8.08 \text{ rad/s}^2$$

等效干擾加速度。

穩態時  $K_p \cdot e = \Delta \ddot{q}$  (在加速度層級)，故：

$$e_{ss} = \Delta \ddot{q}_1 / K_p = 8.08/400 = 0.0202 \text{ rad} = 1.16^\circ$$

穩態誤差約  $1.16$  度。

解 (d)：方式一：負載辨識。讓機器人執行一段激勵軌跡，由實測力矩反推負載參數 (第 13.10 節)，再更新模型。多數協作機器人提供此功能，通常只需十幾秒。

方式二：加入積分項。在外層 PD 加上積分項變成 PID，可消除這個常值誤差。代價是第 16 章提到的響應變慢與積分飽和風險。實務上兩者常並用：先用負載辨識消除大部分誤差，剩餘的小殘差交給積分項。

## 程式範例：計算力矩控制與 PID 的對比模擬

以下 Python 程式在平面 2R 手臂上實作三種控制器 (獨立關節 PID、PD 加重力補償、計算力矩控制)，執行同一段軌跡並比較追隨誤差。同時示範模型誤差與負載變化的影響。

```
import numpy as np

class Arm2R:
    """平面 2R 手臂動力學 (沿用第 13、14 章)"""
    def __init__(s, L1=0.4, L2=0.3, m1=3.0, m2=2.0, mL=0.0, g=9.8):
        s.L1, s.L2, s.g = L1, L2, g
        s.m1, s.m2, s.mL = m1, m2, mL          # mL 為末端額外負載
        s.l1, s.l2 = L1/2, L2/2
        I1, I2 = m1*L1**2/12, m2*L2**2/12
        # 把末端負載併入連桿 2 的等效參數
        m2e = m2 + mL
```

```

l2e = (m2*s.l2 + mL*L2)/m2e if m2e > 0 else s.l2
I2e = I2 + m2*(l2e-s.l2)**2 + mL*(L2-l2e)**2
s.m2e, s.l2e = m2e, l2e
s.alpha = m1*s.l1**2 + I1 + m2e*(L1**2 + l2e**2) + I2e
s.beta = m2e*L1*l2e
s.delta = m2e*l2e**2 + I2e

def M(s, q):
    c2 = np.cos(q[1])
    return np.array([[s.alpha + 2*s.beta*c2, s.delta + s.beta*c2],
                    [s.delta + s.beta*c2, s.delta]])

def Cqd(s, q, qd):
    sn = np.sin(q[1])
    return np.array([-s.beta*sn*qd[1]**2 - 2*s.beta*sn*qd[0]*qd[1],
                    s.beta*sn*qd[0]**2])

def G(s, q):
    c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
    return np.array([(s.m1*s.l1 + s.m2e*s.l1)*s.g*c1 + s.m2e*s.l2e*s.g*c12,
                    s.m2e*s.l2e*s.g*c12])

def forward_dynamics(s, q, qd, tau):
    return np.linalg.solve(s.M(q), tau - s.Cqd(q, qd) - s.G(q))

# ----- 三種控制器 -----
def ctrl_pid(model, q, qd, ref, gains, state):
    """獨立關節 PID (無模型)"""
    qd_ref, qdd_ref, q_ref = ref
    Kp, Kd, Ki = gains
    e, ed = q_ref - q, qd_ref - qd
    state['I'] += e * state['dt']
    return Kp*e + Kd*ed + Ki*state['I']

def ctrl_pd_gravity(model, q, qd, ref, gains, state):
    """PD + 重力補償 (17.5 節)"""
    qd_ref, qdd_ref, q_ref = ref
    Kp, Kd, _ = gains
    return Kp*(q_ref - q) + Kd*(qd_ref - qd) + model.G(q)

def ctrl_ctc(model, q, qd, ref, gains, state):
    """計算力矩控制 (17.2 節)"""
    qd_ref, qdd_ref, q_ref = ref
    wn, zeta = state['wn'], state['zeta']
    Kp_a, Kd_a = wn**2, 2*zeta*wn # 加速度層級的增益
    u = qdd_ref + Kd_a*(qd_ref - qd) + Kp_a*(q_ref - q)
    return model.M(q) @ u + model.Cqd(q, qd) + model.G(q) # 逆動力學

# ----- 軌跡：五次多項式 (第 15 章) -----
def quintic(q0, qf, T, t):
    t = np.clip(t, 0, T); s = t/T
    dq = qf - q0
    return (q0 + dq*(10*s**3 - 15*s**4 + 6*s**5),

```



```

dq*(30*s**2 - 60*s**3 + 30*s**4)/T,
dq*(60*s - 180*s**2 + 120*s**3)/T**2)

# ----- 模擬 -----
def simulate(ctrl, gains, plant, model, T=1.0, dt=1e-4, wn=20.0, zeta=1.0):
    q0 = np.deg2rad([10.0, 20.0]); qf = np.deg2rad([70.0, -30.0])
    q, qd = q0.copy(), np.zeros(2)
    state = {'I': np.zeros(2), 'dt': dt, 'wn': wn, 'zeta': zeta}
    max_err, tau_max = 0.0, 0.0
    n = int(T/dt)
    for k in range(n):
        t = k*dt
        q_r, qd_r, qdd_r = quintic(q0, qf, T*0.8, t)
        tau = ctrl(model, q, qd, (qd_r, qdd_r, q_r), gains, state)
        tau = np.clip(tau, -60, 60) # 力矩飽和
        qdd = plant.forward_dynamics(q, qd, tau) # 真實系統
        qd = qd + qdd*dt; q = q + qd*dt # 半隱式歐拉
        if t > 0.05:
            max_err = max(max_err, np.abs(np.rad2deg(q_r - q)).max())
            tau_max = max(tau_max, np.abs(tau).max())
    final_err = np.abs(np.rad2deg(qf - q)).max()
    return max_err, final_err, tau_max

# ===== 情境一：模型完全準確 =====
print("=== 情境一：模型準確、無額外負載 ===")
plant = Arm2R(); model = Arm2R()
gains_pid = (np.array([300.,300.]), np.array([30.,30.]), np.array([100.,100.]))
gains_pd = (np.array([300.,300.]), np.array([30.,30.]), None)
print(f"{'控制器':<18}{'最大追隨誤差':>14}{'終點誤差':>12}{'最大力矩':>12}")
for name, fn, g in (("獨立關節 PID", ctrl_pid, gains_pid),
                    ("PD + 重力補償", ctrl_pd_gravity, gains_pd),
                    ("計算力矩控制 CTC", ctrl_ctc, gains_pd)):
    me, fe, tm = simulate(fn, g, plant, model)
    print(f"{name:<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

# ===== 情境二：末端加上 1 kg 未建模負載 =====
print("\n=== 情境二：實機多 1.0 kg 負載，控制器模型未更新 ===")
plant2 = Arm2R(mL=1.0) # 真實系統有負載
model2 = Arm2R(mL=0.0) # 控制器模型沒有
for name, fn, g in (("獨立關節 PID", ctrl_pid, gains_pid),
                    ("PD + 重力補償", ctrl_pd_gravity, gains_pd),
                    ("計算力矩控制 CTC", ctrl_ctc, gains_pd)):
    me, fe, tm = simulate(fn, g, plant2, model2)
    print(f"{name:<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

# ===== 情境三：更新模型後的 CTC =====
print("\n=== 情境三：CTC 且模型已更新（負載辨識後） ===")
me, fe, tm = simulate(ctrl_ctc, gains_pd, plant2, Arm2R(mL=1.0))
print(f"{'CTC ( 模型已更新 )':<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

```

```
# ===== 情境四：不同速度下的性能 =====
print("\n=== 情境四：速度對三種控制器的影響（最大追隨誤差） ===")
print(f"{'總時間':>8}{'PID':>12}{'PD+G':>12}{'CTC':>12}")
for T in (2.0, 1.0, 0.5, 0.3):
    r = []
    for fn, g in ((ctrl_pid, gains_pid), (ctrl_pd_gravity, gains_pd),
                  (ctrl_ctc, gains_pd)):
        me, _, _ = simulate(fn, g, plant, model, T=T)
        r.append(me)
    print(f"{T:7.1f}s{r[0]:11.3f}°{r[1]:11.3f}°{r[2]:11.3f}°")
print("→ 速度越快，PID 誤差惡化越明顯；CTC 幾乎不受影響")

# ===== 例題 17-2：穩態誤差驗證 =====
print("\n=== 例題 17-2：純 PD 的重力造成穩態誤差 ===")
arm = Arm2R()
q_test = np.deg2rad([0.0, 90.0])
G = arm.G(q_test)
for Kp in (320.0, 800.0, 1572.0):
    print(f"Kp={Kp:7.1f}: G1={G[0]:6.2f} N·m "
          f"穩態誤差={np.rad2deg(G[0]/Kp):6.3f}°")
print(f"加入重力補償後：穩態誤差 = 0.000° ← 一行模型勝過五倍增益")
```

請試著修改：（一）把情境四的總時間縮短到 0.2 秒，觀察 PID 是否會完全失控。（二）在 CTC 的模型中把慣性參數全部乘上 0.8（模擬 20% 模型誤差），觀察誤差增加多少——你會發現它仍遠優於 PID。（三）把重力補償改成只補償 90%，觀察殘餘的穩態誤差是否與理論值  $0.1G/Kp$  相符。

## 實作：實體手臂的模型式控制

本章實作是在實體手臂上實現重力補償與計算力矩控制，並量化其相對於 PID 的改善。**安全提醒**：模型式控制若參數錯誤可能導致手臂暴衝，務必先在低增益、低速下測試，並確保緊急停止隨手可及。

- 前置作業**：沿用第 13 章實作中辨識出的動力學參數（質量、質心、慣量、摩擦）。若尚未完成，請先回頭完成該實作。
- 重力補償驗證**：把手臂設為只輸出  $G(q)$  的模式（無 PD 項）。逐一測試多個姿態，觀察手臂是否能「浮」在原處。若緩慢下墜代表補償不足、若自行上飄代表過度補償，據此微調參數。
- 零力控制體驗**：在重力補償正確後，用手推動手臂各關節，感受推動的難易。記錄「推動所需的最小力」——這反映了未補償的摩擦大小。

4. **摩擦補償**：加入  $\tau_f = F_v \cdot \dot{q} + F_c \cdot \text{sign}(\dot{q})$  的摩擦補償項，重複上一步，量化推動所需力量的下降幅度。注意  $\text{sign}$  函數在零速附近會抖動，應改用平滑的近似。
5. **穩態誤差比較**：設定一個水平伸展的目標姿態，分別以「純 PD」與「PD + 重力補償」執行，量測穩態位置誤差。驗證例題 17-2 的理論值  $e_{ss} = G/K_p$ 。
6. **軌跡追隨比較**：規劃一段中速軌跡（第 15 章），分別以三種控制器執行，記錄全程的追隨誤差。繪出三條誤差曲線於同一張圖。
7. **速度掃描**：把同一段軌跡的執行時間逐步縮短（如 3、2、1、0.6 秒），記錄各控制器的最大追隨誤差。繪出「速度 vs 誤差」曲線——你應該會看到 PID 的曲線急遽上升，而 CTC 的曲線相對平坦。
8. **負載測試**：在末端加裝已知重量的配重，重複步驟 6。分別測試「模型未更新」與「模型已更新」兩種情況，量化負載辨識的價值。
9. **安全性比較（選作）**：在低速下讓手臂輕觸一個固定障礙物，比較高增益 PID 與低增益 CTC 的接觸力大小。這能直觀展示「模型式控制可用較低增益達成同等精度」對安全性的意義。

表 17-4 控制器性能比較記錄表（第一列為範例）

| 控制器        | 執行時間  | 最大追隨誤差 | 穩態誤差  | 最大力矩     | 備註      |
|------------|-------|--------|-------|----------|---------|
| 獨立關節 PID   | 1.0 s | 2.84°  | 0.42° | 38.2 N·m | 轉彎處明顯落後 |
| （自行填寫其餘測試） |       |        |       |          |         |

## 國中小也看得懂：端一杯水走過房間的兩種方法

想像你要端一杯滿滿的水走過房間。有兩種方法：

**第一種方法：邊走邊看。** 你盯著水面，看到水往左邊晃，就趕快往右邊調整；看到往右晃，就往左調。這樣可以走到，但水會一直晃來晃去，而且你必須非常專心。這就是「PID 控制」——看到偏了才修正。

**第二種方法：先想好再走。** 如果你已經知道「等一下要轉彎，轉彎的時候水會往外甩」，那你可以在轉彎之前就先把杯子往內傾斜一點點，剛好抵消那個甩出去的力。這樣水根本不會晃，你也不用一直緊張地盯著。這就是這一章的「計算力矩控制」——用知識預先準備好。

那第一種方法就沒用了嗎？當然有用！因為總會有你沒想到的事——地板突然有一塊不平、有人突然撞到你。這時候還是要靠「邊走邊看」來救。

所以最好的方法是**兩個一起用**：能預測的事情先算好（這叫前饋），沒預測到的事情再用眼睛修正（這叫回授）。厲害的機器人工程師就是這樣做的。

最後告訴你一個很酷的小知識：這一章有一個叫做「重力補償」的技巧，就是讓機器人的馬達剛好出力抵消自己的重量。做到之後，機器手臂會像在無重力的太空一樣**浮在空中**——你用一根手指就能把它推來推去！很多新型的機器人就是用這個方法，讓工人可以直接用手教它動作，不需要寫程式。

## 叫獸提醒與常見錯誤

- 誤區一：以為調高 PID 增益就能解決高速誤差。系統的有效阻尼比隨姿態變化，固定增益不可能在所有姿態下都最佳。
- 誤區二：前饋項使用實際的  $q$ 、 $\dot{q}$  計算。應使用期望值  $q_d$ 、 $\dot{q}_d$ ，否則感測雜訊會直接進入前饋通道被放大。
- 誤區三：忘記軌跡必須提供加速度。CTC 的前饋需要  $\ddot{q}_d$ ，若軌跡生成器只輸出位置與速度，前饋效果會大打折扣。
- 誤區四：以為模型必須完美才能用 CTC。80% 準確的模型就能把干擾降低五倍——模型的價值不在完美而在大幅減負。
- 誤區五：更換夾具或負載後不更新模型參數。這會造成系統性的前饋偏差，應使用負載辨識功能。
- 誤區六：零力控制未補償摩擦。只補償重力的話，手臂仍會因減速機摩擦而「推不動」，體驗很差。
- 誤區七：在高增益下追求精度而忽略安全。模型式控制的價值之一，就是能用較低增益達成同等精度，這對協作應用至關重要。
- 誤區八：忽略力矩飽和對線性化的破壞。一旦飽和， $\ddot{q} \neq u$ ，CTC 的性能保證即失效，應在規劃階段就避免。
- 誤區九：混淆回授線性化與近似線性化。前者在整個狀態空間精確成立（但需完整模型），後者只在工作點附近有效。

- 誤區十：對驅動不足系統套用回授線性化。人形機器人的浮動基座、無人機的姿態，都無法精確線性化，需要其他方法。
- 誤區十一：在操作空間控制中使用運動學版本的零空間投影。力控制場合必須用動態一致版本，否則次任務會干擾主任務的接觸力。
- 誤區十二：以為滑模控制的顫振可以忽略。高頻切換會激發機構振動、加速磨損並產生噪音，實務上必須用邊界層緩解。

## 課堂討論

1. 開頭故事中，小好調好的增益在換姿態與加負載後都失效。請用  $\omega_n(q)$  與  $\zeta(q)$  的公式，定量解釋這兩種情況下系統動態各自如何改變。
2. 例題 17-2 顯示「一行重力補償勝過五倍增益」。請討論：還有哪些機器人控制問題適合用「模型」而非「增益」來解決？請舉兩例並說明理由。
3. 手動導引示教徹底改變了機器人的易用性。請討論它對「誰能使用機器人」的影響，並分析這個功能需要哪三項技術同時到位（提示：對照第 5、6、11 章）。
4. 四種進階方法（滑模、自適應、魯棒、MPC）各有適用場合。請為以下三個情境各選一種並說明理由：（a）機器人抓取重量未知的物件；（b）在有大幅外力干擾的環境作業；（c）四足機器人的即時步態控制。
5. 本章的哲學是「用模型處理已知、用回授處理未知」。請討論這個原則在機器人以外的領域有何對應（提示：可考慮天氣預報、庫存管理、或第 31 章的機器學習）。

## 章末摘要

- 機器人的四個非線性來源：慣性隨姿態變化、關節耦合、速度平方項、重力隨姿態變化。
- 固定增益 PID 的有效自然頻率與阻尼比隨姿態變化（ $\omega_n = \sqrt{K_p/M(q)}$ 、 $\zeta = K_d/(2\sqrt{K_p \cdot M(q)})$ ），因此不可能在所有姿態下都最佳。
- 計算力矩控制  $\tau = M(q)u + C\dot{q} + G$  使閉環系統成為  $\ddot{q} = u$ ，即  $n$  個獨立的雙積分器。
- 外層設計  $u = \ddot{q}_d + K_d \dot{e} + K_p e$ ，得到誤差動態  $\ddot{e} + K_d \dot{e} + K_p e = 0$ ——不含  $M$ 、 $C$ 、 $G$ ，與姿態負載完全無關。
- 實作上直接呼叫一次逆動力學  $\tau = \text{InverseDynamics}(q, \dot{q}, u)$ ，複雜度  $O(n)$ 。

- 回授線性化是精確的（整個狀態空間成立），不同於只在工作點附近有效的近似線性化；但需要完整模型且僅適用於完全驅動系統。
- 模型誤差造成兩個偏差：有效增益偏移與等效干擾；但 80% 準確的模型仍能把干擾降低五倍。
- 重力補償是投報率最高的改良：消除穩態誤差（純 PD 的  $e_{ss} = G/K_p$ ）、避免依賴積分項、計算量極小。
- Takegaki-Arimoto 定理：PD 加重力補償保證定點調節的全域漸近穩定，不需  $M$  與  $C$  的任何資訊。
- 純重力補償（ $\tau = G(q)$ ）即為零力控制，是協作機器人手動導引示教的原理，實作需同時補償摩擦。
- 三種控制律的選擇：低速固定負載用 PD 加重力補償；高速、負載變化或高精度需求用完整 CTC。
- 操作空間控制  $\tau = J^T[\Lambda \dot{u}_x + \mu + p]$  直接在任務空間設計，其中  $\Lambda = (J M^{-1} J^T)^{-1}$  為末端的等效慣量。
- 操作空間控制達成動態解耦，使各方向可獨立設計剛度——這是第 20 章力控制的基礎。
- 四種進階方法：滑模（強健但顫振）、自適應（估測參數但需持續激勵）、魯棒（保守）、MPC（能處理約束但計算量大）。
- 實務架構為多層複合：軌跡插補、動力學前饋、位置速度回授、電流迴路、安全監控；前饋用期望值、回授用實際值。
- 核心哲學：用模型處理已知的部分，用回授處理未知的部分。

## 觀念題與計算題

### 觀念題

1. 列舉機器人動力學的四個非線性來源，並各說明其對控制的影響。
2. 為何固定增益的 PID 無法在所有姿態下都達到最佳性能？請以  $\omega_n$  與  $\zeta$  的公式說明。
3. 推導計算力矩控制律，並證明其閉環系統為  $\ddot{q} = u$ 。
4. 為何說 CTC 的誤差動態「與姿態、速度、負載無關」？這帶來什麼實務好處？
5. 說明回授線性化與近似線性化的差異，以及各自的適用條件。

6. 什麼是完全驅動與驅動不足系統？為何後者無法精確回授線性化？
7. 分析模型誤差對 CTC 的兩種影響，並說明為何「80% 準確的模型仍然有用」。
8. 推導純 PD 控制在有重力時的穩態誤差公式，並說明重力補償如何消除它。
9. 敘述 Takegaki-Arimoto 定理的內容與適用範圍。
10. 說明零力控制的原理與三個實作要點，以及它對機器人易用性的意義。
11. 比較三種關節空間控制律的模型需求、計算量與適用場合。
12. 說明操作空間控制的優勢，並解釋  $\Lambda = (J M^{-1} J^T)^{-1}$  的物理意義。
13. 為何操作空間控制在奇異點處會失效？請由  $\Lambda$  的定義說明。
14. 比較滑模、自適應、魯棒三種方法的核心思想與主要缺點。
15. 說明 MPC 的四個步驟，以及它相對於前述方法的最大優勢。

### 計算題

1. 某平面 2R 手臂  $\alpha = 0.72$ 、 $\beta = 0.18$ 、 $\delta = 0.09 \text{ kg}\cdot\text{m}^2$ 。目前  $q = (45^\circ, 90^\circ)$ 、 $\dot{q} = (1.5, -0.8) \text{ rad/s}$ ，軌跡要求  $q_d = (48^\circ, 87^\circ)$ 、 $\dot{q}_d = (1.6, -0.7) \text{ rad/s}$ 、 $\ddot{q}_d = (3.0, 1.5) \text{ rad/s}^2$ 。取  $\omega_n = 25 \text{ rad/s}$ 、 $\zeta = 1$ 。(a) 求  $K_p$ 、 $K_d$ 。(b) 求虛擬控制  $u$ 。(c) 求慣性矩陣  $M$ 。(d) 求  $M \cdot u$ 。
2. 承上題手臂 ( $m_1 = 3.5$ 、 $m_2 = 2.2 \text{ kg}$ 、 $L_1 = 0.45$ 、 $L_2 = 0.35 \text{ m}$ 、質心在中點)。(a) 計算  $q = (0^\circ, 0^\circ)$  (水平完全伸展) 姿態下的重力項  $G$ 。(b) 若採純 PD 且  $K_p = 400 \text{ N}\cdot\text{m/rad}$ ，求穩態誤差 (度)。(c) 若要求誤差小於  $0.3^\circ$ ，需要多大的  $K_p$ ？(d) 計算該  $K_p$  下的自然頻率，並判斷是否可能激發  $15 \text{ Hz}$  的機構共振。
3. 某手臂在末端加裝了一個未建模的夾具，質量  $0.8 \text{ kg}$ 、質心距法蘭  $0.12 \text{ m}$ 。已知該姿態下  $M = [[0.85, 0.15], [0.15, 0.08]]$ 、 $L_1 = 0.4 \text{ m}$ 、 $c_1 = 0.7$ 、 $c_{12} = 0.3$ 。(a) 估算重力項的誤差  $\Delta G$ 。(b) 求  $M$  的反矩陣。(c) 計算等效干擾加速度。(d) 若 CTC 的  $K_p = 625$  ( $\omega_n = 25$ )，求穩態位置誤差。
4. 某六軸手臂的控制器需在  $1 \text{ kHz}$  執行。已知：逆動力學需 900 次浮點運算、雅可比需 400 次、 $\Lambda$  矩陣的計算 (含  $M^{-1}$  與  $3 \times 3$  求逆) 需約 1200 次。處理器每秒可執行  $10^8$  次運算。(a) 計算關節空間 CTC 的運算時間與週期佔比。(b) 計算操作空間控制的運算時間與佔比。(c) 若還需加上碰撞偵測 ( $500 \mu\text{s}$ )，判斷兩者是否仍可行。(d) 提出一個降低操作空間控制計算量的策略。



## 動手做與專題延伸

### 本章實作：控制器模組

延續第 13 ~ 16 章的程式庫，建立完整的控制器模組。這將是第 18 ~ 20 章的實驗平台。

- **階段一**：實作三種關節空間控制器（獨立 PID、PD 加重力補償、CTC），統一介面為  $\text{ctrl}(q, \dot{q}, \ddot{q}, \text{ref}) \rightarrow \tau$ 。
- **階段二**：建立模擬環境，整合第 15 章的軌跡生成與第 14 章的正向動力學，能執行完整的閉環模擬並記錄誤差曲線。
- **階段三**：加入模型誤差的模擬能力（真實系統與控制器模型使用不同參數），量化各控制器的強健性。
- **階段四**：實作操作空間控制，包含  $\Lambda$  矩陣計算與  $J^T$  映射，並與關節空間控制比較末端的追隨精度。
- **階段五**：實作力矩飽和與抗飽和機制，觀察飽和對 CTC 性能的破壞程度。

### 專題延伸

- **性能地圖**：對整個工作空間掃描，計算 PID 與 CTC 在各姿態下的追隨誤差，繪成兩張熱力圖並列比較。你會看到 PID 的誤差分布極不均勻（在慣量變化大的區域惡化），而 CTC 的分布相對均勻——這是本章理論最直觀的視覺證明。
- **模型誤差的敏感度分析**：系統性地為每一個動力學參數（各連桿的質量、質心、慣量）加入  $\pm 20\%$  的誤差，分別量測 CTC 的追隨誤差變化。找出「最敏感的參數」——這正是實務上參數辨識時應優先確保精度的項目，可與第 9 章的運動學敏感度分析對照。
- **進階：自適應控制的實作與觀察**。實作 Slotine-Li 自適應控制器，在末端加上未知負載的情境下執行。繪出「參數估計值隨時間收斂」的曲線與「追隨誤差隨時間下降」的曲線。特別觀察：若軌跡過於單調（例如只在單一方向來回），參數可能不會收斂到真值——這就是「持續激勵條件」的實際體現。設計一段「豐富」的軌跡再試一次，比較兩者的差異。

## 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 10 章：非線性控制，本章主要參考)。



- Khatib, O., "A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation," IEEE J. Robotics and Automation, 1987 ( 操作空間控制的奠基論文 ) 。
- Takegaki, M. and Arimoto, S., "A New Feedback Method for Dynamic Control of Manipulators," ASME J. Dynamic Systems, 1981 ( PD 加重力補償的穩定性證明 ) 。
- Slotine, J.-J. E. and Li, W., Applied Nonlinear Control, Prentice Hall ( 滑模與自適應控制的經典教材 ) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley ( 第 8 ~ 10 章：非線性與多變數控制 ) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer ( 第 8 章：運動控制，含各類控制律的完整比較 ) 。
- Slotine, J.-J. E. and Li, W., "On the Adaptive Control of Robot Manipulators," IJRR, 1987 ( Slotine-Li 自適應控制器 ) 。
- Nakanishi, J. et al., "Operational Space Control: A Theoretical and Empirical Comparison," IJRR, 2008 ( 多種操作空間控制方法的實證比較 ) 。
- Rawlings, J. B., Mayne, D. Q. and Diehl, M., Model Predictive Control: Theory, Computation, and Design, Nob Hill ( MPC 的權威教材 ) 。
- Di Carlo, J. et al., "Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control," IROS 2018 ( MPC 在四足機器人上的代表性應用 ) 。

## 第 18 章

# 最佳控制與 Riccati 方程

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】畫面中央為一台小車上立著一根桿子的倒立擺，桿子微微右傾、小車正往右加速追趕，並以虛線標出「支點必須移到重心下方」。左側疊一張取舍曲線圖（橫軸為最大控制力、縱軸為穩定時間），曲線上標出三個點： $Q$  小（省力但慢）、 $Q$  適中、 $Q$  大（快但費力），並註明「這條線上的每一點都是最佳的——選哪一個取決於你在乎什麼」。右側為一個由後往前計算的網格圖，紅色箭頭由終點往起點逐層推算價值函數。標題文字：不再試誤——把「什麼叫好」寫成公式，最佳解就會自己出現。

## 叫獸開講 「輕便車上的倒立搖桿，如何自己找出最省力的平衡之道？」

小翰把第 17 章的計算力矩控制寫完了，模擬跑得很漂亮。但他有一個問題卡了三天。

「老師，公式說  $K_p = \omega n^2$ 、 $K_d = 2\zeta\omega n$ 。可是  $\omega n$  該取多少？我試了 10、20、50、100——每一個看起來都『可以』，但我不知道哪一個是對的。」

叫獸沒有直接回答，而是在白板上畫了兩條曲線。「 $\omega n$  取 100 的時候，誤差收斂得很快對不對？那你有沒有看馬達的力矩？」

小翰想了想：「……很大。有時候會撞到飽和。」

「那  $\omega n$  取 10 呢？」「力矩很小，但誤差要好幾秒才收斂。」

「所以，」叫獸把兩條曲線並排，「這根本不是一個『對或錯』的問題，而是一個**取捨**的問題。你想要收斂快，就要付出大力矩的代價；你想要省力，就要忍受慢。而『哪一個最好』——**取決於你認為快有多重要、省力有多重要。**」

他在白板上寫下一個式子： $J = \int (\text{誤差的代價} + \text{用力的代價}) dt$ 。「工程上有一個很優雅的作法：把你的兩個願望寫成一個**代價函數**。誤差大要罰、用力大也要罰。然後問：有沒有一組控制律，能讓這個總代價最小？」

小好舉手：「這聽起來像最佳化。可是控制是一個隨時間變化的過程，不是一個固定的數值——要怎麼對『一整條軌跡』做最佳化？」

「好問題。這正是最佳控制與一般最佳化最大的差別。」叫獸拿出一顆玻璃彈珠與一段彎曲的軌道，「想像彈珠要從 A 滾到 B。你不是在選一個數字，你是在選**一整條路徑**——而路徑有無限多條。」

他停頓了一下，「1950 年代，一位叫 Bellman 的數學家想出了一個絕妙的辦法。他說：不要一次想整條路。**先問一個問題——如果我已經走到中途某個點，剩下的路該怎麼走最好？**如果你能回答『從任何一點出發的最佳走法』，那整條路的答案就自動出來了。」

「這個想法叫做**動態規劃**，它的核心是一句話：**最佳路徑的任何一段，本身也必定是最佳的。**聽起來像廢話，但它把一個無限維的問題，變成了一個可以遞迴求解的問題。」

叫獸在白板角落畫了一台小車，上面立著一根桿子。「今天結束時，你們會親手算出一個東

西——一個能自動平衡倒立擺的控制器。而且不是靠試誤調參數，是算出來的最佳解。」

## 本章為什麼重要

第 16、17 章教會了我們設計控制器的結構，卻留下一個懸而未決的問題：參數該怎麼選？增益調高則反應快但耗能大、容易飽和、對雜訊敏感；調低則省力但遲鈍。這個取捨沒有唯一答案——除非我們先明確定義「什麼叫好」。

最佳控制提供了這個框架。它要求設計者把目標寫成一個代價函數，然後在數學上求出使代價最小的控制律。這帶來三個層次的價值：

**價值一：從試誤到計算。**LQR（線性二次調節器）能由系統模型與兩個權重矩陣，直接算出最佳的回授增益——不需要 Ziegler-Nichols 那樣的試誤。對多輸入多輸出系統（如六軸手臂、四足機器人），這個優勢尤其明顯，因為手動調參幾乎不可行。

**價值二：處理欠驅動系統。**第 17 章的計算力矩控制只適用於完全驅動系統。但倒立擺、兩輪自平衡車、四旋翼無人機、人形機器人的浮動基座——這些都是欠驅動的，回授線性化行不通。LQR 則能直接處理。

**價值三：連結多個領域。**本章的 Bellman 方程與價值函數，正是第 31 章強化學習的數學基礎；而 MPC（第 17.8.4 節）本質上就是「有限時域的最佳控制反覆求解」。理解本章，等於同時打開了三扇門。

## 學習目標

- 能定義最佳控制問題的三個要素：系統動態、代價函數與邊界條件。
- 能說明離散與連續時間最佳控制問題的差異。
- 能敘述 Bellman 最佳性原理，並說明其為何能簡化問題。
- 能定義價值函數，並說明其物理意義。
- 能推導離散時間的 Bellman 方程並用動態規劃求解簡單問題。
- 能寫出 Hamilton-Jacobi-Bellman 方程並說明其意義與求解困難。
- 能定義線性二次調節器問題，並說明  $Q$ 、 $R$  權重矩陣的作用。
- 能推導 Riccati 微分方程，並說明有限時域與無限時域的差異。

- 能求解代數 Riccati 方程並計算最佳回授增益  $K$ 。
- 能將 LQR 應用於一維點質量與倒立擺，並解釋所得增益的物理意義。
- 能說明最佳控制與逆運動學、軌跡最佳化及強化學習的關係。

## 先備知識

第 13~14 章的動力學模型（提供系統方程）、第 16 章的二階系統分析與增益的物理意義、第 17 章的控制架構。數學方面需要：矩陣運算、偏導數、以及狀態空間表示法的基本概念（本章 18.2 節會完整介紹）。

本章是全書數學密度最高的章節之一。建議讀法：**18.1~18.3 節建立直覺，務必徹底理解 Bellman 原理**；18.4~18.6 節的推導可先掌握結論與意義，細節之後回頭補；18.7 節之後回到具體應用，會重新變得直觀。若時間有限，至少要讀懂 18.6 節的 LQR 與 18.8 節的實例。

## 核心名詞中英對照

表 18-1 本章核心名詞

| 中文            | 英文                               | 說明                  |
|---------------|----------------------------------|---------------------|
| 最佳控制          | Optimal Control                  | 求使代價函數最小的控制律        |
| 代價函數          | Cost Function                    | 衡量控制表現好壞的量化指標       |
| 狀態            | State                            | 完整描述系統當前情況的變數集合     |
| 狀態空間表示        | State-space Representation       | 以一階微分方程組描述系統的形式     |
| 價值函數          | Value Function                   | 由某狀態出發的最小可能代價       |
| 最佳性原理         | Principle of Optimality          | 最佳路徑的任一段本身也是最佳的     |
| 動態規劃          | Dynamic Programming              | 由後往前遞迴求解最佳決策的方法     |
| Bellman 方程    | Bellman Equation                 | 價值函數所滿足的遞迴關係式       |
| HJB 方程        | Hamilton-Jacobi-Bellman Equation | 連續時間版本的 Bellman 方程  |
| 線性二次調節器       | Linear Quadratic Regulator, LQR  | 線性系統加二次代價的最佳控制      |
| Riccati 方程    | Riccati Equation                 | LQR 中決定最佳增益的矩陣方程    |
| 代數 Riccati 方程 | Algebraic Riccati Equation, ARE  | 無限時域下的穩態 Riccati 方程 |
| 狀態回授          | State Feedback                   | 以 $u = -Kx$ 形式的控制律  |
| 權重矩陣          | Weight Matrix                    | 代價函數中調整各項相對重要性的矩陣   |
| 倒立擺           | Inverted Pendulum                | 重心在支點上方的不穩定系統       |

| 中文  | 英文            | 說明            |
|-----|---------------|---------------|
| 欠驅動 | Underactuated | 控制輸入數少於自由度的系統 |

## 18.1 最佳控制問題的定義

### 18.1.1 三個要素

一個最佳控制問題由三個要素構成。

**要素一：系統動態。**描述狀態如何隨控制輸入演變。連續時間形式為：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

$\mathbf{x}$  為狀態向量、 $\mathbf{u}$  為控制輸入。對機器人而言， $\mathbf{x}$  通常包含位置與速度（如  $\mathbf{x} = [q, \dot{q}]^T$ ）， $\mathbf{u}$  則是關節力矩。

**要素二：代價函數。**量化「什麼叫好」。標準形式為：

$$J = \varphi(\mathbf{x}(T)) + \int_0^T L(\mathbf{x}, \mathbf{u}) dt$$

$\varphi$  稱為終端代價（到達終點時的懲罰）、 $L$  稱為運行代價（過程中每一刻的懲罰）。

**要素三：邊界條件與約束。**初始狀態  $\mathbf{x}(0) = \mathbf{x}_0$ 、可能的終端條件、以及控制與狀態的限制（如力矩極限）。

問題就是：找出控制訊號  $\mathbf{u}(t)$ （ $t$  由 0 到  $T$ ），使  $J$  最小。

### 18.1.2 與一般最佳化的差別

這與第 10 章的逆運動學最佳化有一個本質差異。逆運動學是在有限維空間中找一個點（一組關節角度）；最佳控制則是在函數空間中找一條曲線（整條控制訊號  $\mathbf{u}(t)$ ）。這是無限維的問題。

正如叫獸的彈珠比喻：你不是在選一個數字，而是在選一整條路徑，而路徑有無限多條。這使得直接套用梯度下降之類的方法變得困難——你要對「一個函數」求導數。

Bellman 的貢獻，正是提供了一個繞過這個困難的方法。

### 18.1.3 為何機器人需要最佳控制

在機器人上，最佳控制回答的問題形形色色：

- **最省能：**以最少的能量完成搬運（電池供電的移動機器人特別在意）。

- **最快速**：在力矩極限下用最短時間完成動作（第 15.8.3 節的時間最佳軌跡）。
- **最平順**：最小化急跳度以減少振動（第 15.5 節）。
- **最穩定**：在維持平衡的前提下最小化控制量（倒立擺、雙足行走）。
- **多目標權衡**：同時考慮精度、能耗與時間，以權重調整優先順序。

這些看似不同的需求，在最佳控制的框架下都只是「換一個代價函數」而已——這正是這個框架的威力所在。

## 18.2 狀態空間表示法

### 18.2.1 把高階系統化為一階

最佳控制理論建立在狀態空間表示上。它的核心技巧是：把任何高階微分方程，化為一組一階微分方程。

以第 16 章的一維點質量為例，其方程為二階： $m \ddot{x} = f$ 。定義狀態向量：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \text{位置} \\ \text{速度} \end{bmatrix}$$

狀態向量。選擇位置與速度作為狀態，是機械系統的標準作法。

則系統可寫成兩個一階方程：

$$\begin{aligned} \dot{x}_1 &= x_2 \quad (\text{位置的導數是速度}) \\ \dot{x}_2 &= f/m \quad (\text{速度的導數是加速度}) \end{aligned}$$

化為一階系統。

寫成矩陣形式：

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u \quad \text{即} \quad \dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} u$$

線性狀態空間形式。 $\mathbf{A}$  稱為系統矩陣、 $\mathbf{B}$  稱為輸入矩陣。這是控制理論中最標準的表示法。

### 18.2.2 為何要這樣做

這個轉換有三個好處。**好處一：統一形式**。無論原系統是幾階，都化為  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, u)$  的一階形式，理論可以統一處理。**好處二：完整描述**。狀態的定義是「知道現在的狀態與未來的輸入，就能完全決定未來」——這正是 Bellman 原理成立的前提。**好處三：矩陣工具**。線性系統可用線性代數的全套工具分析（特徵值、可控性、可觀測性——第 19 章的主題）。

### 18.2.3 機器人的狀態空間

對  $n$  自由度的機器手臂，狀態向量為  $2n$  維：

$$\mathbf{x} = \begin{bmatrix} \mathbf{q} \end{bmatrix} \in \mathbb{R}^{2n}, \quad \mathbf{u} = \boldsymbol{\tau} \in \mathbb{R}^n$$

狀態含所有關節的位置與速度。六軸手臂的狀態空間是 12 維的。

其動態方程來自第 13 章：

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C} \dot{\mathbf{q}} - \mathbf{G}) \end{bmatrix}$$

機器人的狀態空間形式。注意這是非線性的（因  $\mathbf{M}$ 、 $\mathbf{C}$ 、 $\mathbf{G}$  都是  $\mathbf{q}$  的函數）。

這帶出一個重要的實務策略：若已經套用第 17 章的計算力矩控制，系統就變成了  $\mathbf{q}'' = \mathbf{u}$ ，此時狀態方程化為線性的雙積分器——LQR 就能直接應用。這是模型式控制與最佳控制結合的標準作法。

## 18.3 Bellman 最佳性原理

### 18.3.1 核心洞見

Bellman 於 1950 年代提出的最佳性原理，可以用一句話表達：

#### ◎ Bellman 最佳性原理

最佳路徑的任何一段，本身也必定是最佳的。換句話說：無論之前如何到達當前狀態，剩下的最佳決策只取決於當前狀態，而與過去的歷史無關。

這個原理有一個生活化的驗證。假設從台北到高雄的最快路線經過台中。那麼「台中到高雄」這一段，必定也是台中到高雄的最快路線——否則你可以把它替換成更快的走法，使整條路線更快，這與原路線是最快的假設矛盾。

這個看似顯然的觀察，威力極大：它把「一次決定整條路徑」的無限維問題，轉化為「在每個狀態決定下一步」的遞迴問題。

### 18.3.2 價值函數

為了把原理數學化，定義價值函數（value function）：

$$V(\mathbf{x}, t) = \min_{\mathbf{u}} \int_t^T L(\mathbf{x}, \mathbf{u}) d\tau + \varphi(\mathbf{x}(T))$$

由時刻  $t$ 、狀態  $\mathbf{x}$  出發，若之後都採取最佳決策，所能達到的最小總代價。



價值函數的意義非常直觀：它是「站在這個狀態，未來最少還要付出多少代價」。狀態越好（離目標近、速度合適），價值函數越小。

這個概念在第 31 章的強化學習中會再次出現，且是核心中的核心。強化學習的許多演算法（Q-learning、DQN、Actor-Critic）本質上都是在估計價值函數——只是它們用資料學習，而本章用模型計算。

### 18.3.3 為何價值函數如此有用

如果我們知道了價值函數，最佳控制就變得極為簡單：在每一個時刻，選擇讓「當下代價 + 下一步的價值」最小的控制輸入。這是一個局部的、有限維的最佳化——遠比原本的無限維問題容易。

$$u^*(x, t) = \operatorname{argmin}_u [L(x, u) + (\text{下一步的價值})]$$

*最佳控制律。只需在當前狀態做一次局部最佳化。*

這種「由價值函數導出控制律」的結構稱為**回授形式的解**——它給出的不是一條預定的軌跡，而是一個依狀態決定動作的規則。這比開迴路的軌跡強健得多：即使系統受到干擾偏離了原路徑，控制律仍然知道該怎麼做。

## 18.4 動態規劃與 Bellman 方程

### 18.4.1 離散時間的遞迴關係

在離散時間下，Bellman 原理可以寫成一個精確的遞迴關係。設系統為  $x_{k+1} = f(x_k, u_k)$ ，代價為  $\sum L(x_k, u_k)$ ，則：

$$V_k(x) = \min_u [L(x, u) + V_{k+1}(f(x, u))]$$

*Bellman 方程（離散時間）。當前的價值 = 當下的代價 + 下一步的價值（取最小）。*

這條式子是本章的核心。它的結構極其優雅：把一個跨越  $N$  步的最佳化問題，化為  $N$  次「只看一步」的最佳化。

### 18.4.2 由後往前求解

Bellman 方程的求解方向是由後往前（backward），這是初學者最不習慣的一點。

原因是：終點的價值是已知的（ $V_N(x) = \varphi(x)$ ，即終端代價）。有了  $V_N$ ，就能用

Bellman 方程算出  $V_{N-1}$ ；有了  $V_{N-1}$ ，就能算出  $V_{N-2}$ ——如此逐步往前推。

1. 初始化： $V_N(x) = \varphi(x)$ （終端代價，通常已知）。
2. 遞迴：對  $k = N-1, N-2, \dots, 0$ ，計算  $V_k(x) = \min_u [L(x,u) + V_{k+1}(f(x,u))]$ ，同時記錄使其最小的  $u^*_k(x)$ 。
3. 輸出：得到每個時刻、每個狀態下的最佳控制  $u^*_k(x)$ ——這是一個完整的控制策略。

這個「由後往前」的邏輯有一個直觀的解釋：要知道現在該怎麼走，必須先知道未來的每一步值多少。就像下棋時要往後推算——先想清楚殘局怎麼贏，才能決定現在該走哪一步。

圖 18-1 動態規劃的由後往前求解

建議配圖：一個網格狀的狀態—時間圖。橫軸為時間步  $k = 0$  到  $N$ ，縱軸為離散化的狀態。最右側 ( $k = N$ ) 的每個狀態標註其終端代價值。以紅色箭頭由右往左，逐層計算各狀態的價值函數，每個節點旁標註  $V_k(x)$  的數值與最佳動作  $u^*$ 。左下角另附一個小圖，顯示由  $x_0$  出發沿最佳策略走出的路徑（綠色粗線）。標題文字：先知未來值多少，才知道現在該怎麼走。

### 18.4.3 維數詛咒

動態規劃在理論上完美，但有一個致命的實務問題：必須把狀態空間離散化並逐格計算。

若每個狀態維度分成 100 格，則：一維系統需要 100 格、二維需要  $10^4$  格、六維需要  $10^{12}$  格——而六軸手臂的狀態空間是 12 維的，需要  $10^{24}$  格。這遠超過任何電腦的能力。

這就是 Bellman 自己命名的維數詛咒 (curse of dimensionality)。它是第 26 章路徑規劃放棄格點法、改用取樣法 (PRM、RRT) 的原因，也是第 31 章強化學習用神經網路近似價值函數的原因。

那麼動態規劃還有什麼用？答案是：在特殊結構下，Bellman 方程有封閉解。而最重要的一個特例，就是「線性系統 + 二次代價」——這正是 18.6 節的 LQR。

## 18.5 Hamilton-Jacobi-Bellman 方程

### 18.5.1 由離散走向連續

18.4 節的 Bellman 方程是離散時間的。把時間步長  $\Delta t$  趨近於零，就得到連續時間的版本。

推導的思路是：在一個微小的時間  $\Delta t$  內，代價約為  $L(x,u) \cdot \Delta t$ ，而狀態變化約為  $\dot{x} \cdot \Delta t$ 。代入離散的 Bellman 方程並對價值函數做泰勒展開：

$$V(\mathbf{x}, t) = \min_{\mathbf{u}} [ L(\mathbf{x}, \mathbf{u})\Delta t + V(\mathbf{x} + \mathbf{f}(\mathbf{x}, \mathbf{u})\Delta t, t + \Delta t) ]$$

離散形式。展開右邊的  $V$  並保留一階項。

$$V(\mathbf{x}, t) \approx \min_{\mathbf{u}} [ L\cdot\Delta t + V(\mathbf{x}, t) + (\partial V/\partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u})\Delta t + (\partial V/\partial t)\Delta t ]$$

泰勒展開。兩邊的  $V(\mathbf{x}, t)$  消去，再除以  $\Delta t$  並令  $\Delta t \rightarrow 0$ ：

$$-\partial V/\partial t = \min_{\mathbf{u}} [ L(\mathbf{x}, \mathbf{u}) + (\partial V/\partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u}) ]$$

*Hamilton-Jacobi-Bellman (HJB) 方程。這是連續時間最佳控制的基本方程。*

### 18.5.2 方程的解讀

HJB 方程雖然形式緊湊，但每一項都有明確的意義。

左邊的  $-\partial V/\partial t$  表示「價值隨時間的變化率」——隨著時間推進、剩餘時間減少，價值通常會下降。

右邊括號中的第一項  $L(\mathbf{x}, \mathbf{u})$  是當下的代價；第二項  $(\partial V/\partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u})$  是「因為狀態移動而導致的價值變化」。控制輸入  $\mathbf{u}$  的選擇，就是在這兩者之間權衡：用力大則當下代價高，但能把狀態推向價值更低的方向。

這正是所有控制取舍的數學本質。第 17 章的增益選擇問題，在這裡得到了精確的表述。

### 18.5.3 為何 HJB 難解

HJB 是一個偏微分方程，且是非線性的（因為含  $\min$  運算）。對一般的  $\mathbf{f}$  與  $L$ ，它沒有解析解，數值求解又面臨 18.4.3 節的維數詛咒。

因此實務上有三條路：

- **特例求解**：找出有封閉解的特殊情形——這就是 LQR（18.6 節）。
- **近似求解**：用函數近似器（如神經網路）逼近價值函數——這是第 31 章深度強化學習的路線。
- **局部求解**：不求全域的價值函數，只在當前狀態附近做有限時域的最佳化——這就是 MPC（第 17.8.4 節）。

這三條路涵蓋了現代機器人控制的主要方法。理解它們共同的起點是 HJB，就能看清這些看似不同的技術之間的血緣關係。

## 18.6 線性二次調節器 (LQR)

### 18.6.1 問題設定

LQR 是 HJB 方程有封閉解的最重要特例。它要求兩個條件：**系統是線性的、代價是二次的**。

$$\text{系統：}\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}$$

$$\text{代價：} J = \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt$$

*LQR 問題。Q 為狀態權重矩陣 (半正定)、R 為控制權重矩陣 (正定)。*

代價函數的兩項意義極為直觀：

$\mathbf{x}^T \mathbf{Q} \mathbf{x}$ ：狀態偏離的懲罰。Q 越大，代表越不能容忍誤差，控制器會更積極。若 Q 是對角矩陣，其對角元素分別代表各個狀態變數的重要程度。

$\mathbf{u}^T \mathbf{R} \mathbf{u}$ ：控制用力的懲罰。R 越大，代表越不希望用大力氣，控制器會更保守。R 必須正定 (不能為零)，否則最佳解會是無限大的控制輸入。

因此 Q/R 的比值決定了控制器的積極程度——這正是叫獸開場時說的「取決於你認為快有多重要、省力有多重要」的數學化。

### 18.6.2 猜測解的形式

求解 HJB 的關鍵技巧是「猜」出價值函數的形式。對 LQR 問題，一個合理的猜測是二次型：

$$V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$$

*猜測價值函數為二次型。P 為對稱正定矩陣 (待求)。*

這個猜測的合理性來自：系統是線性的、代價是二次的，因此積分後的總代價「應該」也是二次的。(嚴格的證明需要驗證這個解確實滿足 HJB 方程——下面的推導就是驗證過程。)

代入 HJB 方程 (無限時域下  $\partial V / \partial t = 0$ )：

$$0 = \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + (2\mathbf{P}\mathbf{x})^T (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u})]$$

*代入  $\partial V / \partial \mathbf{x} = 2\mathbf{P}\mathbf{x}$ 。*

### 18.6.3 求最佳控制律

先對 u 求極小。對括號內的式子關於 u 求偏導並令其為零：

$$\partial / \partial \mathbf{u} [\mathbf{u}^T \mathbf{R} \mathbf{u} + 2\mathbf{x}^T \mathbf{P} \mathbf{B} \mathbf{u}] = 2\mathbf{R} \mathbf{u} + 2\mathbf{B}^T \mathbf{P} \mathbf{x} = 0$$

一階必要條件。

$$\mathbf{u}^* = -\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\mathbf{x} \quad \text{即} \quad \mathbf{u}^* = -\mathbf{K}\mathbf{x}, \text{ 其中 } \mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$$

*最佳控制律。它是一個線性的狀態回授——這是 LQR 最重要的結果。*

這個結果值得強調：**最佳控制律是狀態的線性函數**。它不是一條預先算好的軌跡，而是一個「看當前狀態決定用多少力」的規則。這正是 18.3.3 節提到的回授形式解，具有天然的抗干擾能力。

#### 18.6.4 Riccati 方程

把  $\mathbf{u}^*$  代回 HJB 方程，整理後得到  $\mathbf{P}$  必須滿足的條件：

$$\mathbf{A}^T\mathbf{P} + \mathbf{P}\mathbf{A} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{Q} = 0$$

*代數 Riccati 方程 (ARE)。*這是一個矩陣二次方程，解出  $\mathbf{P}$  後即可得增益  $\mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$ 。

這條方程是本章的核心成果。它的意義是：**給定系統模型 (A、B) 與設計者的偏好 (Q、R)，最佳增益就唯一確定了**——不需要試誤，直接解方程即可。

ARE 的求解在數值上是成熟的問題，所有控制軟體都內建 (MATLAB 的 `lqr`、Python 的 `scipy.linalg.solve_continuous_are`)。實務上工程師只需要選擇  $\mathbf{Q}$  與  $\mathbf{R}$ ，剩下交給求解器。

##### ◎ 為何叫 Riccati

這個名稱來自十八世紀義大利數學家 Jacopo Riccati，他研究的是形如  $\dot{y} = a(t) + b(t)y + c(t)y^2$  的一階非線性微分方程。控制理論中的矩陣版本 (含二次項  $\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$ ) 與之結構相同，因此沿用其名。有趣的是，Riccati 本人研究它時純粹出於數學興趣，完全無法預見兩百年後它會成為太空船導航、飛機自動駕駛與機器人控制的核心——這是純數學意外造福工程的又一個例子。

#### 18.6.5 有限時域與微分 Riccati 方程

若代價函數的積分只到有限時間  $T$  (而非無限)，則價值函數會隨時間變化， $\mathbf{P}$  也成為時間的函數。此時 Riccati 方程變成微分方程：

$$-\dot{\mathbf{P}}(t) = \mathbf{A}^T\mathbf{P} + \mathbf{P}\mathbf{A} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{Q}, \quad \text{終端條件 } \mathbf{P}(T) = \mathbf{Q}_f$$

*微分 Riccati 方程。*需由終點往回積分求解 (呼應 18.4.2 節的「由後往前」)。

此時增益也隨時間變化： $\mathbf{K}(t) = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}(t)$ 。一個有趣的現象是：**當  $T$  夠大時， $\mathbf{P}(t)$  在大部分時間內會趨於一個常數值**——正是 ARE 的解。只有在接近終點的一小段時間， $\mathbf{P}$  才會變化。

這解釋了為何實務上多用無限時域版本：對於持續運行的系統（如平衡控制、軌跡追隨），常數增益已經是最佳的，不需要每個時刻重算。

表 18-2 有限時域與無限時域 LQR 的比較

| 面向         | 有限時域              | 無限時域         |
|------------|-------------------|--------------|
| 代價積分       | 0 到 T             | 0 到 $\infty$ |
| Riccati 方程 | 微分方程（需反向積分）       | 代數方程（一次解出）   |
| 增益 K       | 隨時間變化 $K(t)$      | 常數 K         |
| 終端代價       | 可指定 $Q_f$         | 無（隱含於穩定性要求）  |
| 計算量        | 較大（需儲存整條 $K(t)$ ） | 小（一組常數）      |
| 適用場合       | 有明確終點的任務（如著陸）     | 持續運行（平衡、追隨）  |

## 18.7 權重矩陣的選擇

### 18.7.1 從物理意義出發

LQR 把「調增益」的問題轉化為「選  $Q$  與  $R$ 」的問題。這是否只是換湯不換藥？並非如此——因為  $Q$  與  $R$  有明確的物理意義，比抽象的增益容易選擇得多。

實務上最常用的作法是採用對角矩陣，並用 Bryson 法則設定初值：

$$Q_{ii} = 1 / (x_{i,max})^2, \quad R_{jj} = 1 / (u_{j,max})^2$$

Bryson 法則。 $x_{i,max}$  是「第  $i$  個狀態可容許的最大偏差」、 $u_{j,max}$  是「第  $j$  個控制輸入可接受的最大值」。

這個法則的邏輯是：把每一項都正規化，使各項在達到其容許上限時對代價的貢獻相當。例如容許位置誤差 0.01 m、容許力矩 20 N·m，則  $Q_{11} = 1/0.0001 = 10000$ 、 $R_{11} = 1/400 = 0.0025$ 。

Bryson 法則給出的是起點，之後再依實測結果微調。但相較於盲目試誤，它通常能在一兩次調整內得到可用的結果。

### 18.7.2 調整的直覺

有了初值後，調整的方向非常直觀：

表 18-3 LQR 權重調整的效果

| 觀察到的現象   | 原因        | 調整方向                         |
|----------|-----------|------------------------------|
| 收斂太慢     | 狀態懲罰不足    | 提高 $Q$ ( 或降低 $R$ )           |
| 力矩太大、常飽和 | 控制懲罰不足    | 提高 $R$ ( 或降低 $Q$ )           |
| 某個狀態誤差偏大 | 該狀態權重不足   | 提高該狀態對應的 $Q_{ii}$            |
| 振盪       | 速度狀態的懲罰不足 | 提高速度對應的 $Q_{ii}$ ( 等效於加大阻尼 ) |
| 對雜訊過度反應  | 增益過高      | 整體提高 $R$                     |

一個實用的技巧是：只調整  $Q$  與  $R$  的比值，而非絕對值。因為 LQR 的解只取決於  $Q/R$  的相對大小——把兩者同時乘以 10，得到的增益完全相同。因此實務上常固定  $R = I$ ，只調整  $Q$ 。

### 18.7.3 LQR 與 PD 的關係

一個有啟發性的觀察：對一維點質量系統，LQR 的解恰好就是一個 PD 控制器。

回顧 18.2.1 節，一維點質量的狀態為  $x = [\text{位置}, \text{速度}]^T$ 。LQR 的控制律  $u = -Kx$  展開後為：

$$u = -k_1 \cdot (\text{位置}) - k_2 \cdot (\text{速度})$$

這正是 PD 控制的形式： $k_1$  對應  $K_p$ 、 $k_2$  對應  $K_d$ 。

因此 LQR 並沒有發明新的控制器結構，而是告訴我們該用多大的  $K_p$  與  $K_d$ 。第 17 章留下的問題，在此得到了回答。

更重要的是，LQR 的優勢在多變數系統中才完全展現。對六軸手臂（12 維狀態、6 個輸入）， $K$  是一個  $6 \times 12$  的矩陣，含 72 個增益——手動調參幾乎不可能，但 LQR 能一次算出全部，且保證是該代價函數下的最佳解。

## 18.8 LQR 的性質與限制

### 18.8.1 三個優良性質

**性質一：保證穩定。**只要系統可控（第 19 章會嚴格定義）且  $Q$ 、 $R$  選得合理，LQR 閉環系統必定穩定。這是一個強而有力的保證——不需要另外驗證。

**性質二：良好的裕度。**LQR 閉環系統有一個著名的結果：增益裕度至少為  $[0.5, \infty)$ 、相位裕度至少  $60^\circ$ 。這代表即使實際的系統增益是模型的一半或無限倍，系統仍然穩定——這是相當寬裕的容錯能力。

**性質三：解的唯一性。**ARE 的正定解是唯一的，因此最佳增益也是唯一的。不會有「不同的初值收斂到不同解」的問題。

## 18.8.2 四個實務限制

**限制一：需要線性模型。**機器人本質上是非線性的。因應之道有二：在工作點附近做**近似線性化**（第 19 章），或先用第 17 章的計算力矩控制把系統線性化為  $\ddot{\mathbf{q}} = \mathbf{u}$ ，再套用 LQR。後者是機器人上的標準作法。

**限制二：需要完整的狀態量測。**LQR 的控制律  $\mathbf{u} = -\mathbf{K}\mathbf{x}$  需要知道所有狀態。若某些狀態無法直接量測（如速度雜訊太大），必須加入**狀態觀測器**（如 Kalman 濾波器，第 24 章）。LQR 加上 Kalman 濾波器的組合稱為 **LQG**（Linear Quadratic Gaussian），是經典控制理論的巔峰成果之一。

**限制三：無法處理約束。**LQR 的推導假設  $\mathbf{u}$  可以取任意值。實際的力矩極限、關節限位都無法納入。若最佳解超出極限而被截斷，性能保證就失效——這正是 MPC 存在的理由（第 17.8.4 節）。

**限制四：二次代價未必反映真實目標。**有些目標本質上不是二次的，例如「時間最短」（第 15.8.3 節）或「絕不能超過某個誤差」。強行用二次代價近似，可能得到不理想的結果。

**表 18-4 LQR 與其他控制方法的比較**

| 方法      | 增益來源                    | 處理非線性 | 處理約束  | 計算量      | 適用場合         |
|---------|-------------------------|-------|-------|----------|--------------|
| 手動調 PID | 試誤                      | 否     | 否     | 極低       | 單輸入單輸出、低要求   |
| 計算力矩控制  | 指定 $\omega_n$ 、 $\zeta$ | 是（抵消） | 否     | 中        | 完全驅動的機械手臂    |
| LQR     | 解 ARE                   | 需先線性化 | 否     | 低（離線算好）  | 多變數、欠驅動、平衡控制 |
| MPC     | 線上最佳化                   | 是     | 是     | 高        | 有硬約束、需前瞻     |
| 強化學習    | 資料學習                    | 是     | 可（軟性） | 訓練極高、推論低 | 難以建模的複雜任務    |

### ◎ 為何 LQR 在機器人上如此重要

LQR 特別適合三類機器人問題。一、**欠驅動系統的平衡**：倒立擺、兩輪自平衡車、四旋翼的姿態控



制——這些無法用第 17 章的回授線性化處理，LQR 是最標準的解。二、**多變數耦合系統**：當狀態與輸入都是多維時，手動調參不可行，LQR 能一次算出所有增益。三、**作為更複雜方法的基礎**：MPC 的終端代價常用 LQR 的價值函數、iLQR ( 迭代 LQR ) 是軌跡最佳化的主流方法之一、而強化學習的許多演算法在線性情形下會退化為 LQR。理解 LQR，等於掌握了一把通往現代控制的鑰匙。

## 18.9 倒立擺：欠驅動系統的經典範例

### 18.9.1 為何倒立擺是控制理論的試金石

倒立擺 ( cart-pole ) 是控制領域最經典的測試問題：一台可左右移動的小車，上面用鉸鏈立著一根桿子。控制輸入只有小車的水平推力，目標是讓桿子維持直立。

它之所以成為標竿，有三個原因。**原因一：本質不穩定**。放手就會倒，必須持續控制——這排除了所有開迴路方法。**原因二：欠驅動**。系統有兩個自由度 ( 小車位置、桿子角度 )，卻只有一個控制輸入。第 17 章的回授線性化在此完全失效。**原因三：非最小相位**。要讓桿子往右倒回來，小車必須**先往右移** ( 把支點移到重心下方 )——控制的初始反應方向與最終目標相反，這在直覺上極為違反常識。

這三個特性，恰好也是雙足機器人、兩輪自平衡車與火箭垂直著陸的共同特性。因此掌握倒立擺，就掌握了這一大類問題的核心。

### 18.9.2 線性化模型

完整的倒立擺動力學是非線性的，但在直立點 (  $\theta \approx 0$  ) 附近可以線性化。設小車質量  $M$ 、桿質量  $m$ 、桿長  $2l$  ( 質心在中點  $l$  處 )，狀態向量取：

$$\mathbf{x} = [s, \dot{s}, \theta, \dot{\theta}]^T$$

狀態：小車位置、小車速度、桿角度 ( 由垂直向上量起 )、桿角速度。

在  $\theta \approx 0$ 、 $\dot{\theta} \approx 0$  的假設下 (  $\sin \theta \approx \theta$ 、 $\cos \theta \approx 1$ 、 $\theta^2 \approx 0$  )，線性化後的系統矩陣為：

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -mg/M & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & (M+m)g/(Ml) & 0 \end{bmatrix}$$

系統矩陣。第四列的正號是關鍵——它代表「角度越大，角加速度越大」，即不穩定。

$$\mathbf{B} = [0, 1/M, 0, -1/(Ml)]^T$$

輸入矩陣。注意推力對桿角的影響是負的——往右推小車，桿子會相對往左倒，這正是非最小相位的來源。

### 18.9.3 為何 LQR 特別適合

面對這個系統，LQR 有三個優勢。

**優勢一：自然處理欠驅動。**LQR 不要求輸入數等於自由度，它只是求解「用這一個輸入，如何最好地影響四個狀態」。

**優勢二：自動協調多目標。**我們同時希望桿子直立且小車不要跑太遠。這兩個目標會衝突（要扶正桿子就得移動小車），LQR 用  $Q$  矩陣的相對權重自動權衡。

**優勢三：一次算出四個增益。**控制律  $u = -Kx$  中， $K$  是  $1 \times 4$  的矩陣，含四個增益。手動調這四個數字極其困難（它們互相耦合），LQR 直接算出。

例題 18-3 與本章程式範例將完整實作這個系統。

## 18.10 與其他主題的關聯

### 18.10.1 與逆運動學的關係

第 10.8 節把逆運動學寫成一個最佳化問題：最小化任務誤差加上正則項。這與 LQR 的代價函數結構完全相同——都是「目標誤差的二次型 + 控制量的二次型」。

差別在於：逆運動學是靜態的（一次求解一個位形），LQR 是動態的（考慮整條時間軌跡）。但兩者的數學工具（二次型最佳化、偽逆與正則化）是相通的。第 10.7.3 節的阻尼最小平方方法，其實就是一個「單步的 LQR」。

### 18.10.2 與軌跡最佳化的關係

第 15 章的軌跡生成用的是預設的曲線形式（多項式、梯形、S 形）。更一般的作法是軌跡最佳化：直接把整條軌跡當作最佳化變數，最小化某個代價函數（時間、能耗、平順度）並滿足動力學與約束。

這正是一個最佳控制問題。求解方法有兩大類：間接法（由 HJB 或 Pontryagin 最大原理導出必要條件再求解）與直接法（把時間離散化，化為一個大型非線性規劃問題）。實務上直接法更常用，因為容易加入約束。

其中一個重要的方法是 iLQR（迭代 LQR）：對非線性系統，在當前軌跡附近線性化、用

LQR 求解、更新軌跡、再重複。它結合了 LQR 的高效與對非線性的處理能力，是目前四足機器人與操作規劃的主流工具之一。

### 18.10.3 與強化學習的關係

這是本章最重要的連結。第 31 章的強化學習，其數學基礎正是本章的 Bellman 方程。

兩者的對應關係如下：

表 18-5 最佳控制與強化學習的術語對照

| 最佳控制               | 強化學習          | 說明            |
|--------------------|---------------|---------------|
| 狀態 $x$             | 狀態 $s$        | 相同概念          |
| 控制輸入 $u$           | 動作 $a$        | 相同概念          |
| 代價 $L$ (要最小化)      | 獎勵 $r$ (要最大化) | 符號相反，本質相同     |
| 價值函數 $V(x)$        | 狀態價值 $V(s)$   | 相同概念          |
| 系統動態 $f(x,u)$ (已知) | 環境轉移 (未知，需探索) | <b>最關鍵的差異</b> |
| Bellman 方程         | Bellman 方程    | 完全相同          |
| 動態規劃               | 動態規劃 / 時序差分學習 | 後者用取樣估計代替完整計算 |

**核心差異只有一個：模型是否已知。**最佳控制假設我們知道  $f(x,u)$ ，因此可以直接計算；強化學習則假設環境未知，必須透過與環境互動來學習。

這個差異帶來完全不同的方法論，但**目標與數學結構完全相同**。理解本章，第 31 章會輕鬆許多——屆時你會發現許多「新概念」其實是舊識。

## 18.11 由最佳性邁向穩定性

### 18.11.1 本章的成果

本章回答了第 17 章留下的問題：增益不是憑感覺調的，而是可以由代價函數計算出來的。你現在能夠：把控制問題形式化為最佳控制問題、理解 Bellman 最佳性原理與價值函數、寫出並解讀 HJB 方程、推導並求解 LQR 與 Riccati 方程、用 Bryson 法則選擇權重、以及把 LQR 應用於欠驅動系統如倒立擺。

更重要的是，你看見了一個統一的框架。**動態規劃、LQR、MPC、iLQR、強化學習——這些看似不同的技術，全部源自同一條 Bellman 方程。**差別只在於：模型是否已知、時域是有限還

是無限、以及用什麼方法近似求解。

### 18.11.2 接下來：從「最好」到「保證」

但本章有一個微妙的缺口。LQR 保證了「在這個代價函數下最好」，也宣稱閉環系統會穩定——但我們還沒有嚴格證明過任何一個控制器的穩定性。

第 16 章分析二階系統時，我們看了微分方程的解；第 17 章推導計算力矩控制時，我們得到了線性的誤差方程。但這些都是針對特定的、簡化的情形。對於一般的非線性系統，如何證明「這個控制器一定不會失控」？

第 19 章（可控性與穩定性）將提供工具。我們會學到兩個核心概念：可控性（系統是否有可能被控制到任意狀態——這是 LQR 能運作的前提）與 Lyapunov 穩定性（一個不需解出微分方程就能判定穩定性的強大方法）。屆時我們將回頭嚴格證明第 17.5.2 節那個「PD 加重力補償保證全域漸近穩定」的定理，並看見第 13 章那個「 $\dot{M} - 2C$  為斜對稱」的性質如何在證明中扮演關鍵角色。

換句話說：第 18 章告訴你什麼是最好的，第 19 章告訴你為什麼它一定安全。

## 完整計算例題

### 例題 18-1 一維點質量的 LQR 設計

題目：質量  $m = 2 \text{ kg}$  的物體在無摩擦平面上，控制輸入為力  $f$ 。要求設計 LQR 控制器，使物體回到原點。（a）寫出狀態空間模型。（b）取  $Q = \text{diag}(1, 0)$ 、 $R = 1$ ，求解 ARE 並得到最佳增益。（c）計算閉環系統的自然頻率與阻尼比。（d）若改為  $Q = \text{diag}(100, 0)$ ，重新計算並比較。

解（a）：取狀態  $x = [\text{位置}, \text{速度}]^T$ 。由 18.2.1 節：

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

狀態空間模型。

解（b）：設  $P = [[p_1, p_2], [p_2, p_3]]$ （對稱）。代入 ARE： $A^T P + PA - PBR^{-1}B^T P + Q = 0$ 。

先計算各項。 $A^T P + PA$ ：

$$A^T P = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} p_1 & p_2 \\ p_2 & p_3 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ p_1 & p_2 \end{bmatrix}, \quad P A = \begin{bmatrix} 0 & p_1 \\ 0 & p_2 \end{bmatrix}$$

兩項相加得  $[[0, p_1], [p_1, 2p_2]]$ 。

再計算  $PBR^{-1}B^T P \cdot B^T P = [0, 0.5] \cdot P = [0.5p_2, 0.5p_3]$ ：

$$PBR^{-1}B^T P = \begin{bmatrix} 0.5p_2 \\ 0.5p_3 \end{bmatrix} \begin{bmatrix} 0.5p_2 & 0.5p_3 \end{bmatrix} = \begin{bmatrix} 0.25p_2^2 & 0.25p_2p_3 \\ 0.25p_2p_3 & 0.25p_3^2 \end{bmatrix}$$

二次項。

代入 ARE 並逐項比對 ( $Q = \text{diag}(1, 0)$ )：

$$(1,1) \text{ 項: } 0 - 0.25p_2^2 + 1 = 0 \Rightarrow p_2^2 = 4 \Rightarrow p_2 = 2$$

$$(1,2) \text{ 項: } p_1 - 0.25p_2p_3 = 0 \Rightarrow p_1 = 0.5p_3$$

$$(2,2) \text{ 項: } 2p_2 - 0.25p_3^2 + 0 = 0 \Rightarrow p_3^2 = 8p_2 = 16 \Rightarrow p_3 = 4$$

解得  $p_2 = 2$ 、 $p_3 = 4$ 、 $p_1 = 2$ 。(取正定解，故各值取正。)

$$K = R^{-1}B^T P = [0.5p_2, 0.5p_3] = [1.0, 2.0]$$

最佳增益。控制律為  $f = -1.0 \cdot (\text{位置}) - 2.0 \cdot (\text{速度})$ 。

解 (c)：閉環系統為  $\dot{x} = (A - BK)x$ 。代入：

$$A - BK = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -0.5 & -1.0 \end{bmatrix}$$

閉環系統矩陣。

其特徵方程為  $\lambda^2 + 1.0\lambda + 0.5 = 0$ 。對照第 16 章的標準形式  $\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2$ ：

$$\omega_n = \sqrt{0.5} = 0.707 \text{ rad/s}, \quad \zeta = 1.0/(2 \times 0.707) = 0.707$$

自然頻率與阻尼比。

**有趣的觀察：**LQR 自動選出了  $\zeta = 0.707$ ——正是第 16.4.4 節提到的實務最佳值！這不是巧合，而是二次代價函數的自然結果。這個例子生動說明了 LQR 的價值：它把工程師的經驗法則，變成了數學的必然結論。

解 (d)： $Q = \text{diag}(100, 0)$  代表更重視位置精度。重複計算：

$$p_2^2 = 4 \times 100 = 400 \Rightarrow p_2 = 20 \\ p_3^2 = 8p_2 = 80 \Rightarrow p_3 = 8.944$$

新的  $P$  元素。

$$K = [0.5 \times 10, 0.5 \times 8.944] = [5.0, 4.472]$$

新的增益。

$$\omega_n = \sqrt{(0.5 \times 5.0)} = \sqrt{2.5} = 1.58 \text{ rad/s}, \quad \zeta = (0.5 \times 4.472)/(2 \times 1.58) = 0.707$$

新的動態參數。

**關鍵觀察：**Q 增大 100 倍後， $\omega_n$  增為 2.24 倍（ $\sqrt{100}$  的四次方根，即  $100^{0.25}$ ），但阻尼比  $\zeta$  仍然是 0.707。這揭示了 LQR 的一個優美性質：改變 Q/R 的比值會調整系統的速度，但阻尼比保持在最佳值——控制器自動維持了「不振盪」的品質。

### 例題 18-2 Bryson 法則的應用

題目：某機器人關節（等效慣量  $J = 0.5 \text{ kg}\cdot\text{m}^2$ ）採用 LQR 控制。設計要求：位置誤差不超過 0.02 rad、速度誤差不超過 0.5 rad/s、力矩不超過 10 N·m。（a）用 Bryson 法則設定 Q 與 R。（b）說明若實測發現力矩經常飽和，該如何調整。（c）若改為要求位置誤差不超過 0.005 rad，Q 如何改變？增益大約變為幾倍？

解（a）：由 18.7.1 節的 Bryson 法則：

$$Q_{11} = 1/(0.02)^2 = 1/0.0004 = 2500 \quad (\text{位置})$$

$$Q_{22} = 1/(0.5)^2 = 1/0.25 = 4 \quad (\text{速度})$$

$$R = 1/(10)^2 = 1/100 = 0.01 \quad (\text{力矩})$$

初始權重設定。

因此  $Q = \text{diag}(2500, 4)$ 、 $R = 0.01$ 。注意  $Q_{11}$  遠大於  $Q_{22}$ ——這反映了「位置精度要求嚴格得多」這個設計意圖。

解（b）：力矩飽和代表控制器太積極，應提高 R（或等效地降低 Q）。實務作法是把 R 逐步加倍（ $0.01 \rightarrow 0.02 \rightarrow 0.04$ ），每次重新解 ARE 並測試，直到力矩落在極限內。

由於 LQR 的解只取決於 Q/R 的比值，也可以固定  $R = 1$  而縮小 Q——兩種作法等價，選擇習慣的即可。

解（c）：位置容許誤差由 0.02 縮為 0.005（四分之一），則：

$$Q_{11} = 1/(0.005)^2 = 40000 \quad (\text{原本的 16 倍})$$

新的位置權重。

由例題 18-1 的規律，增益大約與  $Q^{0.5}$  的四次方根相關。更精確地說，對此類系統：

$$K_p \propto \sqrt{(Q_{11}/R)} \Rightarrow \text{增益約變為 } \sqrt{16} = 4 \text{ 倍}$$

增益的變化。位置增益約增為 4 倍。

**工程判讀：**要求精度提高 4 倍，需要增益提高約 4 倍，而力矩也會相應增大。若原本的力矩餘裕不足，這個要求可能無法達成——此時應考慮改善機構（降低摩擦、提高剛性）而非一味提高增益。

### 例題 18-3 倒立擺的 LQR 設計

題目：倒立擺參數為：小車質量  $M = 1.0 \text{ kg}$ 、桿質量  $m = 0.2 \text{ kg}$ 、桿長  $2l = 0.6 \text{ m}$ （故  $l = 0.3 \text{ m}$ ）、 $g = 9.8 \text{ m/s}^2$ 。（a）寫出線性化的 A、B 矩陣。（b）驗證開迴路系統不穩定。（c）說明 Q 與 R 的選擇考量。（d）解釋所得增益的物理意義。

解（a）：代入 18.9.2 節的公式：

$$\begin{aligned} -mg/M &= -(0.2)(9.8)/1.0 = -1.96 \\ (M+m)g/(Ml) &= (1.2)(9.8)/((1.0)(0.3)) = 11.76/0.3 = 39.2 \end{aligned}$$

兩個關鍵元素。

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -1.96 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 39.2 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 1.0 \\ 0 \\ -3.333 \end{bmatrix}$$

線性化模型。B 中的  $-1/(Ml) = -1/0.3 = -3.333$ 。

解（b）：求 A 的特徵值。由結構可知有兩個零特徵值（對應小車位置與速度的積分），另兩個滿足：

$$\lambda^2 = 39.2 \Rightarrow \lambda = \pm 6.26$$

其中  $\lambda = +6.26$  為正實部特徵值  $\Rightarrow$  系統不穩定。

這個正特徵值的物理意義是：任何微小的角度偏差，都會以  $e^{(6.26t)}$  指數放大。時間常數為  $1/6.26 = 0.16$  秒——大約每 0.16 秒偏差就放大 2.7 倍。這解釋了為何倒立擺必須以高頻率控制（至少 50 Hz 以上）。

解（c）：四個狀態的權重考量如下：

- **桿角度  $\theta$** ：最重要——倒了就完全失敗。應給最大權重。
- **桿角速度  $\dot{\theta}$** ：次重要——它預示了未來的角度變化，適度懲罰有助於阻尼。
- **小車位置  $s$** ：中等——不希望小車跑太遠（軌道有限），但可以容忍一定範圍。
- **小車速度  $\dot{s}$** ：最不重要——只要最終停下即可，過程中的速度不需嚴格限制。

典型的選擇是  $Q = \text{diag}(1, 1, 100, 10)$ 、 $R = 1$ （角度權重遠大於位置）。

解 (d)：解 ARE 後（見本章程範例）會得到形如  $K = [k_1, k_2, k_3, k_4]$  的增益，控制律為：

$$u = -k_1 \cdot s - k_2 \cdot \dot{s} - k_3 \cdot \theta - k_4 \cdot \dot{\theta}$$

*控制律的四項。*

**物理解讀：** $k_3$  與  $k_4$  是「扶正桿子」的主力——桿子往右倒 ( $\theta > 0$ )，控制器輸出負的力（往左推小車），使支點移到重心下方，桿子就會被「接住」。 $k_1$  與  $k_2$  則負責「把小車拉回原點」，但它們必須很小心——因為要把小車往左移，反而必須先讓桿子往左傾，這正是非最小相位的體現。

實際計算會發現  $k_3$  的符號與直覺相反（往右倒卻要往右推？）——這是因為  $B$  矩陣中角度那一項是負的。這個符號的糾結，正是倒立擺讓初學者困惑的地方，也是為何手動調參極其困難、而 LQR 能直接算出正確答案的價值所在。

## 程式範例：LQR 設計與倒立擺模擬

以下 Python 程式實作 ARE 的求解、LQR 增益計算，並完整模擬倒立擺的平衡控制。同時驗證例題 18-1 的手算結果與  $Q$  權重的影響。

```
import numpy as np
from scipy.linalg import solve_continuous_are

def lqr(A, B, Q, R):
    """求解連續時間 LQR：回傳增益 K、Riccati 解 P、閉環特徵值"""
    P = solve_continuous_are(A, B, Q, R)
    K = np.linalg.solve(R, B.T @ P)          # K = R^-1 B^T P
    eig = np.linalg.eigvals(A - B @ K)
    return K, P, eig

# ===== 驗證一：例題 18-1 一維點質量 =====
print("=== 例題 18-1：一維點質量 LQR (m = 2 kg) ===")
m = 2.0
A = np.array([[0., 1.], [0., 0.]])
B = np.array([[0.], [1/m]])

for q11 in (1.0, 100.0):
    Q = np.diag([q11, 0.0]); R = np.array([[1.0]])
    K, P, eig = lqr(A, B, Q, R)
    wn = np.sqrt(K[0,0]/m)                # 閉環 wn
    zeta = (K[0,1]/m)/(2*wn)              # 閉環 ζ
    print(f"Q11={q11:6.0f}: K = [{K[0,0]:.4f}, {K[0,1]:.4f}] "
```



```

        f"wn = {wn:.4f} rad/s    ζ = {zeta:.4f}")
    print(f"            P = {np.round(P.flatten(), 4)}    "
          f"閉環極點 = {np.round(eig, 4)}")
print("→ 注意：Q 改變 100 倍 · ζ 始終維持 0.707 (最佳阻尼)")

# ===== 驗證二：Q/R 比值的效果 =====
print("\n=== Q/R 比值對性能的影響 ===")
print(f"'Q11/R':>8}'{Kp':>10}'{Kd':>10}'{wn':>10}'{ζ':>8}'{最大力矩':>12}")
for ratio in (0.1, 1, 10, 100, 1000):
    Q = np.diag([ratio, 0.0]); R = np.array([[1.0]])
    K, _, _ = lqr(A, B, Q, R)
    wn = np.sqrt(K[0,0]/m); zeta = (K[0,1]/m)/(2*wn)
    f_max = K[0,0]*1.0 # 初始偏差 1 m 時的力
    print(f"{ratio:8.1f}{K[0,0]:10.3f}{K[0,1]:10.3f}"
          f"{wn:10.3f}{zeta:8.3f}{f_max:11.2f} N")
print("→ 提高 Q 使反應變快 · 但所需力矩也同比例增加")

# ===== 例題 18-3：倒立擺 =====
print("\n=== 例題 18-3：倒立擺 LQR 設計 ===")
M_cart, m_pole, l, g = 1.0, 0.2, 0.3, 9.8

A_ip = np.array([
    [0., 1., 0., 0.],
    [0., 0., -m_pole*g/M_cart, 0.],
    [0., 0., 0., 1.],
    [0., 0., (M_cart+m_pole)*g/(M_cart*l), 0.]])
B_ip = np.array([[0.], [1/M_cart], [0.], [-1/(M_cart*l)]])

print(f"開迴路特徵值: {np.round(np.linalg.eigvals(A_ip), 4)}")
unstable = [e for e in np.linalg.eigvals(A_ip) if e.real > 1e-9]
print(f"→ 有 {len(unstable)} 個正實部特徵值 · 系統不穩定")
print(f"→ 發散時間常數 = {1/max(e.real for e in unstable):.3f} 秒")

# Q 的選擇：角度最重要 (18.9.3 節)
Q_ip = np.diag([1.0, 1.0, 100.0, 10.0])
R_ip = np.array([[1.0]])
K_ip, P_ip, eig_ip = lqr(A_ip, B_ip, Q_ip, R_ip)
print(f"\nQ = diag(1, 1, 100, 10), R = 1")
print(f"最佳增益 K = [{K_ip[0,0]:8.3f} {K_ip[0,1]:8.3f} "
      f"{K_ip[0,2]:8.3f} {K_ip[0,3]:8.3f}]")
print(f"            ( 位置    速度    角度    角速度 )")
print(f"閉環極點 = {np.round(eig_ip, 3)}")
print(f"→ 全部落在左半平面 · 系統已穩定")

# ===== 非線性模擬 (驗證線性化控制器的有效性) =====
def cartpole_dynamics(x, u, M=1.0, mp=0.2, l=0.3, g=9.8):
    """完整的非線性倒立擺動力學"""
    s, sd, th, thd = x
    sin_t, cos_t = np.sin(th), np.cos(th)

```

```

den = M + mp*sin_t**2
sdd = (u + mp*sin_t*(1*thd**2 - g*cos_t)) / den
thdd = (-u*cos_t - mp*1*thd**2*sin_t*cos_t + (M+mp)*g*sin_t) / (1*den)
return np.array([sd, sdd, thd, thdd])

def simulate_cartpole(K, x0, T=5.0, dt=0.001, u_max=20.0):
    x = np.array(x0, float)
    max_angle, max_u, max_s = 0.0, 0.0, 0.0
    for k in range(int(T/dt)):
        u = float(-K @ x)
        u = np.clip(u, -u_max, u_max)
        xd = cartpole_dynamics(x, u)
        x = x + xd*dt
        max_angle = max(max_angle, abs(np.rad2deg(x[2])))
        max_u = max(max_u, abs(u)); max_s = max(max_s, abs(x[0]))
        if abs(x[2]) > np.pi/2:
            return None, max_angle, max_u, max_s      # 倒了
    return x, max_angle, max_u, max_s

print("\n=== 非線性模擬：由不同初始傾角出發 ===")
print(f"{'初始角度':>10}{'結果':>8}{'最大角度':>12}{'最大力':>10}{'小車位移':>12}")
for deg in (5, 10, 20, 30, 45):
    xf, ma, mu, ms = simulate_cartpole(K_ip, [0., 0., np.deg2rad(deg), 0.])
    status = "穩定" if xf is not None else "失敗"
    print(f"{deg:9d}°{status:>8}{ma:11.2f}°{mu:9.2f} N{ms:11.3f} m")
print("> 線性化設計的控制器，在相當大的角度範圍內仍然有效")

# ===== Q 權重對倒立擺行為的影響 =====
print("\n=== Q 權重的影響 ( 初始傾角 15° ) ===")
print(f"{'Q(角度)':>10}{'Q(位置)':>10}{'最大角度':>12}{'最大力':>10}{'小車位移':>12}")
for q_th, q_s in ((10., 1.), (100., 1.), (1000., 1.), (100., 50.)):
    Q_t = np.diag([q_s, 1.0, q_th, 10.0])
    K_t, _, _ = lqr(A_ip, B_ip, Q_t, R_ip)
    xf, ma, mu, ms = simulate_cartpole(K_t, [0., 0., np.deg2rad(15), 0.])
    print(f"{q_th:10.0f}{q_s:10.0f}{ma:11.2f}°{mu:9.2f} N{ms:11.3f} m")
print("> 角度權重高則桿子更穩但小車跑更遠；位置權重高則小車收斂快但角度波動大")

```

請試著修改：（一）把倒立擺的  $R$  由 1 提高到 100，觀察增益如何下降、最大力如何減小、但恢復是否變慢。（二）把初始角度加大到 60 度，觀察線性化控制器何時失效——這說明了線性化的有效範圍。（三）在 `cartpole_dynamics` 中加入摩擦項，觀察 LQR 是否仍然有效（它應該相當強健）。

## 實作：LQR 控制器的設計與驗證

本章實作以模擬與設計為主，並可延伸到實體的兩輪自平衡車或倒立擺平台。

1. 建立狀態空間模型：為你的系統（可選一維點質量、單關節手臂、或倒立擺）寫出  $A$ 、 $B$  矩

陣。若是非線性系統，先在工作點附近線性化。

2. **開迴路分析**：計算  $A$  的特徵值，判斷系統是否穩定。若有正實部特徵值，計算其發散時間常數，據此決定所需的控制頻率（建議至少為時間常數倒數的 10 倍）。
3. **Bryson 法則設定初值**：依 18.7.1 節，由各狀態的容許誤差與控制的可用範圍，設定  $Q$  與  $R$  的初值。
4. **求解 ARE**：用 scipy 或 MATLAB 求得  $P$  與  $K$ ，並檢查閉環特徵值是否全部落在左半平面。
5. **線性模擬**：以線性模型模擬閉環響應，記錄收斂時間、超越量與最大控制量。
6. **非線性模擬**：改用完整的非線性模型重複模擬，比較與線性模擬的差異。逐步加大初始偏差，找出線性化控制器失效的臨界點。
7. **權重掃描**：系統性地改變  $Q$  與  $R$ （建議固定  $R = 1$ ，只掃描  $Q$ ），記錄「收斂時間 vs 最大控制量」的關係，繪出取舍曲線（Pareto front）。這張圖能直觀展示最佳控制的本质——沒有免費的午餐。
8. **強健性測試**：故意讓控制器使用的模型參數與真實系統相差  $\pm 20\%$ （如質量估計錯誤），觀察 LQR 是否仍能穩定。驗證 18.8.1 節提到的增益裕度性質。
9. **（進階）實體驗證**：若有兩輪自平衡車或倒立擺平台，將設計好的  $K$  燒錄進去測試。注意實體系統有感測雜訊、量化誤差與致動器延遲，通常需要把增益降低 20~30% 才能穩定運行。

表 18-6 LQR 設計記錄表（第一列為範例）

| Q 設定       | R 設定 | 增益 K         | 閉環極點            | 收斂時間  | 最大控制量 |
|------------|------|--------------|-----------------|-------|-------|
| diag(1, 0) | 1    | [1.00, 2.00] | $-0.5 \pm 0.5j$ | 6.0 s | 1.0 N |
| （自行填寫其餘設計） |      |              |                 |       |       |

## 國中小也看得懂：在手掌上立掃把的祕密

你有沒有玩過「在手掌上立掃把」的遊戲？（第 16 章我們玩過立竹竿，這次是更難的版本。）

要立住掃把，你的手必須一直移動。但你有沒有想過一件很奇怪的事——當掃把往右倒的時候，你的手要往哪邊移？

答案是：**往右**！這聽起來很怪對不對？掃把往右倒，你卻要往右追。但仔細想想就懂了：你要把「支撐點」移到掃把重心的下方，它才會被「接住」。如果你往左移，掃把只會倒得更快。

這件事有趣的地方在於：**如果你想讓掃把往左移動一點點，你必須先讓它往左倒一下下，然後再追過去。** 也就是說——**想往左，要先往左倒**。這種「動作方向和目的地相反」的系統，是工程師最頭痛的一類。

那要怎麼算出「該移動多快」呢？這一章教的方法很聰明。工程師會先問自己兩個問題：

**問題一：我最不能忍受什麼？** 是掃把倒下去？還是手移動太遠？

**問題二：我願意付出多少力氣？** 用力大一點就穩，但會很累。

把這兩個答案寫成一個「分數」——歪太多要扣分、太用力也要扣分。然後用數學算出「怎麼做才能扣最少分」。算出來的答案，就是最好的控制方法！

最酷的是：這個方法不用試誤。以前的工程師要花好幾天慢慢調參數，現在只要說清楚「我在乎什麼」，電腦就能直接算出最佳答案。

## 叫獸提醒與常見錯誤

- 誤區一：以為 LQR 能取代模型。它需要準確的  $A$ 、 $B$  矩陣；模型錯了，算出的「最佳」增益也是錯的。
- 誤區二： $R$  設為零。 $R$  必須正定，否則最佳解是無限大的控制輸入，ARE 無解。
- 誤區三：只調  $Q$  與  $R$  的絕對值。LQR 的解只取決於  $Q/R$  的比值，兩者同乘一個常數得到完全相同的增益。
- 誤區四：忘記 LQR 需要完整的狀態量測。若某些狀態無法量測，必須加入觀測器（如 Kalman 濾波器）構成 LQG。
- 誤區五：忽略約束。LQR 不處理力矩極限；若最佳解超限而被截斷，穩定性保證即失效，應改用 MPC 或降低增益。
- 誤區六：在遠離工作點處使用線性化的 LQR。線性化只在小範圍內有效，大偏差時可能失控——本章程式範例的角度掃描即在驗證此點。
- 誤區七：用直覺調倒立擺的四個增益。它們高度耦合且部分符號違反直覺（非最小相位），手動調參幾乎不可能成功。

- 誤區八：把有限時域的  $K(t)$  用在持續運行的系統上。持續運行應使用無限時域的常數增益，計算簡單且已是最佳。
- 誤區九：以為動態規劃能直接用在高維系統。維數詛咒使其在超過三、四維時就不可行，必須改用 LQR、MPC 或函數近似。
- 誤區十：把實體系統的增益直接用模擬值。實機有感測雜訊、量化誤差與致動器延遲，通常需要降低 20~30% 才能穩定。
- 誤區十一：忽略 LQR 與強化學習的血緣關係。兩者共用 Bellman 方程，差別只在模型是否已知——理解前者會讓後者容易得多。

## 課堂討論

1. 例題 18-1 發現 LQR 自動選出  $\zeta = 0.707$ ，恰好是第 16 章的實務最佳值。請討論：這是巧合嗎？工程師的經驗法則與數學最佳解之間，通常是什麼關係？
2. 維數詛咒使動態規劃無法直接用於高維系統。請比較本書中三種因應策略——LQR（特例封閉解）、MPC（局部有限時域）、強化學習（函數近似）——各自「犧牲了什麼、換到了什麼」。
3. 倒立擺是非最小相位系統：想往左，要先往左倒。請再舉出兩個生活或工程中的非最小相位例子，並討論它們為何難以直覺控制。
4. LQR 保證增益裕度至少  $[0.5, \infty)$ 、相位裕度至少  $60^\circ$ 。請討論這個保證在實務上的價值，以及它為何在加入觀測器（LQG）後可能失效。
5. 本章的代價函數  $J = \int (x^T Q x + u^T R u) dt$  把所有目標都化為二次型。請討論：哪些機器人任務的目標**不適合**用二次型表達？此時該怎麼辦？

## 章末摘要

- 最佳控制問題由三要素構成：系統動態  $\dot{x} = f(x, u)$ 、代價函數  $J$ 、以及邊界條件與約束。
- 它與一般最佳化的本質差異：在函數空間中找一整條曲線（無限維），而非在有限維空間中找一個點。
- 狀態空間表示把高階系統化為一階形式  $\dot{x} = Ax + Bu$ ，是控制理論的統一語言。
- Bellman 最佳性原理：最佳路徑的任何一段本身也是最佳的；剩餘決策只取決於當前狀態。

與歷史無關。

- 價值函數  $V(x)$  是「由此狀態出發，未來最少還要付出的代價」；知道它就能導出回授形式的最佳控制律。
- 離散時間的 Bellman 方程  $V_k(x) = \min_u [L + V_{k+1}(f)]$  由後往前遞迴求解，稱為動態規劃。
- 動態規劃面臨維數詛咒：狀態每增一維，計算量指數成長，超過三四維即不可行。
- 連續時間版本為 HJB 方程  $-\partial V/\partial t = \min_u [L + (\partial V/\partial x)^T f]$ ，一般無解析解。
- LQR 是 HJB 有封閉解的最重要特例：線性系統加二次代價，價值函數為  $V = x^T P x$ 。
- 最佳控制律為線性狀態回授  $u = -Kx$ ，其中  $K = R^{-1}B^T P$ ， $P$  由代數 Riccati 方程  $A^T P + PA - PBR^{-1}B^T P + Q = 0$  解出。
- $Q$  懲罰狀態偏差、 $R$  懲罰控制用力；解只取決於  $Q/R$  的比值；Bryson 法則以各量的容許上限倒數平方設定初值。
- LQR 的三個優良性質：保證穩定、增益裕度至少  $[0.5, \infty)$  與相位裕度至少  $60^\circ$ 、解唯一。
- 四個限制：需線性模型、需完整狀態量測、無法處理約束、二次代價未必反映真實目標。
- 對一維點質量，LQR 的解就是一個 PD 控制器；其價值在於為多變數系統一次算出所有增益。
- 倒立擺是欠驅動、不穩定且非最小相位的系統，回授線性化失效，LQR 是標準解法。
- 最佳控制與強化學習共用 Bellman 方程，核心差異僅在於系統模型是否已知。

## 觀念題與計算題

### 觀念題

1. 說明最佳控制問題的三個要素，並解釋它與一般最佳化問題的本質差異。
2. 為何要把高階系統化為狀態空間的一階形式？這帶來哪三個好處？
3. 敘述 Bellman 最佳性原理，並以一個生活化的例子說明其正確性。
4. 定義價值函數，並說明為何「知道價值函數就等於解決了問題」。
5. 寫出離散時間的 Bellman 方程，並說明為何必須由後往前求解。
6. 什麼是維數詛咒？本書中有哪三種因應策略？

7. 寫出 HJB 方程並解讀其中每一項的物理意義。
8. 說明 LQR 問題的兩個前提條件，以及  $Q$ 、 $R$  兩個權重矩陣的作用。
9. 為何  $R$  必須正定而  $Q$  可以只是半正定？
10. 寫出代數 Riccati 方程與最佳增益的公式，並說明其工程意義。
11. 比較有限時域與無限時域 LQR 的差異，並說明實務上多用後者的原因。
12. 說明 Bryson 法則的邏輯，並解釋為何它是一個好的起點。
13. 列舉 LQR 的三個優良性質與四個實務限制。
14. 為何說「對一維點質量，LQR 的解就是 PD 控制器」？LQR 的真正價值在哪裡？
15. 說明倒立擺的三個特性（不穩定、欠驅動、非最小相位），以及為何 LQR 特別適合它。
16. 列出最佳控制與強化學習的術語對照，並指出兩者最關鍵的差異。

## 計算題

1. 質量  $m = 4 \text{ kg}$  的物體在無摩擦平面上，控制輸入為力。（a）寫出狀態空間模型  $A$ 、 $B$ 。  
（b）取  $Q = \text{diag}(4, 0)$ 、 $R = 1$ ，手算求解 ARE 得到  $P$  與  $K$ 。（c）計算閉環的自然頻率與阻尼比。（d）驗證阻尼比是否為 0.707。
2. 某關節的等效慣量  $J = 0.8 \text{ kg}\cdot\text{m}^2$ 。設計要求：位置誤差不超過  $0.01 \text{ rad}$ 、速度誤差不超過  $0.4 \text{ rad/s}$ 、力矩不超過  $15 \text{ N}\cdot\text{m}$ 。（a）用 Bryson 法則設定  $Q$  與  $R$ 。（b）若實測發現收斂太慢，說明應如何調整並解釋理由。（c）若把位置容許誤差改為  $0.002 \text{ rad}$ ， $Q_{11}$  變為幾倍？增益約變為幾倍？
3. 某倒立擺  $M = 1.5 \text{ kg}$ 、 $m = 0.3 \text{ kg}$ 、 $l = 0.4 \text{ m}$ 、 $g = 9.8$ 。（a）計算  $A$  矩陣中的  $-mg/M$  與  $(M+m)g/(Ml)$  兩個元素。（b）計算  $B$  矩陣的兩個非零元素。（c）由  $\lambda^2 = (M+m)g/(Ml)$  求不穩定極點與發散時間常數。（d）依此建議最低的控制頻率（取時間常數倒數的 10 倍）。
4. 某二階系統  $\dot{x} = Ax + Bu$ ，其中  $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ 。（a）計算開迴路特徵值並判斷穩定性。（b）取  $Q = I$ 、 $R = 1$ ，寫出 ARE 的三個純量方程式（設  $P = \begin{bmatrix} p_1 & p_2 \\ p_2 & p_3 \end{bmatrix}$ ）。（c）說明求解此方程組的步驟（不需完整求出數值）。（d）若求得  $K = [k_1, k_2]$ ，寫出閉環系統矩陣  $A - BK$ 。

## 動手做與專題延伸



## 本章實作：LQR 設計工具

延續前面章節的程式庫，加入最佳控制模組。

- **階段一**：實作 ARE 求解與 LQR 增益計算（可用 `scipy`，但建議也自行實作一個簡單的迭代解法以理解原理）。
- **階段二**：實作 Bryson 法則的自動權重設定，輸入各狀態與控制的容許範圍，自動產生  $Q$  與  $R$ 。
- **階段三**：整合第 14 章的動力學模擬，建立「設計 → 模擬 → 評估」的完整迴圈，能自動掃描權重並輸出取舍曲線。
- **階段四**：實作有限時域的微分 Riccati 方程（反向積分），觀察  $K(t)$  如何在大部分時間趨於常數。
- **階段五**：把 LQR 與第 17 章的計算力矩控制結合——先用 CTC 線性化為  $\ddot{q} = u$ ，再用 LQR 設計外層增益。比較這個組合與手動指定  $\omega_n$ 、 $\zeta$  的差異。

## 專題延伸

- **取舍曲線（Pareto Front）視覺化**：對一維點質量系統，掃描  $Q/R$  由 0.01 到 10000，記錄每組增益對應的「穩定時間」與「最大控制量」，以這兩者為軸繪出散布圖。你會得到一條凸曲線——這就是「快與省力」的取舍前緣，任何在曲線上的點都是最佳的，選哪一個取決於你的偏好。這張圖是理解最佳控制本質最好的教材。
- **倒立擺的吸引域分析**：對設計好的 LQR 控制器，在「初始角度 × 初始角速度」的平面上取樣數千個初始狀態，模擬並記錄哪些能成功穩定。將成功與失敗以不同顏色繪出，得到一張「吸引域」圖。改變  $Q$  與  $R$  再繪一次，觀察吸引域如何變化——你會發現更積極的控制器（高  $Q$ ）有更大的吸引域，但代價是更大的控制量。
- **進階：iLQR 的實作與應用**。實作迭代 LQR：對非線性系統，在當前軌跡上逐點線性化、用有限時域 LQR 求解、更新軌跡、重複直到收斂。用它求解一個「倒立擺擺起」（swing-up）問題——由下垂位置擺盪到直立位置。這個任務無法用單純的 LQR 解決（因為需要遠離線性化點的大範圍運動），但 iLQR 能找到漂亮的解。這將讓你親身體會第 17.8.4 節的 MPC 與本章 LQR 之間的橋樑。

## 參考文獻與延伸閱讀



- Bellman, R., Dynamic Programming, Princeton University Press, 1957 ( 動態規劃與最佳性原理的原始著作 ) 。
- Kalman, R. E., "Contributions to the Theory of Optimal Control," Bol. Soc. Mat. Mexicana, 1960 ( LQR 與 Riccati 方程在控制上的奠基論文 ) 。
- Bertsekas, D. P., Dynamic Programming and Optimal Control, Athena Scientific ( 動態規劃與最佳控制的權威教材 ) 。
- Anderson, B. D. O. and Moore, J. B., Optimal Control: Linear Quadratic Methods, Prentice Hall ( LQR 與 LQG 的經典專著 ) 。
- Bryson, A. E. and Ho, Y.-C., Applied Optimal Control, Taylor & Francis ( Bryson 法則的出處，工程導向的最佳控制教材 ) 。
- Kirk, D. E., Optimal Control Theory: An Introduction, Dover ( 適合初學者的入門教材，含大量範例 ) 。
- Todorov, E. and Li, W., "A Generalized Iterative LQG Method for Locally-Optimal Feedback Control," ACC 2005 ( iLQR/iLQG 的代表性論文 ) 。
- Tedrake, R., Underactuated Robotics, MIT 課程講義 ( 線上公開 ) ( 欠驅動系統控制的優秀教材，含大量倒立擺與行走機器人範例 ) 。
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin ( 本書系參考的德式講義，含最佳控制與 Riccati 方程的簡潔處理 ) 。
- Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed., MIT Press ( 強化學習的權威教材，第 4 章即為動態規劃，可與本章對照閱讀 ) 。

# 第 19 章

## 可控性與穩定性

編著：李世淵 ( 機器人叫獸 )

助教：李天宇、李宇晴

【章首情境圖建議】畫面中央為一個三維碗狀曲面，橫軸為位置誤差、縱軸為速度，高度代表 *Lyapunov* 函數  $V$ ；一條螺旋軌跡由碗緣持續往下滑向碗底，旁註「 $V \leq 0$ 」。左側疊一張複平面圖，標出左半平面 ( 連續系統穩定區 ) 與單位圓 ( 離散系統穩定區 )，並以箭頭表示極點配置把不穩定極點由右半平面搬到左半平面。右側為一支機械手臂由數個差異極大的初始姿態同時出發、最終全部收斂到同一目標姿態的多重曝光，旁註「全域漸近穩定」。標題文字：不必解出方程，也能證明它一定會到。

## 叫獸開講 「控制器算出了最完美的指令，系統卻仍然翻車——問題出在哪裡？」

小妤把第 18 章的 LQR 用在一台自製的兩輪自平衡車上。她仔細寫出 A、B 矩陣、選好 Q 與 R、解出 ARE、燒錄增益——然後車子在啟動的瞬間就往前撲倒，一路加速衝到桌邊摔了下去。

「我照著公式算的啊，」她把筆電推過來，「Q 和 R 都是正定的，ARE 也解出來了，程式沒有報錯。」

叫獸看了一眼她的 A 矩陣，指著其中一項：「你這裡的符號寫反了。」

「那.....重算一次就好了？」

「當然可以。但我想問你一個更重要的問題。」叫獸把筆電闔上，「在你按下啟動按鈕之前，你怎麼知道這個控制器一定不會失控？」

小妤愣住了。「模擬跑過了啊。」

「模擬跑的是你寫錯的那個模型。它在你的模型裡穩得很，但那個模型不是真實的車子。」叫獸在白板上寫下三個字：保證。「工程師和玩家的差別，就在這裡。玩家說『我試過了，沒問題』；工程師說『我可以證明它不會出問題』。」

他停頓了一下：「更根本的問題是——有些系統根本就控制不了。你有沒有想過，如果那台車的兩個輪子被焊死在同一根軸上，不管你的控制器多聰明，它都轉不了彎？這不是控制器的問題，是系統本身的問題。在動手設計之前，你應該先問：這個系統，究竟能不能被控制到我想要的任何狀態？」

小翰舉手：「那要怎麼問？」

「有一個矩陣可以回答。」叫獸寫下  $C = [B, AB, A^2B, \dots]$ 。「把它算出來，看它的秩夠不夠。夠，代表你想去哪就去得了；不夠，代表有些狀態你永遠到不了——再厲害的演算法也沒用。」

接著他在白板另一邊畫了一個碗，裡面放了一顆珠子。「至於穩定性，有一個更漂亮的方法，連微分方程都不用解。想像你的系統是一顆珠子在碗裡滾動。只要你能找到一個『高度函數』，證明珠子每一刻的高度都在下降，那你根本不需要知道它走的路徑長什麼樣——它終究會滾到碗底。」

「這個想法是一位俄國數學家 Lyapunov 在一百三十年前提出的。今天，我們要用它來證明

一件事：第 17 章那個 PD 加重力補償的控制器，無論初始位置在哪裡、無論機器人多重、無論姿態多複雜，都保證會收斂。而且證明過程中，會用到第 13 章那個看似無用的性質—— $\dot{M} - 2C$  是斜對稱矩陣。」

## 本章為什麼重要

前面三章（16、17、18）教會了我們設計控制器，但都留下了同一個缺口：我們憑什麼相信它一定有效？第 16 章分析了簡化的二階系統、第 17 章假設模型完美、第 18 章宣稱 LQR 保證穩定卻沒有證明。本章要補上這個缺口。

這不是純理論的潔癖。在機器人上，「失控」的代價是實實在在的：手臂暴衝可能傷人、自平衡車摔壞、四足機器人在測試場翻覆。而最危險的失控，往往發生在模擬中沒測到的邊界情況——這正是為何需要「證明」而非「測試」。

本章提供兩把工具。可控性回答「這個系統有沒有可能被控制」——這是設計之前的體檢，能在動手前就篩掉不可能的方案。Lyapunov 穩定性則提供一個不需解微分方程就能判定穩定性的方法，它對非線性系統同樣有效，是機器人控制中最重要的理論工具。

本章也是三個伏筆的回收處：第 13 章的  $\dot{M} - 2C$  斜對稱性、第 17 章的 Takegaki-Arimoto 定理、以及第 18 章 LQR 的穩定性保證，都將在此得到嚴格的證明。

## 學習目標

- 能定義可控性，並說明其在控制系統設計中的角色。
- 能建構可控性矩陣並以其秩判斷系統是否完全可控。
- 能舉例說明不可控系統，並解釋其物理成因。
- 能區分 Lyapunov 穩定、漸近穩定與指數穩定三個定義。
- 能以特徵值判斷線性系統的穩定性。
- 能運用極點配置設計狀態回授增益。
- 能說明 Lyapunov 直接法的核心思想與三個條件。
- 能為簡單系統構造 Lyapunov 函數並證明其穩定性。
- 能運用能量作為 Lyapunov 候選函數，證明機器人控制器的穩定性。

- 能說明  $\dot{M} - 2C$  斜對稱性在穩定性證明中的關鍵作用。
- 能說明可觀測性的意義及其與可控性的對偶關係。

## 先備知識

第 13 章的動力學方程與慣性矩陣性質（尤其是正定性與  $\dot{M} - 2C$  的斜對稱性）、第 16~18 章的控制器設計、第 18 章的狀態空間表示法。

數學方面需要：矩陣的秩與特徵值、正定矩陣的概念、以及基本的微分運算。本章的證明看似抽象，但每一步都有清晰的物理對應——建議邊讀邊回想「碗中的珠子」這個圖像。

## 核心名詞中英對照

表 19-1 本章核心名詞

| 中文            | 英文                             | 說明                              |
|---------------|--------------------------------|---------------------------------|
| 可控性           | Controllability                | 能否以控制輸入將系統帶到任意狀態                |
| 可控性矩陣         | Controllability Matrix         | 判斷可控性的矩陣 $[B, AB, A^2B, \dots]$ |
| 完全可控          | Completely Controllable        | 可控性矩陣滿秩的系統                      |
| 可觀測性          | Observability                  | 能否由輸出推斷出完整的內部狀態                 |
| 平衡點           | Equilibrium Point              | 系統可以永遠停留的狀態                     |
| Lyapunov 穩定   | Lyapunov Stable                | 初始擾動小則後續偏差也小                    |
| 漸近穩定          | Asymptotically Stable          | 不僅有界且最終收斂到平衡點                   |
| 指數穩定          | Exponentially Stable           | 以指數速率收斂                         |
| 全域穩定          | Globally Stable                | 對任意初始狀態皆成立                      |
| Lyapunov 函數   | Lyapunov Function              | 類似能量、沿軌跡遞減的純量函數                 |
| 直接法           | Direct Method                  | 不需解方程即可判定穩定性的方法                 |
| 半負定           | Negative Semi-definite         | 恆小於等於零的二次型                      |
| LaSalle 不變性原理 | LaSalle's Invariance Principle | 處理 $V$ 半負定情形的定理                 |
| 極點配置          | Pole Placement                 | 指定閉環特徵值以設計增益的方法                 |
| 穩定裕度          | Stability Margin               | 系統距離不穩定的餘裕                      |
| 斜對稱矩陣         | Skew-symmetric Matrix          | 滿足 $S^T = -S$ 的矩陣               |

## 19.1 可控性：能不能控制

### 19.1.1 問題的意義

在設計控制器之前，應該先問一個更基本的問題：**這個系統有沒有可能被控制？**

可控性的正式定義是：對於任意的初始狀態  $x_0$  與任意的目標狀態  $x_f$ ，是否存在一個控制訊號  $u(t)$ ，能在有限時間內把系統由  $x_0$  帶到  $x_f$ ？若答案為是，稱系統**完全可控**。

這個問題之所以重要，是因為**不可控是系統的先天缺陷，任何控制器都無法彌補**。正如開頭故事中兩輪被焊死的車子——不管演算法多聰明，它就是轉不了彎。及早發現這一點，能避免把時間浪費在注定失敗的設計上。

### 19.1.2 可控性矩陣

對於線性系統  $\dot{x} = Ax + Bu$ ，Kalman 給出了一個優雅的判準。建構**可控性矩陣**：

$$C = [B, AB, A^2B, \dots, A^{n-1}B]$$

可控性矩陣。 $n$  為狀態的維度。矩陣的大小為  $n \times (n \cdot m)$ ，其中  $m$  為輸入的數目。

$$\text{系統完全可控} \iff \text{rank}(C) = n$$

*Kalman 可控性判準。秩必須等於狀態的維度。*

為什麼是這個形式？直觀的解釋是： $B$  代表「輸入能直接推動的方向」； $AB$  代表「經過一次系統動態演化後，這個推力會傳到哪些方向」； $A^2B$  則是傳兩次之後……把所有這些方向加起來，若能張滿整個狀態空間，就代表沒有任何方向是控制不到的。

為何只算到  $A^{n-1}B$  就停？這來自 Cayley-Hamilton 定理： $A^n$  可以表示為  $I, A, \dots, A^{n-1}$  的線性組合，因此再往後算不會產生新的方向。

### 19.1.3 不可控的例子

**例子一：兩個完全對稱的子系統共用一個輸入。**設有兩台完全相同的馬達，接在同一個訊號上。無論怎麼下指令，它們永遠做一模一樣的動作——「兩者的差」這個狀態永遠是零，無法被控制。這正是開頭故事中焊死的輪子。

**例子二：控制輸入影響不到某個狀態。**若某個關節的馬達壞了（或根本沒裝），該關節的角度就成為不可控狀態。

**例子三：機構的對稱性。**某些並聯機構在特定姿態下會出現對稱的自由度，使控制輸入無法區分兩種運動模式。

#### ◎ 可控性與奇異點的關係

第 12 章的奇異點與本章的不可控性有相似之處，但**不完全相同**。奇異點是**瞬時**的現象：在該位形下，雅可比失去滿秩，末端某個方向的速度無法產生；但只要離開那個位形，能力就恢復了。不可控則是**結構性**的缺陷：無論在哪個狀態，都有一部分狀態空間永遠到不了。用一個比喻：奇異點像車子開到了死巷（倒退出來就好），不可控則像車子根本沒有方向盤。實務上，機器手臂在奇異點附近會出現「暫時性的方向失能」，這在數學上更接近「可控性變差」而非「完全不可控」。

### 19.1.4 可控性與 LQR 的關係

第 18.8.1 節提到「只要系統可控，LQR 必定穩定」。現在可以理解這個條件的意義：若系統不可控，代表有些狀態根本無法被影響——若這些狀態本身又是不穩定的，那麼無論怎麼設計控制器都無法拯救。

嚴格來說，LQR 的穩定性只需要一個較弱的條件：**可穩定 (stabilizable)**——即所有**不穩定的模態**都是可控的。已經穩定的模態即使不可控也無妨，因為它們會自己收斂。

## 19.2 可觀測性與對偶

### 19.2.1 能不能「看見」

可控性問的是「能不能推動系統」，可觀測性則問相反的問題：**能不能由量測到的輸出，推斷出系統完整的內部狀態？**

這在機器人上是實際的問題。第 18.8.2 節提過，LQR 的控制律  $u = -Kx$  需要知道**所有狀態**。但實務上：關節速度往往不直接量測（而是由位置差分得到，雜訊大）、末端負載的狀態未知、柔性關節的變形量無法量測。這時就需要**觀測器**由可量測的訊號估計出完整狀態。

觀測器能否存在，取決於系統是否可觀測。設輸出方程為  $y = Cx$ ，則**可觀測性矩陣**為：

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

可觀測性矩陣。系統完全可觀測  $\Leftrightarrow \text{rank}(\mathcal{O}) = n$ 。

### 19.2.2 優雅的對偶關係

比較兩個矩陣的形式，會發現一個漂亮的對稱性：

$$(A, B) \text{ 可控} \iff (A^T, B^T) \text{ 可觀測}$$

*可控性與可觀測性的對偶原理。*

這代表兩個問題在數學上是同一個問題的兩面。因此**所有關於可控性的結果，都可以直接翻譯成關於可觀測性的結果**——包括設計方法。事實上，Kalman 濾波器（第 24 章）的增益計算與 LQR 的增益計算，用的是形式完全相同的 Riccati 方程，只是把  $A$  換成  $A^T$ 。

這個對偶在工程上的意義是：**估計與控制是同一件事的兩面**。控制是「決定要輸出什麼」，估計是「推斷輸入了什麼」——兩者共用同一套數學。第 24 章討論狀態估測時會再次見到這個結構。

## 19.3 穩定性的三個定義

### 19.3.1 平衡點

談穩定性之前，必須先定義**平衡點**：系統可以永遠停留不動的狀態。數學上：

$$x_e \text{ 是平衡點} \iff f(x_e, 0) = 0$$

*在該狀態下，若不施加控制，狀態的導數為零，因此系統保持不動。*

以倒立擺為例，有兩個平衡點：桿子垂直向下（穩定）與桿子垂直向上（不穩定）。兩者都滿足「不動就一直不動」的條件，但性質截然不同——這正是需要區分不同穩定性等級的原因。

### 19.3.2 三個層次

穩定性有三個層次，由弱到強：

**層次一：Lyapunov 穩定 (Stable)**。「初始擾動夠小，之後的偏差就會夠小。」形式化的說法是：對任意的  $\varepsilon > 0$ ，存在  $\delta > 0$ ，使得  $\|x(0) - x_e\| < \delta$  時，恆有  $\|x(t) - x_e\| < \varepsilon$ 。

注意這個定義**不要求收斂**——它只要求「不會越跑越遠」。一個無阻尼的單擺（永遠等幅擺動）就是 Lyapunov 穩定但不收斂的例子，正如第 16.3.2 節的純比例控制。

**層次二：漸近穩定 (Asymptotically Stable)**。在 Lyapunov 穩定的基礎上，還要求最終收斂：



$$\lim_{t \rightarrow \infty} x(t) = x_e$$

漸近穩定。這是控制設計通常追求的目標。

層次三：指數穩定 ( Exponentially Stable ) 。不僅收斂，還要求收斂速度至少是指數的：

$$\|x(t) - x_e\| \leq \alpha \|x(0) - x_e\| e^{-\lambda t}$$

指數穩定。 $\lambda$  稱為收斂率，它給出了「多快收斂」的定量保證。

三者的關係是：指數穩定  $\Rightarrow$  漸近穩定  $\Rightarrow$  Lyapunov 穩定。反之則不然。

表 19-2 三種穩定性的比較

| 等級          | 要求       | 會收斂嗎 | 有速度保證嗎 | 典型例子          |
|-------------|----------|------|--------|---------------|
| Lyapunov 穩定 | 偏差有界     | 不一定  | 無      | 無阻尼單擺、純 P 控制  |
| 漸近穩定        | 最終收斂到平衡點 | 會    | 無      | 有摩擦的單擺、PD 控制  |
| 指數穩定        | 以指數速率收斂  | 會    | 有      | 線性穩定系統、CTC 閉環 |

### 19.3.3 局部與全域

還有一個重要的區分：穩定性是局部還是全域的。

**局部穩定**：只有當初始狀態夠靠近平衡點時才成立。倒立擺的 LQR 控制器就是局部的——傾角太大時（第 18 章程式範例中約 45 度以上），線性化失效，控制器會失敗。這個「夠靠近」的範圍稱為**吸引域**（region of attraction）。

**全域穩定**：對任意初始狀態都成立。第 17.5.2 節的 PD 加重力補償就是全域漸近穩定——無論手臂一開始擺在哪裡，都保證會收斂。這是一個非常強的保證，也是本章要證明的核心結果。

## 19.4 線性系統的穩定性：特徵值判準

### 19.4.1 由解的形式看穩定性

對線性系統  $\dot{x} = Ax$ ，其解可以用矩陣指數表示： $x(t) = e^{(A)t} x(0)$ 。若  $A$  可對角化，則解可分解為各特徵模態的組合：

$$x(t) = \sum_i c_i v_i e^{\lambda_i t}$$

各項為特徵向量  $v_i$  乘上指數項。 $\lambda_i$  為特徵值。

由此立刻可以看出穩定性的判準：每一項是否衰減，取決於  $e^{\lambda_i t}$  是否趨近於零。若  $\lambda = a$

+ bj 為複數，則  $|e^{(\lambda t)}| = e^{(at)}$ ——只取決於實部。

### 線性系統漸近穩定 $\Leftrightarrow$ 所有特徵值的實部皆為負

*特徵值判準。即所有極點落在複平面的左半平面。*

虛部的作用則是決定振盪：實部決定衰減快慢、虛部決定振盪頻率。這與第 16 章二階系統的分析完全一致—— $\zeta\omega_n$  是實部（衰減）、 $\omega_d$  是虛部（振盪）。

#### 19.4.2 閉環系統的特徵值

加入狀態回授  $u = -Kx$  後，閉環系統為：

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

*閉環系統。穩定性取決於  $A - BK$  的特徵值。*

這給出了控制設計的一個直接視角：設計  $K$ ，就是在移動閉環系統的特徵值。開迴路不穩定的系統（如倒立擺，第 18.9 節算出有  $\lambda = +6.26$ ），可以透過適當的  $K$  把特徵值搬到左半平面。

#### 19.4.3 極點配置

既然穩定性取決於特徵值，一個自然的設計方法是：直接指定期望的閉環特徵值，反推  $K$ 。這稱為極點配置（pole placement）。

有一個重要的定理：若系統完全可控，則可以透過狀態回授把閉環特徵值配置到任意指定的位置。這正是可控性最重要的實用意義——它保證了設計的自由度。

極點配置與 LQR 的比較很有啟發性：

表 19-3 極點配置與 LQR 的比較

| 面向    | 極點配置           | LQR                     |
|-------|----------------|-------------------------|
| 設計者指定 | 期望的閉環極點位置      | 權重矩陣 $Q$ 、 $R$          |
| 直覺性   | 直接（極點決定響應）     | 間接（需理解權重意義）             |
| 多輸入系統 | 解不唯一，需額外準則     | 解唯一                     |
| 控制量大小 | 無保證，可能過大       | 由 $R$ 直接控制              |
| 穩定裕度  | 無保證            | 保證 $[0.5, \infty)$ 增益裕度 |
| 適用場合  | 低階、單輸入、需精確指定響應 | 多變數、需權衡控制量              |

實務上兩者常互補：先用 LQR 得到一組合理的增益，再檢查其閉環極點是否符合預期；或

用極點配置得到初值，再用 LQR 微調。

### 19.4.4 線性化與局部穩定性

對非線性系統  $\dot{x} = f(x)$ ，可以在平衡點附近做一階泰勒展開：

$$\dot{x} \approx A x, \quad \text{其中 } A = \partial f / \partial x |_{x = x_e} \quad (\text{雅可比矩陣})$$

近似線性化。注意這與第 17.3.3 節的回授線性化完全不同——這只在平衡點附近有效。

**Lyapunov 第一法（間接法）**指出：若線性化系統的所有特徵值實部皆為負，則原非線性系統在該平衡點**局部**漸近穩定。

這個方法的優點是簡單（只需算特徵值），缺點有二：**只能保證局部穩定**（不知道吸引域多大），且**當有特徵值實部為零時無法判定**（線性項不足以決定行為，需看高階項）。

這些限制，正是下一節 Lyapunov 直接法要克服的。

## 19.5 Lyapunov 直接法

### 19.5.1 碗中珠子的想法

Lyapunov 於 1892 年提出的方法，核心想法是開頭故事中那個「碗中珠子」的圖像：

#### ◎ Lyapunov 直接法的核心思想

若能找到一個類似「高度」或「能量」的純量函數  $V(x)$ ，滿足：一、它在平衡點為零、在其他地方為正（碗底最低）；二、沿著系統的軌跡，它持續下降（珠子一直往下滾）——那麼系統必定會收斂到平衡點。而且完全不需要解出微分方程。

這個方法的威力在於「不需要解方程」。對非線性系統，解析解幾乎不存在；但只要能找到一個下降的函數，就能斷定收斂。這使 Lyapunov 方法成為非線性控制中最重要的工具。

### 19.5.2 三個條件

形式化地說，若存在一個純量函數  $V(x)$ （稱為 **Lyapunov 函數**）滿足下列條件，則平衡點  $x_e = 0$  是穩定的：

條件一（正定）： $V(0) = 0$ ，且  $V(x) > 0$  對所有  $x \neq 0$

條件二（遞減）： $\dot{V}(x) \leq 0$  對所有  $x$

條件三（徑向無界）：當  $\|x\| \rightarrow \infty$  時  $V(x) \rightarrow \infty$

*Lyapunov 穩定性定理。條件三只在要證明全域穩定時才需要。*

三個條件的物理意義是：**條件一**確保  $V$  像一個「碗」——平衡點在碗底；**條件二**確保狀態沿著碗壁往下滑，不會往上爬；**條件三**確保碗是「無限深」的，不會有狀態從碗的邊緣溜走。

若進一步有  $\dot{V} < 0$ （嚴格負定，而非只是小於等於零），則平衡點是**漸近穩定**——不只有界，還會真正收斂到零。

### 19.5.3 關鍵技術：計算 $\dot{V}$

實際運用時，最關鍵的一步是計算  $V$  沿系統軌跡的時間導數。由鏈鎖律：

$$\dot{V} = (\partial V / \partial \mathbf{x})^T \dot{\mathbf{x}} = (\partial V / \partial \mathbf{x})^T \mathbf{f}(\mathbf{x})$$

*注意這裡代入了系統方程  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ 。這是整個方法的精髓——把系統的動態「灌」進  $V$  的導數中。*

這一步是初學者最容易忽略的： $\dot{V}$  不是隨便的導數，而是「沿著系統實際運動的軌跡」的導數。因此它必然包含系統的動態資訊，也因此才能反映系統的行為。

### 19.5.4 方法的優點與難處

**優點一：適用於非線性系統。**特徵值判準只適用於線性系統，Lyapunov 方法則無此限制。

**優點二：可證明全域穩定。**線性化只能給出局部結果，Lyapunov 方法（配合條件三）能證明全域性質。

**優點三：不需解方程。**這在實務上是決定性的——多數機器人系統的閉環方程根本無法解析求解。

**難處：如何找到  $V$ ？**這是 Lyapunov 方法唯一的、也是最大的困難。定理只說「若能找到」，卻沒說「怎麼找」。對一般的非線性系統，構造 Lyapunov 函數是一門藝術。

但機器人有一個巨大的優勢：它是機械系統，天然就有一個現成的候選——**能量**。這正是下一節的主題。

## 19.6 能量作為 Lyapunov 函數

### 19.6.1 為何能量是天然的候選

物理系統的能量有三個性質，恰好對應 Lyapunov 函數的要求：**一、能量恆為非負**（動能為正、位能可調整基準使其為正）；**二、能量在平衡點最小**（穩定平衡就是能量的局部極小點）；**三、有耗散時能量會下降**（摩擦、阻尼會消耗能量）。

因此對機械系統，「證明穩定」往往等價於「證明能量持續下降」——這是一個極其自然的物理論證，而 Lyapunov 方法把它變成了嚴格的數學工具。

### 19.6.2 最簡單的例子：質量—彈簧—阻尼

先用最簡單的例子熟悉流程。考慮  $m\ddot{x} + c\dot{x} + kx = 0$  (第 16 章的 PD 控制閉環系統)。取總能量為 Lyapunov 候選：

$$V = \frac{1}{2} m \dot{x}^2 + \frac{1}{2} k x^2$$

動能加彈性能。顯然  $V \geq 0$ ，且僅在  $x = \dot{x} = 0$  時為零——滿足條件一。

計算  $\dot{V}$  並代入系統方程 ( $\ddot{x} = -(c\dot{x} + kx)/m$ )：

$$\dot{V} = m \dot{x} \ddot{x} + k x \dot{x} = \dot{x}(m \ddot{x} + k x) = \dot{x}(-c \dot{x}) = -c \dot{x}^2$$

關鍵的一步：彈性項  $kx\dot{x}$  與  $m\dot{x}\ddot{x}$  中的對應項互相抵消，只剩下阻尼項。

$$\dot{V} = -c \dot{x}^2 \leq 0$$

只要阻尼  $c > 0$ ，能量就持續下降——系統穩定。

這個計算揭示了一個深刻的事實：在能量的觀點下，彈簧不消耗能量（只是在動能與位能之間搬運），只有阻尼才真正耗散能量。這正是第 16.3.2 節「純比例控制會永遠振盪」的物理解釋——沒有阻尼項， $\dot{V} = 0$ ，能量守恆，振盪不衰減。

### 19.6.3 $\dot{V}$ 半負定的問題與 LaSalle 原理

上例中  $\dot{V} = -c\dot{x}^2$  只是半負定——當  $\dot{x} = 0$  但  $x \neq 0$  時（例如彈簧被拉開但瞬間靜止）， $\dot{V} = 0$ 。嚴格說來，這只能證明 Lyapunov 穩定，不足以證明漸近穩定。

解決這個問題的工具是 LaSalle 不變性原理：

#### ◎ LaSalle 不變性原理

若  $\dot{V} \leq 0$ ，考慮集合  $E = \{x : V(x) = 0\}$ 。系統的軌跡最終會收斂到  $E$  中最大的不變集（即進去後就不會出來的部分）。若這個不變集只包含平衡點，則系統漸近穩定。

套用到上例： $E = \{(x, \dot{x}) : \dot{x} = 0\}$ 。但若系統停在  $\dot{x} = 0$  而  $x \neq 0$  處，由方程  $m\ddot{x} = -kx \neq 0$ ，代表  $\ddot{x} \neq 0$ ，狀態立刻會離開  $\dot{x} = 0$ ——因此無法「停留」在  $E$  中。 $E$  中唯一的不變集只有原點，故系統漸近穩定。

LaSalle 原理在機器人控制的穩定性證明中幾乎必定會用到，因為機械系統的  $V$  通常只含速

度項（半負定）。

## 19.7 伏筆回收：PD 加重力補償的穩定性證明

### 19.7.1 要證明什麼

現在具備了所有工具，可以嚴格證明第 17.5.2 節提到的 Takegaki-Arimoto 定理。要證明的敘述是：

#### ◎ 待證定理

對於任意的機械手臂，控制律  $\tau = K_p(q_d - q) - K_d \dot{q} + G(q)$ （其中  $K_p$ 、 $K_d$  為任意正定矩陣、 $q_d$  為常數目標）能保證閉環系統全域漸近穩定。也就是說：無論初始姿態在哪裡、無論機器人的慣性參數是多少，關節必定收斂到  $q_d$  且速度趨於零。

值得再次強調這個定理的驚人之處：控制器完全不需要知道  $M(q)$  與  $C(q, \dot{q})$ ，只需要重力項  $G(q)$ 。這是為何重力補償在實務上如此重要。

### 19.7.2 建立閉環方程

由第 13 章的動力學方程與上述控制律：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = K_p e - K_d \dot{q} + G(q) \quad (\text{其中 } e = q_d - q)$$

代入控制律。注意  $G(q)$  在兩邊出現，直接抵消。

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + K_d \dot{q} - K_p e = 0$$

閉環系統方程。重力項已完全消失——這正是重力補償的作用。

注意  $q_d$  為常數，故  $\dot{e} = -\dot{q}$ 、 $\ddot{e} = -\ddot{q}$ 。這個關係在後續計算中會用到。

### 19.7.3 構造 Lyapunov 函數

依 19.6.1 節的思路，取「動能 + 虛擬彈性位能」作為候選：

$$V = \frac{1}{2} \dot{q}^T M(q) \dot{q} + \frac{1}{2} e^T K_p e$$

Lyapunov 候選函數。第一項是機器人的實際動能（第 13.2.2 節）、第二項是控制器產生的「虛擬彈簧」的位能。

驗證條件一（正定）：由第 13.6.1 節， $M(q)$  恆為正定，故第一項  $\geq 0$  且僅在  $\dot{q} = 0$  時為零； $K_p$  為正定矩陣，故第二項  $\geq 0$  且僅在  $e = 0$  時為零。因此  $V \geq 0$ ，且僅在  $(e, \dot{q}) = (0, 0)$  時為零——條件一成立。

驗證條件三（徑向無界）：當  $\|e\|$  或  $\|q\|$  趨於無窮時， $V$  也趨於無窮（因兩個矩陣皆正定）——條件三成立，故可證明全域穩定。

#### 19.7.4 計算 $V$ ：斜對稱性登場

這是整個證明最關鍵的一步。對  $V$  求時間導數：

$$\dot{V} = \dot{q}^T M \dot{q} + \frac{1}{2} \dot{q}^T \dot{M} \dot{q} + e^T K_p \dot{e}$$

注意第二項——因為  $M$  是  $q$  的函數，會隨時間變化，微分時必須包含  $\dot{M}$  這一項。這一項的存在正是機器人動力學的特色。

由閉環方程解出  $M \ddot{q} = -C \dot{q} - K_d \dot{q} + K_p e$ ，代入：

$$\dot{V} = \dot{q}^T (-C \dot{q} - K_d \dot{q} + K_p e) + \frac{1}{2} \dot{q}^T \dot{M} \dot{q} + e^T K_p \dot{e}$$

代入系統動態。

利用  $\dot{e} = -\dot{q}$ ，第三項變為  $-e^T K_p \dot{q}$ 。又因  $K_p$  對稱， $\dot{q}^T K_p e = e^T K_p \dot{q}$ ，故第一項中的  $\dot{q}^T K_p e$  與第三項恰好抵消：

$$\dot{V} = -\dot{q}^T C \dot{q} - \dot{q}^T K_d \dot{q} + \frac{1}{2} \dot{q}^T \dot{M} \dot{q}$$

剩下三項。

整理前兩項中含  $C$  與  $\dot{M}$  的部分：

$$\dot{V} = \frac{1}{2} \dot{q}^T (\dot{M} - 2C) \dot{q} - \dot{q}^T K_d \dot{q}$$

關鍵時刻：括號中出現了第 13.7.3 節那個「看似無用」的組合。

回顧第 13 章的性質：採 Christoffel 符號定義的  $C$  使  $\dot{M} - 2C$  為斜對稱矩陣。而斜對稱矩陣有一個重要性質：

$$\text{對任意向量 } x \text{ 與斜對稱矩陣 } S: \quad x^T S x = 0$$

斜對稱矩陣的二次型恆為零。證明： $x^T S x$  是純量，故等於其轉置  $x^T S^T x = -x^T S x$ ，因此  $2x^T S x = 0$ 。

因此第一項完全消失：

$$\dot{V} = -\dot{q}^T K_d \dot{q} \leq 0$$

得證。由於  $K_d$  正定， $V$  恆為非正——能量持續下降。

#### 19.7.5 由 LaSalle 完成證明

$\dot{V}$  只是半負定（當  $\dot{q} = 0$  時為零），因此需要 LaSalle 原理。考慮集合  $E = \{(e, \dot{q}) : \dot{q} = 0\}$ 。

假設系統能停留在  $E$  中，則恆有  $\dot{q} = 0$  且  $\ddot{q} = 0$ 。代入閉環方程：

$$M \cdot 0 + C \cdot 0 + K_d \cdot 0 - K_p e = 0 \Rightarrow K_p e = 0 \Rightarrow e = 0 \quad (\text{因 } K_p \text{ 正定必可逆})$$

$E$  中唯一的不變集是原點。

因此由 LaSalle 原理，系統全域漸近收斂到  $e = 0$ 、 $\dot{q} = 0$ 。定理得證。

### ◎ 這個證明告訴我們什麼

這個證明有三個值得深思的地方。一、 $\dot{M} - 2C$  的斜對稱性不是數學巧合，而是能量守恆的體現——第 13.7.3 節說過，這個性質的物理意義是「內部的科氏力與離心力不產生也不消耗能量，只是重新分配」。正因如此，它在能量的計算中才會完全消失。二、證明完全不需要知道  $M$  與  $C$  的具體形式——只用到了  $M$  正定與  $\dot{M} - 2C$  斜對稱兩個結構性質。這正是定理如此強大的原因。三、 $K_d$  是唯一的耗散來源——與 19.6.2 節的結論一致：彈簧（ $K_p$ ）不耗能，只有阻尼（ $K_d$ ）才讓系統真正收斂。

圖 19-1 PD 加重力補償的能量下降圖像

建議配圖：左半為一個三維的碗狀曲面，橫軸為位置誤差  $e$ 、縱軸為速度  $\dot{q}$ 、高度為 Lyapunov 函數  $V$ 。碗底標註平衡點  $(0,0)$ 。一條螺旋狀的軌跡由碗緣沿著曲面往下滑向碗底，並以箭頭標註「 $\dot{V} = -\dot{q}^T K_d \dot{q} \leq 0$ 」。右半為對應的機器手臂多重曝光：由一個任意的初始姿態逐步收斂到目標姿態，並在旁列出證明的四個關鍵步驟（正定性、代入動態、斜對稱項消失、LaSalle）。

## 19.8 其他控制器的穩定性

### 19.8.1 計算力矩控制

第 17 章的計算力矩控制，其穩定性證明反而最簡單——因為閉環系統已經是線性的。

回顧第 17.2.2 節，CTC 的誤差動態為：

$$\ddot{e} + K_d \dot{e} + K_p e = 0$$

線性、非時變、且各關節解耦。

這是  $n$  個獨立的二階線性系統。由 19.4.1 節的特徵值判準，只要  $K_p$ 、 $K_d$  為正定矩陣（通常取對角矩陣），所有特徵值的實部皆為負，系統指數穩定。

這也顯示了 CTC 的一個優勢：它提供的是指數穩定（有明確的收斂速率保證），而 PD 加重力補償只能保證漸近穩定（會收斂但速率不定）。代價則是需要完整的動力學模型。



## 19.8.2 LQR 的穩定性

第 18.8.1 節宣稱「LQR 保證穩定」，現在可以看出證明的思路：LQR 的價值函數  $V(x) = x^T P x$  本身就是一個 Lyapunov 函數。

條件一：由 Riccati 方程的正定解  $P$ ， $V = x^T P x$  恆為正定。

條件二：計算閉環系統  $\dot{x} = (A - BK)x$  下的  $V$ ，經過整理可得：

$$\dot{V} = -x^T(Q + K^T R K)x \leq 0$$

由於  $Q$  半正定、 $R$  正定，括號中的矩陣至少半正定，故  $\dot{V} \leq 0$ 。

這個結果有一個優美的解讀： $V$  恰好等於「當下的代價」的負值。也就是說，價值函數（未來的總代價）以「當下正在付出的代價」的速率遞減——這完全符合直覺，因為每過一刻，「剩餘的代價」就少掉了「剛付出的代價」。

因此最佳性與穩定性在 LQR 中是同一件事的兩面。這是第 18 章與本章之間最深刻的連結。

## 19.8.3 三種控制器的穩定性比較

表 19-4 三種控制器的穩定性性質

| 控制器       | 所需模型           | 穩定性等級       | 範圍        | 證明工具                      |
|-----------|----------------|-------------|-----------|---------------------------|
| PD（無重力補償） | 無              | 有穩態誤差，非漸近穩定 | 全域有界      | Lyapunov + LaSalle        |
| PD + 重力補償 | 只需 $G(q)$      | 漸近穩定        | 全域        | Lyapunov + LaSalle + 斜對稱性 |
| 計算力矩控制    | 完整動力學          | 指數穩定        | 全域（模型準確時） | 特徵值判準（閉環為線性）              |
| LQR（線性系統） | $A$ 、 $B$ 矩陣   | 指數穩定        | 全域（線性系統）  | 價值函數即 Lyapunov 函數         |
| LQR（線性化後） | 工作點的 $A$ 、 $B$ | 指數穩定        | 局部（吸引域內）  | Lyapunov 第一法              |

## 19.8.4 模型誤差下的穩定性

實務上還有一個重要問題：若模型有誤差，穩定性還成立嗎？

對 PD 加重力補償：若  $G(q)$  的估計有誤差  $\Delta G$ ，則閉環方程中會殘留一項  $\Delta G$ 。此時系統不

再收斂到  $e = 0$ ，而是收斂到  $K_p e = \Delta G$ ，即  $e_{ss} = K_p^{-1} \Delta G$ 。系統仍然穩定（誤差有界），只是有穩態誤差——這與第 17 章例題 17-2 的分析一致。

對 **計算力矩控制**：模型誤差使閉環方程變為  $\ddot{e} + K_d \dot{e} + K_p e = \delta$ （ $\delta$  為模型誤差造成的等效干擾）。只要  $\delta$  有界，系統就是**輸入—狀態穩定**（ISS, Input-to-State Stable）——誤差有界且正比於  $\delta$  的大小。這正是第 17.4.2 節「80% 準確的模型仍然有用」的嚴格說法。

這類「有界干擾下仍保持有界誤差」的性質，是實務上真正需要的保證。純粹的漸近穩定在有雜訊與模型誤差的現實中並不存在，ISS 才是實用的穩定性概念。

## 19.9 離散時間系統的穩定性

### 19.9.1 數位控制器的現實

第 7 章與第 16.7 節提過，實際的控制器都是數位的：以固定週期取樣、計算、輸出。因此嚴格的分析必須在**離散時間**下進行。

離散系統的形式為：

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$$

離散狀態方程。 $\mathbf{A}_d$  與  $\mathbf{B}_d$  由連續系統的  $\mathbf{A}$ 、 $\mathbf{B}$  與取樣週期  $T$  換算而得。

### 19.9.2 穩定性判準的差異

離散系統的穩定性判準與連續系統形式相似，但判定的區域不同：

連續系統穩定  $\Leftrightarrow$  所有特徵值的實部  $< 0$ （左半平面）

離散系統穩定  $\Leftrightarrow$  所有特徵值的絕對值  $< 1$ （單位圓內）

兩者的穩定區域。這是初學者最容易混淆的一點。

為何是單位圓？因為離散系統的解為  $\mathbf{x}_k = \mathbf{A}_d^k \mathbf{x}_0$ ，各模態的行為由  $\lambda^k$  決定。若  $|\lambda| < 1$ ，則  $\lambda^k \rightarrow 0$ ；若  $|\lambda| > 1$ ，則發散。

兩者的對應關係為  $\lambda_d = e^{(\lambda_c T)}$ 。可以驗證： $\lambda_c$  的實部為負  $\Leftrightarrow |\lambda_d| < 1$ 。連續的左半平面，經過指數映射後恰好變成離散的單位圓內部。

### 19.9.3 取樣週期對穩定性的影響

這裡有一個重要的實務結論：**取樣週期太長會使原本穩定的系統變得不穩定。**

直觀的解釋是：數位控制器在兩次取樣之間是「盲目」的——它保持上一次的輸出不變，看不見這段期間系統的變化。取樣越慢，這段盲目期越長，等效於引入了延遲。而延遲會侵蝕相位裕度，過多的延遲必然導致不穩定。

實務上的經驗法則是：

取樣頻率  $\geq 10 \times$  閉環頻寬 （最低要求）

取樣頻率  $\geq 20 \sim 30 \times$  閉環頻寬 （建議值）

取樣頻率的選擇。閉環頻寬約等於設計的  $\omega_n$ 。

以第 18 章的倒立擺為例：其不穩定極點為 6.26 rad/s (約 1 Hz)，閉環設計的頻寬可能在 3~5 Hz，因此取樣頻率至少應為 50 Hz，建議 100 Hz 以上。這正是第 18 章例題 18-3 中「發散時間常數 0.16 秒  $\Rightarrow$  控制頻率至少 50 Hz」的依據。

反過來說，這也解釋了為何機器手臂的伺服迴路要跑到 1~4 kHz (第 3 章的時間尺度金字塔)——因為要控制的機構共振頻率可達數十至數百 Hz。

## 19.10 自適應控制的穩定性概觀

### 19.10.1 為何自適應控制的證明特別困難

第 17.8.2 節介紹的自適應控制，其穩定性證明是一個經典的挑戰。困難在於：系統中有兩組動態在同時演變——狀態的收斂與參數估計的收斂——而且它們互相影響。

Slotine-Li 自適應控制器的證明思路，正是 Lyapunov 方法的漂亮應用。它的 Lyapunov 候選函數同時包含兩部分：

$$V = \frac{1}{2} s^T M(q) s + \frac{1}{2} \tilde{\pi}^T \Gamma^{-1} \tilde{\pi}$$

第一項是「追隨誤差」的能量 ( $s$  為第 17.8.1 節的滑動面變數)、第二項是「參數估計誤差」 $\tilde{\pi} = \pi - \pi$  的能量。 $\Gamma$  為自適應增益矩陣。

計算  $V$  後，經過與 19.7.4 節類似的操作 (包括斜對稱性的抵消)，可以證明  $\dot{V} \leq 0$ 。因此系統穩定，且追隨誤差收斂到零。

### 19.10.2 一個微妙但重要的區別

這個證明有一個常被誤解的結論：追隨誤差保證收斂到零，但參數估計不保證收斂到真值。

原因是  $V$  只包含追隨誤差的項，不包含參數誤差的項。因此參數誤差只保證有界，不保證趨

於零。要讓參數也收斂，需要額外的**持續激勵條件** ( persistent excitation ) ——軌跡必須「足夠豐富」，才能把所有參數都激發出來讓演算法辨識。

實務上的意義是：若機器人只做單調的重複動作，自適應控制器仍能把軌跡追隨得很好，但它學到的參數可能是錯的。一旦換一個新軌跡，性能可能突然變差。這是第 17 章專題延伸中要求驗證的現象。

## 19.11 由穩定性邁向力控制

### 19.11.1 本章的成果

本章補上了前三章的理論缺口。你現在能夠：判斷系統是否可控與可觀測、區分三個層次的穩定性、以特徵值判斷線性系統的穩定性、運用極點配置設計增益、以 Lyapunov 直接法證明非線性系統的穩定性、以及理解離散化對穩定性的影響。

最重要的是，你完成了三個伏筆的回收：第 13 章的  $\dot{M} - 2C$  斜對稱性在 19.7.4 節成為證明的關鍵；第 17 章的 Takegaki-Arimoto 定理在 19.7 節得到完整證明；第 18 章 LQR 的穩定性保證在 19.8.2 節被揭示為「價值函數即 Lyapunov 函數」。

這三個回收展現了一件事：理論的各個部分不是孤立的知識點，而是互相支撐的結構。第 13 章那個當時看似無用的數學性質，兩百頁後成為證明的樞紐——這正是系統性學習的價值。

### 19.11.2 接下來：當機器人碰到東西

到目前為止，第三篇的所有控制都是**運動控制**：規劃一條軌跡，讓機器人精確地跟隨它。這隱含一個假設——機器人在自由空間中運動，不與環境接觸。

但機器人的價值恰恰在於接觸：抓取物件、插入零件、研磨表面、與人握手。一旦接觸發生，純粹的位置控制就會出問題：若目標位置在工件內部（哪怕只差 0.1 公釐），控制器會不斷加大力矩試圖到達那個「不可能的位置」，結果不是壓壞工件就是損壞機器人。

第 20 章（力控制與阻抗控制）將處理這個問題。我們會學到：如何直接控制接觸力、如何用「混合位置 / 力控制」在不同方向分別控制運動與力、以及最優雅的**阻抗控制**——不直接控制位置或力，而是控制兩者之間的**關係**，讓機器人表現得像一個可調整的彈簧。屆時第 17 章的操作空間控制與本章的穩定性工具都將派上用場。

換句話說：第 19 章保證了機器人**不會失控**，第 20 章讓它學會**溫柔地接觸**。

## 完整計算例題

### 例題 19-1 可控性判定

題目：(a) 判斷第 18 章的一維點質量系統是否完全可控。(b) 某系統  $A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$ 、 $B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ，判斷其可控性並解釋物理意義。(c) 某系統  $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ，判斷其可控性。

解 (a)：一維點質量 ( $m = 2$ ) 的模型為  $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$ 。

$$AB = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix}$$

計算  $AB$ 。

$$C = [B, AB] = \begin{bmatrix} 0 & 0.5 \\ 0.5 & 0 \end{bmatrix}$$

可控性矩陣。

其行列式為  $0 \times 0 - 0.5 \times 0.5 = -0.25 \neq 0$ ，故  $\text{rank}(C) = 2 = n$ 。系統完全可控。

物理意義：施力可以改變速度，速度又改變位置——因此透過適當的施力序列，可以把物體帶到任意的位置與速度組合。

解 (b)：計算  $AB = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 。

$$C = [B, AB] = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

第二列全為零。

行列式為零， $\text{rank}(C) = 1 < 2$ 。系統不可控。

物理意義：這個系統的兩個狀態是完全解耦的 ( $A$  為對角矩陣)，而  $B$  的第二個元素為零——代表控制輸入完全影響不到第二個狀態。 $x_2$  會依自己的動態  $\dot{x}_2 = 2x_2$  發散，而我們無能為力。這正是開頭故事中「有些系統根本控制不了」的數學形式。

解 (c)：計算  $AB = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$ 。

$$C = [B, AB] = \begin{bmatrix} 0 & 1 \\ 1 & -3 \end{bmatrix}$$

可控性矩陣。

行列式為  $0 \times (-3) - 1 \times 1 = -1 \neq 0$ ， $\text{rank} = 2$ 。系統完全可控，因此可以用極點配置或 LQR 把閉環極點放到任意位置。

## 例題 19-2 Lyapunov 穩定性證明

題目：考慮非線性系統  $\dot{x} = -x^3$ 。(a) 求其平衡點。(b) 用線性化判斷穩定性，並說明遇到的困難。(c) 用 Lyapunov 直接法證明其漸近穩定。(d) 判斷是否為指數穩定。

解 (a)：令  $\dot{x} = 0$ ，得  $-x^3 = 0$ ，故唯一平衡點為  $\mathbf{x}_e = 0$ 。

解 (b)：線性化需計算  $\partial f / \partial x = -3x^2$ ，在  $x = 0$  處：

$$A = \partial f / \partial x \big|_{x=0} = -3(0)^2 = 0$$

線性化的系統矩陣為零。

特徵值  $\lambda = 0$ ，實部為零。由 19.4.4 節，Lyapunov 第一法在此無法判定——線性項不足以決定行為，必須看高階項。這正是線性化方法的限制。

解 (c)：取最簡單的 Lyapunov 候選：

$$V(x) = \frac{1}{2} x^2$$

顯然  $V(0) = 0$ 、 $V(x) > 0$  對  $x \neq 0$  (條件一成立)，且  $x \rightarrow \pm\infty$  時  $V \rightarrow \infty$  (條件三成立)。

計算  $\dot{V}$  並代入系統方程：

$$\dot{V} = x \dot{x} = x(-x^3) = -x^4$$

代入  $\dot{x} = -x^3$ 。

對所有  $x \neq 0$ ， $-x^4 < 0$  (嚴格負定)。因此  $\dot{V} < 0$ ，由 Lyapunov 定理，系統全域漸近穩定。

這個例子完美展示了直接法的價值：線性化束手無策的問題，Lyapunov 方法只用兩行就解決了。

解 (d)：雖然漸近穩定，但不是指數穩定。可以解出精確解驗證：

$$x(t) = x_0 / \sqrt{1 + 2x_0^2 t}$$

解析解。當  $t$  很大時， $x(t) \approx 1/\sqrt{2t}$ ——這是代數衰減 (如  $t^{-1/2}$ )，比任何指數衰減都慢。

**工程意義：**漸近穩定只保證「最終會到」，不保證「多快到」。在控制設計中，若要求明確的收斂速率，必須額外驗證指數穩定性——這正是第 17 章的 CTC (指數穩定) 優於 PD 加重力

補償（僅漸近穩定）的地方。

### 例題 19-3 單關節 PD 加重力補償的穩定性驗證

題目：考慮第 13 章的單擺（ $I = (1/3)mL^2$ ，質量  $m = 2 \text{ kg}$ 、長  $L = 0.6 \text{ m}$ ），採用控制律  $\tau = K_p(\theta_d - \theta) - K_d \dot{\theta} + mg(L/2)\sin \theta$ ，取  $K_p = 20 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 4 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。（a）寫出閉環方程。（b）構造 Lyapunov 函數並驗證條件一。（c）計算  $V$ 。（d）用 LaSalle 原理完成證明。（e）驗證此為單自由度情形下 19.7 節定理的特例。

解（a）：單擺的動力學為  $I \ddot{\theta} + mg(L/2)\sin \theta = \tau$ 。代入控制律：

$$I \ddot{\theta} + mg(L/2)\sin \theta = K_p(\theta_d - \theta) - K_d \dot{\theta} + mg(L/2)\sin \theta$$

重力項在兩邊抵消。

$$I \ddot{\theta} + K_d \dot{\theta} - K_p e = 0 \quad (\text{其中 } e = \theta_d - \theta)$$

閉環方程。這是一個線性的二階系統——因為單自由度時  $M$  為常數、 $C$  為零。

代入數值： $I = (1/3)(2)(0.36) = 0.24 \text{ kg}\cdot\text{m}^2$ 。

$$0.24 \ddot{\theta} + 4 \dot{\theta} - 20 e = 0$$

數值化的閉環方程。

解（b）：依 19.7.3 節的形式取：

$$V = \frac{1}{2} I \dot{\theta}^2 + \frac{1}{2} K_p e^2 = \frac{1}{2}(0.24)\dot{\theta}^2 + \frac{1}{2}(20)e^2 = 0.12 \dot{\theta}^2 + 10 e^2$$

Lyapunov 候選函數。

驗證條件一：兩項的係數皆為正（ $I > 0$  且  $K_p > 0$ ），故  $V \geq 0$ ，且僅在  $\dot{\theta} = 0$  且  $e = 0$  時為零。條件一成立。此外  $V$  隨  $|e|$  或  $|\dot{\theta}|$  增大而無界，條件三亦成立。

解（c）：

$$\dot{V} = I \dot{\theta} \ddot{\theta} + K_p e \dot{e}$$

求導。注意單自由度時  $i = 0$ ，故無  $\frac{1}{2} q^T M \dot{q}$  項——這是單擺比多連桿手臂簡單之處。

由閉環方程  $I \ddot{\theta} = K_p e - K_d \dot{\theta}$ ，且  $\dot{e} = -\dot{\theta}$ ：

$$\dot{V} = \dot{\theta}(K_p e - K_d \dot{\theta}) + K_p e(-\dot{\theta}) = K_p e \dot{\theta} - K_d \dot{\theta}^2 - K_p e \dot{\theta} = -K_d \dot{\theta}^2$$

彈性項抵消，只剩阻尼項。

$$V = -4\theta^2 \leq 0$$

半負定。能量持續下降。

解 (d) :  $V = 0$  僅當  $\theta = 0$ 。考慮  $E = \{(e, \theta) : \theta = 0\}$ 。若系統停留在  $E$  中，則  $\dot{\theta} = 0$  且  $\ddot{\theta} = 0$ 。代入閉環方程：

$$0.24(0) + 4(0) - 20e = 0 \Rightarrow e = 0$$

$E$  中唯一的不變集是原點。

由 LaSalle 原理，系統**全域漸近穩定**——無論初始角度為何（即使桿子一開始朝下、目標朝上），最終都會收斂到目標。

解 (e) : 這確實是 19.7 節定理的單自由度特例。對照可知：

- $M(q) \rightarrow I$ ：單自由度時慣量為常數，故  $\dot{M} = 0$ 。
- $C(q, \dot{q}) \rightarrow 0$ ：只有一個關節，沒有科氏力與離心力的耦合。
- $\dot{M} - 2C = 0 - 0 = 0$ ：斜對稱項自動為零（零矩陣既對稱又斜對稱），因此在 19.7.4 節中「消失」的那一項，在單自由度情形下根本不會出現。

這說明了為何多自由度的證明需要斜對稱性：單自由度時運氣好（該項為零），多自由度時該項不為零，必須靠斜對稱性才能消掉。第 13 章那個「看似無用」的性質，正是為此而存在。

## 程式範例：可控性檢查與 Lyapunov 函數的數值驗證

以下 Python 程式實作可控性與可觀測性的檢查、極點配置、以及 Lyapunov 函數沿軌跡的數值驗證——直接「看見」能量下降的過程。

```
import numpy as np
from scipy.linalg import solve_continuous_are

# ===== 可控性與可觀測性 =====
def controllability_matrix(A, B):
    n = A.shape[0]
    cols = [B]
    for i in range(1, n):
        cols.append(A @ cols[-1])
    return np.hstack(cols)

def observability_matrix(A, C):
    n = A.shape[0]
    rows = [C]
    for i in range(1, n):
```



```

        rows.append(rows[-1] @ A)
    return np.vstack(rows)

def check_system(name, A, B, C=None):
    n = A.shape[0]
    Cm = controllability_matrix(A, B)
    r = np.linalg.matrix_rank(Cm)
    print(f"{name}")
    print(f"    可控性矩陣秩 = {r} / {n} → ")
    f"'完全可控' if r == n else '不可控 ( 有 %d 個狀態控制不到 )' % (n-r)}")
    if C is not None:
        Om = observability_matrix(A, C)
        ro = np.linalg.matrix_rank(Om)
        print(f"    可觀測性矩陣秩 = {ro} / {n} → ")
        f"'完全可觀測' if ro == n else '不可觀測'}")
    eig = np.linalg.eigvals(A)
    unstable = [e for e in eig if e.real > 1e-9]
    print(f"    開迴路特徵值 = {np.round(eig, 4)} → ")
    f"'不穩定 (%d 個正實部)' % len(unstable) if unstable else '穩定'}")

print("=== 例題 19-1：可控性判定 ===")
# (a) 一維點質量
check_system("(a) 一維點質量 m=2",
              np.array([[0., 1.], [0., 0.]]),
              np.array([[0.], [0.5]]))
# (b) 解耦系統，輸入影響不到第二狀態
check_system("(b) 解耦系統 ( B 第二列為零 )",
              np.array([[1., 0.], [0., 2.]]),
              np.array([[1.], [0.]])
# (c) 一般二階系統
check_system("(c) 二階系統",
              np.array([[0., 1.], [-2., -3.]]),
              np.array([[0.], [1.]])

# ===== 極點配置 =====
print("\n=== 極點配置：把不穩定系統搬到左半平面 ===")
A = np.array([[0., 1.], [2., 0.]])      # 特徵值 ±1.414 · 不穩定
B = np.array([[0.], [1.]])
print(f"開迴路特徵值: {np.round(np.linalg.eigvals(A), 4)} ← 不穩定")

def place_poles_2nd(A, B, desired):
    """二階單輸入系統的極點配置 ( 可控標準型法 ) """
    # 期望特徵多項式 s^2 + a1 s + a0
    a1 = -(desired[0] + desired[1])
    a0 = desired[0] * desired[1]
    # 開迴路特徵多項式係數
    tr, det = np.trace(A), np.linalg.det(A)
    # 對此結構 A=[[0,1],[a,b]], B=[[0],[1]] · K 可直接推得
    return np.array([[a0 + A[1,0], a1 + A[1,1]]])

```

```

for poles in ([-2., -3.], [-5., -6.]):
    K = place_poles_2nd(A, B, poles)
    eig_cl = np.linalg.eigvals(A - B @ K)
    print(f" 期望極點 {poles}: K = {np.round(K.flatten(), 3)}  "
          f"實際閉環極點 = {np.round(np.sort(eig_cl.real), 4)}")

# ===== Lyapunov 函數的數值驗證 =====
print("\n=== 例題 19-3：單擺 PD+重力補償的 Lyapunov 驗證 ===")
m, L, g = 2.0, 0.6, 9.8
I = m*L**2/3
Kp, Kd = 20.0, 4.0
theta_d = np.deg2rad(90.0)          # 目標：水平

def closed_loop(state):
    """閉環動態：I·θ̈ + Kd·θ̇ - Kp·e = 0"""
    th, thd = state
    e = theta_d - th
    thdd = (Kp*e - Kd*thd) / I
    return np.array([thd, thdd])

def lyapunov(state):
    th, thd = state
    e = theta_d - th
    return 0.5*I*thd**2 + 0.5*Kp*e**2

def lyapunov_dot(state):
    """理論值 Ḃ = -Kd·θ̇²"""
    return -Kd * state[1]**2

print(f"I = {I:.4f} kg·m²,  Kp = {Kp}, Kd = {Kd}")
print(f"\n{'時間':>7}{ 'θ (度)':>10}{ 'θ̇':>10}{ 'V':>12}"
      f"{'Ḃ(數值)':>12}{ 'Ḃ(理論)':>12}")

for th0_deg in (-90.0, 200.0):      # 兩個差異極大的初始角
    print(f"\n 初始角度 = {th0_deg}° ( 目標 90° )")
    s = np.array([np.deg2rad(th0_deg), 0.0])
    dt = 1e-4
    V_prev = lyapunov(s)
    for k in range(int(3.0/dt)):
        sd = closed_loop(s)
        s = s + sd*dt
        if (k+1) % 5000 == 0:
            V = lyapunov(s)
            Vdot_num = (V - V_prev)/(5000*dt)
            print(f"{'(k+1)*dt:6.2f}s':>10}{np.rad2deg(s[0]):10.3f}"
                  f"{'s[1]:10.4f}{V:12.5f}{Vdot_num:12.5f}"
                  f"{'lyapunov_dot(s):12.5f}")
            V_prev = V
    print(f"  → 最終誤差 = {np.rad2deg(theta_d - s[0]):.6f}°  "
          f" ( 收斂到目標 · 驗證全域漸近穩定 ) ")

```

```

# ===== 驗證  $\dot{V}$  恆為非正 =====
print("\n=== 驗證  $\dot{V} \leq 0$  在整個狀態空間成立 ===")
worst = 0.0
for th in np.linspace(-2*np.pi, 2*np.pi, 200):
    for thd in np.linspace(-10, 10, 200):
        worst = max(worst, lyapunov_dot(np.array([th, thd])))
print(f" $\dot{V}$  在取樣範圍內的最大值 = {worst:.2e} "
      f"(應  $\leq 0$ ，僅在  $\theta' = 0$  時等於 0)")

# ===== LQR 的價值函數即 Lyapunov 函數 =====
print("\n=== 19.8.2 節驗證：LQR 的  $V = x^T P x$  為 Lyapunov 函數 ===")
A2 = np.array([[0., 1.], [0., 0.]])
B2 = np.array([[0.], [0.5]])
Q, R = np.diag([1., 0.]), np.array([[1.]])
Pm = solve_continuous_are(A2, B2, Q, R)
K2 = np.linalg.solve(R, B2.T @ Pm)
Acl = A2 - B2 @ K2
print(f"P = \n{np.round(Pm, 4)}")
print(f"P 是否正定: {np.all(np.linalg.eigvals(Pm) > 0)}")
# 理論： $\dot{V} = -x^T(Q + K^T R K)x$ 
S_theory = Q + K2.T @ R @ K2
S_actual = -(Acl.T @ Pm + Pm @ Acl) # 由  $\dot{V} = x^T(Acl^T P + P Acl)x$ 
print(f"Q + K^T R K = \n{np.round(S_theory, 4)}")
print(f"-(Acl^T P + P Acl) = \n{np.round(S_actual, 4)}")
print(f"兩者相等: {np.allclose(S_theory, S_actual)} "
      f"→ 驗證  $\dot{V} = -x^T(Q + K^T R K)x$ ")

```

請試著修改：（一）把  $K_d$  設為 0（去掉阻尼），觀察  $V$  是否停止下降——這會重現第 16.3.2 節的永久振盪。（二）把單擺的初始角度設為 720 度（轉了兩圈），驗證全域穩定性確實成立。（三）把可控性檢查用在第 18 章的倒立擺  $A$ 、 $B$  矩陣上，驗證它是完全可控的——這正是 LQR 能穩定它的前提。

## 實作：穩定性的實驗驗證

本章實作以模擬為主，目標是「親眼看見」抽象的穩定性概念。

- 1. 可控性檢查工具：**為你的系統（單關節、平面 2R、或倒立擺）建立狀態空間模型，實作可控性與可觀測性矩陣的計算，並判斷其秩。試著故意「弄壞」系統（如把  $B$  的某一列設為零），觀察秩如何下降。
- 2. 穩定性等級的視覺化：**分別模擬三個系統——無阻尼單擺（Lyapunov 穩定）、有阻尼單擺（漸近穩定）、線性 PD 閉環（指數穩定）。在同一張圖上繪出三者的狀態隨時間變化，直觀比較三種穩定性的差異。

3. **Lyapunov 函數的數值驗證**：對第 17 章的 PD 加重力補償控制器，計算並繪出  $V(t)$  與  $\dot{V}(t)$  的曲線。驗證  $V$  單調下降、 $\dot{V}$  恆為非正、且  $\dot{V}$  只在速度為零的瞬間觸及零。
4. **相平面軌跡與能量等高線**：在相平面（ $e$  對  $\dot{e}$ ）上繪出 Lyapunov 函數的等高線（同心橢圓），再疊上系統的實際軌跡。你會看到軌跡持續穿越等高線往內收斂——這是本章最直觀的視覺化。
5. **吸引域的探索**：對倒立擺的 LQR 控制器，在「初始角度  $\times$  初始角速度」平面上大量取樣，標記哪些初始條件能成功穩定。繪出吸引域的邊界，並比較不同  $Q$ 、 $R$  設定下吸引域的大小。
6. **取樣週期的影響**：把連續時間的控制器離散化，逐步加大取樣週期（1 ms、5 ms、20 ms、50 ms.....），記錄系統從穩定變成不穩定的臨界週期。與 19.9.3 節的經驗法則比較。
7. **模型誤差下的穩定性**：故意讓控制器的重力模型有 20%、50%、100% 的誤差，觀察穩態誤差如何增大、以及系統是否仍然穩定（應該仍然穩定，只是有穩態誤差）。這驗證了 19.8.4 節的 ISS 性質。
8. **斜對稱性的數值驗證**：對平面 2R 手臂，隨機取樣多組  $(q, \dot{q})$ ，數值計算  $\dot{M}$  與  $C$ ，驗證  $x^T(\dot{M} - 2C)x \approx 0$  對任意向量  $x$  成立。這是第 13 章性質的直接驗證。

表 19-5 穩定性驗證記錄表（第一列為範例）

| 測試項         | 系統 / 設定          | 理論預期                                | 模擬結果                   | 判定 |
|-------------|------------------|-------------------------------------|------------------------|----|
| Lyapunov 遞減 | PD+重力補償, $K_d=4$ | $\dot{V} = -4\dot{\theta}^2 \leq 0$ | 最大 $\dot{V} = -1.2e-9$ | 通過 |
| （自行填寫其餘測試）  |                  |                                     |                        |    |

## 國中小也看得懂：彈珠一定會滾到碗底

想像你把一顆彈珠放進一個碗裡。不管你從碗的哪一邊放進去，彈珠最後都會滾到碗底停下來，對不對？

為什麼你這麼確定？你並沒有算出彈珠會走什麼路線、轉幾圈、花多久時間——你只知道一件事：彈珠會一直往低的地方滾。只要它一直在下降，它就一定會到最低的地方。

這一章教的就是這個道理，只是用在機器人上。工程師想知道「我的機器人會不會失控」，但如果要算出它每一秒鐘的位置，會複雜到算不出來。所以他們改用彈珠的方法：

**第一步：**找一個「高度」——通常是機器人的**能量**（動得越快、離目標越遠，能量就越高）。

**第二步：**證明這個能量**一直在減少**。

**第三步：**既然能量一直減少，機器人最後一定會停在「能量最低」的地方——也就是我們要它去的位置！

這個方法是一百三十年前一位俄國數學家想出來的，聰明的地方在於：**不用知道路線，只要知道方向是往下的就夠了**。

還有一件重要的事：這一章還教我們問一個更基本的問題——「這台機器**有沒有可能**被控制？」想想看，如果一台遙控車的兩個輪子被黏在一起，不管你的遙控器多厲害，它都轉不了彎。這不是遙控器的問題，是車子本身的問題。所以聰明的工程師會先檢查：**我想去的每個地方，這台機器都有辦法去嗎？** 確定可以，才開始寫程式。

## 叫獸提醒與常見錯誤

- 誤區一：跳過可控性檢查就開始設計控制器。不可控是系統的先天缺陷，任何演算法都無法彌補，應在設計前先確認。
- 誤區二：混淆連續與離散系統的穩定區域。連續系統看左半平面（實部為負），離散系統看單位圓內（絕對值小於 1）。
- 誤區三：以為 Lyapunov 穩定就代表會收斂。它只保證「不會跑遠」，收斂需要漸近穩定，速度保證則需要指數穩定。
- 誤區四：計算  $V$  時忘記代入系統方程。 $\dot{V} = (\partial V / \partial x)^T \cdot f(x)$  中的  $f(x)$  必須是實際的系統動態，否則計算毫無意義。
- 誤區五： $V$  只是半負定就宣稱漸近穩定。需要 LaSalle 不變性原理才能完成證明。
- 誤區六：以為線性化的結果就是全部。它只給出局部結論，且當特徵值實部為零時完全無法判定（如例題 19-2）。
- 誤區七：忽略取樣週期對穩定性的影響。取樣太慢等效於引入延遲，會使原本穩定的系統變得不穩定。
- 誤區八：以為自適應控制的參數一定會收斂到真值。只有在持續激勵條件滿足時才會，否則

只保證追隨誤差收斂。

- 誤區九：把奇異點與不可控混為一談。前者是瞬時的位形問題（離開就恢復），後者是結構性的先天缺陷。
- 誤區十：只追求漸近穩定而忽略收斂速率。實務上往往需要指數穩定的速率保證，這是 CTC 優於 PD 加重力補償之處。
- 誤區十一：以為有模型誤差就不穩定。多數控制器具備 ISS 性質——有界的模型誤差只造成有界的追隨誤差，系統仍然穩定。

## 課堂討論

1. 開頭故事中叫獸說「工程師和玩家的差別，在於能不能證明」。請討論：在什麼情況下，模擬測試不足以取代理論證明？請舉出兩個機器人上的具體情境。
2. 19.7 節的證明完全不需要知道  $M(q)$  與  $C(q, \dot{q})$  的具體形式，只用到「 $M$  正定」與「 $\dot{M}-2C$  斜對稱」兩個結構性質。請討論這種「只靠結構性質的證明」有什麼工程價值？
3. 第 13 章的  $\dot{M} - 2C$  斜對稱性在當時看似無用，兩百頁後成為證明的關鍵。請回顧本書，找出另一個類似的「伏筆」——某個當時看似次要、後來卻成為關鍵的性質。
4. LQR 的價值函數恰好就是 Lyapunov 函數，這使得「最佳」與「穩定」成為同一件事的兩面。請討論這個巧合的深層原因（提示：兩者都與「能量 / 代價的遞減」有關）。
5. 例題 19-2 的系統漸近穩定但非指數穩定，收斂速度為  $t^{-1/2}$ 。請討論：在機器人控制中，這種「會到但很慢」的收斂在什麼場合可以接受？什麼場合不行？

## 章末摘要

- 可控性回答「系統能否被帶到任意狀態」；可控性矩陣  $C = [B, AB, \dots, A^{n-1}B]$  滿秩即為完全可控。
- 不可控是結構性的先天缺陷，任何控制器都無法彌補；這與第 12 章的奇異點（瞬時位形問題）本質不同。
- 可觀測性回答「能否由輸出推斷完整狀態」，與可控性形成對偶： $(A, B)$  可控  $\Leftrightarrow (A^T, B^T)$  可觀測。
- 穩定性有三個層次：Lyapunov 穩定（有界）、漸近穩定（收斂）、指數穩定（有收斂速率）。

保證)。

- 線性系統的判準：連續系統看特徵值實部是否為負，離散系統看特徵值絕對值是否小於 1。
- 若系統完全可控，則可用極點配置把閉環特徵值放到任意位置；LQR 則以權重間接決定極點且保證裕度。
- Lyapunov 直接法：若能找到正定的  $V$  且  $\dot{V} \leq 0$ ，則系統穩定；不需解微分方程，適用於非線性系統。
- 計算  $\dot{V}$  時必須代入系統方程，這是把系統動態「灌入」證明的關鍵步驟。
- $V$  只是半負定時，需用 LaSalle 不變性原理：軌跡收斂到  $\{V = 0\}$  中最大的不變集。
- 機械系統的能量是天然的 Lyapunov 候選函數：正定、在平衡點最小、且有耗散時遞減。
- PD 加重力補償的穩定性證明：取  $V = \frac{1}{2} \dot{q}^T M \dot{q} + \frac{1}{2} e^T K_p e$ ，計算  $\dot{V}$  時  $\dot{M} - 2C$  的斜對稱性使該項消失，得  $\dot{V} = -\dot{q}^T K_d \dot{q} \leq 0$ ，再由 LaSalle 完成證明。
- 此證明只用到「 $M$  正定」與「 $\dot{M} - 2C$  斜對稱」兩個結構性質，完全不需要參數的具體數值——這正是定理強大之處。
- 計算力矩控制的閉環為線性，可直接用特徵值判準證明指數穩定；LQR 的價值函數  $V = x^T P x$  本身即為 Lyapunov 函數，且  $\dot{V} = -x^T (Q + K^T R K) x$ 。
- 取樣週期過長等效於引入延遲，會侵蝕穩定裕度；建議取樣頻率為閉環頻寬的 20~30 倍。
- 自適應控制保證追隨誤差收斂，但參數估計的收斂需要額外的持續激勵條件。
- 模型誤差下多數控制器具 ISS 性質：有界的誤差只造成有界的偏差，系統仍然穩定。

## 觀念題與計算題

### 觀念題

1. 定義可控性，並說明為何應在設計控制器之前先檢查它。
2. 寫出可控性矩陣的形式，並解釋  $B$ 、 $AB$ 、 $A^2B$  各代表什麼「方向」。
3. 為何可控性矩陣只需算到  $A^{n-1}B$ ？
4. 舉出兩個不可控系統的例子，並說明其物理成因。
5. 比較不可控性與第 12 章的奇異點，說明兩者的本質差異。
6. 說明可觀測性的意義，以及它與可控性的對偶關係。



7. 區分 Lyapunov 穩定、漸近穩定與指數穩定，並各舉一例。
8. 說明連續與離散系統穩定性判準的差異，並解釋兩者為何以  $\lambda_d = e^{(\lambda_c T)}$  對應。
9. 敘述 Lyapunov 直接法的三個條件，並說明各自的物理意義。
10. 為何計算  $V$  時必須代入系統方程？若不代入會發生什麼？
11. 敘述 LaSalle 不變性原理，並說明它為何在機械系統的證明中幾乎必定會用到。
12. 為何能量是機械系統天然的 Lyapunov 候選函數？請由三個性質說明。
13. 說明  $\dot{M} - 2C$  的斜對稱性在 19.7 節證明中扮演的角色，以及其物理意義。
14. 為何說「LQR 的價值函數就是 Lyapunov 函數」？ $V$  的表達式有何直觀意義？
15. 說明取樣週期對穩定性的影響機制，並給出實務上的選擇準則。
16. 什麼是輸入—狀態穩定 (ISS)？為何說它比純粹的漸近穩定更符合實務需求？

## 計算題

1. 某系統  $A = \begin{bmatrix} 0 & 1 \\ -4 & -5 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$ 、 $C = [1, 0]$ 。(a) 計算可控性矩陣並判斷可控性。(b) 計算可觀測性矩陣並判斷可觀測性。(c) 計算開迴路特徵值並判斷穩定性。(d) 若要把閉環極點配置到  $-4$  與  $-6$ ，求所需的  $K$  (提示：比較特徵多項式係數)。
2. 考慮非線性系統  $\dot{x} = -x + x^3$ 。(a) 求所有平衡點。(b) 在  $x = 0$  處線性化並判斷局部穩定性。(c) 取  $V = \frac{1}{2}x^2$  計算  $\dot{V}$ ，並找出  $\dot{V} < 0$  成立的  $x$  範圍。(d) 由此估計原點的吸引域，並說明為何此系統不是全域穩定。
3. 某單關節手臂  $I = 0.5 \text{ kg}\cdot\text{m}^2$ 、重力力矩為  $mg(L/2)\sin\theta$  且  $mg(L/2) = 8 \text{ N}\cdot\text{m}$ 。採用  $\tau = K_p(\theta_d - \theta) - K_d \dot{\theta} + 8\sin\theta$ ，取  $K_p = 30$ 、 $K_d = 6$ 。(a) 寫出閉環方程。(b) 構造 Lyapunov 函數  $V$  並寫出其表達式。(c) 計算  $\dot{V}$  並驗證其為半負定。(d) 計算閉環系統的特徵值，判斷是欠阻尼、臨界阻尼還是過阻尼。
4. 某控制系統的閉環頻寬設計為  $\omega_n = 30 \text{ rad/s}$ 。(a) 換算為赫茲。(b) 依 19.9.3 節的建議，求最低與建議的取樣頻率。(c) 若實際使用  $100 \text{ Hz}$  取樣，判斷是否足夠。(d) 若系統的機構共振頻率為  $80 \text{ Hz}$ ，討論取樣頻率是否還需提高，並說明理由。

## 動手做與專題延伸

### 本章實作：穩定性分析工具箱



延續前面章節的程式庫，加入穩定性分析模組。這將是驗證所有控制器設計的標準工具。

- **階段一**：實作可控性與可觀測性矩陣的計算與秩判定，並整合為「系統體檢報告」函式。
- **階段二**：實作極點配置（可用 `scipy.signal.place_poles`），並與 LQR 的結果比較。
- **階段三**：實作 Lyapunov 函數的數值驗證工具——給定  $V$  的表達式與系統動態，在狀態空間中大量取樣，檢查  $V > 0$  與  $V \leq 0$  是否處處成立。
- **階段四**：實作吸引域的估計：對非線性系統大量取樣初始條件，判定哪些能收斂，繪出吸引域的邊界。
- **階段五**：實作離散化與臨界取樣週期的自動搜尋，驗證 19.9.3 節的經驗法則。

### 專題延伸

- **相平面與能量等高線的視覺化**：對第 17 章的 PD 加重力補償控制器（單關節版本），在相平面上繪製 Lyapunov 函數的等高線（同心橢圓），並疊上多條由不同初始條件出發的軌跡。你會看到所有軌跡都持續往內穿越等高線、螺旋收斂到原點。這張圖是本章所有理論最直觀的呈現，建議做成動畫。
- **吸引域的比較研究**：對倒立擺，分別用（甲）LQR、（乙）極點配置（極點放得較遠）、（丙）極點配置（極點放得較近）三種方法設計控制器，各自繪出吸引域。比較「控制越積極，吸引域是否越大」，以及代價（最大控制量）如何變化。這個實驗能量化「性能與強健性」的取捨。
- **進階：斜對稱性的完整驗證與其重要性**。對平面 2R 手臂，用符號運算（SymPy）推導  $M(q)$  與採 Christoffel 符號定義的  $C(q, \dot{q})$ ，計算  $\dot{M} - 2C$  並驗證其斜對稱性。接著做一個對照實驗：故意使用一個「錯誤的」 $C$  矩陣分解（仍滿足  $C \cdot \dot{q}$  正確，但不具斜對稱性），重新執行 19.7 節的證明步驟，觀察  $V$  中會殘留什麼項、以及這是否會破壞穩定性結論。這個實驗能讓你深刻理解為何  $C$  的「特定分解方式」如此重要。

### 參考文獻與延伸閱讀

- Khalil, H. K., Nonlinear Systems, 3rd ed., Prentice Hall（非線性系統與 Lyapunov 理論的權威教材）。
- Slotine, J.-J. E. and Li, W., Applied Nonlinear Control, Prentice Hall（工程導向的非線性控制，含機器人應用）。

- Takegaki, M. and Arimoto, S., "A New Feedback Method for Dynamic Control of Manipulators," ASME J. Dynamic Systems, 1981 ( PD 加重力補償的原始證明 ) 。
- Kalman, R. E., "On the General Theory of Control Systems," IFAC Congress, 1960 ( 可控性與可觀測性概念的提出 ) 。
- Lyapunov, A. M., The General Problem of the Stability of Motion, 1892 ( 中譯與英譯本；穩定性理論的原始著作 ) 。
- LaSalle, J. P., "Some Extensions of Liapunov's Second Method," IRE Trans. Circuit Theory, 1960 ( 不變性原理 ) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley ( 第 6、8 章：穩定性與非線性控制 ) 。
- Ortega, R. and Spong, M. W., "Adaptive Motion Control of Rigid Robots: A Tutorial," Automatica, 1989 ( 機器人自適應控制的穩定性分析 ) 。
- Sontag, E. D., "Smooth Stabilization Implies Coprime Factorization," IEEE Trans. Automatic Control, 1989 ( 輸入—狀態穩定 ISS 概念的提出 ) 。
- Tedrake, R., Underactuated Robotics, MIT 課程講義 ( 線上公開 ) ( 含 Lyapunov 方法在欠驅動機器人上的大量應用與吸引域分析 ) 。

## 第 20 章

# 力控制、阻抗控制與柔順操作

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照照片：上格是夾爪以位置控制夾生雞蛋、蛋殼碎裂蛋液流出，旁註「位置誤差  $0.5\text{ mm} \rightarrow$  接觸力  $1500\text{ N}$ 」；下格是同一夾爪以阻抗控制輕握完好的雞蛋，旁註「 $K_d = 500\text{ N/m} \rightarrow$  接觸力  $0.25\text{ N}$ 」。右半為機器人末端與虛擬平衡點之間以三根粗細不同的彈簧與阻尼器相連的示意圖，下方橫軸為剛度的連續頻譜，左端標「純力控制」、右端標「純位置控制」，中間標出協作、裝配、研磨等應用的位置。標題文字：不控制位置，也不控制力——控制兩者之間的關係。

## 叫獸開講 「拿雞蛋不能只知道位置，還必須知道用了多少力。」

叫獸在桌上放了一顆生雞蛋，然後請小翰用機器手臂把它夾起來。

小翰很有信心。他量好了雞蛋的寬度是 44 公釐，於是把夾爪的目標位置設為 43.5 公釐——留 0.5 公釐的「夾持餘量」。按下執行，夾爪緩緩合攏——「啪」的一聲，蛋黃流了一桌。

「再給你一次機會。」叫獸換上第二顆蛋。小翰改設 44.2 公釐，這次夾爪停在蛋殼外側，完全沒碰到；他把手臂抬起來，蛋留在桌上。

「所以，」叫獸說，「44.2 太鬆、43.5 太緊。你要不要試試 43.9？」

小翰試了。這次蛋沒破，但也夾不穩——抬到一半就滑了下來。他開始試 43.8、43.85、43.87.....

「停。」叫獸打斷他，「你發現問題了嗎？你在用位置去控制一件本質上是力的事情。」

他在白板上寫了一個計算：「雞蛋殼的剛度大約是每公釐 3000 牛頓。如果你的位置指令比實際接觸點深了 0.5 公釐，接觸力會是多少？」

「3000 乘以 0.5.....1500 牛頓？」小翰倒抽一口氣，「那不是可以壓碎石頭了嗎。」

「這就是為什麼蛋會破。」叫獸點頭，「而且更糟的是——你的位置控制器不知道自己壓爆了蛋。它只看到『還差 0.5 公釐沒到位』，於是繼續加大力矩。第 17 章我們花了整章讓它精確追隨位置，現在這個優點反而變成致命傷。」

小紓舉手：「那改成控制力就好了？」

「一半對。」叫獸把雞蛋放回桌上，「但想想：在夾爪還沒碰到蛋之前，你要控制什麼力？此時無論怎麼下指令，力都是零——控制器會發瘋似地往前衝，直到撞上為止。純力控制在自由空間中是不穩定的。」

他停頓了一下，「所以真正的答案不是『控制位置』也不是『控制力』，而是控制兩者之間的關係。」

叫獸伸出手，輕輕握住那顆蛋，然後說：「你看我的手。我沒有『把手指移到 43.9 公釐』，也沒有『輸出 2 牛頓的力』。我做的是——讓我的手像一個軟硬適中的彈簧。碰到硬的東西就退讓，碰到軟的東西就順從。這個『像什麼樣的彈簧』，就是今天要學的**阻抗**。」

## 本章為什麼重要

第三篇前七章的所有控制方法，都建立在一個隱含假設上：**機器人在自由空間中運動，不與環境接觸**。這個假設下，位置控制越精確越好，增益越高越理想。

但機器人的價值恰恰在於接觸。抓取、裝配、研磨、拋光、擦拭、與人握手——這些任務中，機器人必須與環境交換力。而一旦接觸發生，前面的邏輯就完全反轉：**高增益的精確位置控制，反而成為最危險的設定**。原因如開頭故事所示——環境的剛度極高，微小的位置誤差會產生巨大的接觸力。

本章要建立三個層次的能力。**第一，直接力控制**：讓機器人輸出指定的接觸力。**第二，混合位置 / 力控制**：在不同的方向分別控制運動與力（例如在垂直方向壓住、在水平方向移動）。**第三，阻抗控制**：不直接控制位置或力，而是控制兩者的**動態關係**——讓機器人表現得像一個程式的彈簧與阻尼器。這是最優雅也最實用的方法，是協作機器人、力覺裝配與人機互動的共同基礎。

本章也是第 11 章對偶性、第 17 章操作空間控制、第 19 章穩定性理論的共同匯流處——這三個工具將在此完全發揮作用。

## 學習目標

- 能說明位置控制在接觸任務中失效的原因，並量化其接觸力。
- 能說明純力控制在自由空間中不穩定的原因。
- 能區分力控制、位置控制與阻抗控制三者的本質差異。
- 能說明力 / 力矩感測的三種方式及其優缺點。
- 能設計直接力控制器並說明其積分項的必要性。
- 能運用自然約束與人為約束分析任務，建構選擇矩陣。
- 能設計混合位置 / 力控制器並說明其在操作空間中的實作。
- 能推導阻抗控制律，並說明質量、阻尼、剛度三個參數的物理意義。
- 能區分阻抗控制與導納控制，並依系統條件選擇適當者。
- 能說明接觸穩定性問題與剛度上限的成因。

- 能說明主動柔順與被動柔順的分工，並舉例說明 RCC 等機構。
- 能設計精密插銷入孔的柔順策略並分析其成功條件。

## 先備知識

第 11 章的雅可比與速度—力對偶 ( $\tau = J^T F$  是本章的基礎)、第 17 章的操作空間控制與  $\Lambda$  矩陣、第 13 章的動力學模型、第 19 章的穩定性概念、第 6.4 節的力覺感測器。

本章的數學不比前幾章困難，但**概念的轉換較大**——必須從「控制位置」的思維切換到「控制關係」。建議讀法：務必先徹底理解 20.1 節為何位置控制會失效，這是後續所有方法的動機來源。

## 核心名詞中英對照

表 20-1 本章核心名詞

| 中文     | 英文                               | 說明                |
|--------|----------------------------------|-------------------|
| 力控制    | Force Control                    | 控制機器人與環境間接觸力的方法   |
| 柔順性    | Compliance                       | 受力時產生位移的性質，剛度的倒數  |
| 剛度     | Stiffness                        | 單位位移所產生的力         |
| 阻抗     | Impedance                        | 描述力與運動之間動態關係的算子   |
| 導納     | Admittance                       | 阻抗的反向關係：由力決定運動    |
| 環境剛度   | Environment Stiffness            | 接觸物體的剛度特性         |
| 自然約束   | Natural Constraint               | 由環境幾何決定、無法違背的限制   |
| 人為約束   | Artificial Constraint            | 由任務指定、可自由選擇的目標    |
| 選擇矩陣   | Selection Matrix                 | 指定各方向由位置或力控制的對角矩陣 |
| 混合控制   | Hybrid Control                   | 不同方向分別做位置與力控制的架構  |
| 主動柔順   | Active Compliance                | 以控制軟體實現的柔順性       |
| 被動柔順   | Passive Compliance               | 以機構彈性實現的柔順性       |
| 遠端順應中心 | Remote Center of Compliance, RCC | 使旋轉中心落在工件尖端的順應機構  |
| 接觸不穩定  | Contact Instability              | 接觸時產生高頻振盪的現象      |
| 彈跳     | Bouncing                         | 接觸與脫離反覆交替的現象      |
| 插銷入孔   | Peg-in-Hole                      | 精密裝配的經典測試任務       |
| 倒角     | Chamfer                          | 孔口的斜面，用於引導插入      |

## 20.1 為何位置控制在接觸中失效

### 20.1.1 剛度的量級問題

開頭故事揭示的問題，本質是一個量級問題。考慮機器人末端與環境接觸時的力：

$$F = K_{\text{env}} \cdot \Delta x$$

接觸力等於環境剛度乘上「侵入深度」。Δx 是位置指令超過實際接觸面的量。

關鍵在於  $K_{\text{env}}$  的量級。下表列出常見環境的剛度：

表 20-2 常見環境的剛度量級

| 環境    | 典型剛度                  | 產生 10 N 力所需的侵入深度          | 產生 100 N 所需深度             |
|-------|-----------------------|---------------------------|---------------------------|
| 海綿、軟墊 | 1 ~ 10 N/mm           | 1 ~ 10 mm                 | 10 ~ 100 mm               |
| 人體軟組織 | 5 ~ 50 N/mm           | 0.2 ~ 2 mm                | 2 ~ 20 mm                 |
| 雞蛋殼   | ~ 3000 N/mm           | 0.003 mm                  | 0.03 mm                   |
| 塑膠工件  | $10^3 \sim 10^4$ N/mm | 0.001 ~ 0.01 mm           | 0.01 ~ 0.1 mm             |
| 金屬工件  | $10^5 \sim 10^6$ N/mm | $10^{-5} \sim 10^{-4}$ mm | $10^{-4} \sim 10^{-3}$ mm |

這張表說明了一切。對金屬工件而言，只要位置誤差達到千分之一公釐，接觸力就可能達到數百牛頓。而第 9.9.2 節告訴我們，機器人的絕對精度只有  $\pm 0.5$  到  $\pm 2$  公釐——比所需的精度差了三個數量級。

換句話說：用位置控制去接觸剛性環境，在物理上是不可能成功的。這不是控制器調得不夠好，而是問題的本質。

### 20.1.2 控制器的「盲目」

更危險的是控制器的反應。位置控制器只看位置誤差，當末端被工件擋住而無法到達目標時，它會判定「還差得遠」，於是持續加大力矩——這正是第 16 章積分飽和的物理版本。

結果有三種：輕則工件變形、中則工件或工具損壞、重則機器人本體受損或觸發過載保護緊急停機。開頭故事中的雞蛋只是最溫和的示範。

### 20.1.3 那麼純力控制呢？

既然位置控制不行，是否應該改用純力控制？答案是「不完全」。

在**接觸狀態**下，力控制確實有效：直接指定期望的接觸力，控制器據此調整力矩。但在**自由空間**中，力控制會出問題：

此時無論機器人怎麼移動，測到的力都是零。若期望力為 5 N，誤差恆為 5 N，控制器會持續加速前進，直到撞上東西為止——而撞上的瞬間，累積的動能會產生巨大的衝擊力。

**自由空間中：  $F_{\text{measured}} = 0$ ， $F_{\text{error}} = F_d - 0 = F_d \neq 0 \Rightarrow$  控制器持續加速**

*純力控制在自由空間中不穩定。這是它無法單獨使用的根本原因。*

因此實務上必須有一套機制，處理「接觸」與「未接觸」兩種狀態的切換——這正是 20.4 節混合控制與 20.5 節阻抗控制要解決的問題。

### ◎ 一個核心的觀念轉換

位置控制與力控制看似對立，其實是**同一個問題的兩個極端**。位置控制是「無限剛硬」——無論受多大的力都不退讓；力控制是「無限柔軟」——無論位置偏多少都只維持指定的力。而真實世界的互動需要的是**中間值**：像人手一樣，既能施力又能退讓。這個「中間值」就是**阻抗**，也是本章的核心概念。理解這一點，後面所有的方法都會變得自然。

## 20.2 力與力矩的感測

### 20.2.1 三種感測方式

要控制力，首先要能量測力。第 6.4 節已介紹過原理，此處從控制的角度比較三種方式。

**方式一：六軸力 / 力矩感測器**。安裝在末端法蘭與工具之間，直接量測三個方向的力與三個方向的力矩。優點是精度高（解析度可達 0.01 N）、頻寬高（可達 1 kHz 以上）、且直接量測到作用點附近的力。缺點是價格昂貴（數萬至數十萬元）、增加末端重量與長度、且本身有柔性會降低系統剛度。

**方式二：關節力矩感測器**。在每個關節內部安裝扭矩感測器。優點是能偵測作用在手臂**任何位置**的碰撞（不只末端）、不影響末端剛性、且可用於全身的碰撞偵測。缺點是成本更高（每個關節都要一個）、且要由關節力矩反推末端力需要經過雅可比（在奇異點附近精度會劣化）。這是高階協作機器人的標準配備。

**方式三：馬達電流估測**。如第 11.6.3 節所述，用  $F = (J^T)^+ \cdot \tau_{\text{ext}}$  由電流反推外力。優點是零



額外成本。缺點是精度受摩擦不確定性限制（第 13.8.3 節），且高減速比會稀釋外力訊號。

表 20-3 三種力感測方式的比較

| 方式          | 精度             | 成本 | 偵測範圍 | 對剛性影響 | 典型應用       |
|-------------|----------------|----|------|-------|------------|
| 六軸力 / 力矩感測器 | 高 ( 0.01 N )   | 高  | 僅末端  | 略降低   | 精密裝配、力控研磨  |
| 關節力矩感測器     | 中高 ( 0.1 N·m ) | 極高 | 全身   | 無     | 協作機器人、人機互動 |
| 馬達電流估測      | 低 ( 1 ~ 5 N )  | 零  | 全身   | 無     | 碰撞偵測、手動導引  |

## 20.2.2 力訊號的處理

力訊號的處理有三個實務要點，處理不當會直接導致控制失敗。

**要點一：重力與慣性補償。**感測器量到的是「工具重量 + 慣性力 + 真實外力」的總和。必須先扣除前兩者：

$$F_{\text{external}} = F_{\text{measured}} - F_{\text{gravity}}(q) - F_{\text{inertia}}(q, \dot{q}, \ddot{q})$$

工具的重力隨姿態變化、慣性力隨加速度變化，兩者都必須即時補償。這正是第 13、14 章動力學模型的直接應用。

**要點二：濾波與延遲的取舍。**力訊號雜訊大（尤其是電流估測法），需要濾波；但濾波會引入延遲，而延遲會嚴重損害接觸穩定性（20.7 節）。實務上濾波器的截止頻率通常設在 50 ~ 200 Hz 之間。

**要點三：座標系的一致性。**感測器輸出的是在感測器自身座標系中的力。要用於控制，必須轉換到任務座標系——這需要第 8 章的座標轉換與第 11.7 節的伴隨變換（因為力矩會隨參考點改變）。

## 20.3 直接力控制

### 20.3.1 最基本的形式

直接力控制的目標是讓接觸力追隨指定值  $F_d$ 。最直觀的作法是模仿位置控制的形式：

$$\tau = J^T [ F_d + K_f (F_d - F_{\text{measured}}) ]$$

直接力控制。第一項是前饋（直接輸出期望的力），第二項是回授修正。

注意  $J^T$  的出現——這正是第 11.6 節速度—力對偶的直接應用：把任務空間中期望的力，映

射為關節空間的力矩。

### 20.3.2 為何需要積分項

純比例的力控制有一個穩態誤差問題。考慮環境為彈簧（剛度  $K_{env}$ ），機器人的位置為  $x$ 、環境表面在  $x_e$ ，則接觸力為  $F = K_{env}(x - x_e)$ 。

在穩態下，控制器輸出的力必須恰好等於接觸力，此時若只有比例項，會有殘餘誤差。加入積分項可以消除：

$$\tau = J^T [F_d + K_f e_F + K_i \int e_F dt], \quad e_F = F_d - F$$

含積分的力控制器。積分項對力控制而言幾乎是**必需的**，這與位置控制中積分項只是「可選」不同。

原因在於：位置控制中，重力等常值干擾可以用模型補償（第 17 章的重力補償）；但力控制中，環境剛度  $K_{env}$  通常**未知且會變化**（不同工件、不同接觸點），無法用模型精確補償，只能靠積分項自動適應。

### 20.3.3 直接力控制的限制

直接力控制有三個限制，使它很少單獨使用。

**限制一：無法處理自由空間**（20.1.3 節）。必須有額外的邏輯偵測接觸狀態並切換控制模式，而切換的瞬間容易產生衝擊。

**限制二：無法控制切線方向的運動**。研磨作業需要「垂直壓住、水平移動」，純力控制只能管前者。

**限制三：對環境剛度敏感**。同一組增益在軟環境（如海綿）與硬環境（如金屬）上的表現差異極大，前者可能反應遲鈍、後者可能振盪。

這三個限制，分別引出了接下來的三種方法：混合控制（解決限制二）、阻抗控制（解決限制一與三）、以及被動柔順（輔助解決限制三）。

## 20.4 混合位置 / 力控制

### 20.4.1 自然約束與人為約束

Mason 於 1981 年提出了一個關鍵的分析框架，用於決定「哪個方向該控位置、哪個方向該控力」。

**自然約束** ( natural constraint )：由環境的幾何形狀決定，機器人**無法違背**的限制。例如末端貼在平面上時，垂直於平面的**位置**被約束（不能穿透）；沿著平面滑動時，切線方向的**力**被約束（無摩擦時為零）。

**人為約束** ( artificial constraint )：由任務指定、設計者可以自由選擇的目標。在自然約束「不管」的方向上，我們才能下指令。

兩者有一個優美的互補關係：

在任一方向上：自然約束了位置  $\Rightarrow$  可人為指定力

自然約束了力  $\Rightarrow$  可人為指定位置

*互補性原理。同一個方向不可能同時控制位置與力——這在物理上是矛盾的。*

以「用板擦黑板」為例：垂直於黑板的方向，位置被自然約束（板擦不能陷進去），因此我們可以指定**壓力**；平行於黑板的方向，力被自然約束（沒有阻擋），因此我們可以指定**移動軌跡**。這正是人類擦黑板時直覺在做的事。

表 20-4 常見接觸任務的約束分析

| 任務        | 自然約束 ( 位置 ) | 自然約束 ( 力 ) | 人為指定 ( 力 ) | 人為指定 ( 位置 ) |
|-----------|-------------|------------|------------|-------------|
| 平面研磨 / 擦拭 | 垂直平面方向      | 平面內兩方向     | 垂直壓力       | 平面內軌跡       |
| 軸孔插入      | 徑向兩方向       | 軸向         | 徑向力為零      | 軸向深度        |
| 旋開螺絲      | 軸向與徑向       | 繞軸旋轉       | 軸向壓力       | 旋轉角度        |
| 沿邊緣描摹     | 垂直邊緣方向      | 沿邊緣方向      | 接觸壓力       | 沿邊緣位置       |
| 拋光曲面      | 曲面法向        | 曲面切向       | 法向壓力       | 曲面上的路徑      |

## 20.4.2 選擇矩陣

把上述分析形式化，引入一個對角的**選擇矩陣**  $S$ ：

$$S = \text{diag}(s_1, s_2, \dots, s_6), \quad s_i \in \{0, 1\}$$

$s_i = 1$  代表第  $i$  個方向做位置控制、 $s_i = 0$  代表做力控制。

控制律則把兩者結合：

$$F_{\text{command}} = S \cdot F_{\text{position\_control}} + (I - S) \cdot F_{\text{force\_control}}$$

*混合控制律。S 選出位置控制的方向、(I-S) 選出力控制的方向，兩者互補。*

以平面研磨為例（設  $Z$  為法向）： $S = \text{diag}(1, 1, 0, 1, 1, 1)$ —— $X$ 、 $Y$  方向與三個旋轉方向做位置控制，只有  $Z$  方向做力控制。

### 20.4.3 在操作空間中實作

完整的混合控制律建立在第 17.7 節的操作空間控制之上：

$$\tau = J^T [\Lambda(q) \cdot S \cdot u_x + (I - S) \cdot F_d + \mu + p]$$

*混合位置／力控制。位置控制的方向經過  $\Lambda$  做動態解耦、力控制的方向直接指定期望力。*

這裡可以清楚看見第 17 章操作空間控制的價值：它使各方向的動態完全解耦，因此才能在不同方向施加不同性質的控制而互不干擾。若在關節空間做混合控制，各方向會透過  $M(q)$  互相耦合，效果會大打折扣。

### 20.4.4 混合控制的困難

混合控制在理論上優雅，但實務上有三個困難。

**困難一：需要準確的環境幾何。**選擇矩陣的方向必須與實際的接觸面法向一致。若工件擺放有 5 度的偏差，控制方向就錯了 5 度，可能導致「本該控力的方向在控位置」——後果如 20.1 節所述。

**困難二：接觸狀態的切換。**從自由空間接近工件時是純位置控制，接觸後才切換為混合控制。切換的瞬間若處理不當會產生衝擊。

**困難三：無法處理不確定的接觸。**若工件位置有誤差、或接觸點在移動（如沿著曲面），選擇矩陣必須即時更新——這需要準確的即時感測。

這些困難促使了另一條路線的發展：不試圖區分方向，而是統一控制「力與位置的關係」——這就是阻抗控制。

## 20.5 阻抗控制

### 20.5.1 控制「關係」而非控制「量」

阻抗控制的核心思想，正是開頭故事中叫獸握住雞蛋時說的那句話：不是控制位置，也不是控制力，而是控制兩者之間的關係。

具體而言，我們希望機器人的末端表現得像一個質量—彈簧—阻尼系統：

$$M_d \ddot{\tilde{x}} + B_d \dot{\tilde{x}} + K_d \tilde{x} = F_{ext} \quad , \text{ 其中 } \tilde{x} = x - x_d$$

目標阻抗關係。 $x_d$  是「虛擬平衡位置」、 $F_{ext}$  是外界施加的力。三個參數分別為期望的慣量、阻尼與剛度。

這條式子的意義極為直觀：當外力推動末端時，末端會像一個彈簧一樣退讓。退讓多少由  $K_d$  決定、退讓的過程有多平順由  $B_d$  決定、對衝擊的反應有多鈍由  $M_d$  決定。

關鍵的洞見是：這一條式子同時涵蓋了位置控制與力控制兩個極端：

$K_d \rightarrow \infty$  : 完全不退讓  $\Rightarrow$  純位置控制

$K_d \rightarrow 0$  : 完全順從外力  $\Rightarrow$  純力控制 (重力補償模式)

阻抗控制是一個連續的頻譜，兩端分別是位置控制與力控制。

## 20.5.2 控制律的推導

如何讓機器人真的表現出上述行為？從第 17.7 節的操作空間控制出發。機器人在任務空間的實際動力學為：

$$\Lambda(q) \ddot{x} + \mu(q, \dot{q}) + p(q) = F_{control} + F_{ext}$$

任務空間的動力學。 $F_{control}$  是控制器施加的等效末端力、 $F_{ext}$  是外界的力。

我們希望它表現為目標阻抗 (設  $M_d = \Lambda$ ，即不改變真實慣量，這是最簡單也最常用的選擇)。比較兩式，可得所需的控制力：

$$F_{control} = \mu + p - B_d \dot{\tilde{x}} - K_d \tilde{x}$$

阻抗控制律。前兩項抵消機器人自身的非線性與重力，後兩項產生虛擬的彈簧與阻尼。

再由對偶性映射為關節力矩：

$$\tau = J^T [\mu + p - B_d(\dot{x} - \dot{x}_d) - K_d(x - x_d)]$$

阻抗控制的完整實作。注意這與第 17.7 節的操作空間控制形式幾乎相同——只是外層的控制律改為彈簧阻尼形式。

## 20.5.3 三個參數的物理意義與選擇

剛度  $K_d$ ：最重要的參數。它決定「被推開多少」：

$$\text{穩態時：} \tilde{x} = F_{ext} / K_d$$

剛度越大越硬。單位為  $N/m$  或  $N/rad$ 。

選擇的原則是與環境剛度匹配或更低。若  $K_d$  遠大於環境剛度，行為接近位置控制 (危險)；

若遠小於環境剛度，機器人幾乎不施力（無法完成任務）。實務上常取  $K_d$  在  $100 \sim 5000 \text{ N/m}$  之間——請注意這比 20.1 節表中的金屬工件剛度（ $10^8 \text{ N/m}$ ）小了四到五個數量級，這正是安全的來源。

**阻尼  $B_d$ ：**決定平順度。它抑制振盪，防止接觸時的彈跳。常用的選擇是讓系統維持臨界阻尼：

$$B_d = 2\zeta\sqrt{K_d \cdot M_d}, \quad \text{通常取 } \zeta = 0.7 \sim 1.0$$

由第 16 章的二階系統理論直接套用。

**慣量  $M_d$ ：**通常取為實際慣量。若刻意設定  $M_d \neq \Lambda$ ，控制器必須「主動抵消」真實慣量並「製造」虛擬慣量——這需要準確量測外力  $F_{\text{ext}}$ ，且對感測雜訊極為敏感。因此除非有特殊需求（如讓沉重的手臂「感覺起來很輕」以利手動導引），實務上多取  $M_d = \Lambda$ 。

表 20-5 阻抗參數的選擇指引

| 應用場景        | 剛度 $K_d$                         | 阻尼比 $\zeta$    | 說明          |
|-------------|----------------------------------|----------------|-------------|
| 自由空間精確定位    | 高（ $5000+ \text{ N/m}$ ）         | 1.0            | 接近位置控制      |
| 接觸剛性工件（裝配）  | 中低（ $100 \sim 500 \text{ N/m}$ ） | $0.7 \sim 1.0$ | 允許順應以吸收位置誤差 |
| 研磨、拋光       | 低（法向）/ 高（切向）                     | 1.0            | 各方向使用不同剛度   |
| 人機協作、手動導引   | 極低（ $0 \sim 50 \text{ N/m}$ ）    | 0.7            | 接近零力控制      |
| 接觸軟物（人體、食品） | 極低（ $10 \sim 100 \text{ N/m}$ ）  | 1.0            | 避免壓傷        |

## 20.5.4 阻抗控制的優勢

**優勢一：統一處理接觸與非接觸。**不需要偵測接觸狀態、不需要切換控制模式。在自由空間中， $F_{\text{ext}} = 0$ ，控制律退化為 PD 位置控制；接觸後  $F_{\text{ext}} \neq 0$ ，末端自然退讓。這徹底解決了 20.3.3 節與 20.4.4 節的切換問題。

**優勢二：本質安全。**剛度是設計出來的上限——無論位置誤差多大，接觸力最多是  $K_d$  乘上誤差。以  $K_d = 500 \text{ N/m}$ 、位置誤差  $5 \text{ mm}$  為例，最大接觸力只有  $2.5 \text{ N}$ 。這使機器人在碰到人時不會造成傷害，是協作機器人安全性的根本保證。

**優勢三：不需要力感測器（基本版本）。**20.5.2 節的控制律中並沒有出現  $F_{\text{ext}}$ ——控制器只需要位置與速度回授。這是因為「像彈簧一樣退讓」這個行為，本身就是由 PD 形式的控制律

自然產生的。（若要精確控制力或改變虛擬慣量，才需要力感測。）

**優勢四：各方向可獨立設定。** $K_d$  是一個矩陣，可以在不同方向設定不同的剛度——這自然涵蓋了混合控制的功能，且不需要硬性的 0/1 選擇矩陣。例如插銷入孔時，軸向設高剛度（推進）、徑向設低剛度（順應對位）。

圖 20-1 阻抗控制的彈簧模型

建議配圖：中央為機器人末端執行器，其與「虛擬平衡位置  $x_d$ 」之間以三個彈簧與阻尼器連接（分別代表  $X$ 、 $Y$ 、 $Z$  三個方向，且彈簧粗細不同以表示剛度不同）。左側標註「自由空間： $F_{ext} = 0$ ，末端停在  $x_d$ 」，右側標註「接觸： $F_{ext} \neq 0$ ，末端退讓  $\tilde{x} = F/K_d$ 」。下方以一條橫軸表示剛度的連續頻譜，左端標「 $K_d \rightarrow 0$ ：純力控制 / 零力模式」、右端標「 $K_d \rightarrow \infty$ ：純位置控制」，中間標出各種應用的典型位置。

## 20.6 阻抗控制與導納控制

### 20.6.1 兩種實作方式

實現「力與運動的目標關係」有兩種相反的實作路徑，容易混淆但差異重要。

**阻抗控制 (Impedance Control)：**「量測運動，輸出力」。控制器讀取位置與速度，依目標阻抗關係算出應施加的力，再透過力矩控制輸出。

$$\text{量測 } x, \dot{x} \Rightarrow \text{計算 } F = -K_d \tilde{x} - B_d \dot{\tilde{x}} \Rightarrow \text{輸出力矩 } \tau = J^T F$$

阻抗控制的資料流。內層是力矩控制。

**導納控制 (Admittance Control)：**「量測力，輸出運動」。控制器讀取外力，依目標關係算出末端「應該移動到哪裡」，再交給一個高剛度的位置控制器執行。

$$\text{量測 } F_{ext} \Rightarrow \text{積分目標阻抗方程求出 } x_{ref} \Rightarrow \text{交給位置控制器追隨}$$

導納控制的資料流。內層是位置控制。

### 20.6.2 該選哪一種

兩者的選擇取決於**機器人本身的特性與環境**，這是一個重要的工程判斷。

**阻抗控制適合：**本身可反向驅動的機器人（低減速比、低摩擦——第 5 章的準直驅設計）、以及**剛性環境**。因為它的內層是力矩控制，天生就「軟」，接觸硬物時能自然退讓。

**導納控制適合：**高減速比、高摩擦、難以做力矩控制的機器人（多數傳統工業機器人）、以及**柔軟環境**。它需要力感測器，但只要有了感測器，即使是很「硬」的機器人也能表現出柔順行

為。

一個有用的判準是：

環境硬 + 機器人軟（可反驅）→ 阻抗控制

環境軟 + 機器人硬（高減速比）→ 導納控制

*互補原則。若兩者都硬（硬機器人接觸硬環境），無論用哪種方法都很難穩定——這是接觸控制中最困難的情形。*

表 20-6 阻抗控制與導納控制的比較

| 面向      | 阻抗控制        | 導納控制         |
|---------|-------------|--------------|
| 資料流     | 量測運動 → 輸出力  | 量測力 → 輸出運動   |
| 內層控制    | 力矩 / 電流控制   | 位置控制         |
| 是否需力感測器 | 基本版不需要      | 必需           |
| 適合的機器人  | 可反向驅動（低減速比） | 高減速比、高摩擦     |
| 適合的環境   | 剛性環境        | 柔軟環境         |
| 自由空間的表現 | 良好（退化為 PD）  | 可能因雜訊而漂移     |
| 典型代表    | 協作機器人、腿式機器人 | 傳統工業機器人加力覺感測 |

### 20.6.3 摩擦的影響

阻抗控制有一個實務上的隱憂：**關節摩擦會破壞柔順性**。若減速機的摩擦力矩相當於末端 10 N 的力，那麼小於 10 N 的外力根本推不動機器人——無論你把  $K_d$  設得多低，機器人「感覺起來」都是硬的。

這正是第 5 章「協作機器人減速比不能太高」與第 11.6.3 節知識框的另一個理由。因應方式有三：使用低減速比的準直驅設計、加入摩擦補償（第 17.5.3 節）、或改用導納控制（因為力感測器直接量到外力，摩擦由內層的位置控制器克服）。

## 20.7 接觸穩定性

### 20.7.1 為何接觸時容易振盪

接觸控制有一個特有的穩定性問題：**機器人在自由空間中穩定，一接觸就開始高頻振盪**。這種現象稱為接觸不穩定（contact instability），嚴重時會產生刺耳的噪音、損傷工件、甚至損壞機構。



成因可以用第 19 章的觀點理解。接觸後，環境的剛度加入了閉環系統，使等效的閉環剛度大幅提高：

$$\text{接觸前的等效剛度} = K_d$$

$$\text{接觸後的等效剛度} = K_d + K_{env} \approx K_{env} \quad (\text{因 } K_{env} \gg K_d)$$

*接觸使系統的剛度暴增數個數量級，自然頻率隨之上升。*

而系統的自然頻率上升後，原本足夠的取樣頻率可能變得不足（第 19.9.3 節），且原本設計的阻尼比也會下降：

$$\omega_n = \sqrt{K/M} \text{ 上升} \Rightarrow \zeta = B/(2\sqrt{KM}) \text{ 下降}$$

*接觸後阻尼比下降，系統從穩定變成欠阻尼甚至不穩定。*

## 20.7.2 三個關鍵因素

**因素一：取樣延遲。**數位控制器的取樣週期造成的延遲，會侵蝕相位裕度。接觸時系統頻寬提高，同樣的延遲造成的相位滯後更嚴重。這是接觸不穩定最主要的原因。

**因素二：力感測器的濾波延遲。**為了降低雜訊而加的低通濾波器，其延遲直接進入控制迴路。這造成一個兩難：濾波太少則雜訊大、太多則不穩定。

**因素三：關節與結構的柔性。**減速機的扭轉柔度、連桿的彎曲柔度，在高剛度接觸時會被激發成振動模態。這是為何研磨、拋光等作業常有明顯的顫振（chatter）。

## 20.7.3 可實現的剛度上限

上述分析導出一個重要的結論：**控制器能實現的最大剛度是有上限的**，且這個上限與取樣頻率、延遲、感測雜訊有關。

$$K_{max} \propto 1 / (\text{取樣週期} \times \text{總延遲})$$

*大致的關係。取樣頻率越高、延遲越小，能穩定實現的剛度上限越高。*

實務上的經驗值是：以 1 kHz 控制、良好的力感測器，可穩定實現的剛度上限約在數千 N/m 的量級——遠低於金屬工件的  $10^8$  N/m。這再次印證了 20.1 節的結論：**用純軟體控制去「模擬」高剛度接觸是不可行的。**

因應之道有三：**一、提高控制頻率與降低延遲**（成本高，且有物理極限）；**二、降低目標剛度**（犧牲精度換穩定，最常用）；**三、加入被動柔順**（在機構上加入彈性，使等效環境剛度降低

——這正是下一節的主題）。

### ◎ 為何研磨作業特別困難

研磨結合了本章所有的困難：環境剛度極高（金屬）、需要維持穩定的法向力（力控制）、同時需要沿曲面移動（位置控制）、且工具旋轉會產生高頻擾動。因此工業上的力控研磨幾乎都採用「主動 + 被動」的混合方案：機器人做粗略的力控制與軌跡追隨，工具端則加裝一個氣壓或彈簧式的**主動順應裝置**（active compliance device），由它負責高頻的力調節。這個分工的原理是：機器人負責大範圍、低頻的運動，順應裝置負責小範圍、高頻的力——各自做自己擅長的事。

## 20.8 被動柔順與 RCC

### 20.8.1 用機構取代控制

主動柔順（用控制器實現）有頻寬限制（20.7.3 節）；被動柔順（用機構彈性實現）則沒有這個問題——**彈簧的反應是瞬間的，不需要感測、計算與致動**。

因此在需要高頻順應的場合，被動柔順往往是更好的選擇。常見的被動柔順元件包括：彈簧、橡膠墊、氣壓缸（空氣可壓縮，天然柔順——第 5.1.2 節）、以及專門設計的柔性機構。

被動柔順的缺點是**不可調整**：彈簧一旦裝上，剛度就固定了。若任務需要在不同階段使用不同的剛度，就必須用主動柔順，或設計可切換的機構。

### 20.8.2 遠端順應中心（RCC）

被動柔順最著名的設計是**遠端順應中心裝置**（Remote Center of Compliance, RCC），由 Draper 實驗室於 1970 年代為軸孔裝配開發。

它的巧妙之處在於：透過特殊的彈性元件配置，使整個裝置的「順應中心」落在**裝置外部**——恰好在插銷的**尖端**位置。

為何這個設計如此關鍵？考慮插銷入孔時的兩種誤差：

**平移誤差**：插銷中心與孔中心有橫向偏移。此時倒角會產生一個橫向力，把插銷推正——需要**橫向的柔順**。

**角度誤差**：插銷有傾斜。此時需要旋轉來修正——但關鍵是**繞哪一點旋轉**？若繞裝置本體旋轉，尖端會產生額外的橫向位移，反而使對位變差；若能繞**尖端**旋轉，就能純粹修正角度而不影響已對好的位置。

RCC 正是讓旋轉中心落在尖端，使兩種修正互不干擾。這使得插銷能「自己找到」孔的位置，成功率大幅提升。

圖 20-2 RCC 的作用原理

建議配圖：左右對照。左圖為「無 RCC」——插銷傾斜時繞著法蘭處旋轉，尖端因而橫向偏移，箭頭標示「角度修正反而破壞了位置對準」。右圖為「有 RCC」——以虛線標出順應中心恰好在插銷尖端，插銷繞尖端旋轉，尖端位置不變，箭頭標示「角度與位置的修正互相獨立」。下方另附兩個小圖，顯示插銷接觸倒角時橫向力如何自動引導對位。

### 20.8.3 主動與被動的分工

實務上最佳的方案通常是兩者結合，各自負責擅長的部分：

表 20-7 主動柔順與被動柔順的分工

| 面向       | 主動柔順（控制）        | 被動柔順（機構）     |
|----------|-----------------|--------------|
| 剛度可調     | 可即時改變           | 固定           |
| 頻寬       | 受取樣與延遲限制（數十 Hz） | 極高（機構的自然頻率）  |
| 可實現的剛度範圍 | 有上限（數千 N/m）     | 可極低或極高       |
| 成本       | 需感測器與運算         | 機構成本，通常較低    |
| 對衝擊的反應   | 有延遲，可能來不及       | 瞬間，天然吸收      |
| 適合處理     | 大範圍、低頻、需調整的順應   | 小範圍、高頻、固定的順應 |

典型的組合是：機器人用阻抗控制處理公分級的位置誤差與低頻的力調節，末端的 RCC 或彈性夾具處理零點幾公釐的高頻對位。這個「粗調靠控制、微調靠機構」的分工，是精密裝配的標準作法。

## 20.9 精密裝配：插銷入孔

### 20.9.1 為何這個任務如此經典

插銷入孔（peg-in-hole）是機器人裝配研究的標準測試任務。它看似簡單，卻濃縮了接觸控制的所有困難：位置誤差不可避免、環境剛度極高、接觸狀態不斷改變、且失敗的代價明確（卡住或損壞）。

問題的核心是一個量級落差：孔與軸的配合間隙通常只有零點幾公釐，而機器人的絕對定位精度是數公釐（第 9.9.2 節）。純靠位置精度是不可能成功的。

## 20.9.2 插入過程的三個階段

**階段一：接近 ( Approach )**。末端在自由空間中移動到孔口上方。此時無接觸，阻抗控制退化為位置控制，可用較高的剛度以求快速精確。

**階段二：搜尋與對位 ( Search & Align )**。插銷接觸到工件表面或倒角。此時需要低的橫向剛度，讓插銷能被倒角的斜面「推」到正確位置。若剛度太高，插銷會頂在表面上而不會滑入。

**階段三：插入 ( Insertion )**。插銷進入孔中，開始沿軸向下滑。此時需要軸向適度的推力 ( 力控制 ) 與橫向的高順應 ( 避免卡住 )。

三個階段的參數需求不同，這正是阻抗控制優於混合控制之處——只需連續調整  $K_d$  矩陣，不需切換控制模式。

表 20-8 插銷入孔各階段的阻抗參數

| 階段   | 軸向剛度      | 橫向剛度 | 旋轉剛度 | 軸向力指令        | 說明          |
|------|-----------|------|------|--------------|-------------|
| 接近   | 高         | 高    | 高    | 無            | 自由空間快速精確定位  |
| 搜尋對位 | 低         | 極低   | 極低   | 小 ( 1~5 N )  | 讓倒角引導，避免頂住  |
| 插入   | 中 ( 或力控 ) | 極低   | 極低   | 中 ( 5~20 N ) | 軸向推進，橫向完全順從 |
| 完成確認 | 高         | 中    | 中    | 無            | 回到位置控制以確認深度 |

## 20.9.3 三種提高成功率的策略

**策略一：倒角 ( Chamfer )**。在孔口做出斜面，把「精確對準」的要求轉化為「落在倒角範圍內」。倒角寬度若為 1 公釐，容許的位置誤差就從零點幾公釐放寬到 1 公釐——這是機構設計為控制減負的典型範例，成本極低而效果顯著。

**策略二：螺旋搜尋 ( Spiral Search )**。若首次接觸未能進入孔中，讓末端在保持軸向壓力的同時，沿一條螺旋路徑緩慢移動。當經過孔口時，軸向的順應會使插銷自動落入。這是一個「用時間換空間」的策略，能大幅提升對位置不確定性的容忍度。

**策略三：主動晃動 ( Wiggle / Dither )**。在插入過程中疊加一個小幅度的旋轉或橫向擺動。這能打破靜摩擦 ( 第 13.8.2 節的 stick-slip )、幫助插銷找到最順暢的方向，並避免「三點卡死」 ( jamming ) 的情形。

## 20.9.4 卡死與楔住

插入失敗有兩種模式，成因不同。

**楔住 (Wedging)**：插銷與孔壁在兩個接觸點上互相楔緊，此時無論施加多大的軸向力都無法推入——力越大楔得越緊。成因通常是插入初期角度誤差過大。

**卡死 (Jamming)**：施加的力與力矩的組合方向不對，使插銷「頂住」而無法前進。與楔住不同，卡死可以透過**改變施力方向**來解除。

兩者的區別在實務上很重要：偵測到插入失敗時，若是卡死，可以嘗試調整力矩方向或加入晃動；若是楔住，則必須退出重來。判斷的依據通常是「軸向力持續增大但深度不變」——這是楔住的特徵。

## 20.10 人機協作的安全

### 20.10.1 阻抗控制作為安全基礎

第 1 章與第 7 章提過協作機器人的安全哲學是「限制」而非「隔離」。本章的阻抗控制，正是這個哲學的技術實現。

關鍵在於 20.5.4 節的優勢二：**剛度是設計出來的上限**。以  $K_d = 300 \text{ N/m}$  為例，即使機器人的位置指令與人體位置差了 10 公分，最大接觸力也只有 30 N——這遠低於造成傷害的門檻。相對地，一個高增益的位置控制器在同樣的情況下可能輸出數千牛頓。

### 20.10.2 碰撞的三個階段與對應措施

國際標準（如 ISO/TS 15066）把人機碰撞分為三個階段，各需不同的技術。

**階段一：碰撞前 (Pre-collision)**。以視覺或接近感測偵測人員接近，主動降速或停止。這屬於第 22、26 章的範疇。

**階段二：碰撞中 (Impact)**。碰撞發生的最初數十毫秒，力量由機器人的**慣性主導**——此時控制器根本來不及反應。降低這個衝擊的唯一方法是**降低有效慣量**（輕量化設計、低減速比）與**加裝被動緩衝**（軟性外殼）。這是機構設計的責任，控制無能為力。

**階段三：碰撞後 (Post-impact)**。控制器偵測到異常力（第 11.6.3 節的無感測器碰撞偵測，或關節力矩感測），立即切換為零力模式或反向退讓。這裡的關鍵指標是**偵測延遲**——從碰撞到反應的時間，通常要求在 10 毫秒以內。

三個階段的分工說明了一個重要觀念：**安全不是單靠控制達成的**。控制只能處理階段一與階段三；階段二的衝擊完全由機構的慣量與柔性決定。這也是第 5 章強調準直驅設計、第 4 章討論輕量化的深層原因。

### 20.10.3 力與壓力的區別

最後一個實務要點：安全標準規範的往往不是**力**，而是**壓力**（力除以接觸面積）。同樣是 50 N 的力，透過一個鈍圓的外殼分散開來可能完全無害，但若集中在一個尖角上就可能造成穿刺傷。

因此協作機器人的設計除了控制之外，還必須注意：外殼的圓角化、避免夾點（pinch point）、以及在可能接觸的部位加裝軟性覆蓋。這再次呼應第 7.9 節的「本質安全」——最可靠的安全來自設計本身，而非主動的控制。

## 20.11 由力控制邁向抓取

### 20.11.1 本章的成果

本章讓機器人學會了「接觸」。你現在能夠：分析位置控制在接觸任務中失效的原因並量化接觸力、比較三種力感測方式、設計直接力控制與混合位置 / 力控制、運用自然與人為約束分析任務、推導並實作阻抗控制、依系統條件選擇阻抗或導納控制、理解接觸穩定性的限制與剛度上限、以及運用主動與被動柔順的分工解決精密裝配問題。

最重要的是那個貫穿全章的觀念轉換：**從控制「量」到控制「關係」**。位置控制與力控制是同一個頻譜的兩個極端，而阻抗控制涵蓋了整個頻譜——這使它成為現代機器人與環境互動的通用語言。

### 20.11.2 接下來：從接觸到抓取

本章處理的是「單點接觸」——機器人用一個末端點與環境互動。但**抓取**是更複雜的問題：多個接觸點同時作用、必須考慮摩擦錐、必須判斷物體會不會滑落或旋轉。

**第 21 章（抓取與操作）**將處理這些問題。我們會學到：**力封閉**（force closure）——判斷一組接觸點能否抵抗任意方向的外力；**形封閉與關入籠**（caging）——不靠摩擦而靠幾何限制物體；**摩擦錐**與正線性跨度的數學；以及**抓取品質指標**——如何量化「這個抓法有多穩」。

屆時本章的柔順概念會再次出現，並帶出一個重要的洞見——第 21 章的「手套思想實驗」將說明：**在抓取中，柔順往往比精準更重要**。這與本章的結論一脈相承。

換句話說：第 20 章讓機器人學會**溫柔地碰**，第 21 章讓它學會**穩穩地抓**。

## 完整計算例題

### 例題 20-1 接觸力的量級分析

題目：某機器人以位置控制接近一個金屬工件（環境剛度  $K_{env} = 10^5 \text{ N/mm}$ ）。（a）若位置指令比實際接觸面深 0.01 mm，求接觸力。（b）若機器人的定位誤差為  $\pm 0.5 \text{ mm}$ ，求最壞情況的接觸力。（c）改用阻抗控制， $K_d = 500 \text{ N/m}$ ，重新計算（b）的接觸力。（d）比較並說明。

解（a）：直接代入  $F = K_{env} \cdot \Delta x$ 。注意單位換算：

$$F = 10^5 \text{ N/mm} \times 0.01 \text{ mm} = 1000 \text{ N}$$

*僅 0.01 公釐的侵入，就產生 1000 牛頓的力（約等於 100 公斤重）。*

解（b）：位置誤差 0.5 mm 時：

$$F = 10^5 \text{ N/mm} \times 0.5 \text{ mm} = 5 \times 10^4 \text{ N} = 50000 \text{ N}$$

*五萬牛頓——這遠超過任何機器人的能力，實際上會先觸發過載保護、或造成工件與機構的永久損壞。*

解（c）：阻抗控制下，機器人表現為一個剛度  $K_d$  的彈簧，與環境的彈簧**串聯**。串聯彈簧的等效剛度為：

$$1/K_{eq} = 1/K_d + 1/K_{env} \Rightarrow K_{eq} = (K_d \cdot K_{env}) / (K_d + K_{env})$$

*串聯剛度公式。*

代入數值（注意單位統一為 N/m： $K_d = 500 \text{ N/m}$ 、 $K_{env} = 10^8 \text{ N/m}$ ）：

$$K_{eq} = (500 \times 10^8) / (500 + 10^8) \approx 500 \text{ N/m}$$

*等效剛度幾乎完全由較軟的那一個決定。這是串聯彈簧的重要性質。*

$$F = 500 \text{ N/m} \times 0.5 \text{ mm} = 500 \times 0.0005 = 0.25 \text{ N}$$

*接觸力僅 0.25 牛頓。*

解（d）：位置控制產生 50000 N，阻抗控制產生 0.25 N——**相差二十萬倍**。



**核心結論：**當一個軟彈簧與一個硬彈簧串聯時，等效剛度接近軟的那一個。因此**只要機器人夠「軟」，環境有多硬都無所謂**。這正是阻抗控制安全性的數學根源，也解釋了為何協作機器人能安全地與人接觸。

## 例題 20-2 阻抗參數的設計

題目：某協作機器人需要執行「把零件輕放到桌面上」的任務。要求：(i) 接觸力不超過 20 N；(ii) 能容忍  $\pm 8$  mm 的位置不確定性；(iii) 接觸過程不振盪。末端的等效慣量  $\Lambda = 5$  kg。(a) 求剛度  $K_d$  的上限。(b) 依臨界阻尼求阻尼  $B_d$ 。(c) 計算接觸時的自然頻率，並判斷所需的最低控制頻率。(d) 若改為接觸人手（要求力不超過 5 N），重新設計  $K_d$ 。

解 (a)：最壞情況下位置誤差為 8 mm，此時接觸力不得超過 20 N：

$$K_d \leq F_{\max} / \Delta x_{\max} = 20 \text{ N} / 0.008 \text{ m} = 2500 \text{ N/m}$$

剛度上限。取  $K_d = 2000 \text{ N/m}$  以保留餘裕。

解 (b)：臨界阻尼 ( $\zeta = 1$ ) 時：

$$B_d = 2\zeta\sqrt{(K_d \cdot M_d)} = 2 \times 1 \times \sqrt{(2000 \times 5)} = 2\sqrt{10000} = 2 \times 100 = 200 \text{ N}\cdot\text{s/m}$$

阻尼係數。

解 (c)：接觸後系統的剛度變為  $K_d$  與  $K_{\text{env}}$  的串聯。若桌面為木質 ( $K_{\text{env}} \approx 10^6 \text{ N/m}$ )，則等效剛度仍約為 2000 N/m（由較軟者主導）：

$$\omega_n = \sqrt{(K_{\text{eq}}/M_d)} = \sqrt{(2000/5)} = \sqrt{400} = 20 \text{ rad/s} \approx 3.18 \text{ Hz}$$

接觸時的自然頻率。

依第 19.9.3 節的準則，取樣頻率應為閉環頻寬的 20~30 倍：

$$f_{\text{sample}} \geq 20 \times 3.18 \approx 64 \text{ Hz} \cdot \text{建議 } 100 \text{ Hz 以上}$$

最低控制頻率。實務上力控制通常以 500~1000 Hz 執行，餘裕充足。

解 (d)：接觸人手時要求力不超過 5 N。但人手本身是柔軟的 ( $K_{\text{hand}} \approx 20 \text{ N/mm} = 2 \times 10^4 \text{ N/m}$ )，且人會退讓，因此位置誤差的定義不同。保守起見仍以 8 mm 計算：

$$K_d \leq 5 \text{ N} / 0.008 \text{ m} = 625 \text{ N/m}$$

接觸人體時的剛度上限，約為接觸桌面時的四分之一。



**實務意義：**這說明了為何協作機器人在偵測到人接近時，除了降速之外，還應**降低剛度**。許多協作機器人提供「協作模式」，其本質就是切換到較低的  $K_d$ 。

### 例題 20-3 插銷入孔的容差分析

題目：某軸孔配合的規格為：孔徑 20.05 mm、軸徑 20.00 mm ( 間隙 0.05 mm )，孔口有 1.0 mm 寬、45° 的倒角。機器人的定位精度為  $\pm 0.8$  mm。( a ) 判斷純位置控制是否可行。( b ) 加入倒角後，容許的位置誤差為多少？( c ) 若定位精度仍不足，計算螺旋搜尋所需的搜尋半徑。( d ) 設計此任務的阻抗參數。

解 ( a )：純位置控制要求軸心與孔心的偏差小於間隙的一半：

$$\text{容許偏差} = (20.05 - 20.00)/2 = 0.025 \text{ mm}$$

而機器人的精度為  $\pm 0.8$  mm——相差 32 倍，完全不可行。

解 ( b )：有倒角時，只要軸的邊緣落在倒角範圍內，斜面就能引導它滑入。容許的偏差為：

$$\text{容許偏差} = \text{倒角寬度} + \text{間隙}/2 = 1.0 + 0.025 = 1.025 \text{ mm}$$

倒角把容許誤差放寬了 41 倍。

由於機器人精度  $\pm 0.8$  mm < 1.025 mm，加入倒角後任務變為可行。這生動說明了 20.9.3 節「機構設計為控制減負」的價值——一個成本幾乎為零的倒角，解決了控制上極其困難的問題。

解 ( c )：假設機器人精度較差 (  $\pm 1.5$  mm )，超出倒角的容許範圍。此時需要螺旋搜尋。搜尋半徑應涵蓋最大可能的誤差：

$$\text{搜尋半徑} \geq \text{定位誤差} - \text{容許偏差} = 1.5 - 1.025 = 0.475 \text{ mm}$$

取 0.6 mm 以保留餘裕。螺旋的螺距應小於容許偏差的兩倍 ( 約 2 mm )，以確保不會「跳過」孔口。

解 ( d )：依 20.9.2 節的階段分析設計參數：

- **軸向 ( Z )：**插入階段用力控制，指定 10 N 的推力。或用中等剛度  $K_z = 1000$  N/m 配合位置指令。
- **橫向 ( X、Y )：**極低剛度  $K_{xy} = 50 \sim 200$  N/m，讓倒角能輕易推動插銷對位。
- **旋轉 ( 繞 X、Y )：**極低剛度，允許插銷自由調整傾角以順應孔的方向。
- **繞 Z 旋轉：**視任務而定；若插銷有方向性 ( 如有鍵槽 ) 則需高剛度，否則可設低值。

**驗證橫向剛度是否適當：**倒角的斜面 ( $45^\circ$ ) 在 10 N 軸向推力下，產生的橫向分力約為 10 N。若  $K_{xy} = 200 \text{ N/m}$ ，則橫向可移動  $10/200 = 0.05 \text{ m} = 50 \text{ mm}$ ——遠大於所需的 1 mm，因此順應性充足。若  $K_{xy}$  設為 10000 N/m，橫向只能移動 1 mm，可能不足以完成對位。

## 程式範例：阻抗控制的模擬與比較

以下 Python 程式模擬機器人末端接觸環境的過程，比較位置控制與阻抗控制的接觸力差異，並示範阻抗參數對行為的影響與接觸穩定性問題。

```
import numpy as np

# ===== 一維接觸模擬環境 =====
class ContactSim:
    """末端沿一維運動，在 x_wall 處遇到剛度 K_env 的牆"""
    def __init__(s, m=5.0, x_wall=0.0, K_env=1e8, B_env=100.0):
        s.m, s.x_wall, s.K_env, s.B_env = m, x_wall, K_env, B_env

    def contact_force(s, x, v):
        """環境施加的力（僅在侵入時作用）"""
        pen = x - s.x_wall          # 侵入深度（正值代表已侵入）
        if pen <= 0:
            return 0.0
        f = s.K_env*pen + s.B_env*max(v, 0.0)
        return -f                  # 方向與運動相反

    def step(s, x, v, u, dt):
        f_ext = s.contact_force(x, v)
        a = (u + f_ext) / s.m
        v = v + a*dt
        x = x + v*dt
        return x, v, f_ext

# ===== 三種控制器 =====
def position_control(x, v, x_d, Kp=50000.0, Kd=500.0):
    """高增益位置控制（第 17 章的思維）"""
    return Kp*(x_d - x) - Kd*v

def impedance_control(x, v, x_d, Kd_imp=500.0, Bd=200.0):
    """阻抗控制（20.5 節）：像一個彈簧阻尼器"""
    return Kd_imp*(x_d - x) - Bd*v

def force_control(x, v, F_d, F_meas, Kf=0.5, Ki=20.0, state=None):
    """直接力控制（20.3 節）·含積分項"""
    e = F_d - abs(F_meas)
    state['I'] += e * state['dt']
    return -(F_d + Kf*e + Ki*state['I'])    # 負號：往牆的方向推
```

```

# ===== 模擬主程式 =====
def run(ctrl, label, x_d=0.005, T=1.5, dt=1e-5, **kw):
    """x_d = 0.005 代表指令位置比牆面深 5 mm (模擬定位誤差)"""
    sim = ContactSim()
    x, v = -0.05, 0.0 # 由牆前 50 mm 出發
    state = {'I': 0.0, 'dt': dt}
    f_max, f_final, contact_t = 0.0, 0.0, None
    for k in range(int(T/dt)):
        f_ext = sim.contact_force(x, v)
        if ctrl is force_control:
            u = ctrl(x, v, kw.get('F_d', 10.0), f_ext, state=state)
        else:
            u = ctrl(x, v, x_d, **kw)
        u = np.clip(u, -3000, 3000) # 致動器力矩上限
        x, v, f_ext = sim.step(x, v, u, dt)
        if f_ext != 0.0:
            if contact_t is None: contact_t = k*dt
            f_max = max(f_max, abs(f_ext))
            f_final = abs(f_ext)
    return label, f_max, f_final, x - sim.x_wall

print("=== 位置控制 vs 阻抗控制：接觸金屬牆 (K_env = 1e8 N/m) ===")
print("指令位置比牆面深 5 mm (模擬定位誤差)")
print(f"{'控制器':<28}{'最大接觸力':>14}{'穩態接觸力':>14}{'侵入深度':>14}")
for ctrl, label, kw in (
    (position_control, "位置控制 (Kp=50000)", {}),
    (impedance_control, "阻抗控制 (Kd=2000)", {'Kd_imp': 2000.0, 'Bd': 200.0}),
    (impedance_control, "阻抗控制 (Kd=500)", {'Kd_imp': 500.0, 'Bd': 100.0}),
    (impedance_control, "阻抗控制 (Kd=100)", {'Kd_imp': 100.0, 'Bd': 45.0})):
    lab, fmax, ffin, pen = run(ctrl, label, **kw)
    print(f"{'lab':<28}{'fmax':12.1f} N{'ffin':12.2f} N{'pen*1000':12.4f} mm")
print("> 剛度越低，接觸力越小 (例題 20-1 的驗證)")

# ===== 例題 20-1：串聯剛度 =====
print("\n=== 例題 20-1：串聯彈簧的等效剛度 ===")
K_env = 1e8
print(f"{'K_d (N/m)':>12}{'K_eq (N/m)':>14}{'誤差 0.5mm 的力':>16}")
for Kd_imp in (1e8, 1e5, 1e4, 2000, 500):
    K_eq = (Kd_imp*K_env)/(Kd_imp + K_env)
    print(f"{'Kd_imp':12.0f}{'K_eq':14.1f}{'K_eq*0.0005':14.2f} N")
print("> 等效剛度幾乎完全由較軟者決定")

# ===== 例題 20-2：阻抗參數設計 =====
print("\n=== 例題 20-2：阻抗參數設計 ===")
Lam, F_max, dx_max = 5.0, 20.0, 0.008
Kd_lim = F_max/dx_max
print(f"剛度上限 K_d ≤ {F_max}/{dx_max} = {Kd_lim:.0f} N/m → 取 2000 N/m")
Kd_use = 2000.0
Bd = 2*1.0*np.sqrt(Kd_use*Lam)
wn = np.sqrt(Kd_use/Lam)

```

```

print(f"臨界阻尼 B_d = 2√(K_d·M_d) = {Bd:.1f} N·s/m")
print(f"自然頻率 ω_n = √(K_d/M_d) = {wn:.1f} rad/s = {wn/(2*np.pi):.2f} Hz")
print(f"建議最低取樣頻率 = 20 × {wn/(2*np.pi):.2f} = {20*wn/(2*np.pi):.0f} Hz")
Kd_human = 5.0/0.008
print(f"接觸人手 (力 ≤ 5 N) : K_d ≤ {Kd_human:.0f} N/m "
      f" ( 僅為接觸桌面的 {Kd_human/Kd_lim*100:.0f}% ) ")

# ===== 阻尼比對接觸行為的影響 =====
print("\n=== 阻尼比對接觸的影響 ( K_d = 2000 N/m, M = 5 kg ) ===")
print(f"'ζ':>6}{ 'B_d':>10}{ '最大接觸力':>14}{ '穩態力':>12} 行為")
Kd_imp = 2000.0
for zeta in (0.2, 0.5, 0.7, 1.0, 2.0):
    Bd_t = 2*zeta*np.sqrt(Kd_imp*5.0)
    _, fmax, ffin, _ = run(impedance_control, "", Kd_imp=Kd_imp, Bd=Bd_t)
    behav = ("嚴重超越 / 彈跳" if zeta < 0.4 else
             "略有超越" if zeta < 0.9 else
             "平順" if zeta <= 1.2 else "反應遲鈍")
    print(f"{zeta:6.1f}{Bd_t:10.1f}{fmax:12.1f} N{ffin:10.2f} N {behav}")
print("→ 阻尼不足會在接觸瞬間產生衝擊與彈跳")

# ===== 例題 20-3：插銷入孔容差 =====
print("\n=== 例題 20-3：插銷入孔容差分析 ===")
hole, peg, chamfer, robot_acc = 20.05, 20.00, 1.0, 0.8
clearance = (hole - peg)/2
print(f"間隙半徑 = {clearance:.3f} mm")
print(f"純位置控制容許偏差 = {clearance:.3f} mm "
      f" ( 機器人精度 ±{robot_acc} mm → {'可行' if robot_acc <= clearance else '不可行'} ) ")
tol_chamfer = chamfer + clearance
print(f"有倒角後容許偏差 = {tol_chamfer:.3f} mm "
      f" ( → {'可行' if robot_acc <= tol_chamfer else '不可行'} . "
      f"容差放寬 {tol_chamfer/clearance:.0f} 倍 ) ")
for acc in (1.5, 2.0):
    r = max(0.0, acc - tol_chamfer)
    print(f"若精度為 ±{acc} mm: 需螺旋搜尋，半徑 ≥ {r:.3f} mm → 建議 {r*1.25:.2f} mm")

# 橫向順應性驗證
print("\n 橫向剛度的順應性驗證 ( 45° 倒角、軸向推力 10 N ) : ")
F_lateral = 10.0
for Kxy in (10000., 1000., 200., 50.):
    disp = F_lateral/Kxy*1000
    print(f" K_xy = {Kxy:7.0f} N/m → 橫向可位移 {disp:7.2f} mm "
          f"{'✓ 充足' if disp > 1.0 else 'X 不足'}")

```

請試著修改：（一）把環境剛度改為  $1e4$  N/m（橡膠），觀察位置控制是否變得可行——這說明「環境軟」時位置控制的困難會大幅降低。（二）在阻抗控制中加入 5 ms 的力回授延遲，

觀察是否出現振盪（驗證 20.7 節的接觸不穩定）。（三）把致動器的力上限降到 100 N，觀察位置控制的行為如何改變。

## 實作：柔順控制的實驗

本章實作是在實體手臂上實現柔順行為並量化其效果。**安全提醒：力控制的參數錯誤可能導致機器人失控或損壞工件，務必從極低的剛度與極低的速度開始測試。**

- 前置作業：**確認第 17 章的重力補償已正確實作（阻抗控制建立在其上）。若有力 / 力矩感測器，完成其歸零與重力補償校正。
- 剛度的直接體驗：**實作阻抗控制，設定不同的  $K_d$ （如 5000、1000、200 N/m），用手推動末端並感受其軟硬差異。用彈簧秤量測「推開 10 mm 所需的力」，驗證是否符合  $F = K_d \times \Delta x$ 。
- 阻尼的影響：**固定  $K_d$ ，改變  $B_d$  使  $\zeta$  分別為 0.3、0.7、1.0、2.0。快速推動末端後放手，觀察其回彈行為——低阻尼會來回振盪、臨界阻尼平順回位、過阻尼緩慢回位。
- 接觸力量測：**在末端裝一支筆，讓機器人以不同的  $K_d$  接觸一個電子秤，設定的目標位置比秤面「深」5 mm。記錄各剛度下的實際接觸力，與理論值  $K_d \times 0.005$  比較。
- 位置控制的對照：**在安全的前提下（用海綿或彈簧秤，勿用硬物），以位置控制重複上一步。觀察接觸力如何暴增，並記錄控制器是否觸發過載保護。
- 方向性剛度：**把  $K_d$  設為非對角矩陣（如 Z 方向 2000 N/m、X-Y 方向 100 N/m），用手推動末端，感受不同方向的軟硬差異。這是混合控制的阻抗版本。
- 接觸穩定性探索：**逐步提高  $K_d$ ，找出開始出現振盪或嗡嗡聲的臨界值。記錄此臨界剛度，並嘗試提高控制頻率或降低力訊號的濾波，觀察臨界值是否上升（驗證 20.7.3 節）。
- 插銷入孔實測：**準備一組有倒角與無倒角的軸孔試件（可 3D 列印）。分別以位置控制與阻抗控制嘗試插入各二十次，記錄成功率。這個對照實驗能最直觀地展示柔順控制的價值。
- （進階）螺旋搜尋：**對定位誤差較大的情形，實作螺旋搜尋策略，記錄搜尋所需的平均時間與成功率的提升。

表 20-9 柔順控制實驗記錄表（第一列為範例）

| 實驗項目 | 設定                                | 理論預期   | 實測結果   | 說明     |
|------|-----------------------------------|--------|--------|--------|
| 剛度驗證 | $K_d = 1000 \text{ N/m}$ ，推 10 mm | 10.0 N | 11.3 N | 誤差來自摩擦 |

| 實驗項目         | 設定 | 理論預期 | 實測結果 | 說明 |
|--------------|----|------|------|----|
| ( 自行填寫其餘實驗 ) |    |      |      |    |

## 國中小也看得懂：讓機器人的手變成一根彈簧

試試看這個實驗：用**兩根手指**輕輕拿起一顆生雞蛋。你有沒有想過，你是怎麼知道該用多大的力氣的？

其實你**沒有**特別去想。你不會說「我要把手指移到 43.9 公釐的位置」，也不會說「我要出 2 牛頓的力」。你只是——輕輕地握住它。

但機器人不一樣。如果我們叫機器人「把手指合到 43.5 公釐」，會發生什麼事？蛋殼只要被壓進去 0.5 公釐，就會產生大約 1500 牛頓的力——那可以壓碎石頭！蛋當然一秒就爆了。

那如果改成「用 2 牛頓的力去夾」呢？也不行。因為在手指還沒碰到蛋之前，力是零，機器人會覺得「還不夠用力」，於是拚命往前衝，撞上去的瞬間反而更慘。

所以聰明的做法是**第三種**：讓機器人的手變得像一根彈簧。

想像你的手指上裝了兩根軟軟的彈簧。當彈簧碰到蛋，蛋會把彈簧壓回去一點點，這時彈簧產生的力剛剛好——不會太大（因為彈簧很軟），也不會是零（因為確實被壓縮了）。而且你**完全不需要知道蛋有多大**，彈簧會自己調整！

這就是這一章最重要的概念：**不要控制位置，也不要控制力，而是控制「軟硬」**。工程師叫它「阻抗控制」。

這個方法還有一個很棒的好處：**安全**。如果機器人的手是「很軟的彈簧」，就算它不小心撞到你，也只會輕輕碰一下——因為軟彈簧根本產生不了大力氣。這就是為什麼現在的新型機器人可以直接和人一起工作，不需要用鐵籠子把它關起來。

## 叫獸提醒與常見錯誤

- 誤區一：用位置控制去接觸剛性環境。環境剛度可達  $10^8$  N/m，微米級的位置誤差就會產生數百牛頓的力，這在物理上不可能成功。
- 誤區二：以為純力控制可以取代位置控制。在自由空間中測到的力恆為零，控制器會持續加

速直到撞上，這是本質上的不穩定。

- 誤區三：忘記扣除工具重力與慣性力。力感測器量到的是總和，必須用動力學模型即時補償，否則會把自重誤判為外力。
- 誤區四：混合控制的方向與實際接觸面不一致。若工件擺放有角度偏差，本該控力的方向變成控位置，後果與誤區一相同。
- 誤區五：同一個方向同時控制位置與力。這在物理上是矛盾的——自然約束與人為約束必定互補。
- 誤區六：把阻抗剛度設得太高。剛度上限受取樣頻率與延遲限制，設得太高會導致接觸振盪而非更精確。
- 誤區七：忽略摩擦對柔順性的破壞。高減速比的關節摩擦會使機器人「感覺起來很硬」，無論  $K_d$  設得多低。
- 誤區八：在硬機器人上用阻抗控制、在軟環境中用導納控制。應遵循互補原則：硬環境配可反驅的機器人用阻抗、軟環境配高減速比機器人用導納。
- 誤區九：力訊號濾波過度。濾波降低雜訊但增加延遲，而延遲是接觸不穩定的主因，必須權衡。
- 誤區十：以為柔順控制能取代機構設計。倒角、RCC 等被動措施成本極低卻效果顯著，且沒有頻寬限制。
- 誤區十一：以為安全只靠控制。碰撞的最初數十毫秒由慣量主導，控制器來不及反應，必須靠輕量化與被動緩衝。
- 誤區十二：只看力而忽略壓力。安全標準規範的是壓力（力除以面積），同樣的力集中在尖角上可能造成傷害。

## 課堂討論

1. 例題 20-1 顯示阻抗控制的接觸力比位置控制小二十萬倍。請用「串聯彈簧」的觀點解釋這個現象，並討論它對協作機器人安全設計的意義。
2. 阻抗控制的基本版本不需要力感測器，卻能產生柔順行為。請解釋這是為什麼，並討論在什麼情況下仍然必須加裝力感測器。
3. 20.6.2 節提出「硬環境配軟機器人用阻抗、軟環境配硬機器人用導納」的互補原則。請討



論：若兩者都硬（傳統工業機器人接觸金屬工件），有哪些可行的解決方案？

4. 一個 1 公釐的倒角，把插銷入孔的容許誤差放寬了 41 倍（例題 20-3）。請再舉出兩個「機構設計為控制減負」的例子，並討論這種思維對系統設計的啟示。
5. 20.10.2 節指出碰撞的第二階段（衝擊瞬間）完全由機構決定，控制無能為力。請討論這對機器人設計的優先順序有何影響——在追求更好的控制演算法之前，應該先做什麼？

## 章末摘要

- 位置控制在接觸剛性環境時必然失效：環境剛度可達  $10^8$  N/m，微米級誤差即產生數百牛頓的力。
- 純力控制在自由空間中不穩定：測得的力恆為零，控制器會持續加速直到撞擊。
- 力感測有三種方式：六軸力 / 力矩感測器（精度高但貴）、關節力矩感測器（可全身偵測）、馬達電流估測（免費但精度低）。
- 力訊號必須扣除工具重力與慣性力，並注意濾波延遲與座標系轉換。
- 直接力控制需要積分項，因為環境剛度未知且會變化，無法用模型補償。
- 自然約束由環境幾何決定、人為約束由任務指定，兩者互補：同一方向不可能同時控制位置與力。
- 混合位置 / 力控制以選擇矩陣  $S$  指定各方向的控制模式，並建立在操作空間的動態解耦之上。
- 阻抗控制的目標是讓末端表現為  $M_d \ddot{x} + B_d \dot{x} + K_d x = F_{ext}$ ，即一個可程式的質量—彈簧—阻尼系統。
- 阻抗控制涵蓋整個頻譜： $K_d \rightarrow \infty$  為純位置控制、 $K_d \rightarrow 0$  為純力控制。
- 阻抗控制的四大優勢：統一處理接觸與非接觸、本質安全（剛度即力的上限）、基本版不需力感測器、各方向可獨立設定剛度。
- 串聯彈簧的等效剛度由較軟者主導——只要機器人夠軟，環境有多硬都無所謂，這是安全性的數學根源。
- 阻抗控制「量測運動輸出力」適合可反驅機器人與硬環境；導納控制「量測力輸出運動」適合高減速比機器人與軟環境。
- 接觸使閉環剛度暴增、自然頻率上升、阻尼比下降，因而容易振盪；可實現的剛度有上限，



與取樣頻率及延遲有關。

- 主動柔順可調但受頻寬限制，被動柔順（如 RCC）不可調但反應瞬間；實務上採「粗調靠控制、微調靠機構」的分工。
- 插銷入孔的三個階段需要不同的阻抗參數；倒角、螺旋搜尋與主動晃動是提高成功率的三種策略。
- 人機碰撞的三階段中，衝擊瞬間由機構慣量主導，控制只能處理碰撞前的偵測與碰撞後的退讓。

## 觀念題與計算題

### 觀念題

1. 說明位置控制在接觸剛性環境時失效的原因，並以量級分析支持你的論點。
2. 為何純力控制在自由空間中不穩定？請描述控制器的行為。
3. 比較三種力感測方式的精度、成本、偵測範圍與適用場合。
4. 力訊號處理的三個要點為何？為何重力補償對力控制特別重要？
5. 為何直接力控制幾乎必須加入積分項？這與位置控制中的積分項有何不同？
6. 區分自然約束與人為約束，並說明兩者的互補性原理。
7. 以「用板擦擦黑板」為例，分析其自然約束與人為約束，並寫出選擇矩陣。
8. 寫出阻抗控制的目標關係式，並說明三個參數的物理意義。
9. 為何說阻抗控制涵蓋了位置控制與力控制兩個極端？
10. 列舉阻抗控制的四大優勢，並說明「本質安全」的數學根據。
11. 說明串聯彈簧的等效剛度性質，以及它對接觸安全的意義。
12. 區分阻抗控制與導納控制，並說明選擇兩者的互補原則。
13. 為何摩擦會破壞阻抗控制的柔順性？有哪三種因應方式？
14. 說明接觸不穩定的三個關鍵因素，以及可實現剛度有上限的原因。
15. 比較主動柔順與被動柔順，並說明 RCC 的設計原理。
16. 說明插銷入孔的三個階段及其阻抗參數需求，並列出三種提高成功率的策略。

## 計算題

1. 某機器人接觸一個塑膠工件 ( $K_{env} = 5 \times 10^3 \text{ N/mm}$ )。(a) 若位置指令深入  $0.2 \text{ mm}$ ，求接觸力。(b) 若改用阻抗控制且  $K_d = 800 \text{ N/m}$ ，求同樣位置誤差下的接觸力。(c) 計算兩者的比值。(d) 求串聯後的等效剛度。
2. 某協作機器人末端等效慣量  $\Lambda = 8 \text{ kg}$ ，要求接觸力不超過  $30 \text{ N}$  且能容忍  $\pm 12 \text{ mm}$  的位置不確定性。(a) 求剛度上限並選定  $K_d$ 。(b) 依  $\zeta = 0.8$  求阻尼  $B_d$ 。(c) 計算自然頻率 ( $\text{rad/s}$  與  $\text{Hz}$ )。(d) 依第 19 章的準則建議最低控制頻率。
3. 某軸孔配合：孔徑  $12.03 \text{ mm}$ 、軸徑  $12.00 \text{ mm}$ ，孔口倒角寬  $0.8 \text{ mm}$ 。機器人定位精度  $\pm 0.6 \text{ mm}$ 。(a) 求間隙半徑與純位置控制的容許偏差。(b) 判斷純位置控制是否可行。(c) 加入倒角後的容許偏差為何？是否可行？(d) 若精度劣化為  $\pm 1.2 \text{ mm}$ ，求所需的螺旋搜尋半徑。
4. 某力控研磨作業：法向要求維持  $25 \text{ N}$  的壓力，工件表面有  $\pm 1.5 \text{ mm}$  的起伏。(a) 若用位置控制且工件剛度為  $2 \times 10^4 \text{ N/mm}$ ，求表面起伏造成的力變化範圍。(b) 改用阻抗控制，求使力變化不超過  $\pm 10 \text{ N}$  所需的  $K_d$  上限。(c) 在此剛度下，維持  $25 \text{ N}$  平均壓力所需的平均侵入深度為何？(d) 討論此深度是否合理。

## 動手做與專題延伸

### 本章實作：柔順控制模組

延續第 17 ~ 19 章的控制器程式庫，加入力控制與阻抗控制模組。

- **階段一**：建立一維的接觸模擬環境 (含環境剛度與阻尼)，實作位置控制與阻抗控制並比較接觸力。
- **階段二**：實作直接力控制 (含積分項) 與混合位置 / 力控制 (含選擇矩陣)，在模擬中驗證各方向的行為。
- **階段三**：實作完整的六自由度阻抗控制，整合第 17 章的操作空間控制與第 11 章的雅可比。
- **階段四**：實作導納控制版本，並在「硬機器人 + 軟環境」與「軟機器人 + 硬環境」兩種配置下比較兩者的表現。
- **階段五**：加入力感測的雜訊與延遲模型，找出各配置下的臨界剛度，驗證 20.7.3 節的結論。

### 專題延伸

- **剛度—接觸力關係圖**：系統性地掃描  $K_d$  ( 由  $10$  到  $10^6$  N/m )，對固定的位置誤差記錄接觸力，繪成對數—對數圖。你會看到曲線在  $K_d$  遠小於  $K_{env}$  時呈線性、在接近  $K_{env}$  時飽和——這張圖直觀展示了串聯彈簧的性質與「軟者主導」的原理。
- **接觸穩定性的邊界圖**：以「控制頻率」為橫軸、「目標剛度」為縱軸，對每個組合模擬接觸過程並判定是否穩定，繪出穩定與不穩定的分界線。加入不同的力感測延遲 (  $0$ 、 $2$ 、 $5$ 、 $10$  ms ) 各繪一條線，量化延遲對可實現剛度的影響。
- **進階：插銷入孔的策略比較**。建立一個二維的插銷入孔模擬 ( 含倒角幾何與摩擦 )，實作四種策略—— ( 甲 ) 純位置控制、 ( 乙 ) 阻抗控制、 ( 丙 ) 阻抗控制加螺旋搜尋、 ( 丁 ) 阻抗控制加主動晃動。對每種策略在不同的初始位置誤差下各模擬  $100$  次，統計成功率與平均插入時間，繪出「位置誤差 vs 成功率」曲線。這個實驗能完整呈現本章各項技術的實際價值。

## 參考文獻與延伸閱讀

- Hogan, N., "Impedance Control: An Approach to Manipulation, Parts I-III," ASME J. Dynamic Systems, 1985 ( 阻抗控制的奠基論文 )。
- Mason, M. T., "Compliance and Force Control for Computer Controlled Manipulators," IEEE Trans. SMC, 1981 ( 自然約束與人為約束的框架 )。
- Raibert, M. H. and Craig, J. J., "Hybrid Position/Force Control of Manipulators," ASME J. Dynamic Systems, 1981 ( 混合位置 / 力控制的原始論文 )。
- Whitney, D. E., "Quasi-Static Assembly of Compliantly Supported Rigid Parts," ASME J. Dynamic Systems, 1982 ( RCC 與插銷入孔的理論分析 )。
- Khatib, O., "A Unified Approach for Motion and Force Control of Robot Manipulators," IEEE J. Robotics and Automation, 1987 ( 操作空間中的統一運動與力控制 )。
- Siciliano, B. and Villani, L., Robot Force Control, Kluwer Academic ( 力控制的完整專著 )。
- Colgate, J. E. and Hogan, N., "Robust Control of Dynamically Interacting Systems," Int. J. Control, 1988 ( 接觸穩定性與被動性理論 )。
- Albu-Schäffer, A. et al., "Soft Robotics: From Torque Feedback-Controlled Lightweight Robots to Intrinsically Compliant Systems," IEEE Robotics & Automation Magazine, 2008 ( DLR 輕量手臂與柔順控制的實務 )。

- ISO/TS 15066:2016, Robots and robotic devices — Collaborative robots ( 人機協作的力與壓力限制規範 ) 。
- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson ( 第 9 章：力控制 ) 。

# 第 21 章

## 抓取與操作

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一隻戴著厚工作手套的手正握住一瓶礦泉水的照片，手套明顯變形貼合瓶身，旁註「摸不清楚，卻抓得穩」。右半為三格對照：上格是剛性兩指夾爪與圓柱體的單點接觸（標註「點接觸，位置差 2 mm 即失敗」）；中格是柔性指墊變形後的面接觸（標註「軟指接觸，可抵抗扭轉」）；下格是欠驅動手指自動包絡不規則物體的示意。畫面下緣以摩擦錐圖示串起——兩個相對的圓錐與其間的連線，標註「連線落在雙錐內  $\Rightarrow$  力封閉」。標題文字：抓取的成功，往往不是來自精準，而是來自順應。

## 叫獸開講 「戴著厚手套、閉著眼睛，為什麼還是抓得起來？」

叫獸拿出一副厚厚的工作手套，遞給小妤：「戴上，然後把眼睛閉起來。」

小妤照做了。叫獸在她面前的桌上放了一瓶礦泉水：「桌上有個東西，把它拿起來。」

小妤的手伸出去，摸索了兩三秒，碰到了瓶身，手掌順著曲面滑動了一下，然後五指收攏——穩穩地把水瓶提了起來。全程不到五秒。

「你剛才知道那是什麼嗎？」叫獸問。「不知道，摸起來像個圓柱體。」「知道它有多重嗎？」「不知道。」「知道它的中心在哪裡嗎？」「也不知道。」

叫獸轉向全班：「她戴著厚手套、閉著眼睛、不知道物體是什麼、多重、重心在哪——但她五秒鐘就拿起來了。現在請你們想想：**如果要一台機器人做同樣的事，需要什麼？**」

學生們開始列舉：高解析度的相機、精確的物體辨識、三維重建、質量與摩擦係數的估計、精密的力覺感測、毫米級的手指定位.....

「你們說的都對，」叫獸點頭，「而且這正是過去三十年機器人抓取研究的主流方向——**把每一件事都測得更準、算得更精**。但小妤剛才什麼都不知道，她是怎麼成功的？」

教室安靜了幾秒。小翰試著回答：「因為.....手套很軟？」

「非常接近了。」叫獸拿起那副手套，「厚手套讓她**摸不清楚**，這是壞事。但厚手套同時也讓她的手指**能夠變形、能夠貼合、能夠在滑動時自動調整**——這是好事。而在抓取這件事上，**後者比前者重要得多**。」

他在白板上寫下一行字：**抓取的成功，往往不是來自精準，而是來自順應**。

「這是第 20 章那個雞蛋的延續。人手抓東西時，手指的軟墊會變形、會攤開成一片接觸面、會在物體要滑動時自動增加摩擦。這些都不是『算出來』的，是**機構本身的性質**。」

小妤把手套脫下來：「那我們今天要學什麼？」

「兩件事。」叫獸豎起兩根手指，「第一，**什麼叫做抓穩了**——這需要嚴格的數學，因為工程師不能只憑感覺說『看起來很穩』。第二，**在什麼情況下，該靠算的、什麼時候該靠機構的柔軟**。學會這兩件事，你就理解了現代機器人抓取的全貌。」

## 本章為什麼重要

抓取是機器人與環境互動最基本、也最普遍的形式。從工廠的取放、倉儲的分揀、到服務型機器人的居家操作，幾乎所有任務都由「抓起某個東西」開始。

但抓取同時也是機器人學中最頑固的難題之一。第 1 章提到的 Moravec 悖論在此表現得最為極端：人類幼兒兩歲就能靈巧地抓取任意物體，而這件事至今仍是機器人研究的前沿。原因在於抓取結合了本書幾乎所有的困難——不確定的物體幾何、未知的摩擦與質量、多點同時接觸、以及接觸力學的本質複雜性（第 14.7 節）。

本章要建立三個層次的理解。**第一，理論基礎**：用力封閉、形封閉與摩擦錐等概念，嚴格定義「什麼叫抓穩了」。**第二，設計思維**：理解柔順性在抓取中的關鍵角色，以及軟性夾爪為何能用簡單的機構解決複雜的問題。**第三，現代方法**：了解學習式抓取如何繞過建模的困難，直接由資料學習「哪裡可以抓」。

本章也是第三篇的收尾。第 13~20 章建立的動力學、控制與力控制工具，都將在此匯聚——抓取正是它們的綜合應用。

## 學習目標

- 能說明抓取問題的三個組成：接觸建模、抓取分析與抓取規劃。
- 能描述三種接觸模型及其可傳遞的力與力矩。
- 能定義摩擦錐並說明其在抓取分析中的角色。
- 能定義力封閉，並判斷一組接觸點是否構成力封閉。
- 能運用正線性跨度的概念判斷抓取的完整性。
- 能區分力封閉、形封閉與關入籠三個概念。
- 能說明常用的抓取品質指標及其物理意義。
- 能敘述抓取合成的經典定理與所需的最少接觸點數。
- 能說明柔順性在抓取中的作用，並解釋為何順應往往勝過精準。
- 能比較剛性夾爪、軟性夾爪與欠驅動手的設計哲學。
- 能說明學習式抓取的基本流程與其相對於解析法的優劣。

## 先備知識

第 11 章的雅可比與速度—力對偶、第 20 章的力控制與柔順概念（本章可視為其在多接觸點情形的推廣）、第 13 章的動力學基礎、第 6.4 節的力覺與觸覺感測。

數學方面需要：向量與矩陣運算、線性獨立與跨度（span）的概念、以及基本的凸集合直覺。本章的數學不深，但**幾何直覺很重要**——建議多用手邊的物品實際比劃，體會各個概念的物理意義。

## 核心名詞中英對照

表 21-1 本章核心名詞

| 中文     | 英文                      | 說明                |
|--------|-------------------------|-------------------|
| 抓取     | Grasp                   | 以多個接觸點限制物體運動的行為   |
| 接觸模型   | Contact Model           | 描述接觸點能傳遞何種力的模型    |
| 點接觸    | Point Contact           | 僅能傳遞法向力的理想接觸      |
| 軟指接觸   | Soft Finger Contact     | 可傳遞法向力、摩擦力與繞法向的力矩 |
| 摩擦錐    | Friction Cone           | 不發生滑動的接觸力方向所構成的錐體 |
| 摩擦係數   | Coefficient of Friction | 最大靜摩擦力與法向力之比      |
| 抓取矩陣   | Grasp Matrix            | 把接觸力映射為物體所受合力的矩陣  |
| 力封閉    | Force Closure           | 能以接觸力抵抗任意方向外力的性質  |
| 形封閉    | Form Closure            | 僅靠幾何即可限制物體的性質     |
| 關入籠    | Caging                  | 物體被包圍而無法逃脫但仍可移動   |
| 正線性跨度  | Positive Linear Span    | 以非負係數線性組合所能達到的集合  |
| 抓取品質   | Grasp Quality           | 量化抓取穩固程度的指標       |
| 欠驅動手   | Underactuated Hand      | 驅動器少於自由度的機械手      |
| 自適應抓取  | Adaptive Grasping       | 手指依物體形狀自動貼合的抓取    |
| 軟性夾爪   | Soft Gripper            | 以柔性材料製成的夾持裝置      |
| 抓取姿勢偵測 | Grasp Pose Detection    | 由感測資料預測可行抓取位姿     |

## 21.1 抓取問題的組成

### 21.1.1 三個層次



「抓取」看似單一動作，實際上包含三個層次的問題，各有不同的研究方法。

**層次一：接觸建模。**單一接觸點能傳遞什麼樣的力與力矩？這取決於接觸的幾何（點、線、面）與摩擦特性。這是最底層的物理基礎，21.2 節處理。

**層次二：抓取分析。**給定一組接觸點，判斷這個抓法是否「穩」——能否抵抗任意方向的外力？有多穩？這是本章數學的核心，21.3 至 21.6 節處理。

**層次三：抓取規劃。**面對一個物體，該把手指放在哪裡？這是搜尋與決策問題，也是現代學習式方法的主戰場，21.9 節處理。

三個層次的關係是：分析回答「這個抓法好不好」，規劃則是「找出好的抓法」。前者是後者的評分函數。

### 21.1.2 為何抓取如此困難

抓取的困難來自五個因素的疊加：

- **物體資訊不完整：**形狀可能只看得到一半（遮蔽）、質量與重心未知、材質與摩擦係數難以由視覺判斷。
- **接觸力學本質複雜：**如第 14.7 節所述，摩擦是非線性的、接觸是不連續的、且某些配置下的接觸力沒有唯一解。
- **多點同時作用：**各接觸點的力互相影響，且必須滿足不滑動的條件——這是一個帶約束的多變數問題。
- **位置誤差不可避免：**機器人的定位精度（第 9.9.2 節）遠大於精確抓取所需的容差。
- **物體會動：**抓取過程中物體可能被推動、旋轉或滑落，使原本規劃的接觸點失效。

這五個因素解釋了為何「精確建模再精確執行」的路線在抓取上特別吃力——不確定性太多，且每一項都難以消除。這也預示了本章的核心洞見：**與其消除不確定性，不如設計一個對不確定性不敏感的系統。**

## 21.2 接觸模型與摩擦錐

### 21.2.1 三種接觸模型

抓取分析的第一步，是決定用什麼模型描述接觸點。常用的有三種，複雜度遞增。

**模型一：無摩擦點接觸 (Frictionless Point Contact)**。只能傳遞沿接觸法向的**推力**（不能拉）。每個接觸點只貢獻 1 個力的自由度。適用於極光滑的表面，是最保守的假設。

**模型二：有摩擦點接觸 (Point Contact with Friction, PCwF)**。除法向力外，還能傳遞切向的摩擦力。每個接觸點貢獻 3 個自由度（三維空間中的力向量）。這是最常用的模型。

**模型三：軟指接觸 (Soft Finger Contact)**。除上述三個力之外，還能傳遞繞接觸法向的**力矩**——因為柔軟的指尖會攤開成一小片接觸面，能抵抗扭轉。每個接觸點貢獻 4 個自由度。

**表 21-2 三種接觸模型的比較**

| 模型     | 可傳遞           | 自由度 | 適用情形     | 保守程度 |
|--------|---------------|-----|----------|------|
| 無摩擦點接觸 | 法向推力          | 1   | 極光滑表面、冰面 | 最保守  |
| 有摩擦點接觸 | 法向力 + 兩個切向摩擦力 | 3   | 一般剛性指尖   | 標準假設 |
| 軟指接觸   | 上述 + 繞法向的力矩   | 4   | 橡膠指墊、人手  | 較樂觀  |

**模型三的重要性常被低估**。能抵抗扭轉，代表物體不容易在手中打轉——這正是人類用軟指墊抓握筆或螺絲起子時能穩定施力的原因。這也是本章一再強調的主題：**柔軟不只是「安全」**，它實際上增加了抓取的能力。

## 21.2.2 摩擦錐

有摩擦的接觸點所能施加的力並非任意——它必須滿足庫倫摩擦定律，否則就會滑動。設接觸法向為  $\hat{n}$ 、摩擦係數為  $\mu$ ，則不滑動的條件是：

$$\|f_t\| \leq \mu \cdot f_n \quad (\text{切向力不超過法向力的 } \mu \text{ 倍})$$

庫倫摩擦條件。 $f_n$  為法向分量（必須為正，即推而非拉）、 $f_t$  為切向分量。

滿足此條件的所有力向量，在空間中構成一個以法線為軸的**圓錐**，稱為**摩擦錐**（friction cone）。錐的半頂角為：

$$\alpha = \arctan(\mu)$$

摩擦錐的半頂角。 $\mu = 0.5$  時  $\alpha \approx 26.6^\circ$ ； $\mu = 1.0$  時  $\alpha = 45^\circ$ 。

摩擦錐是抓取分析的核心幾何物件。它的意義是：**接觸力必須落在錐內**，否則手指會滑動。因此所有的抓取分析，本質上都是在問「錐內的力能組合出什麼效果」。

圖 21-1 摩擦錐與接觸力的限制

建議配圖：左圖為單一接觸點的三維示意——物體表面、接觸法線  $\hat{n}$ 、以及以法線為軸的圓錐（半頂角  $\alpha = \arctan \mu$ ）。錐內畫一個綠色的合法力向量、錐外畫一個紅色的非法力向量並標註「會滑動」。右圖為二維簡化版，顯示兩個手指從兩側夾住物體時的兩個摩擦錐，並以陰影標出「兩錐的正線性組合所能產生的合力範圍」。下方標註  $\mu$  值增大時錐角變寬、可用力的範圍變大。

### 21.2.3 摩擦係數的不確定性

摩擦係數是抓取規劃中最不可靠的參數之一。同樣是「橡膠對塑膠」，實際的  $\mu$  可能因表面粗糙度、清潔度、濕度、接觸壓力而在 0.3 到 1.2 之間變動——變化範圍可達四倍。

這帶來一個重要的工程原則：抓取規劃時應使用保守的  $\mu$  估計值。若實際  $\mu$  大於假設值，抓取只會更穩；反之則可能失敗。實務上常取  $\mu = 0.3 \sim 0.5$  作為設計值，即使指墊材料的標稱值更高。

這也再次呼應第 14.7.3 節：模擬中的抓取常與實機不符，摩擦係數的不確定性是主因之一，而因應之道正是第 31 章的領域隨機化。

## 21.3 抓取矩陣與力封閉

### 21.3.1 抓取矩陣

要分析多個接觸點的共同效果，需要把各接觸力對物體產生的合力與合力矩表達出來。設第  $i$  個接觸點的位置為  $p_i$ （相對於物體質心）、施加的力為  $f_i$ ，則它對物體產生：

力： $f_i$       力矩： $p_i \times f_i$

單一接觸點的貢獻。力矩由力臂與力的外積決定（第 11 章的反對稱矩陣在此再次出現）。

把所有接觸點的貢獻加總，寫成矩陣形式：

$$w = G \cdot f, \quad \text{其中 } G = \begin{bmatrix} I & I & \dots & I \\ [p_1] \times & [p_2] \times & \dots & [p_n] \times \end{bmatrix}$$

抓取矩陣  $G$ 。  $w = [力; 力矩]$  是物體所受的合成扳手（wrench）、 $f$  是所有接觸力串接而成的向量。

抓取矩陣的角色，類似第 11 章的雅可比——它是一個線性映射，把「各接觸點的力」轉換為「物體所受的合力與合力矩」。兩者的對照很有啟發性：雅可比連結關節與末端，抓取矩陣連結手指與物體。

### 21.3.2 力封閉的定義

現在可以嚴格定義「抓穩了」的意思。

### ◎ 力封閉 ( Force Closure ) 的定義

一組接觸點構成功力封閉，若且唯若：對於作用在物體上的任意外力與外力矩，都存在一組滿足摩擦錐約束的接觸力，能夠將其平衡。

關鍵在於「任意」兩個字。這代表無論外力從哪個方向來、多大（在手指出力能力範圍內），抓取都能抵抗。這正是工程上「抓穩了」的嚴格意義。

數學上，力封閉的條件可以表述為：

摩擦錐內的接觸力，經抓取矩陣  $G$  映射後，其「正線性跨度」必須涵蓋整個扳手空間（三維時為  $\mathbb{R}^6$ ）

*力封閉的數學條件。*

### 21.3.3 為何是「正」線性跨度

這裡有一個容易被忽略但極重要的細節：為何是正線性跨度，而非一般的線性跨度？

原因是：手指只能推，不能拉。接觸力的法向分量必須為正（ $f_n \geq 0$ ），否則手指就離開物體了。因此在組合各接觸力時，係數只能是非負的。

一般線性跨度： $\{ \sum c_i v_i : c_i \in \mathbb{R} \}$ （係數可正可負）

正線性跨度： $\{ \sum c_i v_i : c_i \geq 0 \}$ （係數必須非負）

*兩者的差別。正線性跨度要涵蓋整個空間，需要更多的向量。*

這個限制帶來一個直觀的推論：在平面上，要用正線性組合涵蓋二維的所有方向，至少需要三個方向不共面的向量（想像三個互成 120 度的向量）。若只有兩個相反的向量，只能涵蓋一條線，無法涵蓋整個平面。這正是下一節「最少接觸點數」定理的幾何直覺。

## 21.4 最少接觸點數與抓取合成

### 21.4.1 經典定理

抓取理論中有幾個關於「最少需要幾個接觸點」的經典結果。

#### ◎ 力封閉的最少接觸點數

平面（2D）情形：以有摩擦的點接觸，至少需要 2 個接觸點（若兩摩擦錐能互相涵蓋）；以無摩擦接觸則至少需要 4 個。空間（3D）情形：以有摩擦的點接觸，至少需要 3 個接觸點；以無摩擦

接觸則至少需要 7 個。

這些數字揭示了摩擦的價值：**有摩擦時所需的接觸點數大幅減少**。這正是為何人手抓東西時，用兩三根手指就足夠——因為皮膚提供了可觀的摩擦。

### 21.4.2 二指抓取的條件

平面上兩指抓取的力封閉條件有一個優美的幾何判準：

**兩個接觸點的連線，必須同時落在兩個摩擦錐之內**

*二指力封閉條件。這稱為「連線在雙錐內」判準。*

直觀的理解是：兩指相夾時，力必定沿著兩點的連線方向傳遞。若這條連線落在兩個摩擦錐內，代表兩指都能沿此方向施力而不滑動，物體就被夾住了。若連線偏離錐外，手指會滑開。

這個判準有一個實用的推論：**摩擦係數越大，錐角越寬，可行的抓取位置就越多**。當  $\mu = 0$  時，連線必須恰好垂直於兩個接觸面（機率為零），因此無摩擦的二指抓取幾乎不可能——這與 21.4.1 節「無摩擦需要 4 點」的結論一致。

### 21.4.3 抓取合成的實務作法

由理論走向實務，「找出一組好的接觸點」稱為**抓取合成**（grasp synthesis）。傳統的解析法流程為：

1. **取得物體的幾何模型**：由 CAD、三維掃描或點雲重建（第 22 章）。
2. **採樣候選接觸點**：在物體表面均勻或依啟發式規則取樣，並計算各點的法向。
3. **組合並篩選**：列舉所有可能的接觸點組合，用力封閉條件篩掉不可行者。
4. **評分排序**：用 21.6 節的品質指標為可行的抓法評分。
5. **運動學可行性檢查**：確認機械手能以該姿態到達（逆運動學有解、不碰撞）。
6. **選定並執行**：取分數最高且可行的抓法。

這個流程在理論上完整，但實務上有三個問題：**一、組合爆炸**—— $n$  個候選點取 3 個的組合數為  $O(n^3)$ ，計算量大；**二、依賴精確的幾何模型**——而真實場景中物體常被遮蔽；**三、對摩擦係數敏感**——而該值不確定。

這三個問題正是 21.9 節學習式方法興起的原因。但在此之前，21.5 至 21.8 節要先探討另一

條同樣重要的路線：用機構的柔順性繞過這些困難。

## 21.5 形封閉與關入籠

### 21.5.1 形封閉：完全不靠摩擦

力封閉允許使用摩擦力。但摩擦係數不確定（21.2.3 節），若能設計一個完全不依賴摩擦的抓取，可靠度會高得多。這就是**形封閉**（form closure）。

#### ◎ 形封閉的定義

一組接觸點構成**形封閉**，若且唯若：即使所有接觸都是**無摩擦**的，物體仍然無法產生任何運動（包含平移與旋轉）。也就是說，物體被幾何形狀完全鎖死。

形封閉比力封閉**嚴格**得多——它是力封閉的特例（形封閉必定力封閉，反之不然）。代價是需要更多的接觸點：平面需要 4 點、空間需要 7 點（21.4.1 節的無摩擦情形）。

實務上，形封閉主要出現在**治具與夾具設計**中，而非機械手抓取。工廠的定位治具就是典型的形封閉——用數個定位銷與夾持點把工件完全鎖死，即使工件表面很滑也不會移動。這種可靠度正是量產所需要的。

表 21-3 力封閉與形封閉的比較

| 面向        | 力封閉             | 形封閉     |
|-----------|-----------------|---------|
| 是否依賴摩擦    | 是               | 否       |
| 平面最少接觸點   | 2（有摩擦）          | 4       |
| 空間最少接觸點   | 3（有摩擦）          | 7       |
| 對摩擦係數的敏感度 | 高               | 無       |
| 可靠度       | 中（依賴 $\mu$ 的估計） | 高       |
| 典型應用      | 機械手抓取           | 定位治具、夾具 |

### 21.5.2 關入籠：不抓緊也能控制

在力封閉與「完全沒抓住」之間，還有一個重要的中間狀態：**關入籠**（caging）。

#### ◎ 關入籠的定義

物體被手指**包圍**，使其無法逃離某個有限的區域，但在該區域內仍可自由移動與旋轉。物體並未被夾緊，卻也跑不掉。

一個生活化的例子：用手掌與五指圍成一個「碗」的形狀去撈一顆彈珠。彈珠在手中可以滾動，但無論怎麼滾都出不去——這就是關入籠。

關入籠的價值在於三點：

- **對位置誤差極不敏感**：不需要精確對準接觸點，只要「圍住」即可。這大幅降低了對感測與定位精度的要求。
- **不會壓壞物體**：因為沒有夾緊，接觸力可以很小甚至為零——這對易碎物（水果、蛋、玻璃）特別有價值。
- **可作為抓取的中間步驟**：先關入籠確保物體不會跑掉，再逐步收攏成為力封閉。這是一個穩健的兩階段策略。

這個「先圍住、再收緊」的策略，正是人類抓取不熟悉物體時的自然做法——也是開頭故事中小妤戴著手套摸索時所做的事。她的手掌先形成一個包圍的姿態，接觸後才收攏。

### 21.5.3 三個概念的層次關係

把三個概念放在一起，可以看出清楚的層次：

**形封閉  $\subset$  力封閉  $\subset$  關入籠  $\subset$  有接觸**

*由嚴格到寬鬆。越往右對精度的要求越低、可靠度的保證越弱。*

工程上的選擇原則是：**用最寬鬆但足夠的那一個**。若任務只是「把物體搬到旁邊」，關入籠可能就夠了；若要「精確地插入孔中」，則需要力封閉；若要「在振動的環境中固定住」，才需要形封閉。過度追求嚴格的封閉條件，會不必要地提高對感測與控制的要求。

## 21.6 抓取品質指標

### 21.6.1 為何需要量化

力封閉是一個「是或否」的判斷，但實務上我們需要比較：兩個都滿足力封閉的抓法，哪一個更好？這就需要**抓取品質指標**（grasp quality metric）。

品質指標的作用有三：在抓取合成時作為排序的依據（21.4.3 節的步驟四）、在執行前評估風險、以及作為學習式方法的訓練標籤（21.9 節）。

### 21.6.2 四個常用指標



**指標一：最小奇異值 (  $\epsilon$  指標 )**。這是最廣泛使用的指標。它衡量「在最不利的方向上，抓取能抵抗多大的外力」：

$$Q_{\epsilon} = \min_{\|w\|=1} \max_{\|f\| : Gf = w, f \text{ 在摩擦錐內}} \text{的倒數}$$

直觀地說：在扳手空間中，找出「能抵抗的外力集合」的內接球半徑。半徑越大，抓取越穩。

這個指標與第 11.8 節的可操作度有相似的結構——都是在問「最弱的方向有多弱」。若  $Q_{\epsilon} = 0$ ，代表某個方向完全無法抵抗，即不構成力封閉。

**指標二：抓取體積 ( Volume 指標 )**。取「能抵抗的外力集合」的體積，而非內接球半徑。它反映整體的能力而非最弱方向，與可操作度  $w = \sqrt{\det(JJ^T)}$  的思路一致。

**指標三：接觸點分布**。直觀的啟發式指標：接觸點越分散、越接近物體重心的對稱位置，抓取越穩。計算簡單，常用於快速篩選。

**指標四：對誤差的強健性**。刻意為接觸點加入位置擾動（如  $\pm 2 \text{ mm}$ ），檢查抓取是否仍然力封閉。通過擾動測試的比例即為強健性分數。

表 21-4 四種抓取品質指標的比較

| 指標                  | 衡量什麼     | 計算成本     | 優點          | 缺點        |
|---------------------|----------|----------|-------------|-----------|
| 最小奇異值<br>$\epsilon$ | 最弱方向的抵抗力 | 中（需解最佳化） | 有明確物理意義、最通用 | 對接觸點位置敏感  |
| 抓取體積                | 整體抵抗能力   | 中        | 反映全方向能力     | 可能掩蓋單一弱方向 |
| 接觸點分布               | 幾何對稱性    | 極低       | 快速、直觀       | 只是啟發式、不嚴謹 |
| 誤差強健性               | 對位置誤差的容忍 | 高（需多次採樣） | 最貼近實務需求     | 計算量大      |

### 21.6.3 指標與實務的落差

必須誠實指出：理論上的高品質抓取，在實機上未必成功。原因有三。

**原因一：模型與現實的落差**。品質指標建立在「物體幾何已知、摩擦係數已知、接觸點精確」的假設上，而這三者在真實場景中都不成立。

**原因二：只考慮靜力學**。多數指標只分析靜態的力平衡，未考慮抓取過程——手指接近時



可能推動物體、閉合時可能使物體旋轉。

**原因三：忽略柔順性。**指標假設剛性接觸，但真實的指墊會變形，這改變了接觸的性質（從點接觸變成面接觸），使實際的抓取往往比理論預測更穩。

因此近年的趨勢是：**用學習的方式直接由實驗資料預測成功率**，而非依賴解析的品質指標。這正是 21.9 節的主題。但理論指標仍有其價值——它們提供了物理直覺，也是產生訓練資料的重要工具。

## 21.7 手套思想實驗：柔順勝過精準

### 21.7.1 回到開頭的實驗

現在有了理論工具，可以重新分析開頭那個手套實驗。小好戴著厚手套、閉著眼睛，卻能在五秒內拿起未知的水瓶。從本章的角度看，發生了什麼？

**第一，手套增加了接觸面積。**剛性指尖與圓柱體接觸是「點接觸」，柔軟的手套則會變形貼合，形成一小片「面接觸」。依 21.2.1 節的模型，這使接觸從「有摩擦點接觸」升級為「軟指接觸」——多了抵抗扭轉的能力。

**第二，手套吸收了位置誤差。**她的手指位置必定不精確（閉著眼睛），但柔軟的材料可以壓縮數公釐而不產生過大的力——這正是第 20 章阻抗控制的機構版本。剛性手指若位置差 3 公釐，不是碰不到就是把瓶子推倒。

**第三，手套的變形自動增加了穩定性。**當瓶子開始下滑時，接觸面會因剪切而變形，這增加了接觸面積與正向壓力，摩擦力隨之上升——這是一個**自動的負回授**，不需要任何感測或計算。

**第四，多指的自適應包絡。**五指收攏時，先碰到的手指會停止、後碰到的繼續前進，直到全部貼合。這使接觸點自動分布在物體周圍，形成良好的力封閉——而這完全不需要事先知道物體的形狀。

#### ◎ 本章的核心洞見

在抓取中，柔順性不只是「安全考量」，它實際上取代了大量的感測與計算。一個柔軟的、能自適應的手，用極少的資訊就能完成剛性手需要精確建模才能達成的任務。這與第 20 章的結論一脈相承：與其消除不確定性，不如設計一個對不確定性不敏感的系統。工程上，這意味著**機構設計與控制策略必須協同考慮**——把問題丟給控制器解決之前，先問：能不能用機構把問題變簡單？

## 21.7.2 人手的生物力學啟示

人手是這個原則的極致體現，有四個值得工程借鏡的設計。

**設計一：分層的柔軟度。**指尖的皮膚最軟、皮下組織次之、骨頭最硬。這個梯度使指尖能大範圍貼合，同時保有施力的能力。

**設計二：筋腱驅動。**肌肉在前臂，透過肌腱驅動手指（第 5.9.3 節）。這使手指本身極輕，慣量小、反應快、且撞擊時不會造成傷害。

**設計三：欠驅動的耦合。**人手有 27 個自由度，但真正能獨立控制的遠少於此——多數手指的關節是耦合運動的。這大幅簡化了控制，卻不太影響抓取能力。

**設計四：豐富的觸覺。**指尖的機械受器密度極高，能偵測壓力、振動與微小滑動。滑動偵測尤其關鍵——它讓人能在物體開始滑落的瞬間自動加大握力，這個反射的延遲只有約 70 毫秒。

這四項設計中，前三項是**機構**的、只有第四項是**感測**的。這個比例本身就說明了：人手的靈巧，主要來自機構而非感知。

## 21.8 軟性夾爪與欠驅動手

### 21.8.1 三種設計哲學

機械手的設計可以分為三種路線，各自代表不同的哲學。

**路線一：剛性多指手。**模仿人手的形狀與自由度，每個關節獨立驅動，配備力覺與觸覺感測。代表如 Shadow Hand、DLR Hand。優點是理論上能完成任何人手能做的操作；缺點是極其昂貴（數百萬元）、控制複雜、且可靠度低（自由度高、故障點多）。

**路線二：欠驅動手。**自由度高但驅動器少，透過機構的耦合（連桿、彈簧或筋腱）讓手指自動適應物體形狀。代表如 Robotiq 的自適應夾爪、Barrett Hand。優點是成本適中、控制極簡（往往只需一個「閉合」指令）、且抓取穩健；缺點是無法做精細的手內操作（in-hand manipulation）。

**路線三：軟性夾爪。**以矽膠、氣動或柔性材料製成，沒有明確的關節。代表如氣動的軟指（Soft Robotics 公司）、真空夾爪、以及顆粒堵塞式夾爪（jamming gripper）。優點是極其安全、能抓取形狀不規則或易碎的物體、成本低；缺點是出力小、精度低、且難以建模與控制。

表 21-5 三種機械手設計的比較

| 面向     | 剛性多指手       | 欠驅動手       | 軟性夾爪      |
|--------|-------------|------------|-----------|
| 自由度    | 多 ( 15~24 ) | 多 ( 但驅動少 ) | 連續變形      |
| 驅動器數   | 等於自由度       | 1~4        | 1~3       |
| 控制複雜度  | 極高          | 低          | 極低        |
| 適應未知形狀 | 需精確建模       | 機構自動適應     | 天然適應      |
| 可做手內操作 | 可           | 有限         | 幾乎不可      |
| 出力     | 中           | 大          | 小         |
| 成本     | 極高          | 中          | 低         |
| 典型應用   | 研究、遙操作      | 工業取放、物流分揀  | 食品、生鮮、易碎品 |

### 21.8.2 欠驅動的機構智慧

欠驅動手的核心巧思，在於用機構實現「自動分配」。以最常見的連桿式欠驅動手指為例：

手指有兩個關節但只有一個驅動器，兩者之間以連桿與彈簧連接。當手指閉合時：若近端指節未碰到物體，兩個關節一起彎曲（像剛體一樣）；若近端指節碰到物體而停住，驅動力會自動轉移到遠端指節，使其繼續彎曲直到也接觸物體。

結果是：手指自動包絡物體的輪廓，無論物體是什麼形狀。這個行為完全由機構產生，不需要任何感測器、不需要知道物體形狀、也不需要任何計算。

這是「機構智慧」（mechanical intelligence）的典範——把原本需要感測與計算才能達成的功能，用機構的巧妙設計直接實現。它的可靠度遠高於任何軟體方案，因為沒有感測器會壞、沒有程式會當。

圖 21-2 欠驅動手指的自適應包絡

建議配圖：四格連續動作圖，顯示一根雙關節欠驅動手指抓取一個不規則物體的過程。第一格：手指張開接近物體。第二格：近端指節接觸物體並停止。第三格：驅動力自動轉移，遠端指節繼續彎曲。第四格：兩指節都貼合物體表面，形成包絡抓取。每格旁標註驅動器的施力路徑（以紅色箭頭表示力如何在連桿間轉移），並在下方對照一根剛性手指抓同樣物體時只能形成單點接觸的情形。

### 21.8.3 真空與吸附

在工業實務中，還有一類極為重要但常被學術界忽略的方案：真空吸盤。

它的優勢在特定場合極為突出：只需要一個接觸面（不需要環繞物體，因此可以從貨箱中取

出緊密排列的物品)、吸附力大且穩定、成本極低、動作極快。物流業的包裹分揀、電子業的面板搬運、食品業的包裝，大量使用真空方案。

限制則是：物體表面必須平滑且不透氣（多孔材料如布料、粗糙的紙箱難以吸附）、且無法抵抗大的剪切力與扭矩。

**工程上的啟示：**面對一個抓取任務，應先問「能不能用吸盤？」而非直接考慮多指手。這是「用最簡單的方案解決問題」的工程原則——與 21.5.3 節「用最寬鬆但足夠的封閉條件」是同一種思維。

#### ◎ 顆粒堵塞夾爪：一個優雅的巧思

有一種軟性夾爪的設計極具啟發性：一個裝滿咖啡渣（或其他細顆粒）的彈性袋。使用時把袋子壓在物體上，顆粒會流動並包覆物體的輪廓；接著抽真空，顆粒之間互相擠壓而「堵塞」，整個袋子瞬間變得堅硬，牢牢鎖住物體的形狀。放開時只需通氣，袋子又變回柔軟。這個設計的巧妙之處在於：它在同一個裝置中同時實現了「柔軟以適應形狀」與「堅硬以牢固抓持」兩個矛盾的需求，而切換只需要一個真空閥。它能抓取幾乎任何形狀的物體，且完全不需要知道物體是什麼——這是機構智慧的又一個典範。

## 21.9 學習式抓取

### 21.9.1 為何轉向學習

21.4.3 節列出了解析式抓取合成的三個困難：組合爆炸、依賴精確幾何模型、對摩擦係數敏感。而真實場景中還有更根本的問題——**物體常常是沒見過的**。

倉儲分揀是最典型的例子：一個電商倉庫可能有數十萬種商品，不可能為每一種都建立 CAD 模型。傳統方法在此完全失效。

學習式方法的思路是繞過建模：**不去理解物體是什麼，而是直接學習「哪裡看起來可以抓」**。這與人類的直覺相符——我們看到一個陌生物體時，並不會先在腦中重建它的三維模型再計算力封閉，而是「一眼就知道該抓哪裡」。

### 21.9.2 基本流程

典型的學習式抓取系統包含四個步驟：

1. **感測：**以 RGB-D 相機取得場景的彩色影像與深度圖（第 6.7 節、第 22 章）。

2. **抓取姿勢偵測**：以神經網路預測一組候選的抓取位姿（夾爪的位置、方向與開度），並為每個候選給出成功率的預測分數。
3. **篩選與排序**：過濾掉運動學不可達或會碰撞的候選，取分數最高者。
4. **執行與回饋**：執行抓取，並記錄成功或失敗——這個結果可以回饋作為新的訓練資料。

其中步驟二是核心。常見的網路輸出形式有兩種：**離散式**（把影像切成網格，為每個位置與角度預測分數）與**生成式**（直接回歸出抓取位姿的參數）。

### 21.9.3 訓練資料從哪裡來

學習需要大量的標註資料，而「這個抓法會不會成功」的標註成本極高。實務上有三種來源：

**來源一：模擬產生**。在物理模擬器（第 14.8 節）中生成大量物體與抓取嘗試，自動標註成功與否。優點是資料量可以極大（數百萬筆）、完全自動化；缺點是 Sim-to-Real Gap——模擬中的接觸與摩擦不夠真實（第 14.7.3 節）。因應之道是**領域隨機化**：在模擬中隨機改變摩擦係數、質量、物體尺寸與光照，強迫網路學到對這些變異不敏感的特徵。

**來源二：解析式標註**。用 21.6 節的品質指標（如  $\epsilon$  指標）自動為模擬中的抓取評分，作為訓練標籤。這結合了理論與學習的優勢——用理論產生資料，用學習達成泛化。

**來源三：真實機器人自我監督**。讓機器人不斷嘗試抓取，用「抓起來了沒有」作為標籤。優點是資料完全真實；缺點是速度慢（每次嘗試需數秒）、且需要自動化的重置機制。曾有研究讓多台機器人並行運轉數月，收集數十萬次嘗試。

### 21.9.4 優勢與限制

**優勢一：能處理未知物體**。這是相對於解析法最大的突破。網路學到的是「什麼樣的局部幾何適合夾持」這種通用特徵，因此能泛化到訓練時沒見過的物體。

**優勢二：不需完整的三維模型**。可以直接由單一視角的深度圖預測，不需重建完整幾何——這在物體互相遮蔽的雜亂場景中尤其重要。

**優勢三：推論極快**。一次前向傳播即可（數十毫秒），遠快於解析法的組合搜尋。

**限制一：缺乏保證**。網路給出的是「預測的成功率」，沒有力封閉那樣的數學保證。在安全關鍵的場合，這是嚴重的缺點。

**限制二：難以處理特殊需求。**若任務要求「必須從特定方向抓」或「抓取後要能施加特定的力」，純粹的成功率預測無法表達這些約束。

**限制三：對訓練分布外的物體仍會失敗。**若遇到訓練資料中完全沒有的形態（如極細長、極柔軟、透明物體），性能會明顯下降。透明與反光物體尤其困難，因為深度相機在這些表面上根本量不到正確的距離（第 6.7 節）。

**表 21-6 解析式與學習式抓取的比較**

| 面向     | 解析式（力封閉分析） | 學習式（抓取姿勢偵測） |
|--------|------------|-------------|
| 需要物體模型 | 需要精確幾何     | 不需要         |
| 處理未知物體 | 困難         | 擅長          |
| 計算時間   | 秒級（組合搜尋）   | 毫秒級（一次推論）   |
| 理論保證   | 有（力封閉條件）   | 無（僅機率預測）    |
| 對摩擦係數  | 敏感         | 隱含在資料中      |
| 處理雜亂場景 | 困難         | 擅長          |
| 可解釋性   | 高          | 低           |
| 典型應用   | 已知工件的產線取放  | 倉儲分揀、服務型機器人 |

## 21.9.5 混合式方法

實務上最有效的往往是兩者結合：**用學習產生候選、用解析驗證品質。**

具體流程是：神經網路快速給出數十個候選抓取位姿（利用其泛化能力與速度），再對排名前幾名的候選，用 21.6 節的品質指標做嚴格驗證（利用理論的可靠性），最後執行通過驗證且運動學可行者。

這個組合兼顧了泛化能力與安全保證，是目前工業應用的主流架構。它也體現了本書一再出現的原則——**模型與學習不是互相取代，而是各自處理擅長的部分**（呼應第 17 章的「模型處理已知、回授處理未知」）。

## 21.10 由抓取到操作

### 21.10.1 抓取只是開始

本章大部分內容討論的是**抓取（grasping）**——把物體拿起來。但真正的任務往往需要操



作 ( manipulation ) ——拿起來之後，還要對它做些什麼：插入、旋轉、擺放、組裝、使用工具。

兩者的難度差距很大。抓取是一個「靜態」的問題 ( 找到穩定的接觸配置 )，操作則是「動態」的 ( 在維持抓握的同時控制物體的運動與受力 )。

### 21.10.2 三種操作方式

**方式一：手臂操作 ( Arm Manipulation )**。抓穩後，物體與手固連，由手臂的運動帶動物體。這是最簡單也最常用的方式——本書第 9 ~ 20 章的所有技術都直接適用，因為「手加物體」可視為一個新的剛體。

**方式二：手內操作 ( In-hand Manipulation )**。在不放開的前提下，用手指改變物體在手中的姿態——例如把一支筆在手中轉個方向。這極其困難，需要多指手、精確的觸覺、以及對滾動與滑動接觸的建模。人類做這件事毫不費力，機器人至今仍相當笨拙。

**方式三：非抓取式操作 ( Non-prehensile Manipulation )**。不抓起物體，而是用推、撥、翻、滾等方式改變其位置——例如把桌上的書推到桌邊再翻起來。這在物體太大、太重或無法抓取時特別有用，也常作為抓取前的「預處理」 ( 把物體推到方便抓的位置 )。

### 21.10.3 操作中的力控制

操作階段的力控制，正是第 20 章技術的直接應用。以「把抓著的插銷插入孔中」為例：

- **軸向**：以力控制施加推力 ( 第 20.3 節 )。
- **橫向**：以低剛度的阻抗控制允許順應對位 ( 第 20.5 節 )。
- **手指握力**：必須足夠大以防滑落，但不能大到壓壞物體——這本身就是一個力控制問題。
- **滑動偵測**：若觸覺感測到物體開始滑動，立即增加握力 ( 模仿人類的 70 毫秒反射 )。

這說明了第 20 章與本章的關係：第 20 章處理「手與環境」的接觸，本章處理「手與物體」的接觸，而操作任務中兩者同時發生。這也是抓取與操作特別困難的原因——它是一個多重接觸、多重約束的耦合問題。

## 21.11 第三篇的總結

### 21.11.1 九章的軌跡

第 21 章至此結束，第三篇「動力學、軌跡與控制」的九章也全部完成。回顧這條路徑：

第 13、14 章（動力學）建立了「力與運動」的數學模型。前者用拉格朗日法讓我們看懂方程的結構，後者用 Newton-Euler 遞迴讓我們算得夠快。

第 15 章（軌跡生成）決定「該走哪條路」，並首次把動力學模型用於檢驗軌跡的可行性。

第 16、17 章（控制）讓機器人真的走上那條路。前者建立回授的基礎與極限，後者用模型前饋突破 PID 的限制——核心哲學是「用模型處理已知、用回授處理未知」。

第 18、19 章（最佳性與穩定性）回答了兩個更深的問題：參數該怎麼選才是最好的（LQR 與 Riccati 方程），以及我們憑什麼相信它一定有效（Lyapunov 方法）。

第 20、21 章（接觸與抓取）讓機器人走出自由空間，學會與環境互動。核心洞見是：與其消除不確定性，不如設計一個對不確定性不敏感的系統——柔順性不只是安全考量，它實際上取代了大量的感測與計算。

### 21.11.2 三條貫穿的線索

回顧九章，有三條線索貫穿始終。

**線索一：模型的價值不在完美，而在減負。**第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍；第 18 章的 LQR 需要線性模型，卻能為多變數系統一次算出所有增益。反覆出現的訊息是：不完美的模型仍然極有價值。

**線索二：理論的各部分互相支撐。**第 13 章那個「看似無用」的  $\dot{\mathbf{M}} - 2\mathbf{C}$  斜對稱性，在第 19 章成為證明的樞紐；第 11 章的雅可比對偶，在第 17 章成為操作空間控制的核心、在第 20 章成為力映射的基礎、在本章成為抓取矩陣的原型。

**線索三：機構與控制必須協同設計。**第 15 章的軌跡受限於馬達能力、第 17 章的協作機器人需要低減速比、第 20 章的插銷入孔靠一個倒角就把容差放寬 41 倍、本章的欠驅動手用機構取代了感測與計算。反覆出現的教訓是：在把問題丟給控制器之前，先問能不能用機構把問題變簡單。

### 21.11.3 接下來：讓機器人走出實驗室

第三篇的所有技術，都建立在一個前提上：機器人知道自己在哪裡、知道目標在哪裡。固定



基座的手臂靠編碼器就能知道自己的姿態，工件的位置則由治具保證。

但一旦機器人要移動、要面對未知的環境，這個前提就不成立了。第四篇「感知、定位與自主移動」（第 22 ~ 28 章）將處理這些問題：如何用視覺理解環境（第 22 章）、行動機器人如何運動（第 23 章）、如何在充滿不確定性的世界中估計自己的狀態（第 24 章）、如何一邊移動一邊建立地圖（第 25 章）、如何規劃避開障礙的路徑（第 26 章）、以及無人機（第 27 章）與腿式機器人（第 28 章）這兩種特殊的移動形態。

屆時你會發現，第三篇的工具並未被拋棄——控制理論仍然是無人機姿態穩定與四足步態的基礎，而動力學模型則是它們規劃運動的依據。只是問題的重心，從「精確地執行」轉向了「正確地理解」。

## 完整計算例題

### 例題 21-1 摩擦錐與二指抓取判定

題目：一個長方體工件被兩指從相對的兩面夾持。指墊與工件之間的摩擦係數  $\mu = 0.4$ 。（a）求摩擦錐的半頂角。（b）若兩接觸點的連線與接觸面法線夾角為  $15^\circ$ ，判斷是否構成功率封閉。（c）若夾角為  $30^\circ$ ，重新判斷。（d）若要容許  $30^\circ$  的夾角，摩擦係數至少需要多少？

解（a）：由 21.2.2 節的公式：

$$\alpha = \arctan(\mu) = \arctan(0.4) = 21.8^\circ$$

摩擦錐的半頂角。

解（b）：依 21.4.2 節的判準，兩接觸點的連線必須落在兩個摩擦錐之內。連線與法線的夾角為  $15^\circ$ ，而錐的半頂角為  $21.8^\circ$ ：

$$15^\circ < 21.8^\circ \Rightarrow \text{連線落在錐內} \Rightarrow \text{構成功率封閉} \checkmark$$

抓取穩固。

解（c）：夾角  $30^\circ$  時：

$$30^\circ > 21.8^\circ \Rightarrow \text{連線落在錐外} \Rightarrow \text{不構成功率封閉} \times, \text{手指會滑動}$$

抓取失敗。

解（d）：要容許  $30^\circ$  的夾角，需要  $\alpha \geq 30^\circ$ ：

$$\mu \geq \tan(30^\circ) = 0.577$$

所需的最小摩擦係數。

**工程判讀：**一般橡膠指墊對塑膠的摩擦係數約在 0.5 ~ 0.8 之間，因此 0.577 是可以達成的。但考慮到 21.2.3 節提到的摩擦不確定性（表面可能有油污或灰塵使  $\mu$  下降），設計時應留有餘裕——實務上會建議改變夾持位置，使夾角降到  $15^\circ$  以下，或改用有齒紋的指墊以提高  $\mu$ 。

### 例題 21-2 平面三指抓取的力封閉分析

題目：一個圓形物體（半徑  $r = 50 \text{ mm}$ ）被三根手指抓持，接觸點分別位於圓周上  $0^\circ$ 、 $120^\circ$ 、 $240^\circ$  的位置。摩擦係數  $\mu = 0.5$ 。（a）求各接觸點的法線方向與摩擦錐。（b）判斷此抓取能否抵抗任意方向的**平移**外力。（c）判斷能否抵抗**旋轉**外力（力矩）。（d）若把三個接觸點改為  $0^\circ$ 、 $30^\circ$ 、 $60^\circ$ ，重新分析。

解（a）：圓形物體的法線沿半徑方向指向圓心。三個接觸點的法線方向分別為  $180^\circ$ 、 $300^\circ$ 、 $60^\circ$ （即指向圓心）。摩擦錐半頂角為  $\arctan(0.5) = 26.6^\circ$ ，因此各接觸點可施加的力方向範圍為：

接觸點 1 ( $0^\circ$ )：力方向  $180^\circ \pm 26.6^\circ$ ，即  $153.4^\circ \sim 206.6^\circ$

接觸點 2 ( $120^\circ$ )：力方向  $300^\circ \pm 26.6^\circ$ ，即  $273.4^\circ \sim 326.6^\circ$

接觸點 3 ( $240^\circ$ )：力方向  $60^\circ \pm 26.6^\circ$ ，即  $33.4^\circ \sim 86.6^\circ$

三個摩擦錐涵蓋的方向範圍。

解（b）：三個錐的中心方向分別為  $180^\circ$ 、 $300^\circ$ 、 $60^\circ$ ，兩兩相差  $120^\circ$ 。由 21.3.3 節的正線性跨度概念：三個互成  $120^\circ$  的向量，其正線性組合可以涵蓋整個平面。

直觀的驗證：任取一個方向（如  $0^\circ$ ），可以由  $300^\circ$  與  $60^\circ$  兩個方向的正組合產生（兩者的角平分線正是  $0^\circ$ ）。因此能抵抗任意方向的**平移**外力。

解（c）：對於力矩，需要考慮切向力。由於每個接觸點的力可以偏離法線  $\pm 26.6^\circ$ ，其切向分量為：

$$f_t = f_n \cdot \tan(26.6^\circ) = 0.5 f_n$$

最大切向分量。

此切向力對圓心產生的力矩為：

$$M = f_t \times r = 0.5 f_n \times 0.05 = 0.025 f_n \text{ (N}\cdot\text{m, 若 } f_n \text{ 以 N 為單位)}$$

單一接觸點的最大力矩貢獻。三指同向施加切向力時可疊加，故總力矩為  $0.075 f_n$ 。

由於三指可以同時施加順時針或逆時針的切向力，**能抵抗兩個方向的力矩**。因此此抓取構成完整的力封閉。

解 (d)：若三點集中在  $0^\circ$ 、 $30^\circ$ 、 $60^\circ$ ，則三個法線方向為  $180^\circ$ 、 $210^\circ$ 、 $240^\circ$ ——**全部落在同一個半平面內**。

由正線性跨度的性質，這三個方向的非負組合只能產生「指向左方的力」，**無法產生指向右方 ( $0^\circ$  方向) 的力**。因此物體會被往右推出去，**不構成力封閉**。

**核心結論**：接觸點的分布比數量更重要。三個聚集的接觸點不如兩個分散的接觸點。這正是 21.6.2 節「接觸點分布」作為品質指標的依據，也解釋了為何人手抓握時會自然地把拇指與其他手指分置於物體兩側。

### 例題 21-3 柔順性的效益量化

題目：某夾爪抓取一個直徑 40 mm 的圓柱體。夾爪的定位誤差為  $\pm 2$  mm。(a) 若指墊為剛性（等效剛度  $10^5$  N/m），求位置誤差造成的夾持力變化範圍（設標稱夾持力為 20 N）。(b) 若改用柔性指墊（剛度 2000 N/m），重新計算。(c) 比較兩者，並說明柔性指墊如何提升抓取的成功率。(d) 若物體是易碎品，容許的最大夾持力為 30 N，判斷兩種指墊是否可行。

解 (a)：剛性指墊時，位置誤差直接轉換為變形量，進而產生力的變化：

$$\Delta F = K \times \Delta x = 10^5 \text{ N/m} \times 0.002 \text{ m} = 200 \text{ N}$$

位置誤差  $\pm 2$  mm 造成的力變化。

因此實際夾持力範圍為  $20 \pm 200$  N。負值代表夾不到（誤差往外 2 mm 時，指墊根本沒接觸）；正值 220 N 則遠超標稱值。

**這個結果說明剛性指墊在此誤差下根本不可用——不是抓不到，就是夾壞。**

解 (b)：柔性指墊時：

$$\Delta F = 2000 \text{ N/m} \times 0.002 \text{ m} = 4 \text{ N}$$

力的變化僅 4 N。

實際夾持力範圍為  $20 \pm 4 = 16 \sim 24$  N。這是一個完全可接受的範圍。

解 (c)：兩者的力變化相差 50 倍。柔性指墊透過三個機制提升成功率：

- **吸收位置誤差**： $\pm 2\text{ mm}$  的誤差只造成  $\pm 4\text{ N}$  的力變化，遠在可接受範圍內。
- **增加接觸面積**：柔性材料變形後形成面接觸，使接觸模型由「點接觸」升級為「軟指接觸」（21.2.1 節），增加抵抗扭轉的能力。
- **提高等效摩擦**：面接觸的實際摩擦力大於理論的點接觸值，因為材料的微觀嵌合效應。

解 (d)：容許最大力 30 N 時：

**剛性指墊**：實際力最高可達 220 N，遠超 30 N 的上限。**不可行**——物體必定損壞。

**柔性指墊**：實際力最高  $24\text{ N} < 30\text{ N}$ 。**可行**，且還有 6 N 的餘裕。

**工程結論**：這個例題量化了本章的核心洞見。要讓剛性夾爪可行，必須把定位精度提升到  $\pm 0.05\text{ mm}$  以下（使  $\Delta F < 5\text{ N}$ ）——這需要極昂貴的機構與感測。而換一個成本數百元的柔性指墊，就直接解決了問題。**這正是「用機構把問題變簡單」的最佳例證**，也與第 20 章例題 20-3 中「一個倒角把容差放寬 41 倍」是同一種思維。

## 程式範例：摩擦錐與力封閉的計算

以下 Python 程式實作平面抓取的力封閉判定：計算摩擦錐、建構抓取矩陣、以正線性跨度判斷是否力封閉，並計算抓取品質指標。

```
import numpy as np
from itertools import product

# ===== 平面抓取分析 =====
class PlanarGrasp:
    """平面 (2D) 抓取分析：接觸點在物體邊界上"""
    def __init__(s, mu=0.5):
        s.mu = mu
        s.alpha = np.arctan(mu)          # 摩擦錐半頂角
        s.contacts = []                  # [(位置, 法線方向角), ...]

    def add_contact(s, pos, normal_angle_deg):
        """pos: (x, y) 相對物體質心; normal_angle: 法線指向物體內部的角度"""
        s.contacts.append((np.array(pos, float), np.deg2rad(normal_angle_deg)))

    def cone_edges(s):
        """回傳所有摩擦錐的邊界方向 (每個接觸點兩條邊)"""
        edges = []
        for pos, n_ang in s.contacts:
```

```

    for sign in (-1, 1):
        ang = n_ang + sign*s.alpha
        edges.append((pos, np.array([np.cos(ang), np.sin(ang)])))
    return edges

def grasp_matrix(s):
    """建構抓取矩陣 G：把各錐邊的單位力映射為扳手 [Fx, Fy, Mz]"""
    cols = []
    for pos, d in s.cone_edges():
        fx, fy = d
        mz = pos[0]*fy - pos[1]*fx      # 二維外積 = 力矩
        cols.append([fx, fy, mz])
    return np.array(cols).T            # 3 x (2n)

def is_force_closure(s, n_samples=360):
    """
    力封閉判定：檢查錐邊的正線性組合能否涵蓋整個扳手空間。
    方法：對大量方向取樣，檢查每個方向是否都能被正組合達成。
    """
    G = s.grasp_matrix()
    if G.shape[1] < 3:
        return False, 0.0
    # 必要條件：原點必須在錐邊所張凸包的內部
    # 以線性規劃的對偶形式檢查：不存在一個方向 w 使所有 G 的行都在其負半空間
    min_margin = np.inf
    for i in range(n_samples):
        th = 2*np.pi*i/n_samples
        for mz_dir in (-1., 0., 1.):
            w = np.array([np.cos(th), np.sin(th), mz_dir*0.05])
            w = w/np.linalg.norm(w)
            # 檢查是否有任一錐邊在此方向上有正投影
            proj = G.T @ w
            min_margin = min(min_margin, proj.max())
    return min_margin > 1e-9, max(min_margin, 0.0)

def report(s, name):
    fc, q = s.is_force_closure()
    print(f"{name}")
    print(f"  摩擦係數  $\mu$  = {s.mu:.2f} · 摩擦錐半頂角 = "
          f"{np.rad2deg(s.alpha):.1f}°")
    print(f"  接觸點數 = {len(s.contacts)}")
    print(f"  力封閉判定 = {'✓ 成立' if fc else 'X 不成立'}")
    print(f"  品質指標  $\approx$  {q:.4f}")
    return fc, q

# ===== 例題 21-1：二指抓取 =====
print("=== 例題 21-1：二指抓取的夾角分析 ===")
mu = 0.4
print(f" $\mu$  = {mu} → 摩擦錐半頂角 = {np.rad2deg(np.arctan(mu)):.1f}°\n")
print(f"{'連線與法線夾角':>16}{'判定':>10}")
for ang in (0, 10, 15, 21, 25, 30):
    ok = ang < np.rad2deg(np.arctan(mu))

```

```

    print(f"{ang:14d}°{'✓ 力封閉' if ok else 'X 會滑動':>12}")
mu_need = np.tan(np.deg2rad(30))
print(f"\n 要容許 30° 夾角 · 需  $\mu \geq \tan(30^\circ) = \{mu\_need:.3f\}$ ")

# ===== 例題 21-2：三指抓取圓形物體 =====
print("\n=== 例題 21-2：三指抓取圓形物體 ( r = 50 mm ) ===")
r = 0.05

# (b)(c) 三指均勻分布 0°, 120°, 240°
g1 = PlanarGrasp(mu=0.5)
for a in (0, 120, 240):
    th = np.deg2rad(a)
    pos = (r*np.cos(th), r*np.sin(th))
    g1.add_contact(pos, a + 180)      # 法線指向圓心
g1.report("三指均勻分布 (0°, 120°, 240°)")

# (d) 三指聚集 0°, 30°, 60°
g2 = PlanarGrasp(mu=0.5)
for a in (0, 30, 60):
    th = np.deg2rad(a)
    pos = (r*np.cos(th), r*np.sin(th))
    g2.add_contact(pos, a + 180)
g2.report("三指聚集 (0°, 30°, 60°)")
print("> 接觸點的分布比數量更重要")

# ===== 接觸點分布對品質的影響 =====
print("\n=== 接觸點分布角度對抓取品質的影響 ( 三指、 $\mu=0.5$  ) ===")
print(f"'分布跨度':>10}{'接觸點角度':>22}{'力封閉':>10}{'品質':>10}")
for span in (60, 120, 180, 240, 300, 360):
    g = PlanarGrasp(mu=0.5)
    angles = [i*span/3 for i in range(3)]
    for a in angles:
        th = np.deg2rad(a)
        g.add_contact((r*np.cos(th), r*np.sin(th)), a + 180)
    fc, q = g.is_force_closure()
    ang_str = ", ".join(f"{a:.0f}°" for a in angles)
    print(f"{span:9d}°{'ang_str':>22}{'✓' if fc else 'X':>9}{q:10.4f}")
print("> 接觸點越分散 ( 跨度接近 360° ) · 品質越高")

# ===== 摩擦係數的影響 =====
print("\n=== 摩擦係數對二指抓取的影響 ( 夾角固定 20° ) ===")
print(f"{'μ':>8}{'錐角':>10}{'判定':>12}")
for mu_t in (0.1, 0.2, 0.3, 0.36, 0.4, 0.6, 1.0):
    a = np.rad2deg(np.arctan(mu_t))
    print(f"{mu_t:8.2f}{a:9.1f}°{'✓ 力封閉' if a > 20 else 'X 會滑動':>14}")

# ===== 例題 21-3：柔順性的效益 =====
print("\n=== 例題 21-3：指墊剛度對夾持力的影響 ===")
F_nominal, pos_err, F_limit = 20.0, 0.002, 30.0

```

```

print(f"標稱夾持力 = {F_nominal} N · 定位誤差 = ±{pos_err*1000:.0f} mm · "
      f"容許上限 = {F_limit} N\n")
print(f"{'指墊類型':<16}{'剛度 (N/m)':>14}{'力變化':>12}{'力範圍':>18}{'可行?':>8}")
for name, K in (("剛性金屬", 1e5), ("硬塑膠", 2e4),
               ("橡膠墊", 2000.), ("軟矽膠", 500.)):
    dF = K*pos_err
    lo, hi = F_nominal - dF, F_nominal + dF
    ok = (hi <= F_limit) and (lo > 0)
    print(f"{name:<16}{K:14.0f}{dF:10.1f} N"
          f"{lo:9.1f} ~{hi:6.1f} N{'✓' if ok else 'X':>7}")
print("→ 柔性指墊把力的變化縮小 50 倍，使原本不可行的任務變為可行")

# 反推：剛性指墊需要多好的定位精度？
K_rigid = 1e5
err_needed = (F_limit - F_nominal)/K_rigid
print(f"\n 若堅持用剛性指墊 (K = {K_rigid:.0e} N/m) : ")
print(f" 定位精度需達 ±{err_needed*1000:.3f} mm 才能滿足力的上限")
print(f" 這比目前的 ±{pos_err*1000:.0f} mm 嚴格 "
      f"{pos_err/err_needed:.0f} 倍 — 機構成本將大幅上升")

```

請試著修改：（一）把二指抓取的摩擦係數設為 0.1（如濕滑表面），觀察可容許的夾角縮小到多少。（二）為三指抓取加入第四指，觀察品質指標的提升幅度是否顯著（你會發現邊際效益遞減）。（三）把接觸點的位置加入  $\pm 5^\circ$  的隨機擾動，重複計算一百次，統計力封閉成立的比例——這就是 21.6.2 節的「誤差強健性」指標。

## 實作：抓取實驗與柔順性驗證

本章實作以實體抓取實驗為主，重點在於**親手驗證柔順性的價值**。所需器材：任何具備夾爪的機械手臂（或第 5 章的伺服夾爪自製）、不同硬度的指墊材料（金屬、硬塑膠、橡膠、矽膠）、以及一組測試物體。

- 測試物體準備：**準備至少八種物體，涵蓋不同的形狀（圓柱、方塊、球、不規則）、材質（金屬、塑膠、布、海綿）與重量。另準備一顆生雞蛋作為易碎品的極端測試（可先用水煮蛋練習）。
- 摩擦係數量測：**用斜面法量測各種指墊材料與各種物體之間的摩擦係數——把物體放在指墊材料上，緩緩傾斜直到滑動，記錄臨界角度  $\theta$ ，則  $\mu = \tan \theta$ 。製作一張材料配對的摩擦係數表。
- 摩擦錐驗證：**依量得的  $\mu$  計算摩擦錐半頂角，並用例題 21-1 的判準預測「哪些夾持角度會

成功」。實際測試並比對預測的準確度。

4. **指墊硬度比較**：用同一支夾爪，分別裝上四種硬度的指墊，各抓取所有測試物體十次。記錄成功率、並用彈簧秤量測夾持力。製作「指墊硬度 vs 成功率」的圖表。
5. **位置誤差容忍度測試**：刻意讓夾爪的目標位置偏離物體中心（0、1、2、3、5 mm），對各種指墊測試成功率。這是例題 21-3 的實驗版本——你應該會看到柔性指墊的容忍範圍明顯較大。
6. **手套實驗的重現**：讓同學戴上厚手套並蒙眼，嘗試抓取各種物體，記錄成功率與所需時間。再改為戴薄手套或不戴手套（但仍蒙眼）重複。討論結果是否符合本章的預期。
7. **接觸點分布實驗**：若使用三指以上的夾爪，改變手指的分布角度（聚集 vs 分散），測試抓取穩定性。用手施加不同方向的外力，記錄物體開始滑動時的力量大小——這是力封閉品質的實測。
8. **易碎品挑戰**：以生雞蛋為目標，測試各種指墊與夾持力設定。記錄「能成功抓起且不破」的參數範圍寬度——這個範圍越寬，代表系統越穩健。
9. **（進階）欠驅動手指製作**：用紙板、橡皮筋與棉線製作一根雙關節的欠驅動手指（21.8.2 節），驗證其自適應包絡的行為。與剛性手指抓取不規則物體的效果比較。

表 21-7 抓取實驗記錄表（第一列為範例）

| 測試項目       | 指墊材料              | 物體           | 設定      | 成功率   | 觀察       |
|------------|-------------------|--------------|---------|-------|----------|
| 位置誤差容忍     | 橡膠 ( $\mu=0.62$ ) | 圓柱 $\phi 40$ | 偏移 2 mm | 10/10 | 指墊變形吸收誤差 |
| （自行填寫其餘測試） |                   |              |         |       |          |

## 國中小也看得懂：抓東西「軟」比「準」更重要

來玩一個遊戲：戴上一副厚厚的手套，然後閉上眼睛，請旁邊的人在桌上放一樣東西，你去把它拿起來。

你會發現——居然拿得起來！明明你看不見、也摸不清楚，卻還是成功了。為什麼？

秘密就在那副**厚手套**。你可能以為手套讓事情變難了（確實，你摸不清楚形狀），但它同時也幫了你三個大忙：

**第一，手套很軟，可以貼合。** 不管東西是圓的、方的還是奇形怪狀，軟軟的手套都會壓扁



去配合它，變成一大片接觸面。接觸面越大，就抓得越牢。

**第二，手套可以吸收錯誤。** 你的手指位置一定不準（因為閉著眼睛嘛），可能差了好幾公釐。但手套會壓縮，所以差一點點沒關係。如果你的手指是硬邦邦的鐵做的，差 3 公釐不是碰不到、就是把東西撞飛了。

**第三，東西快滑掉的時候，手套會自動抓更緊。** 這是最神奇的——當物體開始往下滑，手套的材料會被拉扯變形，反而讓接觸變得更緊密、摩擦力變大，於是就止住了。這完全不需要你去「想」，是材料自己做到的！

所以這一章最重要的一句話是：抓東西這件事，「軟」比「準」更重要。

科學家花了三十年想讓機器人抓東西，一開始都想用超級精密的相機和超級準確的手指。後來才發現：做一隻軟軟的手，比做一隻超準的手有用多了。現在很多機器人的手都是矽膠做的，軟軟的、可以彎，抓草莓、抓雞蛋都沒問題——而且它根本不知道自己抓的是什麼！

## 叫獸提醒與常見錯誤

- 誤區一：以為接觸點越多抓得越穩。分布比數量重要——三個聚集的接觸點可能不如兩個分散的（例題 21-2）。
- 誤區二：忘記手指只能推不能拉。因此是「正」線性跨度而非一般線性跨度，這使所需的接觸點數增加。
- 誤區三：使用樂觀的摩擦係數。實際  $\mu$  可能因油污、灰塵、濕度而降低數倍，設計時應取保守值並留餘裕。
- 誤區四：混淆力封閉與形封閉。前者允許用摩擦、後者完全不依賴摩擦；形封閉更嚴格但需要更多接觸點。
- 誤區五：忽略關入籠這個中間選項。許多任務不需要完全夾緊，「圍住不讓它跑掉」就足夠且穩健得多。
- 誤區六：過度追求嚴格的封閉條件。應用最寬鬆但足夠的那一個，過度嚴格會不必要地提高感測與控制的要求。
- 誤區七：以為理論品質指標高就一定成功。指標建立在幾何已知、摩擦已知、接觸點精確的假設上，且只考慮靜力學。

- 誤區八：把柔順性只當成安全考量。它實際上增加了抓取能力（面接觸能抵抗扭轉）並取代了大量的感測與計算。
- 誤區九：直接考慮多指手而不先評估簡單方案。真空吸盤、欠驅動手在許多工業場合更快、更便宜、更可靠。
- 誤區十：學習式抓取完全取代解析方法。前者缺乏保證，實務上最佳解是「學習產生候選、解析驗證品質」的混合式。
- 誤區十一：忽略深度相機在透明與反光物體上的失效。這類物體是學習式抓取的主要失敗來源（第 6.7 節）。
- 誤區十二：把抓取與操作混為一談。抓穩是靜態問題，操作是動態問題；手內操作至今仍是研究前沿。

## 課堂討論

1. 開頭的手套實驗中，厚手套同時降低了感知能力並提升了柔順性，而結果是成功的。請討論：這對「該把資源投在感測還是機構」這個工程決策有何啟示？
2. 例題 21-3 顯示柔性指墊把力變化縮小 50 倍，使原本需要  $\pm 0.05$  mm 精度的任務變為只需  $\pm 2$  mm。請再舉出兩個「用機構降低對感測或控制要求」的例子（可回顧第 20 章）。
3. 顆粒堵塞夾爪在同一裝置中實現了「柔軟以適應」與「堅硬以固定」兩個矛盾的需求。請討論這種「可切換的物理性質」還能應用在哪些機器人問題上？
4. 學習式抓取沒有力封閉那樣的數學保證。請討論：在哪些應用中這個缺點是可接受的？在哪些應用中不可接受？如何設計混合式系統來兼顧？
5. 人手的靈巧主要來自機構（分層柔軟度、筋腱驅動、欠驅動耦合）而非感知。請討論：這是否意味著機器人手的研究方向應該調整？多指靈巧手的研究價值何在？

## 章末摘要

- 抓取問題分三個層次：接觸建模（單點能傳遞什麼力）、抓取分析（這個抓法穩不穩）、抓取規劃（該抓哪裡）。
- 三種接觸模型：無摩擦點接觸（1 自由度）、有摩擦點接觸（3 自由度）、軟指接觸（4 自由度，可抵抗扭轉）。

- 摩擦錐的半頂角  $\alpha = \arctan(\mu)$ ；接觸力必須落在錐內，否則會滑動。
- 摩擦係數不確定性大（可差四倍），設計時應取保守值。
- 抓取矩陣  $G$  把各接觸力映射為物體所受的合力與合力矩，角色類似雅可比。
- 力封閉：能以摩擦錐內的接觸力抵抗任意方向的外力；數學條件是接觸力的正線性跨度涵蓋整個扳手空間。
- 因手指只能推不能拉，故為「正」線性跨度，這使所需的接觸點數增加。
- 力封閉的最少接觸點數：平面有摩擦 2 點、空間有摩擦 3 點；無摩擦則分別需 4 點與 7 點。
- 平面二指抓取的判準：兩接觸點的連線必須落在兩個摩擦錐之內。
- 形封閉完全不依賴摩擦（更嚴格、需更多接觸點），關入籠則只包圍不夾緊（最寬鬆、對誤差不敏感）。
- 抓取品質指標有四類：最小奇異值  $\varepsilon$ 、抓取體積、接觸點分布、誤差強健性；但理論指標與實機成功率仍有落差。
- 接觸點的分布比數量更重要——三個聚集的接觸點可能不如兩個分散的。
- 柔順性在抓取中的四個作用：增加接觸面積、吸收位置誤差、滑動時自動增加摩擦、多指自適應包絡。
- 本章核心洞見：抓取的成功往往不是來自精準，而是來自順應；柔順性取代了大量的感測與計算。
- 三種機械手設計：剛性多指手（能力強但昂貴複雜）、欠驅動手（機構自動適應、控制極簡）、軟性夾爪（安全、適應性強但出力小）。
- 學習式抓取直接由感測資料預測可行的抓取位姿，擅長處理未知物體與雜亂場景，但缺乏理論保證；實務最佳解是混合式。

## 觀念題與計算題

### 觀念題

1. 說明抓取問題的三個層次，並指出各自的核心問題。
2. 列舉抓取困難的五個因素，並說明它們為何難以用「精確建模」的路線解決。
3. 比較三種接觸模型可傳遞的力與自由度，並說明軟指接觸為何特別重要。

4. 定義摩擦錐並寫出其半頂角公式；說明摩擦係數的不確定性對設計的影響。
5. 說明抓取矩陣的作用，並將其與雅可比矩陣類比。
6. 定義力封閉，並解釋為何條件中「任意」兩字是關鍵。
7. 為何力封閉的條件是「正」線性跨度而非一般線性跨度？這帶來什麼推論？
8. 寫出力封閉的最少接觸點數（平面與空間、有摩擦與無摩擦），並說明摩擦的價值。
9. 敘述平面二指抓取的力封閉判準，並解釋其幾何直覺。
10. 比較力封閉、形封閉與關入籠三個概念，並說明選擇的原則。
11. 列舉四種抓取品質指標，並說明理論指標與實機成功率落差的三個原因。
12. 說明柔順性在抓取中的四個作用，並解釋「順應勝過精準」的意義。
13. 比較剛性多指手、欠驅動手與軟性夾爪的設計哲學與適用場合。
14. 說明欠驅動手指如何達成自適應包絡，以及這體現了什麼設計原則。
15. 說明學習式抓取的四個步驟與三種訓練資料來源。
16. 比較解析式與學習式抓取的優缺點，並說明混合式方法的架構。

## 計算題

1. 某夾爪的指墊與工件的摩擦係數  $\mu = 0.6$ 。(a) 求摩擦錐半頂角。(b) 若兩接觸點的連線與法線夾角為  $25^\circ$ ，判斷是否力封閉。(c) 求此  $\mu$  值下可容許的最大夾角。(d) 若表面被油污染使  $\mu$  降為 0.25，重新判斷 (b) 的情形。
2. 一個正方形物體（邊長 60 mm）被四指抓持，接觸點位於四個邊的中點。 $\mu = 0.5$ 。(a) 求各接觸點的法線方向。(b) 判斷此抓取能否抵抗任意方向的平移力。(c) 若移除其中一指（剩三指），重新判斷。(d) 若四指改為集中在相鄰的兩個邊上，分析結果。
3. 某夾爪標稱夾持力 15 N，定位誤差  $\pm 3$  mm。(a) 若指墊剛度為  $5 \times 10^4$  N/m，求夾持力的變化範圍。(b) 若物體容許的最大夾持力為 25 N，判斷是否可行。(c) 求使夾持力變化不超過  $\pm 8$  N 所需的指墊剛度上限。(d) 若堅持使用 (a) 的剛性指墊，求所需的定位精度。
4. 某三指夾爪抓取半徑 40 mm 的圓柱體，三指均勻分布（相隔  $120^\circ$ ）， $\mu = 0.45$ ，每指的法向力為 12 N。(a) 求摩擦錐半頂角與各指的最大切向力。(b) 求單指能產生的最大力矩（對圓心）。(c) 求三指同向施力時的總力矩。(d) 若物體重 1.5 kg，判斷此抓取能否在

垂直提起時抵抗重力（提示：比較三指的最大切向力總和與重量）。

## 動手做與專題延伸

### 本章實作：抓取分析工具

延續第 20 章的力控制模組，加入抓取分析功能。

- **階段一**：實作平面抓取的摩擦錐計算與二指力封閉判定，用例題 21-1 驗證。
- **階段二**：實作抓取矩陣的建構與多指的力封閉判定（可用線性規劃或凸包判斷），用例題 21-2 驗證。
- **階段三**：實作抓取品質指標（至少包含  $\epsilon$  指標與接觸點分布指標），並比較不同抓取配置的評分。
- **階段四**：實作簡易的抓取合成——給定一個平面多邊形物體，自動搜尋所有可行的二指抓取，並依品質排序。
- **階段五**：加入誤差強健性分析——對接觸點加入隨機擾動，統計力封閉成立的比例。

### 專題延伸

- **抓取品質地圖**：對一個二維物體（如不規則多邊形），窮舉所有可能的二指抓取位置對，計算每一組的品質指標，繪成熱力圖。你會看到「好抓的位置」形成明顯的區塊——這正是學習式方法試圖用網路學會的東西。比較這張圖與你的直覺是否一致。
- **柔順性的量化實驗**：製作或取得三種不同硬度的指墊，設計一組實驗量化「柔順性換來多少誤差容忍」。具體作法是：對每種指墊，逐步增加夾爪的定位誤差，記錄抓取成功率降到 50% 時的誤差值。繪出「指墊剛度 vs 容許誤差」曲線，並與例題 21-3 的理論預測比較。
- **進階：欠驅動手指的設計與分析**。設計一根雙關節的欠驅動手指（可用 3D 列印或紙板製作），以彈簧與連桿實現力的自動分配。（一）分析其在不同物體形狀下的接觸點分布；（二）量測抓取不同直徑圓柱體時的接觸力分配；（三）與剛性手指比較，量化「自適應」帶來的成功率提升。若有能力，可進一步用第 13 章的靜力學分析推導其力分配的理論值，並與實測比較。

## 參考文獻與延伸閱讀

- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic

Manipulation, CRC Press ( 第 5 章：多指抓取的完整數學理論 ) 。

- Bicchi, A. and Kumar, V., "Robotic Grasping and Contact: A Review," ICRA 2000 ( 抓取理論的經典回顧論文 ) 。
- Nguyen, V.-D., "Constructing Force-Closure Grasps," IJRR, 1988 ( 力封閉抓取合成的奠基工作 ) 。
- Rodriguez, A., Mason, M. T. and Ferry, S., "From Caging to Grasping," IJRR, 2012 ( 關入籠與抓取的關係 ) 。
- Ferrari, C. and Canny, J., "Planning Optimal Grasps," ICRA 1992 (  $\epsilon$  品質指標的提出 ) 。
- Birglen, L., Laliberté, T. and Gosselin, C., Underactuated Robotic Hands, Springer ( 欠驅動手的設計與分析 ) 。
- Brown, E. et al., "Universal Robotic Gripper Based on the Jamming of Granular Material," PNAS, 2010 ( 顆粒堵塞夾爪 ) 。
- Mahler, J. et al., "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," RSS 2017 ( 結合解析指標與深度學習的代表性工作 ) 。
- Levine, S. et al., "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection," IJRR, 2018 ( 大規模真實資料的自我監督學習 ) 。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer ( 抓取與靈巧操作專章 ) 。